ORACLE NETSUITE SUITESCRIPT 2.1

SuiteScript 2.1 Complete API Reference & Script Samples Manual

Authoritative technical reference manual detailing all 55 SuiteScript 2.1 modules, including core business logic, query, workbook, UI widgets, machine learning, and security APIs, enriched with official runnable script samples for each module directly from Oracle NetSuite documentation.

DOCUMENT CREATOR & TECHNICAL AUTHOR

Krushna Gore

NetSuite Solution Architect • SuiteCloud Platform Specialist

55	2,065+	160	250+
TOTAL MODULES	METHODS & MEMBERS	OFFICIAL CODE SAMPLES	OBJECT CLASSES

Created by Krushna Gore • SuiteCloud Platform Technical Documentation • Official NetSuite SuiteScript 2.1 Reference • NetSuite Release 2026

Table of Contents & Quick Navigation

#	Module Name & Link	Code Samples	Supported Script Types	Permissions Required
1	N/action Module	3 sample(s)	Client and server scripts	-
2	N/auth Module	1 sample(s)	Server scripts	-
3	N/cache Module	3 sample(s)	Server scripts	-
4	N/certificateControl Module	5 sample(s)	Server scripts	Certificate Management
5	N/commerce Modules	1 sample(s)	Client and server scripts	-
6	N/compress Module	2 sample(s)	Server scripts	-
7	N/config Module	1 sample(s)	Server scripts	-
8	N/crypto Module	2 sample(s)	Server scripts	-
9	N/crypto/certificate Module	2 sample(s)	Server scripts	Certificate Access
10	N/crypto/random Module	1 sample(s)	Client and server scripts; server scripts support SuiteScript 2.1 only	-
11	N/currency Module	1 sample(s)	Client and server scripts	-
12	N/currentRecord Module	2 sample(s)	Client scripts	-
13	N/dataset Module	2 sample(s)	Server scripts	SuiteAnalytics Workbook
14	N/documentCapture Module	4 sample(s)	Server scripts SuiteScript 2.1 only	-
15	N/email Module	1 sample(s)	Client and server scripts	-
16	N/encode Module	1 sample(s)	Server scripts	-
17	N/error Module	2 sample(s)	Server scripts	-
18	N/file Module	6 sample(s)	Server scripts	-
19	N/format Module	4 sample(s)	Client and server scripts	-
20	N/format/i18n Module	8 sample(s)	Client and server scripts	-
21	N/http Module	2 sample(s)	Client and server scripts	-
22	N/https Module	10 sample(s)	Client and server scripts	-
23	N/https/clientCertificate Module	1 sample(s)	Server scripts	Certificate Access
24	N/keyControl Module	2 sample(s)	Server scripts	Key Management
25	N/llm Module	9 sample(s)	Server scripts SuiteScript 2.1 only	-
26	N/log Module	1 sample(s)	Client and server scripts Limitations apply to client scripts	-
27	N/machineTranslation Module	1 sample(s)	Server scripts SuiteScript 2.1 only	-
28	N/manufacturing/productionCharges Module	1 sample(s)	Â	Â
29	N/pgp Module	4 sample(s)	Server scripts SuiteScript 2.1 only	-
30	N/piremoval Module	1 sample(s)	Server scripts	Remove Personal Information Create Remove Personal Information Run
31	N/plugin Module	2 sample(s)	Server scripts	-
32	N/portlet Module	1 sample(s)	Client scripts	-
33	N/query Module	6 sample(s)	Client and server scripts	SuiteAnalytics Workbook
34	N/record Module	6 sample(s)	Client and server scripts	-
35	N/recordContext Module	1 sample(s)	Client and server scripts	-
36	N/redirect Module	1 sample(s)	Server scripts	-
37	N/render Module	4 sample(s)	Server scripts	Advanced PDF/HTML Templates Custom PDF Layouts
38	N/runtime Module	2 sample(s)	Client and server scripts	Â

#	Module Name & Link	Code Samples	Supported Script Types	Permissions Required
39	N/scriptTypes/restlet Module	1 sample(s)	RESTlet scripts	-
40	N/search Module	9 sample(s)	Client and server scripts	Perform Search Persist Search Publish Search
41	N/sftp Module	3 sample(s)	Server scripts	Key Access (needed when public key authentication is used)
42	N/sso Module	1 sample(s)	Client and server scripts	-
43	N/suiteAppInfo Module	1 sample(s)	Client and server scripts	-
44	N/task Module	5 sample(s)	Server scripts	Tasks
45	N/task/accounting/recognition Module	3 sample(s)	Server scripts	(Transactions) Revenue Arrangement
46	N/transaction Module	1 sample(s)	Client and server scripts	-
47	N/translation Module	5 sample(s)	Client and server scripts	-
48	N/ui/dialog Module	4 sample(s)	Client scripts	-
49	N/ui/message Module	1 sample(s)	Client scripts	-
50	N/ui/serverWidget Module	10 sample(s)	Server scripts	-
51	N/url Module	4 sample(s)	Client and server scripts	-
52	N/util Module	1 sample(s)	Client and server scripts	-
53	N/workbook Module	2 sample(s)	Server scripts	SuiteAnalytics Workbook
54	N/workflow Module	1 sample(s)	Server scripts	Workflow
55	N/xml Module	1 sample(s)	Client and server scripts	-

1. N/action Module

Load the N/action module to execute business logic to update the state of a record. Action APIs emulate NetSuite user interface buttons.

`define(['N/action'], (action) => { ... })` ✓ 3 Code Sample(s) Scripts: Client and server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/action'], (action) => { // Access methods and properties on action });
```

Use the N/action module to execute business logic to update the state of records in view mode. To execute business logic on records in edit mode, use the record macro APIs, which are included in the N/record Module module. See Record Object Members and Macro Object Members. Action and Macro APIs are the programmatic equivalent to clicking a button in the UI. To learn more, see Overview of Record Action and Macro APIs.

The changes that you make to records with N/action module APIs are persisted in the database immediately. For example, consider the timebill record. After you click the Approve button in the UI, the timebill and its entries are saved in an approved state, and this change is immediately updated in the database.

Governance for action module APIs varies for actions and record types. See the action help for governance information specific to actions and record types.

A limited number of individual actions for specific record types are supported. For details, see Supported Record Actions.

For supported script types, see individual member topics listed below.

N/action Module Members

Return Type / Value Type

Supported Script Types

Client and server scripts

Encapsulates a NetSuite record action.

Plain JavaScript Object

Client and server scripts

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	`action.Action`	`Object`	Client and server scripts	Encapsulates a NetSuite record action.
OBJECT	`Plain JavaScript Object`	`Object`	Client and server scripts	A plain JavaScript object of actions available for a record type.
METHOD	`action.execute(options)`	`Object`	Client and server scripts	Executes the record action and returns action results in an object.
METHOD	`action.execute.promise(options)`	`Promise`	Client scripts	Asynchronously executes the record action and returns the action results in an object.
METHOD	`action.executeBulk(options)`	`string`	Client and server scripts	Executes an asynchronous bulk record action and returns its task ID for later status inquiry.
METHOD	`action.find(options)`	`Object`	Client and server scripts	Returns a plain JavaScript object of available record actions for the given record type.
METHOD	`action.find.promise(options)`	`Promise`	Client scripts	Asynchronously returns a plain JavaScript object of available record actions for the given record type.
METHOD	`action.get(options)`	`action.Action`	Client and server scripts	Returns an executable record action for the given record type.
METHOD	`action.get.promise(options)`	`Promise`	Client scripts	Asynchronously returns an executable record action for the given record type.

Action Object Members

The following members are called on action.Action.

Return Type/Value Type

Supported Script Types

Client and server scripts

Executes the action and returns the action results in an object.

Action.promise(options)

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	`Action(options)`	`Object`	Client and server scripts	Executes the action and returns the action results in an object.
METHOD	`Action.promise(options)`	`Promise`	Client scripts	Executes the action asynchronously and returns the action results in an object.

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Action.execute(options)	Object	Client and server scripts	Executes the action and returns the action results in an object.
METHOD	Action.execute.promise(options)	Promise	Client scripts	Executes the action asynchronously and returns the action results in an object.
METHOD	Action.executeBulk(options)	string	Client and server scripts	Executes an asynchronous bulk record action and returns its task ID for later status inquiry.
METHOD	action.getBulkStatus(options)	Object	Client and server scripts	Returns the current status of action.executeBulk(options) with the given task ID.
PROPERTY	Action.description	string	Client and server scripts	The action description.
PROPERTY	Action.id	string	Client and server scripts	The ID of the action. For a list of action IDs, see Supported Record Actions.
PROPERTY	Action.label	string	Client and server scripts	The action label.
PROPERTY	Action.parameters	Object	Client and server scripts	The action parameters.
PROPERTY	Action.recordType	string	Client and server scripts	The type of the record on which the action is to be performed. For a list of record types, see record.Type.

▶ Official NetSuite Script Samples (3 Sample(s))

📄 Sample 1: Locate and Execute an Action on a Timebill Record

```
/**
 * @NApiVersion 2.x
 */
require(['N/action', 'N/record'], function(action, record) {
    // create timebill record
    var rec = record.create({
        type: 'timebill',
        isDynamic: true
    });
    rec.setValue({
        fieldId: 'employee',
        value: 104
    });
    rec.setValue({
        fieldId: 'location',
        value: 312
    });
    rec.setValue({
        fieldId: 'hours',
        value: 5
    });
    var recordId = rec.save();

    var actions = action.find({
        recordType: 'timebill',
        recordId: recordId

    });

    log.debug("We've got the following actions: " + Object.keys(actions));
    if (actions.approve) {
        var result = actions.approve();
        log.debug("Timebill has been successfully approved");
    } else {
        log.debug("The timebill is already approved");
    }
});

// Outputs the following:
// We've got the following actions: approve, reject
// Timebill has been successfully approved
```

📄 Sample 2: Find Actions Available for the Timebill Record Asynchronously Using Promise Methods

```
/**
 * @NApiVersion 2.x
 * @NScriptType ClientScript
 */
require(['N/action', 'N/record'], function(action, record) {
    // create timebill record
    var rec = record.create({
        type: 'timebill',
        isDynamic: true
    });
    rec.setValue({
        fieldId: 'employee',
        value: 104
    });
    rec.setValue({
        fieldId: 'location',
        value: 312
    });
    rec.setValue({
        fieldId: 'hours',
        value: 5
    });
    var recordId = rec.save();

    // find all qualified actions and then execute approve if available
    action.find.promise({
        recordType: 'timebill',
        recordId: recordId
    }).then(function(actions) {
        console.log("We've got the following actions: " + Object.keys(actions));
        if (actions.approve) {
            actions.approve.promise().then(function(result) {
                console.log("Timebill has been successfully approved");
            });
        } else {
            console.log("The timebill is already approved");
        }
    });
});

// Outputs the following:
// We've got the following actions:
// The timebill has been successfully approved
```

📄 Sample 3: Execute a Bulk Action on a Timebill Record

```
/**
 * @NApiVersion 2.x
 */
require(['N/action', 'N/util']function(action, util) {

    // 1a) Bulk execute the specified action on a provided list of record IDs.
    // The params property is an array of parameter objects where each object contains required recordId and arbitrary additional parameters.
    var handle = action.executeBulk({
        recordType: "timebill",
        id: "approve",
        params: [{
                recordId: 1,
                note: "this is a note for 1"
            },
            {
                recordId: 5,
                note: "this is a note for 5"
            },
            {
                recordId: 23,
                note: "this is a note for 23"
            }
        ]
    })
});

// 1b) Bulk execute the specified action on a provided list of record IDs.
// The parameters in the previous sample are similar and can be generated programatically using the map function.
var searchResults = /* result of a search, for example, [1, 5, 23] */ ;
var handle = action.executeBulk({
    recordType: "timebill",
    id: "approve",
    params: searchResults.map(function(v) {
        return {
            recordId: v,
            note: "this is a note for " + v
        };
    })
});

// 2a) Bulk execute the specified action on a provided list of record IDs.
// This time with homogenous parameters, that is, all parameter objects are equal except recordId.
var handle = action.executeBulk({
    recordType: "timebill",
    id: "approve",
    params: searchResults.map(function(v) {
        return {
            recordId: v,
            foo: "bar",
            name: "John Doe"
        };
    })
});

// 2b) Bulk execute the specified action on a provided list of record IDs.
// This time with homogenous parameters. Equivalent to the previous sample.
var commonParams = {
    foo: "bar",
    name: "John Doe"
};
var handle = action.executeBulk({
    recordType: "timebill",
    id: "approve",
    params: searchResults.map(function(v) {
        return util.extend({
            recordId: v
        }, commonParams);
    })
});

// 3) Bulk execute the specified action on a provided list of record IDs.
// This is the simplest usage with no extra parameters besides the record ID.
var handle = action.executeBulk({
    recordType: "timebill",
    id: "approve",
    params: searchResults.map(function(v) {
        return {
            recordId: v
        }
    })
});

// 4) Bulk execute the specified action on all record instances that qualify.
// Since we don't have a list of recordIds in hand, we only provide the callback
// that will later be used to transform a recordId to the corresponding parameters object.
var handle = action.executeBulk({
    recordType: "timebill",
    id: "approve",
    condition: action.ALL_QUALIFIED_INSTANCES,
    paramCallback: function(v) {
        return {
```

```
            recordId: v,
            note: "this is a note for " + v
        };
    }
});

// 5) Get a particular action for a particular record type.
var approveTimebill = action.get({
    recordType: "timebill",
    id: "approve"
});

// 6) Bulk execute the previously obtained action on a provided list of record IDs.
// Params are generated the same way as above in action.executeBulk().
var handle = approveTimebill.executeBulk({
    params: searchResults.map(function(v) {
        return {
            recordId: v,
            note: "this is a note for " + v
        };
    })
});

// 7) Bulk execute the previously obtained action on all record instances that qualify.
var handle = approveTimebill.executeBulk({
    condition: action.ALL_QUALIFIED_INSTANCES,
    paramCallback: function(v) {
        return {
            recordId: v,
            note: "this is a note for " + v
        };
    }
});

// 8) Get status of a bulk action execution.
var res = action.getBulkStatus({
    taskId: handle
}); // returns a RecordActionTaskStatus object
log.debug(res.status);
});
```

2. N/auth Module

Load the N/auth module to change your NetSuite login credentials.

`define(['N/auth'], (auth) => { ... })` ✓ 1 Code Sample(s) Scripts: Server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/auth'], (auth) => { // Access methods and properties on auth });
```

Use the N/auth module to change your NetSuite login credentials.

N/auth Module Script Sample

N/auth Module Members

Return Type / Value Type

Supported Script Types

auth.changeEmail(options)

Changes the current user's NetSuite email address (user name).

auth.changePassword(options)

Changes the current user's NetSuite password.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	auth.changeEmail(options)	void	Server scripts	Changes the current user's NetSuite email address (user name).
METHOD	auth.changePassword(options)	void	Server scripts	Changes the current user's NetSuite password.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: N/auth Module Script Sample

```
/**
 * @NApiVersion 2.x
 */

// This script changes the currently logged-in user's NetSuite email address and password.
require(['N/auth'],function(auth) {
    function changeEmailAndPassword() {
        var password = 'myCurrentPassword';
        auth.changeEmail({
            password: password,
            newEmail: 'auth_test@newemail.com'
        });
        auth.changePassword({
            currentPassword: password,
            newPassword: 'myNewPa55Word'
        });
    }
    changeEmailAndPassword();
});
```

3. N/cache Module

Load the N/cache module to enable the caching of needed data and improve performance.

`define(['N/cache'], (cache) => { ... })` ✓ 3 Code Sample(s) Scripts: Server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/cache'], (cache) => { // Access methods and properties on cache });
```

Use the N/cache module to enable temporary, short-term storage of data. Using a cache improves performance by eliminating the need for scripts to repeatedly retrieve the same piece of data. You can use this module to build a cache to store and retrieve string values using a specific key.

You can create a cache that is available (1) to the current script only, (2) to all server scripts in the current bundle, or (3) to all server scripts in your NetSuite account. Data is stored in the cache according to its time to live (ttl) specified in the Cache.put(options) method.

N/cache Module Members

Return Type / Value Type

Supported Script Types

Encapsulates a cache which is a segment of memory that can be used to store data on a temporary, short-term basis.

cache.getCache(options)

Checks for a cache object with the specified name. If the cache object exists, this method returns it. If the cache object does not exist, the system creates and returns a new cache object.

Holds the string values that describe the availability of the cache. Use this enum to set the value of the Cache.scope property and the options.scope parameter of the cache.getCache(options) method.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	**cache.Cache**	`Object`	Server scripts	Encapsulates a cache which is a segment of memory that can be used to store data on a temporary, short-term basis.
METHOD	**cache.getCache(options)**	`cache.Cache`	Server scripts	Checks for a cache object with the specified name. If the cache object exists, this method returns it. If the cache object does not exist, the system creates and returns a new cache object.
ENUM	**cache.Scope**	`enum`	Server scripts	Holds the string values that describe the availability of the cache. Use this enum to set the value of the Cache.scope property and the options.scope parameter of the cache.getCache(options) method.

Cache Object Members

The following members are called on cache.Cache.

Return Type/Value Type

Supported Script Types

Cache.get(options)

Retrieves a value from the cache based on a key that you provide. If the requested value is not in the cache, the method calls the user-defined function identified by a method parameter.

Cache.put(options)

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**Cache.get(options)**	`string`	Server scripts	Retrieves a value from the cache based on a key that you provide. If the requested value is not in the cache, the method calls the user-defined function identified by a method parameter.
METHOD	**Cache.put(options)**	`string`	Server scripts	Puts a value into the cache.
METHOD	**Cache.remove(options)**	`string`	Server scripts	Removes a value from the cache.
PROPERTY	**Cache.name**	`string`	Server scripts	The name of the cache.
PROPERTY	**Cache.scope**	`string`	Server scripts	The availability of the cache. A cache can be made available to the current script only, to all scripts in the current bundle, or to all scripts in your NetSuite account. Set this value using the cache.Scope enum.

▶ Official NetSuite Script Samples (3 Sample(s))

📄 Sample 1: Look Up Folder IDs

```
const FOLDER_CACHE_NAME = 'folder_cache';

function folderCacheLoader(context) {
    const PARENT_FOLDER_ID = 0;
    const FOLDER_NAME = 1;
    const folderCacheKey = context.key.split('/');
    const parentFolderId = folderCacheKey[PARENT_FOLDER_ID];
    const folderName = folderCacheKey[FOLDER_NAME];

    var folderId = null;
    search.create ({
        type: search.Type.FOLDER,
        columns: ['internalid'],
        filters: [
            ['parent', search.Operator.ANYOF, parentFolderId],
            'AND',
            ['name', search.Operator.IS, folderName]
        ]
    }).run()
    .each(function(folder) {
        folderId = folder.id;
        return false;
    });

    if (!folderId) {
        var folder = record.create({
            type: record.Type.FOLDER
        });
        folder.setValue({
            fieldId: 'parent',
            value: parentFolderId
        });
        folder.setValue({
            fieldId: 'name',
            value: folderName
        });
        folderId = folder.save();
    }
    return folderId;
}

function getFolderCache() {
    return cache.getCache({
        name: FOLDER_CACHE_NAME
    });
}

function folderKey(folderName, parentFolderId) {
    return[parentFolderId, folderName].join('/');
}

function getFolder(folderName, parentFolderId) {
    return getFolderCache().get({
        key: folderKey(folderName, parentFolderId),
        loader: folderCacheLoader
    });
}
```

📄 Sample 2: Retrieve Name of a City Based on a ZIP Code Using Cache and a Custom Loader Function

```
/**
 * @NApiVersion 2.1
 * @NScriptType Suitelet
 */

// This script retrieves the name of a city based on a ZIP code from a cache.
define(['N/cache', '/SuiteScripts/zipToCityIndexCacheLoader'], function(cache, lib) {
    const ZIP_CODES_CACHE_NAME = 'ZIP_CODES_CACHE';
    const ZIP_TO_CITY_IDX_JSON = 'ZIP_TO_CITY_IDX_JSON';

    function getZipCodeToCityLookupObj() {
        const zipCache = cache.getCache({
            name: ZIP_CODES_CACHE_NAME
        });
        const zipCacheJson = zipCache.get({
            key: ZIP_TO_CITY_IDX_JSON,
            loader: lib.zipCodeDatabaseLoader
        });
        return JSON.parse(zipCacheJson);
    }

    function findCityByZipCode(options) {
        return getZipCodeToCityLookupObj()[String(options.zip)];
    }

    function onRequest(context) {
        const start = new Date();
        if (context.request.parameters.purgeZipCache === 'true') {
            const zipCache = cache.getCache({
                name: ZIP_CODES_CACHE_NAME
            });
            zipCache.remove({
                key: ZIP_TO_CITY_IDX_JSON
            });
        }
        const cityName = findCityByZipCode({
            zip: context.request.parameters.zipcode
        });

        context.response.writeLine(cityName || 'Unknown :(');

        if (context.request.parameters.auditPerf === 'true') {
            context.response.writeLine('Time Elapsed: ' + (new Date().getTime() - start.getTime()) + ' ms');
        }
    }
    return {
        onRequest: onRequest
    };
});
```

📄 Sample 3: Retrieve Name of a City Based on a ZIP Code Using Cache and a Custom Loader Function

```
/**
 * zipToCityIndexCacheLoader.js
 * @NApiVersion 2.1
 * @NModuleScope Public
 */

//This custom module is a loader function that uses a CSV file to retrieve a value that was missing from a cache.
define(['N/file', 'N/cache'], function(file, cache) {
    const ZIP_CODES_CSV_PATH = '/SuiteScripts/Resources/free-zipcode-CA-database-primary.csv';

    function trimOuterQuotes(str) {
        return (str || '').replace(/^"+/, '').replace(/"+$/, '');
    }

    function zipCodeDatabaseLoader(context) {
        log.audit({
            title: 'Loading Zip Codes',
            details: 'Loading Zip Codes for ZIP_CODES_CACHE'
        });
        const zipCodesCsvText = file.load({
            id: ZIP_CODES_CSV_PATH
        }).getContents();
        const zipToCityIndex = {};
        const csvLines = zipCodesCsvText.split('\n');
        util.each(csvLines.slice(1), function(el) {
            var cells = el.split(',');
            var key = trimOuterQuotes(cells[0]);
            var value = trimOuterQuotes(cells[2]);
            if (parseInt(key, 10))
                zipToCityIndex[String(key)] = value;
        });
        return zipToCityIndex;
    }

    return {
        zipCodeDatabaseLoader: zipCodeDatabaseLoader
    }
});
```

4. N/certificateControl Module

Load the N/certificateControl module to enable scripting access to the Digital Certificates list found at Setup > Company > Certificates. You can use this module to find the correct certificate for a subsidiary and check the file type. For more information, see Digital Signing and Uploading Digital Certificates.

`define(['N/certificateControl'], (certificateControl) ⇒ { ... })` ✓ 5 Code Sample(s) Scripts: Server scripts Permissions: Certificate Management

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/certificateControl'], (certificateControl) ⇒ { // Access methods and properties on certificateControl });
```

Use the N/certificateControl module to enable scripting access to the Digital Certificates list found in the UI at Setup > Company > Certificates. You can use this module to find, create, update, read and delete certificate records. For more information, see Digital Signing and Uploading Digital Certificates.

To access the N/certificateControl module, you must use the Execute As Role field on the script deployment record. Select either the Administrator role or a custom role with the Certificate Access permission. For more information, see Access to Digital Certificates.

The certificate record holds information for a digital certificate, but it is not a standard NetSuite record and cannot be accessed with the N/record.

N/certificateControl Module Members

Return Type / Value Type

Supported Script Types

certificateControl.Certificate

Encapsulates a digital certificate record.

certificateControl.findCertificates(options)

Returns metadata about the certificate(s).

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	certificateControl.Certificate	object	Server scripts	Encapsulates a digital certificate record.
METHOD	certificateControl.findCertificates(options)	object	Server scripts	Returns metadata about the certificate(s).
METHOD	certificateControl.findUsages(options)	object[]	Server scripts	Returns an audit trail of how a certificate has been used. Includes operations performed with time stamps.
METHOD	certificateControl.createCertificate(options)	certificateControl.Certificate	Server scripts	Creates a certificate record using a file from the File Cabinet. After saving with Certificate.save(), the certificate is accessible on the Certificates.
METHOD	certificateControl.deleteCertificate(options)	string	Server scripts	Deletes a certificate record that has been uploaded to the Certificates list in the UI or created using certificateControl.createCertificate(options) and saved with Certificate.save().
METHOD	certificateControl.loadCertificate(options)	certificateControl.Certificate	Server scripts	Loads a certificate record that has been uploaded to the Certificates list in the UI or created using certificateControl.createCertificate(options).
METHOD	certificateControl.lock(options)	string	Server scripts	Locks a certificate record so that it cannot be edited.
METHOD	certificateControl.unlock(options)	string	Server scripts	Unlocks a certificate record that has been locked with certificateControl.lock(options).
ENUM	certificateControl.Operation	enum	Server scripts	Holds the values for the operation when searching for certificates with certificateControl.findUsages(options).
ENUM	certificateControl.Operator	enum	Server scripts	Holds the values for search operators to use with the name and description parameters of the certificateControl.findCertificates(options) method.
ENUM	certificateControl.Type	enum	Server scripts	Holds the values for the certificate file type to use with the type parameter of the certificateControl.findCertificates(options) method.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Certificate.save()	object containing the script ID of the new certificate record	Server scripts	Saves a certificate record.
PROPERTY	Certificate.description	string	Server scripts	Describes the certificate record.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Certificate.file	File Object Members object	Server scripts	Includes the properties of the file uploaded to create the certificate.
PROPERTY	Certificate.name	string	Server scripts	The name of the certificate record.
PROPERTY	Certificate.monthReminder	boolean	Server scripts	The setting of the Month box for Expiration Reminders on the certificate record.
PROPERTY	Certificate.notifications	number[]	Server scripts	The internal IDs of the employees selected in the Copy Employees field on the certificate record.
PROPERTY	Certificate.password	string (write-only)	Server scripts	The password for the digital certificate. You can create a GUID for the password using Form.addSecretKeyField(options) or you can create an API secret for the secret at Setup > Company > API Secrets.
PROPERTY	Certificate.restrictions	number[]	Server scripts	The internal IDs of the employees selected in the Restrict to Employees field of the certificate record.
PROPERTY	Certificate.scriptId	string	Server scripts	The ID of the certificate record.
PROPERTY	Certificate.subsidiaries	number[]	Server scripts	The internal IDs of the subsidiaries associated with the certificate record.
PROPERTY	Certificate.threeMonthsReminder	boolean	Server scripts	Indicates the setting of the 3 Months box for Expiration Reminders on the certificate record.
PROPERTY	Certificate.weekReminder	boolean	Server scripts	Indicates the setting of the Week box for Expiration Reminders on the certificate record.

▶ Official NetSuite Script Samples (5 Sample(s))

📄 Sample 1: Filter the Digital Certificate List by Subsidiary and File Type

```
/**
 * @NApiVersion 2.1
   */
require(['N/certificateControl'],
      function(certificateControl){
         var all = certificateControl.findCertificates();
         var specificType = certificateControl.findCertificates({
              type: 'PFX'
         });
         var specificSub = certificateControl.findCertificates({
              subsidiary: 93
         });
         var specificTypeAndSub = certificateControl.findCertificates({
               type: 'PFX',
               subsidiary: 93
         });
      });;
```

📄 Sample 2: Find the Audit Trail of POST Operations for a Certificate Record Based on ID

```
/**
 * @NApiVersion 2.1
   */

require(['N/certificateControl'], function(cc){
    var usages = cc.findUsages({
        id: 'custcertificate_china',
            operation: cc.Operation.POST
       });
})
```

📄 Sample 3: Create, Modify, and Save Certificate Record Based on a File in the File Cabinet

```
/**
 * @NApiVersion 2.1
   */

require(['N/certificateControl','N/file'],function(cc, file){
    var fileObj = file.load({
        id: 'SuiteScripts/dsa.p12'
    });
    var options = {
        file : fileObj,
        password : '022b490ad4334c7e86a8304f937ec68f',
        name : 'testCert',
        description : 'testDescription',
        scriptId : '_testid',
        subsidiaries : [1,3],
        weekReminder : false,
        monthReminder : true,
        threeMonthsReminder : false
    };
    var newCertificate = cc.createCertificate(options);
    newCertificate.save();

    var loadedCertificate = cc.loadCertificate({
            scriptId : 'custcertificate_testid'
    });
    fileObj = file.load({
        id: 'SuiteScripts/ecdsa.p12'
    });
    loadedCertificate.file = fileObj;
    loadedCertificate.password = '022b490ad4334c7e86a8304f937ec68f';
    loadedCertificate.save();
})
```

📄 Sample 4: Find and Use an Existing Certificate Record

```
/**
 * @NApiVersion 2.1
   */

require(['N/certificateControl','N/https/clientCertificate'],function(cc, cert){
    var yodlee = cc.findCertificates({
        name: 'Yodlee',
        description: 'Yodlee certificate'
    });
    cert.post({
        certId:yodlee[0].id,
        url:'url',
        body:'body',
        headers:'headers'
    });
})
```

📄 Sample 5: Establish an SFTP Connection Using an SSH Key

```
/**
 * @NApiVersion 2.1
 */

require(['N/file', 'N/sftp', 'N/certificateControl'], function(file, sftp, certificateControl) {
    var certPath = 'yyy/certificates'
    var certName = 'apiclient_cert.p12';

    // Establish SFTP connection
    var connection = sftp.createConnection({
        username: 'sftpuser',
        keyId: 'custkeysftp_nft_demo_key',
        url: 'my.sftp.example.com',
        port: 22,
        directory: 'inbound',
        hostKey: 'AAAAB3NzaC1yc2EAAAADAQABAAABAQC4gYD1K4lE9QnuYEgRRQChjrAM1+bTT95e71Xv0oQ60ywVQEiedhRqSMbPiCPPB4pjpBdOmPIQCCkug+3XwAQ6uNj3UM11zoGGmg86tyEJT6q
GB0SsrQJzHTb3EG38BSrBO0WEzOWeJ8E8YODT3oAj1Nrf8Ls3JbGObRF+0uwJDIllSrFkYS3kWCV27NhBnaytGe7iLBgrJdNVlitNqkxZfK0NsAYCaJWQjQLtz+GFfN5zTbKKNsDa6s/YW7oAMMOI3Q5GQAqd
XtKY728WvxYTjr2FsYS/KM6nbq/csTvZHWLE0z2TQtB2H0IIofvEP/QvXwmgEnCeVPcNgRwdHWQf'
    });
// -------------------------------------------------------------------------

// Create new certificate
    var certScriptId = '_' + (Math.random().toString(36).substring(2, 10));
    var cert = certificateControl.createCertificate();
    cert.name = 'Test Certificate China API';
    cert.description = 'Test Certificate China created using API';
// custcertificate prefix will be added automatically
    cert.scriptId = certScriptId;
    cert.file = connection.download({
        directory: certPath,
        filename: certName
    });
//guid corresponding to the certificate's password';
    var pwd = '022b490ad4334c7e86a8304f937ec68f';
    cert.password = pwd;
    cert.save();
/**/certScriptId = 'custcertificate' + certScriptId;
    log.debug('Certificate "' + cert.name + '" successfuly created with id "' + certScriptId + '"!');
// -------------------------------------------------------------------------
// Rename certificate
    cert = certificateControl.loadCertificate({scriptId: certScriptId});
    cert.name = 'Test Certificate China API TEMP';
    cert.save();
// Verify new certificate name
/**/cert = certificateControl.loadCertificate({scriptId: certScriptId});
    log.debug('Certificate successfully renamed to "' + cert.name + '"');
// -------------------------------------------------------------------------
// Delete certificate
    certificateControl.deleteCertificate(certScriptId);
    log.debug('Certificate deleted!');
// -------------------------------------------------------------------------
// Load the deleted certificate
    // attempt to load the deleted certificate - this should error
    try {
        cert = certificateControl.loadCertificate({scriptId: certScriptId});
    }
    catch (e) {
        log.error(e.message);
    }
})
```

5. N/commerce Modules

Load modules in the N/commerce namespace to access different assets in the web store context, such as items and shopping cart. The modules within the N/commerce namespace are supported by the latest version of SuiteCommerce and by SuiteCommerce Advanced 2019.2 onwards.

`define(['N/commerces'], (commerces) ⇒ { ... })` ✓ 1 Code Sample(s) Scripts: Client and server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/commerces'], (commerces) ⇒ { // Access methods and properties on commerces });
```

Modules in the N/commerce namespace have been designed for use with SSP applications written in SuiteScript 2.x. Their primary objective is to provide web developers a homogeneous development experience when working with Commerce and SuiteScript APIs. Note that N/commerce itself is not a module.

Developers can use modules in the N/commerce namespace to access different assets in the web store context, such as items and shopping cart. The modules within the N/commerce namespace are supported by the latest version of SuiteCommerce and by SuiteCommerce Advanced 2019.2 onwards.

Before you can load N/commerce modules, you must have an active shopping session.

The modules available within the N/commerce namespace are:

Module

Description

N/commerce/recordView Module

Use the N/commerce/recordView module to provide fast, cached, and public access to the item fields and website settings.

Commerce modules are not yet available for all web store interactions. To fully customize your web store on the SuiteCloud platform, you can use Commerce APIs with SSP applications written in SuiteScript 1.0. However, Commerce APIs are not available for use with SSP applications written in SuiteScript 2.x. For more information about the Commerce APIs, see Commerce API.

General Notices

N/commerce/recordView Module Members

Provides fast, cached, and public access to item fields and website settings in commerce context.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	`recordView.viewItems(options)`	`Object | array Returns one or more Items with requested items fields as an object with field:value pairs. See the Returns section.`	Client and server scripts	Retrieves one or more Items with requested items fields from an Item Record.
METHOD	`recordView.viewWebsite(options)`	`Object Returns website and website fields as an object with field:value pairs. See the Returns section.`	Client and server scripts	Retrieves the website details with requested website fields.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: Retrieve Item and Website Data via N/commerce/recordView

```
/**
 * @NApiVersion 2.x
 * @NScriptType Suitelet
 */
define(['N/commerce/recordView'], function (recordView) {
    function service(context) {
        var result = {};
        try {
            // Retrieve item fields from web store cache
            var items = recordView.viewItems({
                ids: [101, 102],
                fields: ['itemid', 'displayname', 'salesdescription', 'baseprice']
            });

            // Retrieve website configuration and settings
            var website = recordView.viewWebsite({
                fields: ['displayname', 'entryform']
            });

            result.items = items;
            result.website = website;
            context.response.write(JSON.stringify(result));
        } catch (e) {
            log.error('Error viewing commerce records', e);
        }
    }
    return { onRequest: service };
});
```

6. N/compress Module

Load the N/compress module to compress, decompress, and archive files.

`define(['N/compress'], (compress) => { ... })` ✓ 2 Code Sample(s) Scripts: Server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/compress'], (compress) => { // Access methods and properties on compress });
```

Use the N/compress module to compress and decompress files. You can also use this module to archive multiple files in a single file archive such as TAR or ZIP file.

You can compress and decompress individual files by using compress.gzip(options) and compress.gunzip(options).

You can create an archive by using compress.createArchiver() and add multiple files to the archive.

N/compress Module Script Sample

N/compress Module Members

Return Type / Value Type

Supported Script Types

compress.Archiver

The functionality for creating archive files.

Use compress.createArchiver() to create this object.

compress.createArchiver()

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	**compress.Archiver**	Object	Server scripts	The functionality for creating archive files. Use compress.createArchiver() to create this object.
METHOD	**compress.createArchiver()**	compress.Archiver	Server scripts	Creates a compress.Archiver object.
METHOD	**compress.gunzip(options)**	file.File	Server scripts	Decompresses a file and returns it as a temporary file object.
METHOD	**compress.gzip(options)**	file.File	Server scripts	Compresses a file and returns it as a temporary file object.
ENUM	**compress.Type**	enum	Server scripts	Holds the string values for the archive types. Use this enum to set the value of the type parameter of the method Archiver.archive(options).

Archiver Object Members

The following members are called on the compress.Archiver object.

Return Type/Value Type

Supported Script Types

Archiver.add(options)

Adds a file to be archived.

Archiver.archive(options)

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**Archiver.add(options)**	void	Server scripts	Adds a file to be archived.
METHOD	**Archiver.archive(options)**	file.File	Server scripts	Creates an archive with the added files and returns it as a temporary file object.

▶ Official NetSuite Script Samples (2 Sample(s))

📄 Sample 1: N/compress Module Script Sample

```
require(['N/compress', 'N/file'], function(compress, file) {
    var unsavedTxtFile = file.create({
        fileType: 'PLAINTEXT',
        name: 'file.txt',
        contents: 'This is a sample content. This is a sample content. This is a sample content. This is only a sample.'
    });

    log.debug('Name, size, and contents of the original, uncompressed file are: ');
    log.debug('Name: ' + unsavedTxtFile.name);
    log.debug('Size: ' + unsavedTxtFile.size + 'b');
    log.debug('Contents: ' + unsavedTxtFile.getContents());

    // gzip the file with max compression (level = 9)
    var gzippedFile = compress.gzip({
        file: unsavedTxtFile,
        level: 9
    });

    log.debug('Name, size, and contents of the gzipped file are: ');
    log.debug('Name: ' + gzippedFile.name);
    log.debug('Size: ' + gzippedFile.size + 'b');
    log.debug('Contents: ' + gzippedFile.getContents().substring(0, 100));

    // gunzip the file
    var gunzippedFile = compress.gunzip({
        file: gzippedFile
    });

    log.debug('Name, size, and contents of the gunzipped file are: ');
    log.debug('Name: ' + gunzippedFile.name);
    log.debug('File size: ' + gunzippedFile.size + 'b');
    log.debug('Contents: ' + gunzippedFile.getContents());
});
```

📄 Sample 2: N/compress Module Script Sample

```
require(['N/compress', 'N/file'], function(compress, file) {
    // load/create files to be archived
    var binaryFile = file.load({
        id: 200
    });
    var textFile = file.create({
        name: 'file.txt',
        fileType: 'PLAINTEXT',
        contents: 'This is sample content.'
    });

    // create an archive as a temporary file object
    var archiver = compress.createArchiver();
    archiver.add({
        file: binaryFile
    });
    archiver.add({
        file: textFile,
        directory: 'txt/'
    });
    var zipFile = archiver.archive({
        name: 'myarchive.zip'
    });

    // save the archive to file cabinet
    zipFile.folder = 123;
    zipFile.save();
});
```

7. N/config Module

Load the N/config module to access NetSuite configuration settings. See config.Type for a list of supported configuration pages.

`define(['N/config'], (config) => { ... })` ✓ 1 Code Sample(s) Scripts: Server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/config'], (config) => { // Access methods and properties on config });
```

Use the N/config module to access NetSuite configuration settings. The config.load(options) method returns a record.Record object. Use the record.Record object members to access configuration settings. You do not need to load the N/record module to do this.

See config.Type for a list of supported configuration objects.

N/config Module Script Sample

N/config Module Members

Return Type / Value Type

Supported Script Types

config.load(options)

Loads a record.Record object that encapsulates the specified configuration page.

Holds the string values for supported configuration objects. Use this enum to set the value of the NetSuite configuration page you want to access in the config.load(options) method.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**config.load(options)**	record.Record	Server scripts	Loads a record.Record object that encapsulates the specified configuration page.
ENUM	**config.Type**	enum	Server scripts	Holds the string values for supported configuration objects. Use this enum to set the value of the NetSuite configuration page you want to access in the config.load(options) method.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: N/config Module Script Sample

```
/**
 * @NApiVersion 2.1
 */

require(['N/config'],
    function(config) {
        function setTaxAndEmployerId() {
            var companyInfo = config.load({
                type: config.Type.COMPANY_INFORMATION
            });
            companyInfo.setValue({
                fieldId: 'taxid',
                value: '1122334455'
            });
            companyInfo.setValue({
                fieldId: 'employerid',
                value: '123456789'
            });
            companyInfo.save();
            companyInfo = config.load({
                type: config.Type.COMPANY_INFORMATION
            });
            var taxid = companyInfo.getValue({
                fieldId: 'taxid'
            });
        }
        setTaxAndEmployerId();
    });
```

8. N/crypto Module

Load the N/crypto module to work with hashing, hash-based message authentication (hmac), and symmetrical encryption. You can access a set of wrappers for OpenSSL's hash, hmac, cipher, and decipher methods.

`define(['N/crypto'], (crypto) => { ... })` ✓ 2 Code Sample(s) Scripts: Server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/crypto'], (crypto) => { // Access methods and properties on crypto });
```

Use N/crypto module to perform hashing, hash-based message authentication (hmac), and symmetrical encryption functions.

When the N/crypto module is used, SuiteScript also loads N/encode Module.

N/crypto Module Members

Return Type / Value Type

Supported Script Types

Encapsulates a cipher.

crypto.CipherPayload

Encapsulates a cipher payload.

Encapsulates a decipher.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	crypto.Cipher	Object	Server scripts	Encapsulates a cipher.
OBJECT	crypto.CipherPayload	Object	Server scripts	Encapsulates a cipher payload.
OBJECT	crypto.Decipher	Object	Server scripts	Encapsulates a decipher.
OBJECT	crypto.Hash	Object	Server scripts	Encapsulates a hash.
OBJECT	crypto.Hmac	Object	Server scripts	Encapsulates an hmac.
OBJECT	crypto.SecretKey	Object	Server scripts	Encapsulates a secret key handle.
METHOD	crypto.checkPasswordField(options)	boolean	Server scripts	Checks whether a password in a record corresponds to the password entered by the user.
METHOD	crypto.createCipher(options)	Object	Server scripts	Creates and returns a new crypto.Cipher Object.
METHOD	crypto.createDecipher(options)	Object	Server scripts	Creates and returns a new crypto.Decipher object.
METHOD	crypto.createHash(options)	Object	Server scripts	Creates and returns a new crypto.Hash Object.
METHOD	crypto.createHmac(options)	Object	Server scripts	Creates and returns a new crypto.Hmac Object.
METHOD	crypto.createSecretKey(options)	Object	Server scripts	Creates and returns a new crypto.SecretKey Object.
ENUM	crypto.EncryptionAlg	string (read-only)	Server scripts	Holds the string values for supported encryption algorithms. Use this enum to set the options.algorithm parameter for crypto.createCipher(options).
ENUM	crypto.HashAlg	string (read-only)	Server scripts	Holds the string values for supported hashing algorithms. Use this enum to set the options.algorithm parameter for crypto.createHash(options) and crypto.createHmac(options).
ENUM	crypto.Padding	string (read-only)	Server scripts	Holds the string values for supported cipher padding. Use this enum to set the options.padding parameter for crypto.createCipher(options) and crypto.createDecipher(options).

Cipher Object Members

The following members are called on crypto.Cipher.

Return Type / Value Type

Supported Script Types

Cipher.update(options)

Updates the clear data with the specified encoding.

Cipher.final(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Cipher.update(options)	void	Server scripts	Updates the clear data with the specified encoding.
METHOD	Cipher.final(options)	Object	Server scripts	Returns the cipher data.

CipherPayload Object Members

The following members are called on crypto.CipherPayload.

Return Type / Value Type

Supported Script Types

CipherPayload.ciphertext

The result of the ciphering process.

CipherPayload.iv

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	CipherPayload.ciphertext	string	Server scripts	The result of the ciphering process.
PROPERTY	CipherPayload.iv	number	Server scripts	An initialization vector.

Decipher Object Members

The following members are called on crypto.Decipher.

Return Type / Value Type

Supported Script Types

Decipher.final(options)

Returns the clear data.

Decipher.update(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Decipher.final(options)	string	Server scripts	Returns the clear data.
METHOD	Decipher.update(options)	void	Server scripts	Updates decipher data with the specified encoding.

Hash Object Members

The following members are called on crypto.Hash.

Return Type / Value Type

Supported Script Types

Hash.digest(options)

Calculates the digest of the data to be hashed.

Hash.update(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Hash.digest(options)	string	Server scripts	Calculates the digest of the data to be hashed.
METHOD	Hash.update(options)	void	Server scripts	Updates the clear data with the encoding specified.

Hmac Object Members

The following members are called on crypto.Hmac.

Return Type / Value Type

Supported Script Types

Hmac.digest(options)

Gets the computed digest.

Hmac.update(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Hmac.digest(options)	string	Server scripts	Gets the computed digest.
METHOD	Hmac.update(options)	void	Server scripts	Updates the clear data with the encoding specified.

SecretKey Object Members

The following members are called on crypto.SecretKey.

Return Type / Value Type

Supported Script Types

The GUID associated with the secret key.

SecretKey.encoding

The encoding used for the clear text value of the secret key.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Secretkey.guid**	string	Server scripts	The GUID associated with the secret key.
PROPERTY	**SecretKey.encoding**	string	Server scripts	The encoding used for the clear text value of the secret key.
PROPERTY	**SecretKey.secret**	string	Server scripts	The script ID of an API secret stored at Setup > Company > API Secrets.

▶ Official NetSuite Script Samples (2 Sample(s))

📄 Sample 1: Create a Secure Key Using SHA512

```
/**
 * @NApiVersion 2.1
 */
/**This sample demonstrates the APIs needed to generate a secure key using the SHA512 hashing algorithm.
*  The mySecret variable is a placeholder that must be replaced with a valid secret from your NetSuite account.
*  For information about  secrets management page, see Account Administration &gt; Authentication &gt; Secrets Management
 * in the Help Center.
**/

require(['N/crypto', 'N/encode', 'N/runtime'], (crypto, encode, runtime) =&gt; {
    function createSecureKeyWithHash() {
        let mySecret = 'custsecret_my_secret';    //secret id take from secrets management page

        let sKey = crypto.createSecretKey({
            secret: mySecret,
            encoding: encode.Encoding.UTF_8
        });

        let hmacSHA512 = crypto.createHmac({
            algorithm: crypto.HashAlg.SHA512,
            key: sKey
        });

        hmacSHA512.update({
            input: inputString,
            inputEncoding: encode.Encoding.BASE_64
        });

        let digestSHA512 = hmacSHA512.digest({
            outputEncoding: encode.Encoding.HEX
        });
    }
    createSecureKeyWithHash();
});
```

📄 Sample 2: Create a Suitelet to Request User Credentials, Create a Secret Key, and Encode a Sample String

```
/**
 * @NApiVersion 2.1
 * @NScriptType Suitelet
 */
define(['N/ui/serverWidget', 'N/runtime', 'N/crypto', 'N/encode'], (serverWidget, runtime, crypto, encode) =&gt; {
    function onRequest(option) {
        if (option.request.method === 'GET') {
            let form = serverWidget.createForm({
                title: 'My Credential Form'
            });
            let skField = form.addSecretKeyField({
                id: 'mycredential',
                label: 'Credential',
                restrictToScriptIds: [runtime.getCurrentScript().id],
                restrictToCurrentUser: false
            })
            skField.maxLength = 200;

            form.addSubmitButton();

            option.response.writePage(form);
        } else {
            let form = serverWidget.createForm({
                title: 'My Credential Form'
            });

            const inputString = "YWJjZGVmZwo=";
            let myGuid = option.request.parameters.mycredential;

            // Create the key
            let sKey = crypto.createSecretKey({
                guid: myGuid,
                encoding: encode.Encoding.UTF_8
            });

            try {
                let hmacSha512 = crypto.createHmac({
                    algorithm: 'SHA512',
                    key: sKey
                });
                hmacSha512.update({
                    input: inputString,
                    inputEncoding: encode.Encoding.BASE_64
                });
                let digestSha512 = hmacSha512.digest({
                    outputEncoding: encode.Encoding.HEX
                });
            } catch (e) {
                log.error({
                    title: 'Failed to hash input',
                    details: e
                });
            }

            form.addField({
                id: 'result',
                label: 'Your digested hash value',
                type: 'textarea'
            }).defaultValue = digestSha512;

            option.response.writePage(form);
        }
    }
    return {
        onRequest: onRequest
    };
});
```

9. N/crypto/certificate Module

Load the N/certificate module to sign XML documents or strings with digital certificates using asymmetric cryptography. In addition to signing XML documents, you can create signer and verifier objects and verify signed documents with this module.

`define(['N/crypto/certificate'], (certificate) ⇒ { ... })` ✓ 2 Code Sample(s) Scripts: Server scripts Permissions: Certificate Access

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/crypto/certificate'], (certificate) ⇒ { // Access methods and properties on certificate });
```

Use the N/crypto/certificate module to sign XML documents or strings with digital certificates using asymmetric cryptography. You can also use this module to create signer and verifier objects and verify signed documents.

N/crypto/certificate Module Members

Return Type/Value Type

Supported Script Types

certificate.SignedXml

Encapsulates an XML string that has been digitally signed.

Use certificate.signXml(options) to create this object.

certificate.Signer

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	**certificate.SignedXml**	Object	Server scripts	Encapsulates an XML string that has been digitally signed. Use certificate.signXml(options) to create this object.
OBJECT	**certificate.Signer**	Object	Server scripts	Encapsulates a created signature (signer) for plain strings. Use certificate.createSigner(options) to create this object.
OBJECT	**certificate.Verifier**	Object	Server scripts	Encapsulates a created verifier for verifying plain string signatures. Use certificate.createVerifier(options) to create this object.
METHOD	**certificate.createSigner(options)**	certificate.Signer	Server scripts	Creates a certificate.Signer object for signing plain strings.
METHOD	**certificate.createVerifier(options)**	certificate.Verifier	Server scripts	Creates a certificate.Verifier object for verifying signatures of plain strings.
METHOD	**certificate.signXml(options)**	certificate.SignedXml	Server scripts	Signs the input XML string using the Certificate ID. Returns the certificate.SignedXml as a string. Note: Formatting, such as line breaks, is disabled in signatures.
METHOD	**certificate.verifyXmlSignature(options)**	void	Server scripts	Verifies the signature in the SignedXml.asFile() file.
ENUM	**certificate.HashAlg**	enum	Server scripts	Holds the string values for hash algorithm types. Use this enum to set the option.algorithm property values for the certificate.createSigner(options), certificate.createVerifier(options), certificate.signXml(options) methods.

SignedXml Object Members

The following members are called on the certificate.SignedXml object.

Return Type / Value Type

Supported Script Types

SignedXml.asFile()

Returns the signed XML as a file object.

SignedXml.asString()

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**SignedXml.asFile()**	file.File	Server scripts	Returns the signed XML as a file object.
METHOD	**SignedXml.asString()**	string	Server scripts	Returns the signed XML as a string.
METHOD	**SignedXml.asXml()**	xml.Document	Server scripts	Returns the signed XML as an XML document. You can use the N/xml Module with this document to access elements and attributes in the XML.

Signer Object Members

The following members are called on the certificate.Signer object.

Return Type / Value Type

Supported Script Types

Signer.sign(options)

Signs the string and returns the signature.

Signer.update(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**Signer.sign(options)**	void	Server scripts	Signs the string and returns the signature.
METHOD	**Signer.update(options)**	void	Server scripts	Updates the input string to be signed. The string can be encoded.

Verifier Object Members

The following members are called on the certificate.Verifier object.

Return Type / Value Type

Supported Script Types

Verifier.update(options)

Updates the string to be verified against specified certificate.

Verifier.verify(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**Verifier.update(options)**	void	Server scripts	Updates the string to be verified against specified certificate.
METHOD	**Verifier.verify(options)**	void	Server scripts	Verifies the string against a provided signature using specified certificate.

▶ Official NetSuite Script Samples (2 Sample(s))

📄 Sample 1: Load an XML File from the File Cabinet and Sign It Using a Digital Certificate

```
/**
 * @NApiVersion 2.1
 */

require(['N/crypto/certificate','N/file'],(certificate, file) =&gt; {
    // Load the file from the File Cabinet.
    // Note that the id value is hard-coded in this sample, and you should use
    // a valid file id from your account.
    let infNFe = file.load({
        id: 922
    });
    let signedXml = certificate.signXml({
        algorithm: certificate.HashAlg.SHA256,
        certId: 'custcertificate1',
        rootTag: 'infNFe',
        xmlString: infNFe.getContents()
    });
    certificate.verifyXMLSignature({
        signedXml:signedXml,
        rootTag: 'infNFe'
    });
});
```

📄 Sample 2: Create Signer and Verifier Objects

```
/**
 * @NApiVersion 2.1
 */

 require(['N/crypto/certificate'], (certificate) =&gt; {
    let signer = certificate.createSigner({
        certId: 'custcertificate1',
        algorithm: certificate.HashAlg.SHA256
    });
    signer.update('test');

    let result = signer.sign();
    let verifier = certificate.createVerifier({
        certId: 'custcertificate1',
        algorithm: certificate.HashAlg.SHA256
    });
    verifier.update('test');
    verifier.verify(result);
})
```

10. N/crypto/random Module

Load the N/crypto/random module to work with cryptographically-secure pseudo-random generator methods.

`define(['N/crypto/random'], (random) => { ... })` ✓ 1 Code Sample(s) Scripts: Client and server scripts; server scripts support SuiteScript 2.1 only

Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/crypto/random'], (random) => { // Access methods and properties on random });
```

The N/crypto/random module is available for both client and server scripts, but server scripts need to use SuiteScript 2.1.

If you cannot update your server script code to SuiteScript 2.1, consider implementing a small RESTlet in SuiteScript 2.1 that uses N/crypto/random module and consuming it from your script with https.requestRestlet(options)

N/crypto/random Module Samples

N/crypto/random Module Members

Return Type / Value Type

Supported Script Types

random.generateBytes(options)

Client and server scripts

Generates cryptographically strong pseudorandom set of bytes.

random.generateInt(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**random.generateBytes(options)**	Uint8Array	Client and server scripts	Generates cryptographically strong pseudorandom set of bytes.
METHOD	**random.generateInt(options)**	number	Client and server scripts	Method used to generate cryptographically strong pseudorandom number.
METHOD	**random.generateUUID()**	string	Client and server scripts	Method used to generate a v4 Universally Unique Identifier using a cryptographically secure random number generator.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: N/crypto/random Module Samples

```
/**
 * @NApiVersion 2.1
 * @NScriptType restlet
*/
define(['N/crypto/random'], function(random) {
    return {
     get : function() {
          return JSON.stringify({
            number: random.generateInt({min: 1, max:7})
        })
   }
  }
 });
```

11. N/currency Module

Load the N/currency module to work with exchange rates within your NetSuite account. You can use this module to find the exchange rate between two currencies based on a certain date.

`define(['N/currency'], (currency) => { ... })` ✓ 1 Code Sample(s) Scripts: Client and server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/currency'], (currency) => { // Access methods and properties on currency });
```

Use the N/currency module to work with exchange rates within your NetSuite account. You can use this module to find the exchange rate between two currencies based on a certain date.

To use multiple currencies, the Multiple Currencies feature must be enabled. For information about enabling this feature, see Enabling the Multiple Currencies Feature.

N/currency Module Script Sample

Currency formatting is handled by the N/format Module.

N/currency Module Member

Return Type / Value Type

Supported Script Types

currency.exchangeRate(options)

Client and server scripts

Returns an exchange rate between two currencies.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	currency.exchangeRate(options)	number	Client and server scripts	Returns an exchange rate between two currencies.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: N/currency Module Script Sample

```
/**
 * @NApiVersion 2.1
 */

require(['N/currency'], function(currency) {
    function getUSDFromCAD() {
        var canadianAmount = 100;
        var rate = currency.exchangeRate({
            source: 'CAD',
            target: 'USD',
            date: new Date('7/28/2015')
        });

        var usdAmount = canadianAmount * rate;
    }

    getUSDFromCAD();
});;
```

12. N/currentRecord Module

Load the N/currentRecord module to access the record instance that you are currently working on. You can then use the record instance in a client context.

`define(['N/currentRecord'], (currentRecord) => { ... })` ✓ 2 Code Sample(s) Scripts: Client scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/currentRecord'], (currentRecord) => { // Access methods and properties on currentRecord });
```

Use the N/currentRecord module to access the record that is active in the current client context. This module is always a dynamic object and mode of work is always dynamic, not deferred dynamic/standard. For more information, see SuiteScript 2.x Standard and Dynamic Modes. Be aware that when the current record is in view mode it cannot be edited; it is a read-only record when in view mode. As such, any set APIs do not work on the current record in view mode.

You can use the currentRecord module in the following types of scripts:

Entry point client scripts - These scripts use the @NScriptType ClientScript annotation. (For details, see SuiteScript 2.1 JSDoc Validation.) The system automatically provides this type of script with a currentRecord.CurrentRecord object that represents the current record. For this reason, an entry point client script does not have to explicitly load the N/currentRecord module. To access the currentRecord object, create a variable and initialize it to the value of the scriptContext.currentRecord property, which is available in each of the SuiteScript 2.1 Client Script Entry Points and API. For an example, see SuiteScript Client Script Sample.

Client custom modules - These scripts do not use an @NScriptType annotation (see SuiteScript 2.1 Custom Modules). For these scripts, you must manually load the N/currentRecord module by naming it in the script's define statement. Additionally, you must actively retrieve a currentRecord.CurrentRecord object by using the currentRecord.get() or currentRecord.get.promise() method. For an example, see N/currentRecord Module Script Samples.

Like the N/record Module, the currentRecord module provides access to body and sublist fields. However, you should use the record module for server scripts and for cases where a client script needs to interact with a record other than the currently active record. You should use the currentRecord module for client scripts that need to interact with the currently active record.

Additionally, the functionality of the two modules varies slightly. For example, the currentRecord module does not permit the editing of subrecords, although subrecords can be retrieved in view mode. For additional details, see the following topics:

N/currentRecord Module Members

Return Type / Value Type

Supported Script Types

currentRecord.Column

Encapsulates a column of a sublist on the current record.

currentRecord.CurrentRecord

Represents the record active on the current page.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	currentRecord.Column	Object	Client scripts	Encapsulates a column of a sublist on the current record.
OBJECT	currentRecord.CurrentRecord	Object	Client scripts	Represents the record active on the current page.
OBJECT	currentRecord.Field	Object	Client scripts	Represents a body or sublist field.
OBJECT	currentRecord.Sublist	Object	Client scripts	Represents a sublist.
METHOD	currentRecord.get()	currentRecord.CurrentRecord	Client scripts	Retrieves a record object that represents the current record.
METHOD	currentRecord.get.promise()	Promise	Client scripts	Retrieves a promise for an object that represents the current record.

Column Object Members

The following members are called on the currentRecord.Column object.

Return Type / Value Type

Supported Script Types

string (read-only)

Returns the internal ID of the column.

Column.isDisabled

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Column.id	string (read-only)	Client scripts	Returns the internal ID of the column.
PROPERTY	Column.isDisabled	boolean	Client scripts	Indicates whether the column is disabled.
PROPERTY	Column.isMandatory	boolean	Client scripts	Indicates whether the column is required.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Column.label	string (read-only)	Client scripts	Returns the UI label for the column.
PROPERTY	Column.sublistId	string (read-only)	Client scripts	Returns the internal ID of the standard or custom sublist that contains the column.
PROPERTY	Column.type	string (read-only)	Client scripts	Returns the column type.

CurrentRecord Object Members

The following members are called on the currentRecord.CurrentRecord object.

Return Type / Value Type

Supported Script Types

CurrentRecord.cancelLine(options)

currentRecord.CurrentRecord

Cancels the changes made to the currently selected line.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	CurrentRecord.cancelLine(options)	currentRecord.CurrentRecord	Client scripts	Cancels the changes made to the currently selected line.
METHOD	CurrentRecord.commitLine(options)	currentRecord.CurrentRecord	Client scripts	Commits the currently selected line.
METHOD	CurrentRecord.findMatrixSublistLineWithValue(options)	number	Client scripts	Returns the line number of the first line that contains the specified value in the matrix column.
METHOD	CurrentRecord.findSublistLineWithValue(options)	number	Client scripts	Gets the line number for the first occurrence of a field value in a sublist.
METHOD	CurrentRecord.getCurrentMatrixSublistValue(options)	number | Date | string | array | boolean	Client scripts	Gets the value for the currently selected line in the matrix.
METHOD	CurrentRecord.getCurrentSublistIndex(options)	number	Client scripts	Gets the line number of the currently selected line.
METHOD	CurrentRecord.getCurrentSublistSubrecord(options)	currentRecord.CurrentRecord	Client scripts	Gets the subrecord for the associated sublist field on the current line. The subrecord object is retrieved in view mode.
METHOD	CurrentRecord.getCurrentSublistText(options)	number | Date | string | array | boolean	Client scripts	Gets the value of the field in the currently selected line by text representation.
METHOD	CurrentRecord.getCurrentSublistValue(options)	number | Date | string | array | boolean	Client scripts	Gets the value of the field in the currently selected line.
METHOD	CurrentRecord.getField(options)	currentRecord.Field	Client scripts	Gets a field object from the record.
METHOD	CurrentRecord.getLineCount(options)	number	Client scripts	Returns the number of lines in the sublist.
METHOD	CurrentRecord.getMatrixHeaderCount(options)	number	Client scripts	Returns the number of columns for the specified matrix.
METHOD	CurrentRecord.getMatrixHeaderField(options)	currentRecord.Field	Client scripts	Gets the field for the specified header in the matrix.
METHOD	CurrentRecord.getMatrixHeaderValue(options)	number | Date | string | array | boolean	Client scripts	Gets the value for the associated header in the matrix.
METHOD	CurrentRecord.getMatrixSublistField(options)	currentRecord.Field	Client scripts	Gets the field for the specified sublist in the matrix.
METHOD	CurrentRecord.getMatrixSublistValue(options)	number | Date | string | array | boolean	Client scripts	Gets the value for the associated field in the matrix.
METHOD	CurrentRecord.getSublist(options)	currentRecord.Sublist	Client scripts	Gets the specified sublist object.
METHOD	CurrentRecord.getSublistField(options)	currentRecord.Field	Client scripts	Gets the specified field object from the sublist.
METHOD	CurrentRecord.getSublistText(options)	string	Client scripts	Gets the value of the field in a sublist by a string representation.
METHOD	CurrentRecord.getSublistValue(options)	number | Date | string | array | boolean	Client scripts	Gets the value of the field in a sublist.
METHOD	CurrentRecord.getSubrecord(options)	currentRecord.CurrentRecord	Client scripts	Gets the subrecord associated with the field. The subrecord object is retrieved in view mode.
METHOD	CurrentRecord.getText(options)	string	Client scripts	Gets the value of the field by a string representation.
METHOD	CurrentRecord.getValue(options)	number | Date | string | array | boolean	Client scripts	Gets the value of the field. Not to be used on custom password fields. Use crypto.checkPasswordField(options) instead.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	CurrentRecord.hasCurrentSublistSubrecord(options)	boolean	Client scripts	Returns a value indicating whether the associated sublist field has a subrecord on the current line.
METHOD	CurrentRecord.hasSublistSubrecord(options)	boolean	Client scripts	Returns a value indicating whether the associated sublist field contains a subrecord.
METHOD	CurrentRecord.hasSubrecord(options)	boolean	Client scripts	Indicates whether the field has a subrecord.
METHOD	CurrentRecord.insertLine(options)	currentRecord.CurrentRecord	Client scripts	Inserts a new line in a sublist.
METHOD				
METHOD	CurrentRecord.removeCurrentSublistSubrecord(options)	currentRecord.CurrentRecord	Client scripts	Removes the subrecord for the associated sublist field on the current line.
METHOD	CurrentRecord.removeLine(options)	currentRecord.CurrentRecord	Client scripts	Removes a line from a sublist.
METHOD	CurrentRecord.removeSubrecord(options)	currentRecord.CurrentRecord	Client scripts	Removes the subrecord associated with the field.
METHOD	CurrentRecord.selectLine(options)	void	Client scripts	Selects a line item in a sublist.
METHOD	CurrentRecord.selectNewLine(options)	currentRecord.CurrentRecord	Client scripts	Selects a new line at the end of the sublist.
METHOD	CurrentRecord.setCurrentMatrixSublistValue(options)	currentRecord.CurrentRecord	Client scripts	Sets the value for the currently selected line in the matrix.
METHOD	CurrentRecord.setCurrentSublistText(options)	currentRecord.CurrentRecord	Client scripts	Sets the value of the field in the currently selected line using a string representation.
METHOD	CurrentRecord.setCurrentSublistValue(options)	currentRecord.CurrentRecord	Client scripts	Sets the value of the field in the currently selected line.
METHOD	CurrentRecord.setMatrixHeaderValue(options)	currentRecord.CurrentRecord	Client scripts	Sets the value for the associated header in the matrix.
METHOD	CurrentRecord.setMatrixSublistValue(options)	currentRecord.CurrentRecord	Client scripts	Sets the value for the associated field in the matrix.
METHOD	CurrentRecord.setText(options)	currentRecord.CurrentRecord	Client scripts	Sets the value of the field using a string representation.
METHOD	CurrentRecord.setValue(options)	currentRecord.CurrentRecord	Client scripts	Sets the value of the field.
PROPERTY	CurrentRecord.id	number (read-only)	Client scripts	Returns the internal record ID.
PROPERTY	CurrentRecord.isDynamic	boolean (read-only)	Client scripts	Indicates whether the record is dynamic.
PROPERTY	CurrentRecord.type	string (read-only)	Client scripts	Returns the record type.

Field Object Members

The following members are called on the currentRecord.Field object.

Return Type / Value Type

Supported Script Types

Field.getSelectOptions(options)

Returns an array of available options on a standard or custom select, multiselect, or radio field as key-value pairs. Only the first 1,000 available options are returned.

Field.insertSelectOption(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Field.getSelectOptions(options)	array	Client scripts	Returns an array of available options on a standard or custom select, multiselect, or radio field as key-value pairs. Only the first 1,000 available options are returned.
METHOD	Field.insertSelectOption(options)	void	Client scripts	Inserts an option into certain types of select and multiselect fields. This method is usable only in fields that were added by a front-end Suitelet or beforeLoad user event script.
METHOD	Field.removeSelectOption(options)	void	Client scripts	Removes an option from certain types of select and multiselect fields. This method is usable only in fields that were added by a front-end Suitelet or beforeLoad user event script. It is supported only in client scripts.
OBJECT	Field.id	string (read-only)	Client scripts	Returns the internal ID of a standard or custom body or sublist field.
OBJECT	Field.isDisabled	boolean	Client scripts	Returns true if the standard or custom field is disabled on the record form, or false otherwise.
OBJECT	Field.isDisplay	boolean	Client scripts	Returns true if the field is set to display on the record form, or false otherwise. This property is read-only for sublist fields.
OBJECT	Field.isMandatory	boolean	Client scripts	Returns true if the standard or custom field is mandatory on the record form, or false otherwise.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	**Field.isPopup**	boolean (read-only)	Client scripts	Returns true if the field is a popup list field, or false otherwise.
OBJECT	**Field.isReadOnly**	boolean	Client scripts	Returns true if the field on the record form cannot be edited, or false otherwise. For textarea fields, this property can be read or written to. For all other fields, this property is read-only.
OBJECT	**Field.isVisible**	boolean (read-only)	Client scripts	Returns true if the field is visible on the record form, or false otherwise.
OBJECT	**Field.label**	string (read-only)	Client scripts	Returns the UI label for a standard or custom field body or sublist field.
OBJECT	**Field.sublistId**	string (read-only)	Client scripts	Returns the ID of the sublist associated with the specified sublist field.
OBJECT	**Field.type**	string (read-only)	Client scripts	Returns the type of a body or sublist field.

Sublist Object Members

The following members are called on the currentRecord.Sublist object.

Return Type / Value Type

Supported Script Types

Sublist.getColumn(options)

currentRecord.Column

Returns a column in the sublist.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**Sublist.getColumn(options)**	currentRecord.Column	Client scripts	Returns a column in the sublist.
PROPERTY	**Sublist.id**	string (read-only)	Client scripts	Returns the internal ID of the sublist.
PROPERTY	**Sublist.isChanged**	boolean (read-only)	Client scripts	Indicates whether the sublist has changed on the current record form.
PROPERTY	**Sublist.isDisplay**	boolean (read-only)	Client scripts	Indicates whether the sublist is displayed on the current record form.
PROPERTY	**Sublist.type**	string (read-only)	Client scripts	Returns the sublist type.

▶ Official NetSuite Script Samples (2 Sample(s))

📄 Sample 1: Perform Field Sourcing Synchronously

```
/**
 * @NApiVersion 2.1
 * @NScriptType ClientScript
 */

define(['N/currentRecord'], (currentRecord) =&gt; {
    const pageInit = () =&gt; {
        const rec = currentRecord.get();

        rec.selectNewLine({
            sublistId: 'item'
        });

        rec.setCurrentSublistValue({
            sublistId: 'item',
            fieldId: 'item',
            value: 39,
            forceSyncSourcing: true
        });

        rec.setCurrentSublistValue({
            sublistId: 'item',
            fieldId: 'quantity',
            value: 1,
            forceSyncSourcing: true
        });

        rec.commitLine({
            sublistId: 'item'
        });
    };

    return {
        pageInit
    };
});
```

📄 **Sample 2: SuiteScript Client Script Sample**

```
/**
 *@NApiVersion 2.x
 *@NScriptType ClientScript
 */
define(['N/error'], function(error) {
    function pageInit(context) {
        if (context.mode != 'create')
            return;
        var currentRecord = context.currentRecord;
        currentRecord.setValue({
            fieldId: 'entity',
            value: 107
        });
    }
    function saveRecord(context) {
        var currentRecord = context.currentRecord;
        if (!currentRecord.getValue({fieldId: 'entity'}) || currentRecord.getLineCount({sublistId: 'item'}) &lt; 1)
            throw error.create({
                name: 'MISSING_REQ_ARG',
                message: 'Please enter all the necessary fields on the salesorder before saving'
             });
        return true;
    }
    function validateField(context) {
        var currentRecord = context.currentRecord;
        var sublistName = context.sublistId;
        var sublistFieldName = context.fieldId;
        var line = context.line;
        if (sublistName === 'item') {
            if (sublistFieldName === 'quantity') {
                if (currentRecord.getCurrentSublistValue({
                    sublistId: sublistName,
                    fieldId: sublistFieldName
                }) &lt; 3)
                    currentRecord.setValue({
                        fieldId: 'otherrefnum',
                        value: 'Quantity is less than 3'
                    });
                else
                    currentRecord.setValue({
                        fieldId: 'otherrefnum',
                        value: 'Quantity accepted'
                    });
                }
            }
       return true;
    }
    function fieldChanged(context) {
        var currentRecord = context.currentRecord;
        var sublistName = context.sublistId;
        var sublistFieldName = context.fieldId;
        var line = context.line;
        if (sublistName === 'item' &amp;&amp; sublistFieldName === 'item')
            currentRecord.setValue({
                fieldId: 'memo',
                value: 'Item: ' + currentRecord.getCurrentSublistValue({
                    sublistId: 'item',
                    fieldId: 'item'
                }) + ' is selected'
            });
    }
    function postSourcing(context) {
        var currentRecord = context.currentRecord;
        var sublistName = context.sublistId;
        var sublistFieldName = context.fieldId;
        var line = context.line;
        if (sublistName === 'item' &amp;&amp; sublistFieldName === 'item')
            if (currentRecord.getCurrentSublistValue({
                sublistId: sublistName,
                fieldId: sublistFieldName
            }) === '39')
                if (currentRecord.getCurrentSublistValue({
                    sublistId: sublistName,
                    fieldId: 'pricelevels'
                }) != '1-1')
                    currentRecord.setCurrentSublistValue({
                        sublistId: sublistName,
                        fieldId: 'pricelevels',
                        value: '1-1'
                    });
    }
    function lineInit(context) {
        var currentRecord = context.currentRecord;
        var sublistName = context.sublistId;
        if (sublistName === 'partners')
            currentRecord.setCurrentSublistValue({
                sublistId: sublistName,
                fieldId: 'partner',
                value: '55'
            });
    }
```

```
    function validateDelete(context) {
        var currentRecord = context.currentRecord;
        var sublistName = context.sublistId;
        if (sublistName === 'partners')
            if (currentRecord.getCurrentSublistValue({
                sublistId: sublistName,
                fieldId: 'partner'
            }) === '55')
                currentRecord.setValue({
                    fieldId: 'memo',
                    value: 'Removing partner sublist'
                });
        return true;
    }
    function validateInsert(context) {
        var currentRecord = context.currentRecord;
        var sublistName = context.sublistId;
        if (sublistName === 'partners')
            if (currentRecord.getCurrentSublistValue({
                sublistId: sublistName,
                fieldId: 'contribution'
            }) !== '100.0%')
                currentRecord.setCurrentSublistValue({
                    sublistId: sublistName,
                    fieldId: 'contribution',
                    value: '100.0%'
                });
        return true;
    }
    function validateLine(context) {
        var currentRecord = context.currentRecord;
        var sublistName = context.sublistId;
        if (sublistName === 'partners')
            if (currentRecord.getCurrentSublistValue({
                sublistId: sublistName,
                fieldId: 'contribution'
            }) !== '100.0%')
                currentRecord.setCurrentSublistValue({
                    sublistId: sublistName,
                    fieldId: 'contribution',
                    value: '100.0%'
                });
        return true;
    }
    function sublistChanged(context) {
        var currentRecord = context.currentRecord;
        var sublistName = context.sublistId;
        var op = context.operation;
        if (sublistName === 'item')
            currentRecord.setValue({
                fieldId: 'memo',
                value: 'Total has changed to ' + currentRecord.getValue({fieldId: 'total'})
            });
    }

    return {
        pageInit: pageInit,
        fieldChanged: fieldChanged,
        postSourcing: postSourcing,
        sublistChanged: sublistChanged,
        lineInit: lineInit,
        validateField: validateField,
        validateLine: validateLine,
        validateInsert: validateInsert,
        validateDelete: validateDelete,
        saveRecord: saveRecord
    };
});
```

13. N/dataset Module

Load the N/dataset module to create and manage datasets in SuiteAnalytics Workbook. Use this module with the N/workbook module to manage all aspects of your datasets and workbooks in SuiteAnalytics Workbook.

`define(['N/dataset'], (dataset) ⇒ { ... })` ✓ 2 Code Sample(s) Scripts: Server scripts Permissions: SuiteAnalytics Workbook

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/dataset'], (dataset) ⇒ { // Access methods and properties on dataset });
```

Use the N/dataset module to create, load, list, or save datasets. You can only use this module in server scripts.

With this module, you can do things like:

Create columns, joins, and conditions within a dataset.

Delete a dataset using the N/query Module.

Execute a dataset and obtain results-similar to running a query definition with the N/query Module.

Use string aliasing to identify columns, which is helpful when linking dataset columns to a workbook.

Apply column labels, which are the string descriptions shown in the UI.

Save a dataset.

Datasets are the foundation for workbooks and their components. In a dataset, you combine record type fields and filters to make a query. You can use the results as source data for any workbook, and you can use one dataset in several workbooks.

To learn more about datasets in SuiteAnalytics, see Defining a Dataset. For details on workbooks, see N/workbook Module.

The N/dataset module doesn't work in unauthenticated client-side contexts.

For details, see the SuiteAnswers Outbound HTTPs in an unauthenticated client-side context.

N/dataset Module Members

Return Type / Value Type

Supported Script Types

A column in the dataset, which usually represents a record field.

dataset.Condition

A condition or set of conditions to apply to a column.

A representation of the entire dataset, including columns, conditions, and joins.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	**dataset.Column**	Object	Server scripts	A column in the dataset, which usually represents a record field.
OBJECT	**dataset.Condition**	Object	Server scripts	A condition or set of conditions to apply to a column.
OBJECT	**dataset.Dataset**	Object	Server scripts	A representation of the entire dataset, including columns, conditions, and joins.
OBJECT	**dataset.Join**	Object	Server scripts	A joined record used in the dataset.
METHOD	**dataset.create(options)**	dataset.Dataset	Server scripts	Creates a dataset.
METHOD	**dataset.createColumn(options)**	dataset.Column	Server scripts	Creates a dataset column.
METHOD	**dataset.createCondition(options)**	dataset.Condition	Server scripts	Creates a dataset condition (criteria). A condition is applied to a dataset column and includes an operator.
METHOD	**dataset.createJoin(options)**	dataset.Join	Server scripts	Creates a dataset join.
METHOD	**dataset.createTranslation(options)**	workbook.Expression	Server scripts	Creates a translation expression based on a Translation Collection.
METHOD	**dataset.describe(options)**	Object[]	Server scripts	Retrieves descriptive information about a dataset, including name, description, and a list of columns or formulas with their labels and types.
METHOD	**dataset.describe.promise(options)**	Promise Object	Server scripts	Asynchronously retrieves descriptive information about a dataset, including name, description, and a list of columns or formulas with their labels and types..
METHOD	**dataset.list()**	Object[]	Server scripts	Lists all existing datasets.
METHOD	**dataset.listPaged(options)**	PagedInfoData	Server scripts	Returns metadata about datasets as a set of paged results.
METHOD	**dataset.load(options)**	dataset.Dataset	Server scripts	Loads an existing dataset.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	dataset.load.promise(options)	Promise Object	Server scripts	Asynchronously loads an existing dataset.

Column Object Members

This object encapsulates the record fields in the dataset. Columns are equivalent to the fields you use when you build a dataset in SuiteAnalytics. For more information about datasets in SuiteAnalytics, see Custom Workbooks and Datasets.

The following members are available for a dataset.Column object.

Return Type / Value Type

Supported Script Types

string (read-only)

The alias of the column.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Column.alias	string (read-only)	Server scripts	The alias of the column.
PROPERTY	Column.fieldId	string (read-only)	Server scripts	The ID of the record field associated with the column.
PROPERTY	Column.formula	string (read-only)	Server scripts	The formula of the column.
PROPERTY	Column.id	number (read-only)	Server Scripts	The ID of the column.
PROPERTY	Column.join	dataset.Join (read-only)	Server scripts	The join for the column. Used only when the column is from a joined record.
PROPERTY	Column.label	string (read-only)	Server scripts	The label of the column.
PROPERTY	Column.type	string (read-only)	Server scripts	The return type of the formula.

Condition Object Members

This object encapsulates the criteria in the dataset. Conditions are equivalent to the criteria you use when you build a dataset in SuiteAnalytics. For more information about criteria used in datasets in SuiteAnalytics, see Dataset Criteria Filters.

The following members are available for a dataset.Condition object.

Return Type / Value Type

Supported Script Types

Condition.caseSensitive

Indicates whether the condition in a sort is case sensitive.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Condition.caseSensitive	boolean	Server scripts	Indicates whether the condition in a sort is case sensitive.
PROPERTY	Condition.children	dataset.Condition[] (read-only)	Server scripts	The children of the condition (for example, subconditions AND'd or OR'd).
PROPERTY	Condition.column	dataset.Column (read-only)	Server scripts	The column on which the condition is placed.
PROPERTY	Condition.operator	string (read-only)	Server scripts	The operator of the condition.
PROPERTY	Condition.values	string[] | number[] | boolean[] | Date[] | Object[] (read-only)	Server scripts	The values for the condition.

Dataset Object Members

This object encapsulates the entire dataset. For more information about datasets in SuiteAnalytics, see Custom Workbooks and Datasets.

The following members are available for a dataset.Dataset object.

Return Type / Value Type

Supported Script Types

Dataset.getExpressionFromColumn(options)

workbook.Expression

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Dataset.getExpressionFromColumn(options)	workbook.Expression	Server scripts	Returns an expression which can be used in a workbook.
METHOD	Dataset.run()	query.ResultSet	Server scripts	Executes the dataset and returns the result set (the same as in N/query Module).

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**Dataset.run.promise()**	Promise Object	Server scripts	Asynchronously executes the dataset and returns the result set (the same as in N/query Module).
METHOD	**Dataset.runPaged(options)**	query.PagedData	Server scripts	Executes the dataset and returns the result set in paginated data form (the same as in N/query Module).
METHOD	**Dataset.save(options)**	Object	Server scripts	Saves a dataset.
PROPERTY	**Dataset.columns**	dataset.Column[]	Server scripts	The columns in the dataset.
PROPERTY	**Dataset.condition**	dataset.Condition	Server scripts	The condition (criteria) for the entire dataset.
PROPERTY	**Dataset.description**	string	Server scripts	The description of the dataset.
PROPERTY	**Dataset.id**	string	Server scripts	The ID of the dataset.
PROPERTY	**Dataset.name**	string	Server scripts	The name of the dataset.
PROPERTY	**Dataset.type**	string	Server scripts	The internal ID for the base record type for the dataset.

Join Object Members

Encapsulates a joined record used in the dataset. For more information about using joins in a dataset, see Joining Record Types in a Dataset.

The following members are available for a dataset.Join object.

Return Type / Value Type

Supported Script Types

string (read-only)

The ID of the field on which the join is performed.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Join.fieldId**	string (read-only)	Server scripts	The ID of the field on which the join is performed.
PROPERTY	**Join.join**	dataset.Join (read-only)	Server scripts	The child join, if the join is a multilevel join.
PROPERTY	**Join.source**	string (read-only)	Server scripts	The internal ID for the source record type of the join.
PROPERTY	**Join.target**	string (read-only)	Server scripts	The polymorphic target of the join.

▶ Official NetSuite Script Samples (2 Sample(s))

📄 Sample 1: Create a Dataset, Run the Dataset, and List All Existing Datasets

```
/**
 * @NApiVersion 2.x
 */

// The following script creates a dataset with three columns, one of which is based on a join, and a condition. Then, all datasets are listed and the newly c
reated dataset is loaded and reviewed.

require(['N/dataset', 'N/query'], function(dataset, query){
    var myTransactionDateColumn = dataset.createColumn({
        fieldId: 'trandate',
        alias: 'date'
    });
    var myJoin = dataset.createJoin({
        fieldId: 'createdby',
        target: 'entity'
    });
    var myNameColumn = dataset.createColumn({
        fieldId: 'lastname',
        alias: 'name',
        join: myJoin
    });
    var myTransactionIdColumn = dataset.createColumn({
        fieldId: 'tranid',
        alias: 'id'
    });
    var myTotalColumn = dataset.createColumn({
        fieldId: 'foreigntotal',
        alias: 'total'
    });
    var myColumns = [myTransactionIdColumn, myNameColumn, myTransactionDateColumn, myTotalColumn];
    var myCondition = dataset.createCondition({
        column: myNameColumn,
        operator: query.Operator.EMPTY
    });
    var myDataset = dataset.create({
        type: 'transaction',
        columns: myColumns,
        condition: myCondition
    });

    // List all created datasets
    var allDatasets = dataset.list();
    log.audit({
        title: 'All datasets:',
        details: allDatasets
    });

    // Review the newly created dataset components (in the log)
    log.audit({
        title: 'My Dataset = ',
        details: myDataset
    });

    // Now run the dataset
    var runResult = myDataset.run();
    var runPagedResult = myDataset.runPaged({
        pageSize: 2
    });
    log.audit({
        title: 'myDataset runResult: ',
        details: runResult
    });
    log.audit({
        title: 'myDataset runPagedResult: ',
        details: runPagedResult
    });
});
```

📄 Sample 2: List All Datasets and Load the First Dataset

```
/**
 * @NApiVersion 2.x
 */

// The following script lists all existing datasets and then loads the first dataset.

require(['N/dataset'], function(dataset){
    // List all created datasets
    var allDatasets = dataset.list();
    log.audit({
        title: 'All datasets:',
        details: allDatasets
    });

    // Load the first dataset
    var myFirstDataset = dataset.load({
        id: allDatasets[0].id
    });
    log.audit('myFirstDataset:', myFirstDataset);
});
```

14. N/documentCapture Module

Load the N/documentCapture module to extract text content from supported documents. This module lets you programmatically extract structured content and key information from a variety of document types (such as invoices, receipts, contracts, and so on).

`define(['N/documentCapture'], (documentCapture) => { ... })` ✓ 4 Code Sample(s) Scripts: Server scripts SuiteScript 2.1 only Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/documentCapture'], (documentCapture) => { // Access methods and properties on documentCapture });
```

The content in this help topic pertains to SuiteScript 2.1.

Load the N/documentCapture module to extract text content from supported documents.

The N/documentCapture module lets you programmatically extract structured content and key information from a variety of document types (such as invoices, receipts, contracts, and so on) directly within NetSuite. This module uses the AI-driven capabilities of the Oracle Cloud Infrastructure (OCI) Document Understanding service and can automate document processing, reduce manual data entry, and enhance business workflows. For more information about the OCI Document Understanding service, refer to Document Understanding in the Oracle Cloud Infrastructure Documentation.

This module provides the following features and benefits:

Automated content extraction - Extracts text, tables, and key-value pairs from scanned documents, PDFs, and images.

AI-powered data recognition - Uses advanced machine learning models from OCI to accurately identify and extract relevant information.

Support for multiple document types - Works with invoices, receipts, tax forms, and other business documents in PDF, PNG, JPG, and TIFF formats.

Synchronous and asynchronous requests - Supports synchronous requests (for documents up to five pages in length) and asynchronous requests (for documents longer than five pages).

Document classification - Automatically classifies documents by type, enabling use cases such as intelligent routing and processing.

Support for multiple languages and layouts - Supports documents in multiple languages and using various layouts, increasing flexibility.

Error handling and confidence scores - Provides confidence scores for extracted data and error handling for improved reliability.

This module is available in NetSuite by default when the Server SuiteScript feature is enabled. For more information, see Enabling Features.

To learn how to get started with the N/documentCapture module, see Getting Started with the N/documentCapture Module.

N/documentCapture Module Members

Return Type / Value Type

Supported Script Types

documentCapture.Cell

An extracted table cell from a document.

documentCapture.Document

The extracted data from a document.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	documentCapture.Cell	Object	Server scripts	An extracted table cell from a document.
OBJECT	documentCapture.Document	Object	Server scripts	The extracted data from a document.
OBJECT	documentCapture.Field	Object	Server scripts	An extracted field from a document.
OBJECT	documentCapture.FieldLabel	Object	Server scripts	An extracted field label from a document.
OBJECT	documentCapture.FieldValue	Object	Server scripts	An extracted field value from a document.
OBJECT	documentCapture.Line	Object	Server scripts	An extracted line of text from a document.
OBJECT	documentCapture.Page	Object	Server scripts	An extracted page from a document.
OBJECT	documentCapture.Table	Object	Server scripts	An extracted table from a document.
OBJECT	documentCapture.TableRow	Object	Server scripts	An extracted table row from a document.
OBJECT	documentCapture.Word	Object	Server scripts	An extracted word from a document.
METHOD	documentCapture.documentToStructure(options)	documentCapture.Document	Server scripts	Extracts content from a document.
METHOD	documentCapture.documentToStructure.promise(options)	Promise	Server scripts	Asynchronously extracts content from a document.
METHOD	documentCapture.documentToText(options)	string	Server scripts	Extracts text content from a PDF file.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	documentCapture.documentToText.promise(options)	Promise	Server scripts	Asynchronously extracts text content from a PDF file.
METHOD	documentCapture.getRemainingConcurrency()	number	Server scripts	Returns the number of available concurrent requests remaining.
METHOD	documentCapture.getRemainingConcurrency.promise()	Promise	Server scripts	Asynchronously returns the number of available concurrent requests remaining.
METHOD	documentCapture.getRemainingFreeUsage()	number	Server scripts	Deprecated. Calls llm.getRemainingUsage().
METHOD	documentCapture.getRemainingFreeUsage.promise()	Promise	Server scripts	Deprecated. Calls llm.getRemainingUsage.promise().
METHOD	documentCapture.parseResult(options)	documentCapture.Document	Server scripts	Converts a JSON file into a documentCapture.Document object.
ENUM	documentCapture.DocumentType	enum	Server scripts	Holds values for the document type.
ENUM	documentCapture.Feature	enum	Server scripts	Holds values for the feature to extract.
ENUM	documentCapture.FieldType	enum	Server scripts	Holds values for the type of a field.
ENUM	documentCapture.Language	enum	Server scripts	Holds values for the language of a document.

Cell Object Members

Return Type / Value Type

Supported Script Types

The confidence level for the cell.

The extracted text of the cell.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Cell.confidence	number	Server scripts	The confidence level for the cell.
PROPERTY	Cell.text	string	Server scripts	The extracted text of the cell.

Document Object Members

Return Type / Value Type

Supported Script Types

Document.mimeType

The MIME type of the document.

documentCapture.Page[]

The pages of the document.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Document.mimeType	string	Server scripts	The MIME type of the document.
PROPERTY	Document.pages	documentCapture.Page[]	Server scripts	The pages of the document.
METHOD	Document.getText()	string	Server scripts	Returns the entire text of the document.

Field Object Members

Return Type / Value Type

Supported Script Types

documentCapture.FieldLabel

The label (name) of the field.

The type of the field.

documentCapture.FieldValue

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Field.label	documentCapture.FieldLabel	Server scripts	The label (name) of the field.
PROPERTY	Field.type	string	Server scripts	The type of the field.
PROPERTY	Field.value	documentCapture.FieldValue	Server scripts	The value of the field.

FieldLabel Object Members

Return Type / Value Type

Supported Script Types

FieldLabel.confidence

The confidence level for the field label.

The name of the field label.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	`FieldLabel.confidence`	`number`	Server scripts	The confidence level for the field label.
PROPERTY	`FieldLabel.name`	`string`	Server scripts	The name of the field label.

FieldValue Object Members

Return Type / Value Type

Supported Script Types

FieldValue.confidence

The confidence level for the field value.

The text of the field value.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	`FieldValue.confidence`	`number`	Server scripts	The confidence level for the field value.
PROPERTY	`FieldValue.text`	`string`	Server scripts	The text of the field value.

Line Object Members

Return Type / Value Type

Supported Script Types

The confidence level for the line.

The text of the line.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	`Line.confidence`	`number`	Server scripts	The confidence level for the line.
PROPERTY	`Line.text`	`string`	Server scripts	The text of the line.

Page Object Members

Return Type / Value Type

Supported Script Types

Page.detectedDocumentTypes

A set of confidence levels indicating whether the page represents a particular type of document.

documentCapture.Field[]

The extracted fields from the page of a document.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	`Page.detectedDocumentTypes`	`Object[]`	Server scripts	A set of confidence levels indicating whether the page represents a particular type of document.
PROPERTY	`Page.fields`	`documentCapture.Field[]`	Server scripts	The extracted fields from the page of a document.
PROPERTY	`Page.lines`	`documentCapture.Line[]`	Server scripts	The extracted lines from the page of a document.
PROPERTY	`Page.tables`	`documentCapture.Table[]`	Server scripts	The extracted tables from the page of a document.
PROPERTY	`Page.words`	`documentCapture.Word[]`	Server scripts	The extracted words from the page of a document.
METHOD	`Page.getText()`	`string`	Server scripts	Returns the entire text of the page.

Table Object Members

Return Type / Value Type

Supported Script Types

documentCapture.TableRow[]

The extracted body rows from the table in a document.

Table.columnCount

The number of extracted columns from the table in a document.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Table.bodyRows**	documentCapture.TableRow[]	Server scripts	The extracted body rows from the table in a document.
PROPERTY	**Table.columnCount**	number	Server scripts	The number of extracted columns from the table in a document.
PROPERTY	**Table.confidence**	number	Server scripts	The confidence level for the table.
PROPERTY	**Table.footerRows**	documentCapture.TableRow[]	Server scripts	The extracted footer rows from the table in a document.
PROPERTY	**Table.headerRows**	documentCapture.TableRow[]	Server scripts	The extracted header rows from the table in a document.
PROPERTY	**Table.rowCount**	number	Server scripts	The number of extracted rows from the table in a document.

TableRow Object Members

Return Type / Value Type

Supported Script Types

documentCapture.Cell[]

The extracted cells in the table row.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**TableRow.cells**	documentCapture.Cell[]	Server scripts	The extracted cells in the table row.

Word Object Members

Return Type / Value Type

Supported Script Types

The confidence level for the word.

The extracted text of the word.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Word.confidence**	number	Server scripts	The confidence level for the word.
PROPERTY	**Word.text**	string	Server scripts	The extracted text of the word.

▶ Official NetSuite Script Samples (4 Sample(s))

📄 Sample 1: Extract Text from a PDF File

```
require(['N/file', 'N/documentCapture', 'N/llm'],
    function(file, documentCapture, llm) {
        // "14" is the unique ID of a PDF stored in the NetSuite File Cabinet
        const fileObj = file.load({
            id: "14"
        });
        const extractedData = documentCapture.documentToText({
            file: fileObj
        });

        const response = llm.generateText({
            prompt: "What is this invoice for?",
            documents: [{
                id: '14',
                data: extractedData
            }]
        });

        log.debug("Answer: ", response.text);
        log.debug("Citations: ", response.citations);
    }
);
```

📄 Sample 2: Extract Feature Content from a Document Synchronously

```
require(['N/file', 'N/documentCapture', 'N/llm'],
    function(file, documentCapture, llm) {
        // "14" is the internal ID of a file stored in the NetSuite File Cabinet
        const extractedData = documentCapture.documentToStructure({
            file: file.load(
                id: "14"
            ),
            features: [
                documentCapture.Feature.TEXT_EXTRACTION,
                documentCapture.Feature.TABLE_EXTRACTION,
                documentCapture.Feature.FIELD_EXTRACTION
            ],
            documentType: documentCapture.DocumentType.INVOICE
        });

        const documentText = extractedData.getText();

        const response = llm.chat({
            preamble: "Your task is to parse the provided document and answer questions about that document.",
            prompt: "What is this invoice for?",
            documents: [{
                id: "1",
                data: documentText
            }]
        });

        log.debug("Answer: ", response.text);
        log.debug("Citations: ", response.citations);
    }
);
```

📄 Sample 3: Extract Content from a Document Asynchronously

```
require(['N/task', 'N/file', 'N/documentCapture'],
    function(task, file, documentCapture) {
        // Create the document capture task
        var docTask = task.create(task.TaskType.DOCUMENT_CAPTURE);

        // Specify task parameters
        docTask.inputFile = file.load("443");
        docTask.outputFilePath="SuiteScripts/result.json";
        docTask.documentType = documentCapture.DocumentType.INVOICE;

        // Submit the task
        const submissionId = docTask.submit();
        log.debug("Submission ID: ", submissionId);
});
```

📄 Sample 4: Extract Content from a Document Asynchronously

```
/**
 * @NApiVersion 2.1
 * @NScriptType ScheduledScript
 * @NModuleScope SameAccount
 */
define(['N/documentCapture', 'N/file', 'N/llm'],
    function(documentCapture, file, llm) {
        function execute(scriptContext) {
            const resultFile = file.load("SuiteScripts/result.json");
            const resultDocument = documentCapture.parseResult(resultFile);
            const resultText = resultDocument.getText();

            const response = llm.chat({
                preamble: "Your task is to parse the provided document and answer questions about that document.",
                prompt: "What is this invoice for?",
                documents:[{
                    id: "1",
                    data: resultText
                }]
            });
            log.debug("Response: ", response.text);
        }
        return {
            execute: execute
        };
    }
);
```

15. N/email Module

Load the N/email module to send email messages from within NetSuite. You can use the email module to send regular, bulk, and campaign email.

`define(['N/email'], (email) ⇒ { ... })` ✓ 1 Code Sample(s) Scripts: Client and server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/email'], (email) ⇒ { // Access methods and properties on email });
```

Use the N/email module to send email messages from within NetSuite. You can use the N/email module to send regular, bulk, and campaign email.

N/email Module Script Sample

N/email Module Members

Return Type / Value Type

Supported Script Types

email.send(options)

Client and server scripts

Sends transactional email to an individual or group of recipients and receives bounceback notifications.

To send email on another user's behalf, the user triggering the email send must have a role with the Vicarious Email (ADMI_VICARIOUS_EMAIL) permission. If a user without this permission executes SuiteScript that sends email, a Permission Violation error is returned.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**email.send(options)**	void	Client and server scripts	Sends transactional email to an individual or group of recipients and receives bounceback notifications. Note: To send email on another user's behalf, the user triggering the email send must have a role with the Vicarious Email (ADMI_VICARIOUS_EMAIL) permission. If a user without this permission executes SuiteScript that sends email, a Permission Violation error is returned.
METHOD	**email.send.promise(options)**	void	Client scripts	Sends transactional email asynchronously to an individual or group of recipients and receives bounceback notifications.
METHOD	**email.sendBulk(options)**	void	Client and server scripts	Sends bulk email (for use when a bounceback notification is not required).
METHOD	**email.sendBulk.promise(options)**	void	Client scripts	Sends bulk email asynchronously (for use when a bounceback notification is not required).
METHOD	**email.sendCampaignEvent(options)**	number	Client and server scripts	Sends a single 'on-demand' campaign email to a specified recipient and return a campaign response ID.
METHOD	**email.sendCampaignEvent.promise(options)**	number	Client and server scripts	Sends a single 'on-demand' campaign email asynchronously to a specified recipient and return a campaign response ID.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: N/email Module Script Sample

```
/**
 * @NApiVersion 2.1
 */

// This script sends an email with an attachment.

require(['N/email', 'N/record', 'N/file'], (email, record, file) =&gt; {
    const senderId = -515;
    const recipientEmail = 'notify@myCompany.com';
    let timeStamp = new Date().getUTCMilliseconds();

    let recipient = record.create({
        type: record.Type.CUSTOMER,
        isDynamic: true
    });
    recipient.setValue({
        fieldId: 'subsidiary',
        value: '1'
    });
    recipient.setValue({
        fieldId: 'companyname',
        value: 'Test Company' + timeStamp
    });
    recipient.setValue({
        fieldId: 'email',
        value: recipientEmail
    });

    let recipientId = recipient.save();

    let fileObj = file.load({
        id: 88
    });

    email.send({
        author: senderId,
        recipients: recipientId,
        subject: 'Test Sample Email Module',
        body: 'email body',
        attachments: [fileObj],
        relatedRecords: {
            entityId: recipientId,
            customRecord: {
                id: recordId,
                recordType: recordTypeId
            }
        }
    });
});
```

16. N/encode Module

Load the N/encode module to convert a string to another type of encoding. See encode.Encoding for a list of supported character set encoding.

`define(['N/encode'], (encode) => { ... })` ✓ 1 Code Sample(s) Scripts: Server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/encode'], (encode) => { // Access methods and properties on encode });
```

Use the N/encode module to convert a string to another type of encoding. This module exposes string encoding and decoding functionality.

N/encode Module Members

Return Type / Value Type

Supported Script Types

encode.convert(options)

Converts a string to another type of encoding and returns the re-encoded string.

Holds the string values for the supported character set encoding. Use this enum to set the inputEncoding and outputEncoding parameter values in N/crypto Module or N/encode Module.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	encode.convert(options)	string	Server scripts	Converts a string to another type of encoding and returns the re-encoded string.
ENUM	encode.Encoding	enum	Server scripts	Holds the string values for the supported character set encoding. Use this enum to set the inputEncoding and outputEncoding parameter values in N/crypto Module or N/encode Module.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: N/encode Module Script Samples

```
/**
 * @NApiVersion 2.x
 */

require(['N/encode'], function(encode) {
    function convertStringToDifferentEncoding() {
        var stringInput = "TÃƒÂ©st StriÃƒÂ±g Input";
        var base64EncodedString = encode.convert({
            string: stringInput,
            inputEncoding: encode.Encoding.UTF_8,
            outputEncoding: encode.Encoding.BASE_64
        });
        var hexEncodedString = encode.convert({
            string: stringInput,
            inputEncoding: encode.Encoding.UTF_8,
            outputEncoding: encode.Encoding.HEX
        });
    }

    convertStringToDifferentEncoding();
});
```

17. N/error Module

Load the N/error module to create your own custom SuiteScript errors. Use these custom errors in try-catch statements to abort script execution.

`define(['N/error'], (error) ⇒ { ... })` ✓ 2 Code Sample(s) Scripts: Server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/error'], (error) ⇒ { // Access methods and properties on error });
```

Use the N/error module to create custom SuiteScript errors that you can use in try-catch statements to abort script execution. Note that this module doesn't provide functionality to throw custom errors. However, you can include logic such as try-catch statements in your script to throw custom SuiteScript errors.

N/error Module Members

Return Type / Value Type

Supported Script Types

error.SuiteScriptError

Encapsulates a custom SuiteScript error for any server script type.

error.create(options)

error.SuiteScriptError

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	**error.SuiteScriptError**	Object	Server scripts	Encapsulates a custom SuiteScript error for any server script type.
METHOD	**error.create(options)**	error.SuiteScriptError	Server scripts	Creates a new error.SuiteScriptError object.
ENUM	**error.Type**	enum	Server scripts	Holds the string values for error types. Use this enum to set the value for the SuiteScriptError.name parameter of the error.create(options) method. This sets the value of the SuiteScriptError.type property.

SuiteScriptError Object Members

Return Type / Value Type

Supported Script Types

SuiteScriptError.cause

string (read-only)

Cause of the error.

SuiteScriptError.id

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**SuiteScriptError.cause**	string (read-only)	Server scripts	Cause of the error.
PROPERTY	**SuiteScriptError.id**	string (read-only)	Server scripts	Error ID that is automatically generated when a new error is created.
PROPERTY	**SuiteScriptError.message**	string (read-only)	Server scripts	Error message text displayed in the Details column of the Execution Log. Set from the options.message parameter when you create a new error using error.create(options)
PROPERTY	**SuiteScriptError.name**	string (read-only)	Server scripts	Error name or error code. Set from the options.name parameter when you create a new error using error.create(options).
PROPERTY	**SuiteScriptError.notifyOff**	boolean (read-only)	Server scripts	Suppresses email notification when set to true. Set from the options.notifyOff parameter when you create a new error using error.create(options)
PROPERTY	**SuiteScriptError.stack**	Array of strings (read-only)	Server scripts	List of method calls that the script is executing when the error is thrown.
PROPERTY	**SuiteScriptError.type**	error.Type (read-only)	Server scripts	Error type (error.SuiteScriptError).

▶ Official NetSuite Script Samples (2 Sample(s))

📄 Sample 1: Create a Custom Error

```
/**
 * @NApiVersion 2.x
 */

// This script creates a custom error.
require(['N/error'], function(error) {
    function createError() {
        var myCustomError = error.create({
            name: 'MY_ERROR_CODE',
            message: 'My custom error details',
            notifyOff: true
        });
    }
    createError();
});
```

📄 Sample 2: Create an Error Based on a Condition

```
/**
 * @NApiVersion 2.x
 */

// This script conditionally creates and throws an error.
require(['N/error'], function(error) {
    function showError() {
        var someVariable = false;

        if (!someVariable) {
            var myCustomError = error.create({
                name: 'WRONG_PARAMETER_TYPE',
                message: 'Wrong parameter type selected.',
                notifyOff: false
            });

            // This will write 'Error: WRONG_PARAMETER_TYPE Wrong parameter type selected' to the log
            log.error('Error: ' + myCustomError.name , myCustomError.message);
            throw myCustomError;
        }
    }
    showError();
});
```

18. N/file Module

Load the N/file module to work with files in NetSuite.

`define(['N/file'], (file) => { ... })` ✓ 6 Code Sample(s) Scripts: Server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/file'], (file) => { // Access methods and properties on file });
```

Use the N/file module to work with files within NetSuite. You can use this module to upload files to the NetSuite File Cabinet, as well as send files as attachments without uploading them to the File Cabinet. You can also use this module to copy files in the File Cabinet, and you can use conflict resolution options to handle conflicts (such as when a file with the same name already exists in the same folder in the File Cabinet).

When dealing with file naming conventions, especially when uploading files to the File Cabinet using SuiteScript, it is important to follow best practices to prevent file duplication. If your system works with multiple files simultaneously, consider including timestamps with millisecond precision in the file names. Additionally, generating a random number as part of the file name can further reduce the chances of duplicates.

A file.Reader object, which is returned by File.getReader(), can be used for special read operations. Use File.getSegments(options) to retrieve an iterator of custom segments of a file.

Methods that load content in memory, such as File.getContents(), have a 10 MB size limit. This limit does not apply when content is streamed, such as when File.save() is called.

The File.appendLine(options) method inserts a line to the end of a CSV or text file, which increases the file size accordingly. All files are screened for malicious content when they are uploaded or updated, and increased file size can slow down the process. Consider splitting large files into several smaller ones if you experience performance issues.

N/file Module Members

Return Type / Value Type

Supported Script Types

Encapsulates a file within NetSuite.

Encapsulates a reader that you can use to perform special read operations.

file.copy(options)

Copies an existing file in the File Cabinet.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	**file.File**	Object	Server scripts	Encapsulates a file within NetSuite.
OBJECT	**file.Reader**	Object	Server scripts	Encapsulates a reader that you can use to perform special read operations.
METHOD	**file.copy(options)**	file.File	Server scripts	Copies an existing file in the File Cabinet.
METHOD	**file.create(options)**	file.File	Server scripts	Creates a new file.File.
METHOD	**file.delete(options)**	void	Server scripts	Deletes an existing file.File from the NetSuite File Cabinet.
METHOD	**file.load(options)**	file.File	Server scripts	Loads an existing file.File from the NetSuite File Cabinet.
ENUM	**file.Encoding**	enum	Server scripts	Holds character encoding values for file contents. Use this enum to set the value of the File.encoding property.
ENUM	**file.NameConflictResolution**	enum	Server scripts	Holds conflict resolution values that apply when copying a file. Use this enum to specify how to resolve conflicts when copying files and to set the value of the conflict resolution parameter in file.copy(options)
ENUM	**file.Type**	enum	Server scripts	Holds file type values. Use this enum to set the value of the File.fileType property.

File Object Members

The following members are available for a file.File object.

Return Type / Value Type

Supported Script Types

File.appendLine(options)

Inserts a line to the end of a CSV or text file.

File.getContents()

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**File.appendLine(options)**	file.File	Server scripts	Inserts a line to the end of a CSV or text file.
METHOD	**File.getContents()**	string	Server scripts	Returns the content of a file in string format.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	File.lines.iterator()	boolean	Server scripts	Calls a developer-defined function for each line. Returns false when line processing stops.
METHOD	File.resetStream()	void	Server scripts	Resets the file stream to its previous state.
METHOD	File.save()	number	Server scripts	Saves a new or updated file to the File Cabinet.
METHOD	File.getReader()	Object	Server scripts	Returns reader object for read operations.
METHOD	File.getSegments(options)	Object	Server scripts	Returns an iterator of segments that are delimited by the specified separator.
PROPERTY	File.description	string	Server scripts	Description of a file.
PROPERTY	File.encoding	string	Server scripts	Character encoding on a file.
PROPERTY	File.fileType	enum	Server scripts	File type of a file.
PROPERTY	File.folder	number	Server scripts	Internal ID of the folder that houses a file within the NetSuite File Cabinet.
PROPERTY	File.id	number (read-only)	Server scripts	Internal ID of a file in the NetSuite File Cabinet.
PROPERTY	File.isInactive	boolean	Server scripts	Inactive status of a file. If set to true, the file is inactive.
PROPERTY	File.isOnline	boolean	Server scripts	'Available without Login' status of a file. If set to true, users can download the file outside of a current NetSuite login session.
PROPERTY	File.isText	boolean (read-only)	Server scripts	Indicates whether a file type is text-based.
PROPERTY	File.name	string	Server scripts	Name of a file.
PROPERTY	File.path	string (read-only)	Server scripts	Relative path to a file in the NetSuite File Cabinet.
PROPERTY	File.size	number (read-only)	Server scripts	Size of a file in bytes.
PROPERTY	File.url	string (read-only)	Server scripts	URL of a file.

Reader Object Members

The following members are available for a file.Reader object.

Return Type / Value Type

Supported Script Types

Reader.readChars(options)

Returns the next options.number characters from the current position.

Reader.readUntil(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Reader.readChars(options)	string	Server scripts	Returns the next options.number characters from the current position.
METHOD	Reader.readUntil(options)	string	Server scripts	Returns string from current position to the next occurrence of options.tag.

▶ Official NetSuite Script Samples (6 Sample(s))

📄 Sample 1: Create and Save a File to the File Cabinet

```
/**
 * @NApiVersion 2.1
 */
require(['N/file'], file =&gt; {
    // Create a file containing text
    let fileObj = file.create({
        name: 'testHelloWorld.txt',
        fileType: file.Type.PLAINTEXT,
        contents: 'Hello World\nHello World'
    });

    // Set the folder for the file
    // Note that this value is hard-coded in this sample, and you should use
    // a valid folder ID from your account
    fileObj.folder = -15;

    // Save the file
    let id = fileObj.save();

    // Load the same file to ensure it was saved correctly
    fileObj = file.load({
        id: id
    });
});
```

📄 Sample 2: Create a File, Set Property Values, and Save It to the File Cabinet

```
/**
 * @NApiVersion 2.1
 */
require(['N/file'], file =&gt; {
    // Create a file containing text
    // Note that the folder value is hard-coded in this sample, and you should
    // use a valid folder ID from your account
    let fileObj = file.create({
        name: 'testHelloWorld3.txt',
        fileType: file.Type.PLAINTEXT,
        contents: 'Hello World\nHello World',
        folder: -15,
        isOnline: true
    });

    // Save the file
    let id = fileObj.save();

    // Load the same file to ensure it was saved correctly
    fileObj = file.load({
        id: id
    });
});
```

📄 Sample 3: Create and Save a CSV File then Reload the File and Parse Its Contents

```
/**
 * @NApiVersion 2.x
 */

require(['N/file', 'N/error', 'N/log'], function (file, error, log) {
    // This sample calculates the total for the
    // second column value in a CSV file.
    //
    // Each line in the CSV file has the following format:
    // date,amount
    //
    // Here is the data that the script adds to the file:
    // 10/21/14,200.0
    // 10/21/15,210.2
    // 10/21/16,250.3

    // Create the CSV file
    var csvFile = file.create({
        name: 'data.csv',
        contents: 'date,amount\n',
        folder: 39,
        fileType: 'CSV'
    });

    // Add the data
    csvFile.appendLine({
        value: '10/21/14,200.0'
    });
    csvFile.appendLine({
        value: '10/21/15,210.2'
    });
    csvFile.appendLine({
        value: '10/21/16,250.3'
    });

    // Save the file
    var csvFileId = csvFile.save();

    // Create a variable to store the calculated total
    var total = 0.0;

    // Load the file
    var invoiceFile = file.load({
        id: csvFileId
    });

    // Obtain an iterator to process each line in the file
    var iterator = invoiceFile.lines.iterator();

    // Skip the first line, which is the CSV header line
    iterator.each(function () {return false;});

    // Process each line in the file
    iterator.each(function (line) {
        // Update the total based on the line value
        var lineValues = line.value.split(',');
        var lineAmount = parseFloat(lineValues[1]);
        if (!lineAmount) {
            throw error.create({
                name: 'INVALID_INVOICE_FILE',
                message: 'Invoice file contained non-numeric value for total: ' + lineValues[1]
            });
        }
        total += lineAmount;
        return true;
    });

    // At the completion of the iteration.each function, the total is 660.5

});
```

📄 Sample 4: Read and Log File Contents Using Commas and New Lines as Separators

```
/**
 * @NApiVersion 2.x
 * @NScriptType bankStatementParserPlugin
 */

define(['N/file', 'N/log'], function(file, log)   {
    return {
        parseBankStatement: function(context) {
            var reader = context.input.file.getReader();

            var textUntilFirstComma = reader.readUntil(',');
            var next10Characters = reader.readChars(10);
            var textUntilNextNewLine = reader.readUntil('\n');
            var next100Characters = reader.readChars(100);

            log.debug({
                title: 'STATEMENT TEXT',
                details: textUntilFirstComma
            });

            log.debug({
                title: 'STATEMENT TEXT',
                details: next10Characters
            });

            log.debug({
                title: 'STATEMENT TEXT',
                details: textUntilNextNewLine
            });

            log.debug({
                title: 'STATEMENT TEXT',
                details: next100Characters
            })
        }
    }
});
```

📄 Sample 5: Read and Log Segments of a File Using a Set of Characters as Separators

```
/**
 * @NApiVersion 2.x
 * @NScriptType bankStatementParserPlugin
 */

define(['N/file', 'N/log'], function(file, log)   {
    return {
        parseBankStatement: function(context) {
            var statementFile = context.input.file;

            var statementSegmentIterator = statementFile.getSegments({separator: '\\|_|/'}).iterator();
            statementSegmentIterator.each(function (segment) {
                log.debug({
                    title: 'STATEMENT TEXT',
                    details: segment.value
                });
                return true;
            });
        }
    }
});
```

📄 Sample 6: Copy a File Using Conflict Resolution

```
/**
 * @NApiVersion 2.x
 */
require(['N/file'], function(file) {
    var fileObj = file.create({
        name: 'test.txt',
        fileType: file.Type.PLAINTEXT,
        contents: 'Hello World\nHello World'
    });
    fileObj.folder = 2667;
    var id = fileObj.save();

    fileObj = file.copy({
        id: id,
        folder: 2670,
        conflictResolution: file.NameConflictResolution.RENAME_TO_UNIQUE
    });
});
```

19. N/format Module

Load the N/format module to convert strings into a specified format and to parse formatted data into strings.

`define(['N/format'], (format) ⇒ { ... })` ✓ 4 Code Sample(s) Scripts: Client and server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/format'], (format) ⇒ { // Access methods and properties on format });
```

Use the format module to parse formatted data into strings and to convert strings into a specified format. The format module formats data according to personal preferences set on the Set Preferences page, accessible from Home > Set Preferences. See Setting Personal Preferences.

N/format Module Members

Return Type / Value Type

Supported Script Types

format.format(options)

Client and server scripts

Takes a raw value and returns a formatted value.

This method is overloaded when you format a datetime or datetimetz value.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**format.format(options)**	`string | Date`	Client and server scripts	Takes a raw value and returns a formatted value. Note: This method is overloaded when you format a datetime or datetimetz value.
METHOD	**format.parse(options)**	`Date | string | number`	Client and server scripts	Takes a formatted value and returns a raw value. Note: This method is overloaded when you format a datetime or datetimetz value.
ENUM	**format.Timezone**	`enum`	Client and server scripts	Holds the string values for supported time zone formats. Use this enum to set the value of the options.timezone parameter.
ENUM	**format.Type**	`enum`	Client and server scripts	Holds the string values for the supported field types. Use this enum to set the value of the options.type parameter when calling format.format(options) or format.parse(options).

▶ Official NetSuite Script Samples (4 Sample(s))

📄 Sample 1: Parse a String to a Date Object

```
/**
 * @NApiVersion 2.1
 * @NScriptType Suitelet
 */
define(['N/ui/serverWidget', 'N/format'], function(serverWidget, format) {
    function parseAndFormatDateString() {
        // Assuming Date format is MM/DD/YYYY
        var initialFormattedDateString = "07/28/2015";
        var parsedDateStringAsRawDateObject = format.parse({
            value: initialFormattedDateString,
            type: format.Type.DATE
        });
        var formattedDateString = format.format({
            value: parsedDateStringAsRawDateObject,
            type: format.Type.DATE
        });
        return [parsedDateStringAsRawDateObject, formattedDateString];
    }
    function onRequest(context) {
        var data = parseAndFormatDateString();

        var form = serverWidget.createForm({
            title: "Date"
        });

        var fldDate = form.addField({
            type: serverWidget.FieldType.DATE,
            id: "date",
            label: "Date"
        });
        fldDate.defaultValue = data[0];

        var fldString = form.addField({
            type: serverWidget.FieldType.TEXT,
            id: "dateastext",
            label: "Date as text"
        });
        fldString.defaultValue = data[1];

        context.response.writePage(form);
    }
    return {
        onRequest: onRequest
    };
});
```

📄 Sample 2: Parse a String to a Number

```
/**
 * @NApiVersion 2.x
 */

require(['N/format'],
    function(format){
         function parseToValue() {
              // Assume number format is 1.000.000,00 and negative format is -100
              var formattedNum = "-20.000,25"
              return format.parse({value:formattedNum, type: format.Type.FLOAT})
              }
         var rawNum = parseToValue(); // -20000.25 -- a number
   });
```

📄 Sample 3: Format a Number as a String

```
/**
 * @NApiVersion 2.x
 */

require(['N/format'],
    function(format){
         function formatToString() {
              // Assume number format is 1.000.000,00 and negative format is (100)
              var rawNum2 =  -44444.44
              return format.format({value:rawNum2, type: format.Type.FLOAT})
              }
         var formattedNum2 = formatToString(); // "44.444,44" -- a string
   });
```

📄 Sample 4: Format Time of Day as a String

```
/**
 * @NApiVersion 2.1
 */

require(['N/format'],
    function(format){
         function formatTimeOfDay() {
              // Instantiate a new Date object assuming it's now 7:01PM.
              const now = new Date();
              // Select a format from the format.Type enum. In this case TIMEOFDAY
              return format.format({value: now, type: format.Type.TIMEOFDAY})
              }
         const formattedTime = formatTimeOfDay(); // "7:01 pm" -- a string
  }); const formattedTime = formatTimeOfDay(); // "7:01 pm" -- a string
 });
```

20. N/format/i18n Module

Load the N/format/i18n module to format currency.

`define(['N/format/i18n'], (i18n) => { ... })` ✓ 8 Code Sample(s) Scripts: Client and server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/format/i18n'], (i18n) => { // Access methods and properties on i18n });
```

Use the N/format/i18n module to format strings in international context and format numbers to currency or number strings. You can also use this module to format phone number to strings and parse strings to phone number.

N/format/i18n Module Members

Return Type / Value Type

Supported Script Types

format.CurrencyFormatter

Client and server scripts

Represents the object that formats the number to currency string.

format.NumberFormatter

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	format.CurrencyFormatter	Object	Client and server scripts	Represents the object that formats the number to currency string.
OBJECT	format.NumberFormatter	Object	Client and server scripts	Represents the object that formats the number to string.
OBJECT	format.PhoneNumberFormatter	Object	Client and server scripts	Represents the object that formats the phone number to string.
OBJECT	format.PhoneNumberParser	Object	Client and server scripts	Represents the object that parses the string to phone number.
METHOD	format.spellOut(options)	string	Client and server scripts	Creates a string containing the spelled-out version of the specified number in a specified locale.
METHOD	format.getCurrencyFormatter(options)	object	Client and server scripts	Creates a format.CurrencyFormatter object to format numbers to currency strings.
METHOD	format.getNumberFormatter(options)	object	Client and server scripts	Creates a format.NumberFormatter object to format numbers to strings.
METHOD	format.getPhoneNumberFormatter(options)	object	Client and server scripts	Creates a format.PhoneNumberFormatter object to format phone numbers to strings.
METHOD	format.getPhoneNumberParser(options)	object	Client and server scripts	Parses a phone number from a string. Returns a format.PhoneNumberParser object.
ENUM	format.NegativeNumberFormat	enum	Client and server scripts	Holds the values for the negative number format. Used to set the value of the options.negativeNumberFormatter parameter of the format.getNumberFormatter(options).
ENUM	format.Currency	enum	Client and server scripts	Holds the values for the currency code. Used to set the value of the options.currency parameter of the format.getCurrencyFormatter(options) method.

Currency Formatter Object Members

The following members are called on the format.CurrencyFormatter object.

Return Type / Value Type

Supported Script Types

CurrencyFormatter.currency

Client and server scripts

Indicates the currency code.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	CurrencyFormatter.currency	string	Client and server scripts	Indicates the currency code.
PROPERTY	CurrencyFormatter.locale	string	Client and server scripts	The locale of the currency formatter.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	CurrencyFormatter.symbol	string	Client and server scripts	Indicates the currency symbol.
PROPERTY	CurrencyFormatter.numberFormatter	object	Client and server scripts	Contains the format.NumberFormatter object derived from format.CurrencyFormatter with the same number formatting parameters without currency symbol.
PROPERTY	CurrencyFormatter.format(options)	string	Client and server scripts	Formats the number to the currency string.

Number Formatter Object Members

The following members are called on the format.NumberFormatter object.

Return Type / Value Type

Supported Script Types

NumberFormatter.groupSeparator

Client and server scripts

Indicates the group separator.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	NumberFormatter.groupSeparator	string	Client and server scripts	Indicates the group separator.
PROPERTY	NumberFormatter.decimalSeparator	string	Client and server scripts	Indicates the decimal separator.
PROPERTY	NumberFormatter.locale	string	Client and server scripts	The locale of the number formatter.
PROPERTY	NumberFormatter.precision	number	Client and server scripts	Indicates the precision.
PROPERTY	format.NegativeNumberFormat	enum	Client and server scripts	Indicates the negative number format.
PROPERTY	NumberFormatter.format(options)	string	Client and server scripts	Formats the number to string.

Phone Number Formatter Object Members

The following members are called on the format.PhoneNumberFormatter object.

Return Type / Value Type

Supported Script Types

format.PhoneNumberFormatter

Client and server scripts

The object that formats the phone number to string.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	format.PhoneNumberFormatter	Object	Client and server scripts	The object that formats the phone number to string.
METHOD	PhoneNumberFormatter.format(options)	string	Client and server scripts	Formats phone number object to string.
ENUM	format.PhoneNumberFormatType	enum	Client and server scripts	Holds the values for the phone number format type. Used to set the value of the options.formatType parameter of the format.getPhoneNumberFormatter(options) method.
ENUM	format.Country	enum	Client and server scripts	Hold the values for the countries. Used to set the value of the options.defaultCountry parameter in the format.getPhoneNumberParser(options) method.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	format.PhoneNumberParser	Object	Client and server scripts	The object that parses the string to phone number.
METHOD	PhoneNumberParser.parse(options)	Object of type PhoneNumber	Client and server scripts	Parses string to phone number.

▶ Official NetSuite Script Samples (8 Sample(s))

📄 Sample 1: Format 12345 as a German String

```
/**
 * @NApiVersion 2.x
 */
require(['N/format/i18n'], function(format) {
     var spellOut = format.spellOut({
        number: 12345,
        locale: "DE"
    });
    // spellOut is 'zwölftausenddreihundertfünfundvierzig'
});
```

📄 Sample 2: Format a Number as a String Using N/format/i18n

```
/**
 * @NApiVersion 2.x
 */
require(['N/format/i18n'], function(format) {
    var numberFormatter = format.getNumberFormatter();

    var gs = numberFormatter.groupSeparator;         // gs is ','
    var ds = numberFormatter.decimalSeparator;       // ds is '.'
    var precision = numberFormatter.precision;       // precision is '2'
    var nnf = numberFormatter.negativeNumberFormat;  // nnf is 'BRACKETS'

    var formatNum1 = numberFormatter.format({
        number: 12.53
    }); // formatNum1 is '12.53'

    var formatNum2 = numberFormatter.format({
        number: 12845.22
    }); // formatNum2 is '12,845.22'

    var formatNum3 = numberFormatter.format({
        number: -5421
    }); // formatNum3 is '(5,421.00)'

    var formatNum4 = numberFormatter.format({
        number: 0.00
    }); // formatNum4 is '0.00'

    var formatNum5 = numberFormatter.format({
        number: 0.3456789
    }); // formatNum5 is '0.35'
});
```

📄 Sample 3: Format Numbers as Currency Strings

```
/**
 * @NApiVersion 2.1
 */
require(['N/format/i18n'], function(format) {
    var curFormatter = format.getCurrencyFormatter({
        currency: "EUR"
    });

    var curCur = curFormatter.currency;                 // curCur is 'EUR'
    var numberFormat = curFormatter.numberFormatter;

    var curSymbol = curFormatter.symbol;                // curSymbol is '€'
    var curSeparator = numberFormat.groupSeparator;     // curSeparator is ' '
    var curDecimalSep = numberFormat.decimalSeparator;  // curDecimalSep is ','
    var curPrecision = numberFormat.precision;          // curPrecision is 2
    var curNNF = numberFormat.negativeNumberFormat;     // curNNF is MINUS

    curFormatNum1 = curFormatter.format({
        number: 12.53
    }); // curFormat1 is '€12,53'

    curFormatNum2 = curFormatter.format({
        number: -5421
    }); // curFormat2 is '€-5 421,00'

    curFormatNum3 = curFormatter.format({
        number: 0.00
    }); // curFormat3 is '€0,00'

    curFormatNum4 = curFormatter.format({
        number: 0.3456789
    }); // curFormat4 is '€0,35'
});
```

📄 Sample 4: Parse a Czech Phone Number

```
/**
 * @NApiVersion 2.x
 */
require(['N/format/i18n'], function(format) {
    var origNumberStr = "602547854ext.154";
    var pnParser = format.getPhoneNumberParser({
        defaultCountry: format.Country.CZECH_REPUBLIC
    });
    var phoneNumber = pnParser.parse({
        number: origNumberStr
    });

    /* after parsing:
       phoneNumber.countryCode is '420'
       phoneNumber.extension is '154'
       phoneNumber.nationalNumber is '602547854'
       phoneNumber.numberofLeadingZeros is 1
       phoneNumber.carrierCode is ' '
       phoneNumber.rawInput is '602547854ext.154'
    */

    var pnFormatter = format.getPhoneNumberFormatter({});
    var strNumber = pnFormatter.format({
        number: phoneNumber
    });  // strNumber is '+420 602 547 854 ext. 154'
});
```

📄 Sample 5: Parse a U.S. Phone Number

```
/**
 * @NApiVersion 2.x
 */
require(['N/format/i18n'], function(format) {
    var origNumberStr = "7524105210ext.154";
    var pnParser = format.getPhoneNumberParser({
        defaultCountry: format.Country.UNITED_STATES
    });
    var phoneNumber = pnParser.parse({
        number: origNumberStr
    });

    /* after parsing:
       phoneNumber.countryCode is '1'
       phoneNumber.extension is '154'
       phoneNumber.nationalNumber is '7524105210'
       phoneNumber.numberofLeadingZeros is 1
       phoneNumber.carrierCode is ' '
       phoneNumber.rawInput is '7524105210ext.154'
    */

    var pnFormatter = format.getPhoneNumberFormatter({
        formatType: format.PhoneNumberFormatType.NATIONAL
    });
    var strNumber = pnFormatter.format({
        number: phoneNumber
    }); // strNumber is '(752) 410-5210 ext. 154'
});
```

📄 Sample 6: Format Numbers Based on the Locale Parameter

```
/**
 * @NApiVersion 2.x
 */
require(['N/format/i18n'], function(format) {
    var numberFormatter = format.getNumberFormatter({
        locale: "fr_FR"
    });

    var gs = numberFormatter.groupSeparator;        // gs is ' '
    var ds = numberFormatter.decimalSeparator;      // ds is ','
    var prec = numberFormatter.precision;           // prec is 2
    var nnf = numberFormatter.negativeNumberFormat; // nnf is MINUS

    var number1 = numberFormatter.format({
        number: 123456.55
    });  // number1 is '123 456,55'

    var number2 = numberFormatter.format({
        number: -123456.55
    });  // number2 is '-123 456,55'
});
```

📄 Sample 7: Format Currency Based on the Locale Parameter

```
/**
 * @NApiVersion 2.1
 */
require(['N/format/i18n'], function(format) {
    var curFormatter = format.getCurrencyFormatter({
        locale: "IT_IT"
    });

    var curCur = curFormatter.currency;   // curCur is 'EUR'
    var sym = curFormatter.symbol;        // sym is '€'

    var numberFormat = curFormatter.numberFormatter;

    var gs = numberFormat.groupSeparator;         // gs is '.'
    var ds = numberFormat.decimalSeparator;       // ds is ','
    var prec = numberFormat.precision;            // prec is 2
    var nnf = numberFormat.negativeNumberFormat;  // nnf is MINUS

    var currency1 = curFormatter.format({
        number: 123456.55
    });// currency1 is '€123.456,55'

    var currency2 = curFormatter.format({
        number: -123456.55
    });  // currency2 is '€-123.456,55'
});
```

📄 Sample 8: Format Numbers and Currencies Based on the English-India Locale Parameter

```
/**
 * @NApiVersion 2.1
 */
require(['N/format/i18n'], function(format) {
    var curFormatter = format.getCurrencyFormatter({
        locale: "en_IN"
    });

    var curCur = curFormatter.currency;  // curCur is 'INR'

    var numberFormat = curFormatter.numberFormatter;
    var sym = curFormatter.symbol;                  // sym is '₹'
    var gs = numberFormat.groupSeparator;           // gs is ','
    var ds = numberFormat.decimalSeparator;         // ds is '.'
    var prec = numberFormat.precision;              // prec is 2
    var nnf = numberFormat.negativeNumberFormat;    // nnf is MINUS

    var currency1 = curFormatter.format({
        number: 12345678.55
    });  // currency1 is '₹1,23,45,678.55'

    var currency2 = curFormatter.format({
        number: -345678.55
    });  // currency2 is '₹-3,45,678.55'
});
```

21. N/http Module

Load the N/http module to make http calls.

`define(['N/http'], (http) => { ... })` ✓ 2 Code Sample(s) Scripts: Client and server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/http'], (http) => { // Access methods and properties on http });
```

Use the N/http module to make HTTP calls from server or client scripts. For client scripts, this module also provides the ability to make cross-domain HTTP requests using NetSuite servers as proxies.

All HTTP content types are supported.

The N/http module does not accept the HTTPS protocol. Use the N/https Module for that purpose.

HTTP Header Information

HTTP headers can be used to pass additional information with an HTTP request or response. Each HTTP header consists of its case-insensitive name followed by a colon (:), then by its value (without line breaks). If you use custom headers, make sure the names of these headers do not contain underscores. For a general list of all HTTP headers, visit http://developer.mozilla.org/en-US/docs/Web/HTTP/Headers.

Some headers are not supported in NetSuite and are blocked. These are listed below as either general HTTP headers or Suitelet response headers.

Be aware that certain headers cannot be set manually when using the N/http module methods. If a script attempts to set values for any of the following headers, the values are discarded. These headers are listed in the following table.

Transfer-Encoding

In addition to the headers described in General Blocked HTTP Headers, certain headers cannot be set manually when interacting with the http.ServerResponse Objects sent by Suitelets. If a script attempts to set values for any of these headers, the system throws an SSS_INVALID_HEADER error. These headers are listed in the following table.

Content-Location

CONNECTION CONTENT-LENGTH HOST JSESSIONID	TRAILER TRANSFER-ENCODING UPGRADE VIA

ALLOW CONTENT-LOCATION CONTENT-MD5 CONTENT-RANGE DATE	LOCATION PROXY-AUTHENTICATE PUBLIC-KEY-PINS PUBLIC-KEY-PINS-REPORT-ONLY RETRY-AFTER	SERVER STRICT-TRANSPORT-SECURITY UPGRADE-INSECURE-REQUESTS WARNING WWW-AUTHENTICATE

N/http Module Members

Return Type / Value Type

Supported Script Types

http.ClientResponse

Object (read-only)

The response from the server to an HTTP client request (for example, http.get(options)).

http.ServerRequest

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	**http.ClientResponse**	`Object (read-only)`	Server scripts	The response from the server to an HTTP client request (for example, http.get(options)).
OBJECT	**http.ServerRequest**	`Object (read-only)`	Server scripts	HTTP request information sent to an HTTP server. For example, a request sent to/received by a Suitelet or RESTlet.
OBJECT	**http.ServerResponse**	`Object`	Server scripts	The response from an HTTP server to an HTTP request. For example, a response from a Suitelet or RESTlet.
METHOD	**http.delete(options)**	`http.ClientResponse or http.ServerResponse`	Client and server scripts	Sends an HTTP DELETE request and returns the response.
METHOD	**http.delete.promise(options)**	`Promise Object`	Client and server scripts	Sends an HTTP DELETE request asynchronously and returns the response.
METHOD	**http.get(options)**	`http.ClientResponse or http.ServerResponse`	Client and server scripts	Sends an HTTP GET request and returns the response.
METHOD	**http.get.promise(options)**	`Promise Object`	Client and server scripts	Sends an HTTP GET request asynchronously and returns the response.
METHOD	**http.post(options)**	`http.ClientResponse or http.ServerResponse`	Client and server scripts	Sends an HTTP POST request and returns the response.
METHOD	**http.post.promise(options)**	`Promise Object`	Client and server scripts	Sends an HTTP POST request asynchronously and returns the response.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**http.put(options)**	http.ClientResponse or http.ServerResponse	Client and server scripts	Sends an HTTP PUT request and returns the response.
METHOD	**http.put.promise(options)**	Promise Object	Client and server scripts	Sends an HTTP PUT request asynchronously and returns the response.
METHOD	**http.request(options)**	http.ClientResponse or http.ServerResponse	Client and server scripts	Sends an HTTP request and returns the response.
METHOD	**http.request.promise(options)**	Promise Object	Client and server scripts	Sends an HTTP request asynchronously and returns the response.
ENUM	**http.CacheDuration**	enum	Server scripts	Holds the string values for supported cache durations. Use this enum to set the value of the type parameter in ServerResponse.setCdnCacheable(options).
ENUM	**http.Method**	enum	Server scripts	Holds the string values for supported HTTP requests. Use this enum to set the value of method parameter in http.request(options).
ENUM	**http.RedirectType**	enum	Server scripts	Holds the string values for supported NetSuite resources that you can redirect to. Use this enum to set the value of the type parameter for ServerResponse.sendRedirect(options).

ClientResponse Object Members

The following members are called on the http.ClientResponse Object.

Return Type / Value Type

Supported Script Types

ClientResponse.body

string (read-only)

The client response body.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**ClientResponse.body**	string (read-only)	Server scripts	The client response body.
PROPERTY	**ClientResponse.code**	number (read-only)	Server scripts	The client response code.
PROPERTY	**ClientResponse.headers**	Object (read-only)	Server scripts	The client response headers.

ServerRequest Object Members

The following members are called on the http.ServerRequest Object.

Return Type / Value Type

Supported Script Types

ServerRequest.getLineCount(options)

Returns the number of lines in a sublist.

ServerRequest.getSublistValue(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**ServerRequest.getLineCount(options)**	number	Server scripts	Returns the number of lines in a sublist.
METHOD	**ServerRequest.getSublistValue(options)**	string	Server scripts	Returns the value of a sublist line item.
PROPERTY	**ServerRequest.body**	string (read-only)	Server scripts	The server request body.
PROPERTY	**ServerRequest.files**	Object (read-only)	Server scripts	The server request files.
PROPERTY	**ServerRequest.headers**	Object (read-only)	Server scripts	The server request headers.
PROPERTY	**ServerRequest.clientIpAddress**	String (read-only)	Server scripts	The remote client IP address.
PROPERTY	**ServerRequest.method**	http.Method	Server scripts	The server request HTTP method.
PROPERTY	**ServerRequest.parameters**	Object (read-only)	Server scripts	The server request parameters.
PROPERTY	**ServerRequest.url**	string (read-only)	Server scripts	The server request URL.

ServerResponse Object Members

The following members are called on the http.ServerResponse Object.

Return Type / Value Type

Supported Script Types

ServerResponse.addHeader(options)

Adds a header to the response.

ServerResponse.getHeader(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	ServerResponse.addHeader(options)	void	Server scripts	Adds a header to the response.
METHOD	ServerResponse.getHeader(options)	string | string[]	Server scripts	Returns the value of a response header.
METHOD	ServerResponse.renderPdf(options)	void	Server scripts	Generates and renders a PDF directly to the response.
METHOD	ServerResponse.sendRedirect(options)	void	Server scripts	Sets the redirect URL by resolving to a NetSuite resource.
METHOD	ServerResponse.setCdnCacheable(options)	void	Server scripts	Sets CDN caching for a period of time.
METHOD	ServerResponse.setHeader(options)	void	Server scripts	Sets the value of a response header.
METHOD	ServerResponse.write(options)	void	Server scripts	Writes information (text, xml, html) to the response.
METHOD	ServerResponse.writeFile(options)	void	Server scripts	Writes a file to the response.
METHOD	ServerResponse.writeLine(options)	void	Server scripts	Writes line information (text, xml, html) to the response.
METHOD	ServerResponse.writePage(options)	void	Server scripts	Generates a page.
PROPERTY	ServerResponse.headers	Object (read-only)	Server scripts	The server response headers.

▶ Official NetSuite Script Samples (2 Sample(s))

📄 Sample 1: Request a URL

```
/**
 * @NApiVersion 2.1
 */

// This script uses an HTTP GET request for a URL.
require(['N/http'], (http)=&gt; {
    function sendGetRequest() {
        let response = http.get({
            url: 'http://www.google.com'
        });
    }
    sendGetRequest();
});
```

📄 Sample 2: Redirect a record

```
/**
 * @NApiVersion 2.1
 * @NScriptType Suitelet
 */

// This script redirects a new sales order record and sets the entity.
define(['N/record', 'N/http'], (record, http)=&gt; {
    function onRequest(context) {
        context.response.sendRedirect({
            type: http.RedirectType.RECORD,
            identifier: record.Type.SALES_ORDER,
            parameters: ({
                entity: 6
            })
        });
    }
    return {
        onRequest: onRequest
    };
});
```

22. N/https Module

Load the N/https module to make https calls. You can also use this module to encode binary content or securely access a handle to the value in a NetSuite credential field.

`define(['N/https'], (https) => { ... })` ✓ 10 Code Sample(s) Scripts: Client and server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/https'], (https) => { // Access methods and properties on https });
```

Use the N/https module to manage content sent to a third party using HTTPS calls. This module encapsulates all the functionality of the N/http Module, but does not allow the HTTP protocol. You can make HTTPS calls from client and server scripts.

You can use the N/https module to encode binary content or access a handle to the value in a NetSuite credential field.

You can use the N/https module to communicate between SuiteScript scripts, RESTlets, and SuiteTalk REST APIs without having to reauthenticate, using the https.requestRestlet(options) and https.requestSuiteTalkRest(options) methods.

When the N/https module is used, SuiteScript also loads the N/crypto Module and N/encode Module.

Use TLS 1.2 for HTTPS requests. SuiteScript 2.0 requests such https.delete(options), https.get(options), https.post(options), https.put(options), and https.request(options) usually go to third-party servers. Management of these servers is not within the control of your company. These HTTPS requests now fail the handshake when they attempt to connect to servers that do not support TLS 1.2. You should communicate with those who manage any third-party servers to which you connect, and ensure their servers support the TLS 1.2 protocol.

NetSuite supports the same list of trusted third-party certificate authorities (CAs) as the Mozilla Included CA Certificate List.

The target endpoint, domain, or server must use one of these trusted third-party CAs, or the connection cannot be established. Oracle NetSuite requires that the endpoints that you are connecting to from NetSuite provide a full certification chain, including intermediate certificates.

For a list of certificate authorities, see https://wiki.mozilla.org/CA/Included_Certificates.

Using plain text or other unencrypted user credentials is unsafe and can pose a security threat. Whenever possible, use Token-based Authentication (TBA) or OAuth 2.0 to specify user credentials.

HTTPS Header Information

HTTP headers add extra information to a request or response. A header has a case-insensitive name, a colon (:), and a value on the same line. When you define custom headers, don't include underscores in the name. For a list of standard HTTP and HTTPS headers, see https://developer.mozilla.org/docs/Web/HTTP/Headers.

If you call https.post(options) without a Content-Type header, NetSuite sets a default value:

POST Request Body Type

Default Content-Type Header

application/octet-stream

application/x-www-form-urlencoded; charset=UTF-8

POST REQUEST BODY TYPE	DEFAULT CONTENT-TYPE HEADER
OBJECT	`application/octet-stream`
UINT8ARRAY	`application/x-www-form-urlencoded; charset=UTF-8`
OTHER TYPE	`text/xml; charset=UTF-8`

CONNECTION CONTENT-LENGTH HOST JSESSIONID	TRAILER TRANSFER-ENCODING UPGRADE VIA

ALLOW CONTENT-LOCATION CONTENT-MD5 CONTENT-RANGE DATE	LOCATION PROXY-AUTHENTICATE PUBLIC-KEY-PINS PUBLIC-KEY-PINS-REPORT-ONLY RETRY-AFTER	SERVER STRICT-TRANSPORT-SECURITY UPGRADE-INSECURE-REQUESTS WARNING WWW-AUTHENTICATE

N/https Module Members

Return Type / Value Type

Supported Script Types

https.SecureString

Encapsulates data that may be sent to a third-party using an HTTPS call.

https.ClientResponse

Object (read-only)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	**https.SecureString**	Object	Server scripts	Encapsulates data that may be sent to a third-party using an HTTPS call.
OBJECT	**https.ClientResponse**	Object (read-only)	Server scripts	Encapsulates the response to an HTTPS client request.
OBJECT	**https.ServerRequest**	Object (read-only)	Server scripts	Encapsulates the HTTPS request information sent to an HTTPS server. For example, a request received by a Suitelet or RESTlet.
OBJECT	**https.ServerResponse**	Object	Server scripts	Encapsulates the response from an HTTPS server to an HTTPS request. For example, a response from a Suitelet or RESTlet.
METHOD	**https.createSecretKey(options)**	Object	Server scripts	Creates a key for the contents of a credential field.
METHOD	**https.createSecureString(options)**	Object	Server scripts	Creates an https.SecureString Object.
METHOD	**https.delete(options)**	https.ClientResponse or https.ServerResponse	Client and server scripts	Sends an HTTPS DELETE request and returns the response.
METHOD	**https.delete.promise(options)**	Promise Object	Client and server scripts	Sends an HTTPS DELETE request asynchronously and returns the response.
METHOD	**https.get(options)**	https.ClientResponse or https.ServerResponse	Client and server scripts	Sends an HTTPS GET request and returns the response.
METHOD	**https.get.promise(options)**	Promise Object	Client and server scripts	Sends an HTTPS GET request asynchronously and returns the response.
METHOD	**https.post(options)**	https.ClientResponse or https.ServerResponse	Client and server scripts	Sends an HTTPS POST request and returns the response.
METHOD	**https.post.promise(options)**	Promise Object	Client and server scripts	Sends an HTTPS POST request asynchronously and returns the response.
METHOD	**https.put(options)**	https.ClientResponse or https.ServerResponse	Client and server scripts	Sends an HTTPS PUT request and returns the response.
METHOD	**https.put.promise(options)**	Promise Object	Client and server scripts	Sends an HTTPS PUT asynchronously request and returns the response.
METHOD	**https.request(options)**	https.ClientResponse or https.ServerResponse	Client and server scripts	Sends an HTTPS request and returns the response. If a request fails, an error.SuiteScriptError is thrown.
METHOD	**https.request.promise(options)**	Promise Object	Client and server scripts	Sends an HTTPS request asynchronously and returns the response. If a request fails, a Promise.reject is thrown with a parameter Error.
METHOD	**https.requestRestlet(options)**	https.ClientResponse	Server scripts	Sends an HTTPS request to a RESTlet and returns the response. Authentication headers are automatically added. The RESTlet will run with the same privileges as the calling script.
METHOD	**https.requestRestlet.promise(options)**	Promise Object	Client and server scripts	Sends an HTTPS request to a Restlet and returns the response.
METHOD	**https.requestSuitelet(options)**	https.ClientResponse	Client and server scripts	Sends an HTTPS request to a Suitelet and returns the response.
METHOD	**https.requestSuitelet.promise(options)**	Promise Object	Client and server scripts	Sends an HTTPS request asynchronously to a Suitelet and returns the response.
METHOD	**https.requestSuiteTalkRest(options)**	https.ClientResponse	Server scripts	Sends an HTTPS request to a SuiteTalk REST endpoint and returns the response. Authentication headers are automatically added.
ENUM	**https.CacheDuration**	enum	Server scripts	Holds the string values for supported cache durations. Use this enum to set the value of the type parameter in ServerResponse.setCdnCacheable(options).
ENUM	**https.Encoding**	enum	Server scripts	Holds the string values for supported encoding types. Use this enum to set the value of parameters in SecureString.appendString(options), SecureString.convertEncoding(options), https.createSecureString(options).
ENUM	**https.HashAlg**	enum	Server scripts	Holds the string values for supported hashing algorithms. Use this enum to set the value of parameters in SecureString.hash(options) and SecureString.hmac(options).
ENUM	**https.Method**	enum	Server scripts	Holds the string values for supported HTTPS requests. Use this enum to set the value of method parameter in https.request(options).
ENUM	**https.RedirectType**	enum	Server scripts	Holds the string values for supported NetSuite resources that you can redirect to. Use this enum to set the value of the type parameter for ServerResponse.sendRedirect(options).

SecureString Object Members

SecureString functionality is supported only in server scripts.

The following members are called on the https.SecureString Object.

Return Type / Value Type

Supported Script Types

SecureString.appendSecureString(options)

https.SecureString

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	SecureString.appendSecureString(options)	https.SecureString	Server scripts	Appends one https.SecureString to another https.SecureString.
METHOD	SecureString.appendString(options)	https.SecureString	Server scripts	Appends a string to a https.SecureString.
METHOD	SecureString.convertEncoding(options)	https.SecureString	Server scripts	Converts the content of a https.SecureString between two encodings.
METHOD	SecureString.hash(options)	https.SecureString	Server scripts	Creates a hash for a https.SecureString.
METHOD	SecureString.hmac(options)	https.SecureString	Server scripts	Creates an hmac for a https.SecureString.
METHOD	SecureString.replaceString(options)	https.SecureString	Server Scripts	Replaces all occurrences of a pattern string inside a https.SecureString with a replacement string.

ClientResponse Object Members

The following members are called on the https.ClientResponse Object.

Return Type / Value Type

Supported Script Types

ClientResponse.body

string (read-only)

The response body.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	ClientResponse.body	string (read-only)	Server scripts	The response body.
PROPERTY	ClientResponse.code	number (read-only)	Server scripts	The response code.
PROPERTY	ClientResponse.headers	Object (read-only)	Server scripts	The response body.

ServerRequest Object Members

The following members are called on the https.ServerRequest Object.

Return Type / Value Type

Supported Script Types

ServerRequest.getLineCount(options)

Returns the number of lines in a sublist.

ServerRequest.getSublistValue(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	ServerRequest.getLineCount(options)	number	Server scripts	Returns the number of lines in a sublist.
METHOD	ServerRequest.getSublistValue(options)	string	Server scripts	Returns the value of a sublist line item.
PROPERTY	ServerRequest.body	string (read-only)	Server scripts	The server request body
PROPERTY	ServerRequest.files	Object (read-only)	Server scripts	The server request files represented as object in ID-file.File pair.
PROPERTY	ServerRequest.headers	Object (read-only)	Server scripts	The server request headers.
PROPERTY	ServerRequest.method	https.Method	Server scripts	The HTTPS method for the server request.
PROPERTY	ServerRequest.parameters	Object (read-only)	Server scripts	The server request parameters.
PROPERTY	ServerRequest.url	string (read-only)	Server scripts	The server request URL.

ServerResponse Object Members

The following members are called on the https.ServerResponse Object.

Return Type / Value Type

Supported Script Types

ServerResponse.addHeader(options)

Adds a header to the response.

ServerResponse.getHeader(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	ServerResponse.addHeader(options)	void	Server scripts	Adds a header to the response.
METHOD	ServerResponse.getHeader(options)	string | string[]	Server scripts	Returns the value of a response header.
METHOD	ServerResponse.renderPdf(options)	void	Server scripts	Generates and renders a PDF directly to the response.
METHOD	ServerResponse.sendRedirect(options)	void	Server scripts	Sets the redirect URL by resolving to a NetSuite resource.
METHOD	ServerResponse.setCdnCacheable(options)	void	Server scripts	Sets CDN caching for a period of time.
METHOD	ServerResponse.setHeader(options)	void	Server scripts	Sets the value of a response header.
METHOD	ServerResponse.write(options)	void	Server scripts	Writes information (text/xml/html) to the response.
METHOD	ServerResponse.writeFile(options)	void	Server scripts	Writes a file to the response.
METHOD	ServerResponse.writeLine(options)	void	Server scripts	Writes line information (text/xml/html) to the response.
METHOD	ServerResponse.writePage(options)	void	Server scripts	Generates a page.
PROPERTY	ServerResponse.headers	Object (read-only)	Server scripts	The server response headers.

▶ Official NetSuite Script Samples (10 Sample(s))

📄 Sample 1: Generate a Secure Token and a Secret Key

```
/**
 * @NApiVersion 2.1
 */

// This script uses a GUID to generate a secure token and a secret key.
require(['N/https', 'N/runtime'], (https, runtime) =&gt; {
    function createSecureString() {
        const passwordGuid = '{284CFB2D225B1D76FB94D150207E49DF}';
        let secureToken = https.createSecureString({
            input: passwordGuid
        });
        let secretKey = https.createSecretKey({
            input: passwordGuid
        });
        secureToken = secureToken.hmac({
            algorithm: https.HashAlg.SHA256,
            key: secretKey
        });
    }
    createSecureString();
});
```

📄 Sample 2: Create a Form with a Credential Field

```
/**
 * @NApiVersion 2.1
 * @NScriptType Suitelet
 */

// This script creates a form with a credential field.
define(['N/ui/serverWidget', 'N/https', 'N/url'], (serverWidget, https, url) =&gt; {
    function onRequest(context) {
        if (context.request.method === 'GET') {
            const form = serverWidget.createForm({
             title: 'Password Form'
            });

            const credField = form.addCredentialField({
                id: 'password',
                label: 'Password',
                restrictToDomains: ['&lt;accountID&gt;.app.netsuite.com'],
                restrictToCurrentUser: false,
                restrictToScriptIds: 'customscript_my_script'
            });

            credField.maxLength = 32;

            form.addSubmitButton();

            context.response.writePage({
                pageObject: form
                });
        }
        else {
            // Request to an existing Suitelet with credentials
            let passwordGuid = context.request.parameters.password;

            // Replace SCRIPTID and DEPLOYMENTID with the internal ID of the suitelet script and deployment in your account
            let baseUrl = url.resolveScript({
                scriptId: SCRIPTID,
                deploymentId: DEPLOYMENTID,
                returnExternalURL: true
            });

            let authUrl = baseUrl + '&amp;pwd={' + passwordGuid + '}';

            let secureStringUrl = https.createSecureString({
                input: authUrl
            });

            let headers = ({
               'pwd': passwordGuid
            });

            let response = https.post({
                credentials: [passwordGuid],
                url: secureStringUrl,
                body: {authorization:' '+ passwordGuid + '', data:'anything can be here'},
                headers: headers
            });
        }
    }
    return {
        onRequest: onRequest
    };
});
```

📄 Sample 3: Create an Authentication Header Using a Secure String

```
/**
 * @NApiVersion 2.1
 * @NScriptType Suitelet
 */

// This script creates an authentication header using an https.SecureString.
define(['N/https', 'N/encode'], (https, encode) =&gt; {
    function onRequest(context) {

        // Secrets with these two Script IDs must be existing and allowed for this script
        const nameToken = "custsecret_myName";
        const passwordToken = "custsecret_mypPassword";

        // Create BASE-64 encoded name:password pair
        const secStringKeyInBase64 = https.createSecureString({
            input: "{" + nameToken + "}:{" + passwordToken + "}"
        });

        secStringKeyInBase64.convertEncoding({
            toEncoding: encode.Encoding.BASE_64,
            fromEncoding: encode.Encoding.UTF_8
        });

        // Construct the Authorization header
        const secStringBasicAuthHeader = https.createSecureString({
            input: "Basic "
        });

        secStringBasicAuthHeader.appendSecureString({
            secureString: secStringKeyInBase64,
            keepEncoding: true
        });

        // Send the request to third party with the Authorization header
        const resp = https.get({
            url: "myUrl",
            headers: {
                "Authorization": secStringBasicAuthHeader
            }
        });
    };
    return {
        onRequest: onRequest
    };
});
```

📄 Sample 4: Concatenate API Secrets with Strings

```
/**
 * @NApiVersion 2.1
 */

// This script uses appendSecureString to concatenate strings to use as an API secret.
require(['N/https', 'N/runtime'], (https, runtime) =&gt; {
    function concatToCreateSecureString() {
        let baseUrl = https.createSecureString({
            input: 'www.someurl.com/add?apikey='
         });
        let apiKey = https.createSecureString({
            input: '{CUSTSECRET_SOME_INTEGRATION}'
        });
        let url = baseUrl.appendSecureString({
            secureString: apiKey
        });
    }
    concatToCreateSecureString();
});
```

📄 Sample 5: Create a JWT Token Using a SecureString

```
/**
 * @NApiVersion 2.1
 * @NScriptType Suitelet
 */

// This script creates a JWT token using https.SecureString.
define(['N/https', 'N/encode'], (https, encode) =&gt; {
    function onRequest(context) {
        let nameToken = "custsecret_myName";
        let passwordToken = "custsecret_myPassword";
        let headerObj = {
            "alg": "HS256",
            "typ": "JWT"
        }
        let payloadObj = {
            "sub": "1234567890",
            "name": "John Doe",
            "iat": 1516239002
        }

        let headerJSON = JSON.stringify(headerObj);
        let payloadJSON = JSON.stringify(payloadObj);
        let headerBASE64 = encode.convert({
            string: headerJSON,
            inputEncoding: encode.Encoding.UTF_8,
            outputEncoding: encode.Encoding.BASE_64_URL_SAFE
        });

        let payloadBASE64 = encode.convert({
            string: payloadJSON,
            inputEncoding: encode.Encoding.UTF_8,
            outputEncoding: encode.Encoding.BASE_64_URL_SAFE
        });

        headerBASE64 = headerBASE64.replace(/=/g, ""); // remove = padding as per JWT spec 'base64UrlEncode' - URL-safe BASE-64 without padding
        payloadBASE64 = payloadBASE64.replace(/=/g, ""); // remove = padding as per JWT spec 'base64UrlEncode' - URL-safe BASE-64 without padding

        let secStringJwtSignature = https .createSecureString({
            input: headerBASE64 + "." + payloadBASE64
        })
        .hmac({
            algorithm: https.HashAlg.SHA256,
            key: https.createSecretKey({
                     secret: passwordToken,
                     encoding: encode.Encoding.UTF_8
            }),
           resultEncoding: encode.Encoding.BASE_64_URL_SAFE
        })
        .replaceString({ // remove = padding as per JWT spec 'base64UrlEncode' - URL-safe BASE-64 without padding
             pattern: "=",
             replacement: ""
        })

        let secStringJwtAuthHeader = https .createSecureString({
            input: "Bearer " + headerBASE64 + "." + payloadBASE64 + "."
        })
        .appendSecureString({
            secureString: secStringJwtSignature,
            keepEncoding: true
        })

        // Reflect the response using a echo-request suitelet
        let resp = https.get({
            url: "myURL",
            headers: {
                "Authorization": secStringJwtAuthHeader
            }
        });

        {
            log.debug("resp-code", resp.code);
            log.debug("resp-body", resp.body);

            let respAuth = JSON.parse(resp.body)["headers"]["Authorization"];

            log.debug("reps-head-auth", respAuth);
            log.debug("reps-head-auth-expected",
                "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.uel3RLILSJ9Q9W2Gomh8vAJQAgdb
nd6TS4b7plyFOtA" ); // see https://jwt.io/#debugger-io
        }
    }
    return {
        onRequest: onRequest
    };
});
```

📄 Sample 6: Retrieve Employee Information Using a Suitelet and a RESTlet Script with a Defined Content-Type Header

```
/**
 * @NApiVersion 2.1
 * @NScriptType Suitelet
 */
define(['N/https'],

    (https) =&gt; {
        /**
         * Defines the Suitelet script trigger point.
         * @param {Object} scriptContext
         * @param {ServerRequest} scriptContext.request - Incoming request
         * @param {ServerResponse} scriptContext.response - Suitelet response
         * @since 2015.2
         */
        const onRequest = (scriptContext) =&gt; {
            // Get the employee ID from the parameters
            const employeeId = scriptContext.request.parameters.employeeId || '0';

            // Define output variable
            let output = '';

            // Retrieve the employee name
            const restletResponse = https.requestRestlet({
                body: JSON.stringify({ employeeId }),
                deploymentId: 'customdeploy_samples_rs_requestrestlet',
                scriptId: 'customscript_samples_rs_requestrestlet',
                headers: { 'Content-Type': 'application/json' },
                method: 'POST'
            });

            // Check if the RESTlet call is successful
            if (restletResponse.code === 200) {
                if (restletResponse.body !== '') {
                    // Parse the RESTlet response
                    const restletBody = JSON.parse(restletResponse.body);

                    // Write the suitelet output
                    output = `Employee Name is: ${restletBody.name}`;
                } else {
                    // Write error message.
                    output = 'No employee found.';
                }

            } else {
                // Write error message.
                output = 'Unexpected error. Please check script logs.';
            }

            // Write output to response object.
            scriptContext.response.write(output);
        }

        return {onRequest}

    });
```

📄 Sample 7: Retrieve Employee Information Using a Suitelet and a RESTlet Script with a Defined Content-Type Header

```
/*
 * Copyright (c) 2022, Oracle and/or its affiliates.
 */

/**
 * @NApiVersion 2.1
 * @NScriptType Restlet
 */
define(['N/scriptTypes/restlet', 'N/query'], function (restlet, query) {
    // Restlet entry point
    const post = function (requestBody) {
        // Define the output variable
        let returnValue = '';

        // Create the Employee Query
        const qObj = query.create({
            type: query.Type.EMPLOYEE
        });

        // Check if the requestBody and the employeeId field is defined
        if (!!requestBody &amp;amp;&amp;amp; !!requestBody.employeeId) {
            // Define the query condition for employee ID field in Employee record
            qObj.condition = qObj.createCondition({
                fieldId: 'id',
                operator: query.Operator.EQUAL,
                values: requestBody.employeeId
            });

            // Add the desired columns in the Query
            qObj.columns = [
                qObj.createColumn({
                    fieldId: 'firstname'
                }),
                qObj.createColumn({
                    fieldId: 'lastname'
                })
            ];

            // Retrieve the results
            const results = qObj.run().asMappedResults();
            for (let i = 0; i &lt; results.length; i++) {
                const result = results[i];
                returnValue = {
                    name: result.firstname + ' ' + (result.lastname || '')
                };
            }
        }

        return returnValue;
    }

    return { post };
});
```

📄 Sample 8: Call a Suitelet from a Client Script

```
/**
 * @NApiVersion 2.1
 * @NScriptType UserEventScript
 */

// This script adds a button to a form and calls a custom function defined in a client script.
define(['N/ui/serverWidget'],

    function (serverWidget) {
        function beforeLoad(scriptContext) {

            // Add a button that calls the callSuitelet function
            scriptContext.form.addButton({
                id: 'custpage_call_suitelet_button',
                label: 'Click to Open Suitelet',
                functionName: 'callSuitelet'
            })

            // Specify the path to the client script where the callSuitelet function is defined
            scriptContext.form.clientScriptModulePath = './cs_request_suitelet.js' // Replace with the path to your client script
        }

        return {beforeLoad}
    });
```

📄 Sample 9: Call a Suitelet from a Client Script

```
/**
 * @NApiVersion 2.x
 * @NScriptType ClientScript
 * @NModuleScope SameAccount
 */

// This script defines a custom function to call a Suitelet.
define(['N/https', 'N/ui/dialog'],

    function(https, dialog) {
        function pageInit(scriptContext) {}

        // Create a custom function that calls a Suitelet and displays the response in a dialog
        function callSuitelet() {
            const response = https.requestSuitelet({
                scriptId: 'customscript_sl_plf_requestsuitelet', // Replace with your Suitelet script ID
                deploymentId: 'customdeploy_sl_plf_requestsuitelet', // Replace with your Suitelet deployment ID
            });
            dialog.alert({
                title: 'Suitelet Response',
                message: response.body
            });
        }

        return {
            pageInit: pageInit,
            callSuitelet: callSuitelet
        };

    });
```

📄 Sample 10: Call a Suitelet from a Client Script

```
/**
 * @NApiVersion 2.1
 * @NScriptType Suitelet
 */

// This Suitelet shows the text 'Hello World!' in plain text.
define([],

    function() {
        function onRequest(context) {
            context.response.setHeader({name: 'Content-Type', value: 'text/plain'});
            context.response.write('Hello World!');
        }

        return {onRequest};

    });
```

23. N/https/clientCertificate Module

Load the N/clientCertificate module to send SSL requests with a digital certificate.

`define(['N/https/clientCertificate'], (clientCertificate) ⇒ { ... })` ✓ 1 Code Sample(s) Scripts: Server scripts Permissions: Certificate Access

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/https/clientCertificate'], (clientCertificate) ⇒ { // Access methods and properties on clientCertificate });
```

Use the clientCertificate module to send SSL requests with a digital certificate.

N/https/clientCertificate Module Script Sample

N/https/clientCertificate Module Members

Return Type/Value Type

Supported Script Types

clientCertificate.post(options)

https.ClientResponse

Sends a SSL secured POST request to a remote server.

clientCertificate.get(options)

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	clientCertificate.post(options)	https.ClientResponse	Server scripts	Sends a SSL secured POST request to a remote server.
METHOD	clientCertificate.get(options)	https.ClientResponse	Server scripts	Sends a SSL secured GET request to a remote server.
METHOD	clientCertificate.put(options)	https.ClientResponse	Server scripts	Sends a SSL secured PUT request to a remote server.
METHOD	clientCertificate.delete(options)	https.ClientResponse	Server scripts	Sends a SSL secured DELETE request to a remote server.
METHOD	clientCertificate.request(options)	https.ClientResponse	Server scripts	Sends a SSL secured REQUEST request to a remote server.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: N/https/clientCertificate Module Script Sample

```
/**
 * @NApiVersion 2.1
 */

require(['N/https/clientCertificate'],(cert)=&gt; {
    // Set the URL
    const url = "https://nfe.fazenda.sp.gov.br/ws/cadconsultacadastro4.asmx";

    let data = "&lt;?xml version=\"1.0\" encoding=\"utf-8\"?&gt;&lt;soapenv:Envelope xmlns:soapenv=\"http://www.w3.org/2003/05/soap-envelope\"&gt;&lt;soapenv:Body&gt;&lt;ns1:nfeDadosMsg xmlns:ns1=\"http://www.portalfiscal.inf.br/nfe/wsdl/CadConsultaCadastro4\"&gt;&lt;ConsCad xmlns=\"http://www.portalfiscal.inf.br/nfe\" versao=\"2.00\"&gt;&lt;infCons&gt;&lt;xServ&gt;CONS-CAD&lt;/xServ&gt;&lt;UF&gt;SP&lt;/UF&gt;&lt;CNPJ&gt;47508411000156&lt;/CNPJ&gt;&lt;/infCons&gt; &lt;/ConsCad&gt;&lt;/ns1:nfeDadosMsg&gt;&lt;/soapenv:Body&gt;&lt;/soapenv:Envelope&gt;";

    const key = "custcertificate1";

    let headers = {
        "Content-Type": "application/soap+xml"
         };

    let response = cert.post({
        url: url,
        certId: key,
        body: data,
        headers: headers
    });
})
```

24. N/keyControl Module

Load the N/keyControl module to access key storage.

`define(['N/keyControl'], (keyControl) => { ... })` ✓ 2 Code Sample(s) Scripts: Server scripts Permissions: Key Management

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/keyControl'], (keyControl) => { // Access methods and properties on keyControl });
```

Use the N/keyControl module to use SSH keys and access key storage. You can also access keys in the UI at Setup > Company > Preferences > Keys.

By using the SSH keys, you can manage files and directories by using the SSH file transfer (SFTP) protocol. For more information, see SSH Keys for SFTP. For more information about SFTP, see N/sftp Module.

N/keyControl Module Members

Return Type / Value Type

Supported Script Types

Represents the key object.

keyControl.findKeys(options)

Searches and returns a list of keys based on criteria set. If no options are set for criteria, the full list of keys stored in NetSuite is returned.

keyControl.createKey(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	`keyControl.Key`	`Object`	Server scripts	Represents the key object.
METHOD	`keyControl.findKeys(options)`	`Object`	Server scripts	Searches and returns a list of keys based on criteria set. If no options are set for criteria, the full list of keys stored in NetSuite is returned.
METHOD	`keyControl.createKey(options)`	`keyControl.Key`	Server scripts	Creates a key.
METHOD	`keyControl.deleteKey(options)`	`Object`	Server scripts	Marks the key as deleted in database. The history is retained.
METHOD	`keyControl.loadKey(options)`	`keyControl.Key`	Server scripts	Loads a key.
METHOD	`keyControl.lock(options)`	`string`	Server scripts	Locks a key so that it cannot be edited in the UI.
METHOD	`keyControl.unlock(options)`	`string`	Server scripts	Unlocks a key that has been locked by keyControl.lock(options).
ENUM	`keyControl.Operator`	`enum`	Server scripts	Holds the values for the key operators of keyControl.findKeys(options).

Key Object Members

The following members are called on the keyControl.Key object.

Return Type / Value Type

Supported Script Types

File object of the key.

Password of the key (write-only).

You can create a GUID using Form.addSecretKeyField(options). This property also accepts the script ID of an API secret stored at Setup > Company > API Secrets.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	`Key.file`	`file.File`	Server scripts	File object of the key.
PROPERTY	`Key.password`	`string`	Server scripts	Password of the key (write-only). You can create a GUID using Form.addSecretKeyField(options). This property also accepts the script ID of an API secret stored at Setup > Company > API Secrets.
PROPERTY	`Key.scriptId`	`string`	Server scripts	Script ID of the key. NetSuite prepends this ID with custkey.
PROPERTY	`Key.name`	`string`	Server scripts	Name of the key.
PROPERTY	`Key.description`	`string`	Server scripts	Description of the key.
PROPERTY	`Key.restrictions`	`string`	Server scripts	The internal IDs of the employees selected in the Restrict to Employees field of the key record.
METHOD	`Key.save()`	`Object`	Server scripts	Saves the key.

▶ Official NetSuite Script Samples (2 Sample(s))

📄 Sample 1: Create a Secret Key

```
/**
 * @NApiVersion 2.x
 */

require(['N/keyControl','N/file'],function(keyControl,file){
            var key = keyControl.createKey();
            key.file = file.load(422);
         //id of file containing private key (id_ecdsa or id_rsa)
            key.name = "SFTP key";
            key.save();
})
```

📄 Sample 2: Add a Secret Key Field to a Form

```
/**
 * @NApiVersion 2.1
 * @NScriptType Suitelet
 */
define(['N/ui/serverWidget', 'N/file', 'N/keyControl', 'N/runtime'], function(serverWidget, file, keyControl, runtime) {
    function onRequest(context) {
        var request = context.request;
        var response = context.response;

        if (request.method === 'GET') {
            var form = serverWidget.createForm({
                title: 'Enter Password'
            });

            var credField = form.addSecretKeyField({
                id: 'custfield_password',
                label: 'Password',
                restrictToScriptIds: [runtime.getCurrentScript().id],
                restrictToCurrentUser: true //Depends on use case
            });
            credField.maxLength = 64;

            form.addSubmitButton();
            response.writePage(form);
        } else {
            // Read the request parameter matching the field ID we specified in the form
            var passwordToken = request.parameters.custfield_password;

            var pem = file.load({
                id: 422
            });

            var key = keyControl.createKey();
            key.file = pem;
            key.name = 'Test';
            key.password = passwordToken;
            key.save();
        }
    }
    return {
        onRequest: onRequest
    };
});
```

25. N/llm Module

Load the N/llm module to use generative artificial intelligence (AI) capabilities. You can use this module to send requests to the large language models (LLMs) supported by NetSuite and to receive LLM responses to use in your scripts.

`define(['N/llm'], (llm) => { ... })` ✓ 9 Code Sample(s) Scripts: Server scripts SuiteScript 2.1 only Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/llm'], (llm) => { // Access methods and properties on llm });
```

The content in this help topic pertains to SuiteScript 2.1.

The N/llm module supports generative artificial intelligence (AI) capabilities in SuiteScript. You can use this module to send requests to the large language models (LLMs) supported by NetSuite and receive responses to use in your scripts.

Methods in this module that send requests to an LLM consume AI Units. For more information, see NetSuite AI Units and NetSuite Features and AI Units FAQ.

If you're new to using generative AI in SuiteScript, see SuiteScript 2.1 Generative AI APIs. That topic contains essential information about this feature.

The following list summarizes the main features that are available using the N/llm module:

Content generation - You can request generative AI content from a supported LLM using llm.generateText(options). You can provide a prompt that describes the content you want to generate, and the module sends the request to the Oracle Cloud Infrastructure (OCI) Generative AI service to generate a response.

Prompt evaluation - If you use Prompt Studio to manage existing prompts in your NetSuite account, you can use llm.evaluatePrompt(options) to send a prompt from Prompt Studio to the LLM for evaluation. This method uses the information from the prompt definition in Prompt Studio (such as the model and model parameters), and it lets you provide values for any variables the prompt uses before sending it for evaluation. For more information about Prompt Studio, see Prompt Studio.

Prompt and Text Enhance action management - By using the N/record module, you can create, update, and delete prompts and Text Enhance actions in your scripts. For more information, see Managing Prompts and Text Enhance Actions Using the N/llm Module.

Retrieval-augmented generation (RAG) support - You can give source documents to the LLM when calling llm.generateText(options). The LLM uses information from the source documents to augment its response. The LLM also returns citations that identify which source documents it used. For an example of how to implement a RAG use case using the N/llm module, see Provide Source Documents When Calling the LLM.

Image support - When using the Cohere Command A Vision model, you can send an image to the LLM when calling llm.generateText(options) or llm.generateTextStreamed(options). You can ask questions about the image and receive information such as advanced image captions, a detailed description of the image, or information about charts and graphs in the image.

Embedding support - The llm.embed(options) method converts text to vector embeddings. Your SuiteScript applications can use vector embeddings for use cases such as semantic searches, recommender systems, text classification, or text clustering. For an example of how to generate and use embeddings, see Find Similar Items Using Embeddings. For more information about embedding models, refer to Offered Pretrained Foundational Models in Generative AI in the Oracle Cloud Infrastructure Documentation.

Embeddings include the semantic information of the original data and should be treated with the same level of sensitivity. Embeddings created from sensitive data must be protected, stored, logged, shared, retained, and deleted according to the same rules as the original data.

Streaming support - Using the llm.generateTextStreamed(options) and llm.evaluatePromptStreamed(options) methods, your code gets content as the LLM generates it, instead of waiting to receive it all at the same time. For an example of how to work with streamed content, see Receive a Partial Response from the LLM.

Tooling support - You can define custom tools using llm.createTool(options) that let you extend and enhance LLM interactions. Tools can run business-specific logic in your scripts (such as SuiteQL queries), receive inputs from the LLM, and return structured outputs. The LLM can request tool calls as needed and integrate tool results into its generated responses, which can provide richer and more dynamic responses.

For more information, see Tooling in the N/llm Module. For an example of how to create tools and provide them to the LLM, see Use Tools to Enhance the Response from the LLM.

Method aliases - You can use the following aliases in your code instead of the method names:

llm.chat(options) is an alias for llm.generateText(options).

llm.executePrompt(options) is an alias for llm.evaluatePrompt(options).

llm.chatStreamed(options) is an alias for llm.generateTextStreamed(options).

llm.executePromptStreamed(options) is an alias for llm.evaluatePromptStreamed(options).

Promise versions are also available for these methods.

When aliases are available for methods, you'll see them listed in the main table of the method's help topic. For an example, see llm.generateTextStreamed(options).

This module is available in NetSuite by default when the Server SuiteScript feature is enabled. For more information, see Enabling Features.

As you work with this module, keep the following considerations in mind:

Generative AI features, such as the N/llm module, use creativity in their responses. Make sure you validate the AI-generated responses for accuracy and quality. Oracle NetSuite isn't responsible or liable for the use or interpretation of AI-generated content.

N/llm Module Members

Return Type / Value Type

Supported Script Types

The chat message.

A citation returned from the LLM.

A document to be used as source content when calling the LLM.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	**llm.ChatMessage**	Object	Server scripts	The chat message.
OBJECT	**llm.Citation**	Object	Server scripts	A citation returned from the LLM.
OBJECT	**llm.Document**	Object	Server scripts	A document to be used as source content when calling the LLM.
OBJECT	**llm.EmbedResponse**	Object	Server scripts	The embeddings response returned from the LLM.
OBJECT	**llm.Response**	Object	Server scripts	The response returned from the LLM.
OBJECT	**llm.StreamedResponse**	Object	Server scripts	The streamed response returned from the LLM.
OBJECT	**llm.Tool**	Object	Server scripts	A tool the LLM can request.
OBJECT	**llm.ToolCall**	Object	Server scripts	A tool call request from the LLM.
OBJECT	**llm.ToolParameter**	Object	Server scripts	A parameter for a tool.
OBJECT	**llm.ToolResult**	Object	Server scripts	A tool result to send back to the LLM.
OBJECT	**llm.Usage**	Object	Server scripts	The token usage for a request to the LLM.
METHOD	**llm.createChatMessage(options)**	llm.ChatMessage	Server scripts	Creates a chat message based on a specified role and text.
METHOD	**llm.createDocument(options)**	llm.Document	Server scripts	Creates a document to be used as source content when calling the LLM.
METHOD	**llm.createTool(options)**	llm.Tool	Server scripts	Creates a tool the LLM can request.
METHOD	**llm.createToolParameter(options)**	llm.ToolParameter	Server scripts	Creates a tool parameter for a tool.
METHOD	**llm.createToolResult(options)**	llm.ToolResult	Server scripts	Creates a tool result to send back to the LLM.
METHOD	**llm.embed(options)**	llm.EmbedResponse	Server scripts	Returns the embeddings from the LLM for a given input.
METHOD	**llm.embed.promise(options)**	Promise	Server scripts	Asynchronously returns the embeddings from the LLM for a given input.
METHOD	**llm.evaluatePrompt(options)**	llm.Response	Server scripts	Takes the ID of an existing prompt and values for variables used in the prompt and returns the response from the LLM.
METHOD	**llm.evaluatePrompt.promise(options)**	Promise	Server scripts	Takes the ID of an existing prompt and values for variables used in the prompt and asynchronously returns the response from the LLM.
METHOD	**llm.evaluatePromptStreamed(options)**	llm.StreamedResponse	Server scripts	Takes the ID of an existing prompt and values for variables used in the prompt and returns the streamed response from the LLM.
METHOD	**llm.evaluatePromptStreamed.promise(options)**	Promise	Server scripts	Takes the ID of an existing prompt and values for variables used in the prompt and asynchronously returns the streamed response from the LLM.
METHOD	**llm.generateText(options)**	llm.Response	Server scripts	Takes a prompt and parameters for the LLM and returns the response from the LLM.
METHOD	**llm.generateText.promise(options)**	Promise	Server scripts	Takes a prompt and parameters for the LLM and asynchronously returns the response from the LLM.
METHOD	**llm.generateTextStreamed(options)**	llm.StreamedResponse	Server scripts	Takes a prompt and parameters for the LLM and returns the streamed response from the LLM.
METHOD	**llm.generateTextStreamed.promise(options)**	Promise	Server scripts	Takes a prompt and parameters for the LLM and asynchronously returns the streamed response from the LLM.
METHOD	**llm.getRemainingFreeUsage()**	number	Server scripts	Deprecated. Calls llm.getRemainingUsage().
METHOD	**llm.getRemainingFreeUsage.promise()**	Promise	Server scripts	Deprecated. Calls llm.getRemainingUsage.promise().
METHOD	**llm.getRemainingFreeEmbedUsage()**	number	Server scripts	Deprecated. Calls llm.getRemainingUsage(). Embed usage is no longer tracked separately.
METHOD	**llm.getRemainingFreeEmbedUsage.promise()**	Promise	Server scripts	Deprecated. Calls llm.getRemainingUsage.promise(). Embed usage is no longer tracked separately.
METHOD	**llm.getRemainingUsage()**	number	Server scripts	Returns the number of AI Units remaining.
METHOD	**llm.getRemainingUsage.promise()**	Promise	Server scripts	Asynchronously returns the number of AI Units remaining.
ENUM	**llm.ChatRole**	enum	Server scripts	Holds the string values for the author (role) of a chat message. Use this enum to set the value of the options.role parameter in llm.createChatMessage(options).
ENUM	**llm.EmbedModelFamily**	enum	Server scripts	Holds the string values for the large language model to be used to generate embeddings. Use this enum to set the value of the options.embedModelFamily parameter in llm.embed(options).
ENUM	**llm.ModelFamily**	enum	Server scripts	Holds the string values for the large language model to be used. Use this enum to set the value of the options.model parameter in llm.generateText(options).

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
ENUM	`llm.SafetyMode`	`enum`	Server scripts	Holds the string values for the safety mode to be used for LLM requests. Use this enum to set the value of the options.safetyMode parameter in llm.generateText(options) and llm.generateTextStreamed(options).
ENUM	`llm.ToolParameterType`	`enum`	Server scripts	Holds the data type for a tool parameter. Use this enum to set the value of the options.type parameter in llm.createToolParameter(options).
ENUM	`llm.Truncate`	`enum`	Server scripts	Holds the string values for the truncation method to use when embeddings input exceeds 512 tokens. Use this enum to set the value of the options.truncate parameter in llm.embed(options).

ChatMessage Object Members

Return Type / Value Type

Supported Script Types

ChatMessage.role

The author (role) of the chat message.

ChatMessage.text

Text of the chat message.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	`ChatMessage.role`	`string`	Server scripts	The author (role) of the chat message.
PROPERTY	`ChatMessage.text`	`string`	Server scripts	Text of the chat message. This text can be either the prompt sent by the script or the response returned by the LLM.

Citation Object Members

Return Type / Value Type

Supported Script Types

Citation.documentIds

The IDs of the documents where the cited text is located.

The ending position of the cited text.

The starting position of the cited text.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	`Citation.documentIds`	`string[]`	Server scripts	The IDs of the documents where the cited text is located.
PROPERTY	`Citation.end`	`number`	Server scripts	The ending position of the cited text.
PROPERTY	`Citation.start`	`number`	Server scripts	The starting position of the cited text.
PROPERTY	`Citation.text`	`string`	Server scripts	The cited text from the documents.

Document Object Members

Return Type / Value Type

Supported Script Types

The content of the document.

The ID of the document.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	`Document.data`	`string`	Server scripts	The content of the document.
PROPERTY	`Document.id`	`string`	Server scripts	The ID of the document.

EmbedResponse Object Members

Return Type / Value Type

Supported Script Types

EmbedResponse.embeddings

The embeddings returned from the LLM.

EmbedResponse.inputs

The list of inputs used to generate the embeddings response.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	EmbedResponse.embeddings	number[]	Server scripts	The embeddings returned from the LLM.
PROPERTY	EmbedResponse.inputs	string[]	Server scripts	The list of inputs used to generate the embeddings response.
PROPERTY	EmbedResponse.model	string	Server scripts	The model used to generate the embeddings response.

Response Object Members

Return Type / Value Type

Supported Script Types

Response.chatHistory

llm.ChatMessage[]

List of chat messages.

Response.citations

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Response.chatHistory	llm.ChatMessage[]	Server scripts	List of chat messages.
PROPERTY	Response.citations	llm.Citation[]	Server scripts	List of citations used to generate the response.
PROPERTY	Response.documents	llm.Document[]	Server scripts	List of documents used to generate the response.
PROPERTY	Response.model	string	Server scripts	Model used to produce the LLM response.
PROPERTY	Response.text	string	Server scripts	Text returned by the LLM.
PROPERTY	Response.toolCalls	llm.ToolCall[]	Server scripts	Tool calls requested by the LLM.
PROPERTY	Response.usage	llm.Usage	Server scripts	Token usage for a request to the LLM.

StreamedResponse Object Members

Return Type / Value Type

Supported Script Types

StreamedResponse.chatHistory

llm.ChatMessage[]

List of chat messages.

StreamedResponse.citations

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	StreamedResponse.chatHistory	llm.ChatMessage[]	Server scripts	List of chat messages.
PROPERTY	StreamedResponse.citations	llm.Citation[]	Server scripts	List of citations used to generate the streamed response.
PROPERTY	StreamedResponse.documents	llm.Document[]	Server scripts	List of documents used to generate the streamed response.
PROPERTY	StreamedResponse.model	string	Server scripts	Model used to produce the streamed response.
PROPERTY	StreamedResponse.text	string	Server scripts	Text returned by the LLM.
PROPERTY	StreamedResponse.toolCalls	llm.ToolCall[]	Server scripts	Tool calls requested by the LLM.

Tool Object Members

Return Type / Value Type

Supported Script Types

Tool.description

The description of the tool.

The name of the tool.

llm.ToolParameter[]

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Tool.description	string	Server scripts	The description of the tool.
PROPERTY	Tool.name	string	Server scripts	The name of the tool.
PROPERTY	Tool.parameters	llm.ToolParameter[]	Server scripts	The parameters of the tool.

ToolCall Object Members

Return Type / Value Type

Supported Script Types

The name of the requested tool.

ToolCall.parameters

llm.ToolParameter[]

The parameters of the requested tool.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**ToolCall.name**	`string`	Server scripts	The name of the requested tool.
PROPERTY	**ToolCall.parameters**	`llm.ToolParameter[]`	Server scripts	The parameters of the requested tool.

ToolParameter Object Members

Return Type / Value Type

Supported Script Types

ToolParameter.description

The description of the tool parameter.

ToolParameter.name

The name of the tool parameter.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**ToolParameter.description**	`string`	Server scripts	The description of the tool parameter.
PROPERTY	**ToolParameter.name**	`string`	Server scripts	The name of the tool parameter.
PROPERTY	**ToolParameter.type**	`string`	Server scripts	The type of the tool parameter.

ToolResult Object Members

Return Type / Value Type

Supported Script Types

The originating tool call request from the LLM.

ToolResult.outputs

The outputs from running the tool specified in the tool call request.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**ToolResult.call**	`llm.ToolCall`	Server scripts	The originating tool call request from the LLM.
PROPERTY	**ToolResult.outputs**	`Object[]`	Server scripts	The outputs from running the tool specified in the tool call request.

Usage Object Members

Return Type / Value Type

Supported Script Types

Usage.completionTokens

The number of tokens in the response from the LLM.

Usage.promptTokens

The number of tokens in the request to the LLM.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Usage.completionTokens**	`number`	Server scripts	The number of tokens in the response from the LLM.
PROPERTY	**Usage.promptTokens**	`number`	Server scripts	The number of tokens in the request to the LLM.
PROPERTY	**Usage.totalTokens**	`number`	Server scripts	The total number of tokens for the entire request to the LLM.

▶ Official NetSuite Script Samples (9 Sample(s))

📄 Sample 1: Send a Prompt to the LLM and Receive a Response

```
/**
*@NApiVersion 2.1
*/
// This example shows how to query the default LLM
require(['N/llm'],
    function(llm) {
        const response = llm.generateText({
            // modelFamily is optional. When omitted, the Cohere Command A model is used.
            prompt: "Hello World!",
            modelParameters: {
                maxTokens: 1000,
                temperature: 0.2,
                topK: 3,
                topP: 0.7,
                frequencyPenalty: 0.4,
                presencePenalty: 0
            }
        });
        const responseText = response.text;
        const remainingUsage = llm.getRemainingUsage(); // View remaining AI Units
    });
```

📄 Sample 2: Clean Up Content for Text Area Fields After Saving a Record

```
/**
 * @NApiVersion 2.1
 * @NScriptType UserEventScript
 */
define(['N/llm'], (llm) =&gt; {
    /**
     * @param {Object} scriptContext The updated inventory item
     * record to clean up typo errors for purchase description and
     * sales description fields. The values are set before the record
     * is submitted to be saved.
     */
    function fixTypos(scriptContext) {
        const purchaseDescription = scriptContext.newRecord.getValue({
            fieldId: 'purchasedescription'
        })
        const salesDescription = scriptContext.newRecord.getValue({
            fieldId: 'salesdescription'
        })

        const p1 = llm.generateText.promise({
            prompt: `Please clean up typos in the following text:
                     ${purchaseDescription} and return just the corrected text.
                     Return the text as is if there's no typo
                     or you don't understand the text.`
        })
        const p2 = llm.generateText.promise({
            prompt: `Please clean up typos in the following text:
                     ${salesDescription} and return just the corrected text.
                     Return the text as is if there's no typo
                     or you don't understand the text.`
        })

        // When both promises are resolved, set the updated values for the
        // record
        Promise.all([p1, p2]).then((results) =&gt; {
            scriptContext.newRecord.setValue({
                fieldId: 'purchasedescription',
                value: results[0].value.text
            })
            scriptContext.newRecord.setValue({
                fieldId: 'salesdescription',
                value: results[1].value.text
            })
        })
    }

    return { beforeSubmit: fixTypos }
})
```

📄 Sample 3: Provide an LLM-based ChatBot for NetSuite Users

```
/**
 * @NApiVersion 2.1
 * @NScriptType Suitelet
 */

define(['N/ui/serverWidget', 'N/llm'], (serverWidget, llm) =&gt; {
  /**
   * Creates NetSuite form to communicate with LLM
   */
  function onRequest (context) {
    const form = serverWidget.createForm({
      title: 'Chat Bot'
    })
    const fieldgroup = form.addFieldGroup({
      id: 'fieldgroupid',
      label: 'Chat'
    })
    fieldgroup.isSingleColumn = true
    const historySize = parseInt(
      context.request.parameters.custpage_num_chats || '0')
    const numChats = form.addField({
      id: 'custpage_num_chats',
      type: serverWidget.FieldType.INTEGER,
      container: 'fieldgroupid',
      label: 'History Size'
    })
    numChats.updateDisplayType({
      displayType: serverWidget.FieldDisplayType.HIDDEN
    })

    if (context.request.method === 'POST') {
      numChats.defaultValue = historySize + 2
      const chatHistory = []
      for (let i = historySize - 2; i &gt;= 0; i -= 2) {
        const you = form.addField({
          id: 'custpage_hist' + (i + 2),
          type: serverWidget.FieldType.TEXTAREA,
          label: 'You',
          container: 'fieldgroupid'
        })
        const yourMessage = context.request.parameters['custpage_hist' + i]
        you.defaultValue = yourMessage
        you.updateDisplayType({
          displayType: serverWidget.FieldDisplayType.INLINE
        })

        const chatbot = form.addField({
          id: 'custpage_hist' + (i + 3),
          type: serverWidget.FieldType.TEXTAREA,
          label: 'ChatBot',
          container: 'fieldgroupid'
        })
        const chatBotMessage =
          context.request.parameters['custpage_hist' + (i + 1)]
        chatbot.defaultValue = chatBotMessage
        chatbot.updateDisplayType({
          displayType: serverWidget.FieldDisplayType.INLINE
        })
        chatHistory.push({
          role: llm.ChatRole.USER,
          text: yourMessage
        })
        chatHistory.push({
          role: llm.ChatRole.CHATBOT,
          text: chatBotMessage
        })
      }

      const prompt = context.request.parameters.custpage_text
      const promptField = form.addField({
        id: 'custpage_hist0',
        type: serverWidget.FieldType.TEXTAREA,
        label: 'You',
        container: 'fieldgroupid'
      })
      promptField.defaultValue = prompt
      promptField.updateDisplayType({
        displayType: serverWidget.FieldDisplayType.INLINE
      })
      const result = form.addField({
        id: 'custpage_hist1',
        type: serverWidget.FieldType.TEXTAREA,
        label: 'ChatBot',
        container: 'fieldgroupid'
      })
      result.defaultValue = llm.generateText({
        prompt: prompt,
        chatHistory: chatHistory
      }).text
      result.updateDisplayType({
        displayType: serverWidget.FieldDisplayType.INLINE
```

```
      })
    } else {
      numChats.defaultValue = 0
    }

    form.addField({
      id: 'custpage_text',
      type: serverWidget.FieldType.TEXTAREA,
      label: 'Prompt',
      container: 'fieldgroupid'
    })

    form.addSubmitButton({
      label: 'Submit'
    })

    context.response.writePage(form)
  }

  return {
    onRequest: onRequest
  }
})
```

📄 Sample 4: Evaluate an Existing Prompt and Receive a Response

```
/**
* @NApiVersion 2.1
*/
require(['N/llm'],
    function(llm) {
        const response = llm.evaluatePrompt({
            id: 'stdprompt_gen_purch_desc_invt_item',
            variables: {
                "form": {
                    "itemid": "My Inventory Item",
                    "stockdescription": "This is the stock description of the item.",
                    "vendorname": "My Item Vendor Inc.",
                    "isdropshipitem": "false",
                    "isspecialorderitem": "true",
                    "displayname": "My Amazing Inventory Item"
                },
                "text": "This is the purchase description of the item."
            }
        });
        const responseText = response.text;
        const remainingUsage = llm.getRemainingUsage(); // View remaining AI Units
    });
```

📄 Sample 5: Create a Prompt and Evaluate It

```
/**
* @NApiVersion 2.1
*/
require(['N/record', 'N/llm'], function(record, llm) {
    const rec = record.create({
        type: "prompt"
    });

    rec.setValue({
        fieldId: "name",
        value: "Test"
    });
    rec.setValue({
        fieldId: "prompttype",
        value: "GENERIC"
    });
    rec.setValue({
        fieldId: "modelfamily",
        value: "COHERE_COMMAND"
    });
    rec.setValue({
        fieldId: "template",
        value: "${mandatoryVariable} &lt;#if optionalVariable?has_content&gt;${optionalVariable}&lt;#else&gt;World&lt;/#if&gt;"
    });

    const id = rec.save();

    try {
        llm.evaluatePrompt({
            id: id
        });
    }
    catch (e) {
        if (e.name === "TEMPLATE_PROCESSING_EXCEPTION")
            log.debug("Exception", "Expected exception was thrown");
    }

    const response = llm.evaluatePrompt({
        id: id,
        variables: {
            mandatoryVariable: "Hello",
            optionalVariable: "People"
        }
    });
    if ("Hello People" === response.chatHistory[0].text)
        log.debug("Evaluation", "Correct prompt got evaluated");

    llm.evaluatePrompt.promise({
        id: id,
        variables: {
            mandatoryVariable: "Hello",
            optionalVariable: "World"
        }
    }).then(function(response) {
        if ("Hello World" === response.chatHistory[0].text)
            log.debug("Evaluation", "Correct prompt got evaluated");
        record.delete({
            type: "prompt",
            id: id
        });
        debugger;
    })
});
```

📄 Sample 6: Provide Source Documents When Calling the LLM

```
/**
 * @NApiVersion 2.1
 */
require(['N/llm'], function(llm) {
    const doc1 = llm.createDocument({
        id: "doc1",
        data: "Emperor penguins are the tallest."
    });
    const doc2 = llm.createDocument({
        id: "doc2",
        data: "Emperor penguins only live in the Sahara desert."
    });

    llm.generateText({
        prompt: "Where do the tallest penguins live?",
        documents: [doc1, doc2],
        modelFamily: llm.ModelFamily.COHERE_COMMAND,
        modelParameters: {
            maxTokens: 1000,
            temperature: 0.2,
            topK: 3,
            topP: 0.7,
            frequencyPenalty: 0.4,
            presencePenalty: 0
        }
    });
});
```

📄 Sample 7: Receive a Partial Response from the LLM

```
/**
 * @NApiVersion 2.1
 */
require(['N/llm'], function(llm) {
    const response = llm.generateTextStreamed({
       preamble: "You are a script writer for TV shows.",
       prompt: "Write a 300 word pitch for a TV show about tigers.",
       modelFamily: llm.ModelFamily.COHERE_COMMAND,
       modelParameters: {
          maxTokens: 1000,
          temperature: 0.8,          // High temperature values result in more varied and
                                     // creative responses
          topK: 3,
          topP: 0.7,
          frequencyPenalty: 0.4,
          presencePenalty: 0
       }
    });

    var iter = response.iterator();
    iter.each(function(token){
        log.debug("token.value: " + token.value);
        log.debug("response.text: " + response.text);
        return true;
    })
});
```

📄 Sample 8: Find Similar Items Using Embeddings

```
/**
 * @NApiVersion 2.1
 * @NScriptType Suitelet
 * @NModuleScope SameAccount
 */
define(['N/ui/serverWidget','N/query', 'N/llm'],
    function(serverWidget, query, llm) {

        function cosineSimilarity(array1, array2) {
            const dotProduct = array1.reduce((sum, value, index) =&gt; sum + value
                                        * array2[index], 0);
            const magnitude1 = Math.sqrt(array1.reduce((sum, value) =&gt; sum +
                                    value * value, 0));
            const magnitude2 = Math.sqrt(array2.reduce((sum, value) =&gt; sum +
                                    value * value, 0));
            return dotProduct / (magnitude1 * magnitude2);
        }

        function onRequest(context) {
            if (context.request.method === 'GET') {
                var form = serverWidget.createForm({
                    title: 'Item Similarity Form'
                });

                var selectField = form.addField({
                    id: 'custpage_myselect',
                    type: serverWidget.FieldType.SELECT,
                    label: 'Select an Item to find similar items for'
                });

                var res = query.runSuiteQL("SELECT id, case when displayname
                is not null then displayname else itemid end name from item
                where rownum &lt;= 96 order by id").asMappedResults();

                for (let i = 0; i &lt; res.length; i++)
                {
                    selectField.addSelectOption({
                        value: res[i]['id'],
                        text: res[i]['name']
                    });
                }

                form.addSubmitButton({
                    label: 'Submit'
                });

                context.response.writePage(form);
            } else {
                var selectedValue = context.request.parameters.custpage_myselect;
                var res = query.runSuiteQL("SELECT id, case when displayname
                is not null then displayname else itemid end name from item
                where rownum &lt;= 96 order by id").asMappedResults();

                const inputs = [];
                let selectedName;
                let selectedIndex = 0;

                for (let i = 0; i &lt; res.length; i++)
                {
                    if (res[i]['id'] == selectedValue)
                    {
                        selectedName = res[i]["name"];
                        selectedIndex = i;
                    }
                    inputs.push(res[i]["name"]);
                }

                var embeddingResult = llm.embed({
                    embedModelFamily: llm.EmbedModelFamily.COHERE_EMBED,
                    inputs: inputs
                });

                var item = embeddingResult.embeddings[selectedIndex];
                var similarityResults = [];

                for (var j = 0; j &lt; embeddingResult.inputs.length; j++)
                    similarityResults.push({
                        itemName: embeddingResult.inputs[j],
                        similarity: cosineSimilarity(item, embeddingResult.embeddings[j])
                    });

                similarityResults.sort((a,b) =&gt; b.similarity - a.similarity);
                context.response.write(JSON.stringify(similarityResults, null, 2));
            }
        }

        return {
            onRequest: onRequest
        };
```

```
    }
);
```

📄 **Sample 9: Use Tools to Enhance the Response from the LLM**

```
/**
 * @NApiVersion 2.1
 * @NScriptType Suitelet
 */

define(['N/ui/serverWidget', 'N/llm', 'N/query'], (serverWidget, llm, query) =&gt; {
    const TOOL_HANDLERS = {
      getNumberOfCustomerPurchasesForItem: (options) =&gt; {
        const res = query.runSuiteQL({
          query: "SELECT count(*) cnt from transactionline where entity=? and item=?",
          params: [options.userId, options.itemId]
        }).asMappedResults();
        return res[0]["cnt"];
      },

      findItemId: (options) =&gt; {
        const res = query.runSuiteQL({
          query: "SELECT id from item where itemid=?",
          params: [options.itemName]
        }).asMappedResults();
        return res.length === 0 ? -1 : res[0]["id"];
      },

      findUserId: (options) =&gt; {
        const res = query.runSuiteQL({
          query: "SELECT id from entity where fullname=?",
          params: [options.userName]
        }).asMappedResults();
        return res.length === 0 ? -1 : res[0]["id"];
      }
    };

    const TOOL_DEFINITIONS = {
        'findUserId': llm.createTool({
            name: "findUserId",
            description: "Looks up user id based on his/her name",
            parameters: [
                llm.createToolParameter({ name: "userName", description: "name of the user", type: "STRING" })
            ]
        }),
        'findItemId': llm.createTool({
            name: "findItemId",
            description: "Looks up item id based on its name",
            parameters: [
                llm.createToolParameter({ name: "itemName", description: "name of the item", type: "STRING" })
            ]
        }),
        'getNumberOfCustomerPurchasesForItem': llm.createTool({
            name: "getNumberOfCustomerPurchasesForItem",
            description: "Looks up how many times did user purchase given item, both user and item are specified by id.",
            parameters: [
                llm.createToolParameter({ name: "userId", description: "id of the user", type: "INTEGER" }),
                llm.createToolParameter({ name: "itemId", description: "id of the item", type: "INTEGER" })
            ]
        })
    }

    const createFromHeader = (context) =&gt; {
        const form = serverWidget.createForm({ title: 'Tool Use Example' });

        const fieldgroup = form.addFieldGroup({
            id: 'fieldgroupid',
            label: 'Chat'
        });
        fieldgroup.isSingleColumn = true;

        const history = form.addField({
            id: 'custpage_history',
            type: serverWidget.FieldType.LONGTEXT,
            container: 'fieldgroupid',
            label: "History"
        });
        history.updateDisplayType({ displayType: serverWidget.FieldDisplayType.HIDDEN });

        const numChats = form.addField({
            id: 'custpage_num_chats',
            type: serverWidget.FieldType.INTEGER,
            container: 'fieldgroupid',
            label: "History Size"
        });
        numChats.updateDisplayType({ displayType: serverWidget.FieldDisplayType.HIDDEN });
        return form;
    }

    const displayHistory = (context, form) =&gt; {
        const historySize = parseInt(context.request.parameters["custpage_num_chats"] || "0", 10);
        form.getField({id: 'custpage_num_chats'}).defaultValue = String(historySize + 2);
        //numChats.defaultValue = String(historySize + 2);

        // Recreate the chat history fields (in reverse order)
        for (let i = historySize - 2; i &gt;= 0; i -= 2) {
```

```
            const you = form.addField({
                id: 'custpage_hist' + (i + 2),
                type: serverWidget.FieldType.TEXTAREA,
                label: 'You',
                container: 'fieldgroupid'
            });
            const yourMessage = context.request.parameters["custpage_hist" + i];
            you.defaultValue = yourMessage;
            you.updateDisplayType({ displayType: serverWidget.FieldDisplayType.INLINE });

            const chatbot = form.addField({
                id: 'custpage_hist' + (i + 3),
                type: serverWidget.FieldType.TEXTAREA,
                label: 'ChatBot',
                container: 'fieldgroupid'
            });
            const chatBotMessage = context.request.parameters["custpage_hist" + (i + 1)];
            chatbot.defaultValue = chatBotMessage;
            chatbot.updateDisplayType({ displayType: serverWidget.FieldDisplayType.INLINE });
        }
    }

    const runToolLoop = (prompt, chatHistory) =&gt; {
        const preamble = "You are an AI assistant. Don't be chatty. If you're asking for further
               information, keep it very brief and instructive. All the input variables for tools
               have to be explicitly stated.";
        const tools = Object.values(TOOL_DEFINITIONS);

        let llmResult = llm.generateText({
            modelFamily: llm.ModelFamily.COHERE_COMMAND,
            preamble,
            tools,
            prompt,
            chatHistory: chatHistory ? JSON.parse(chatHistory) : undefined
        });

        while (llmResult.toolCalls.length &gt; 0) {
            const toolResults = [];
            for (const call of llmResult.toolCalls) {
                const resultValue = TOOL_HANDLERS[call.name](call.parameters);
                const toolResult = llm.createToolResult({
                    call,
                    outputs: [{ result: resultValue }]
                });
                toolResults.push(toolResult);
            }

            llmResult = llm.generateText({
                modelFamily: llm.ModelFamily.COHERE_COMMAND,
                tools,
                toolResults,
                chatHistory: llmResult.chatHistory
            });
        }
        return llmResult;
    }

    const displayResult = (prompt, llmResult, form) =&gt; {
        const promptField = form.addField({
                id: 'custpage_hist0',
                type: serverWidget.FieldType.TEXTAREA,
                label: 'You',
                container: 'fieldgroupid'
            });
            promptField.defaultValue = prompt;
            promptField.updateDisplayType({ displayType: serverWidget.FieldDisplayType.INLINE });

            const resultField = form.addField({
                id: 'custpage_hist1',
                type: serverWidget.FieldType.TEXTAREA,
                label: 'ChatBot',
                container: 'fieldgroupid'
            });
            resultField.defaultValue = llmResult.text;
            resultField.updateDisplayType({ displayType: serverWidget.FieldDisplayType.INLINE });

            form.getField({id: 'custpage_history'}).defaultValue = JSON.stringify(llmResult.chatHistory);
            //history.defaultValue = JSON.stringify(llmResult.chatHistory);
    }

    const createFormFooter = (form) =&gt; {
    // Field for new prompt input
        form.addField({
            id: 'custpage_text',
            type: serverWidget.FieldType.TEXTAREA,
            container: 'fieldgroupid',
            label: 'Prompt'
        });

        form.addSubmitButton({ label: 'Submit' });
    }

    const onRequest = (context) =&gt; {

        const form = createFromHeader(context);
```

```
        if (context.request.method === 'POST') {
            displayHistory(context, form);

            const prompt = context.request.parameters["custpage_text"];
            const chatHistory = context.request.parameters["custpage_history"];

            const llmResult = runToolLoop(prompt, chatHistory)

            displayResult(prompt, llmResult, form);

        } else { // GET
            form.getField({id: 'custpage_num_chats'}).defaultValue = '0';
            //numChats.defaultValue = "0";
        }

        createFormFooter(form);
        context.response.writePage(form);
    };

    return { onRequest };
});
```

26. N/log Module

Load the N/log module to access methods for logging script execution details. Module members are also supported by the global log Object.

`define(['N/log'], (log) ⇒ { ... })` ✓ 1 Code Sample(s) Scripts: Client and server scripts Limitations apply to client scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/log'], (log) ⇒ { // Access methods and properties on log });
```

Use the N/log module to manually access methods for logging script execution details. These methods can also be accessed using the global log object. For more information about the global log object, see log Object.

Log messages appear on the Execution Log subtab of the script deployment for deployed scripts, or on the Execution Log subtab of the SuiteScript Debugger if you are debugging a script. Log messages also appear on the Script Execution Logs page at Customization > Scripting > Script Execution Logs.

N/log Module Script Sample

N/log Module Guidelines

The following guidelines are provided for use of the N/log module:

NetSuite governs the amount of logging that can be done in any specific 60 minute time period. A company is allowed to make up to 100,000 log object method calls across all of their scripts. Script owners are notified if NetSuite detects that one script is logging excessively and automatically adjusts the log level.

NetSuite purges logs older than 30 days.

The Server Script Log search and the Execution Log subtab of script records and script deployment records have a log storage limit of 5 million per database. Because log persistence is not guaranteed, You should use custom records if you want to store script execution logs for extended periods. The Script Execution Logs page at Customization > Scripting > Script Execution Logs does not share the same database limit.

The Execution Log subtab also lists notes returned by NetSuite such as error messages. For more information, see N/error Module.

If you deploy a client script to a form using Form.clientScriptFileId or Form.clientScriptModulePath, using the N/log module adds the logs to the deployment of the parent script. The parent script can be either a beforeLoad user event script or a SuiteScript 2.1 Suitelet Script Type.

Using Log Levels

Use the log methods along with the Log Level field on the script deployment to specify which log messages are written to the Execution Log on the script deployment. This is useful during the debugging of a script or for providing useful execution notes for auditing or tracking purposes.

Log levels on the script deployment and N/log module methods act as a filter on the amount of information logged. The following Log Levels are supported:

Use Debug to show all messages.

This type of logging is suitable only for testing scripts. To avoid excessive logging, don't use Debug level for active scripts in production.

Use Audit to shows a record of events that have occurred during the processing of the script (for example, 'A request was made to an external site.').

Use Error to show only unexpected script errors.

LOG LEVEL	EXAMPLE USES
DEBUG	Use Debug to show all messages. This type of logging is suitable only for testing scripts. To avoid excessive logging, don't use Debug level for active scripts in production.
AUDIT	Use Audit to shows a record of events that have occurred during the processing of the script (for example, 'A request was made to an external site.').
ERROR	Use Error to show only unexpected script errors.
EMERGENCY	Use Emergency to show only the most critical errors in the script log.

LOG LEVEL	LOG METHOD USED IN YOUR SCRIPT			
	log.emergency(options)	log.error(options)	log.audit(options)	log.debug(options)
EMERGENCY		-	-	-
ERROR			-	-
AUDIT				-
DEBUG				

Viewing Script Execution Logs

Log messages written in client scripts that are attached to a form are viewed in the browser debug console tab when the client script runs on the form. Log messages written in server scripts and client scripts (that are deployed at the record level) are viewed in NetSuite.

Log messages for a specific script are shown on the Execution Log of the script deployment for the script. These logs are not guaranteed to persist for 30 days and may be purged to enhance performance if volume is high.

To view script execution log details for server scripts, go to Customization > Scripting > Script Execution Logs. This list of script execution logs is an enhanced repository that stores all log details for 30 days.

On this page, you can perform the following tasks:

Search for specific logs using filter options, such as log level, execution date range, and script name.

Download the list as a CSV file or an Excel spreadsheet.

N/log Module Members

Return Type / Value Type

Supported Script Types

log.audit(options)

Client* and server scripts

Logs an Audit message.

log.debug(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	log.audit(options)	void	Client* and server scripts	Logs an Audit message.
METHOD	log.debug(options)	void	Client* and server scripts	Logs a Debug message.
METHOD	log.emergency(options)	void	Client* and server scripts	Logs an Emergency message.
METHOD	log.error(options)	void	Client* and server scripts	Logs an Error message.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: N/log Module Script Sample

```
/**
 * @NApiVersion 2.1
 * @NScriptType UserEventScript
 */

// This script creates each type of log message.
define(['N/log'],function(log) {
    function beforeLoad(context) {
        var myValue = 'value';

        var myObject = {
            name: 'Jane',
            id: '123'
        };

        // An audit log message
        log.audit({
            title: 'Audit Entry',
            details: myObject
        });

        // A debug log message
        log.debug({
            title: 'Debug Entry',
            details: 'Value of myValue is: ' + myValue
        });

        // An emergency log message
        log.emergency({
            title: 'Emergency Entry',
            details: 'Value of myValue is: ' + myValue
        });

        // An error log message
        log.error({
            title: 'Error Entry',
            details: 'Value of myValue is: ' + myValue
        });
    }
    return {
        beforeLoad: beforeLoad
    };
});
```

27. N/machineTranslation Module

Load the N/machineTranslation module to translate text into supported languages using generative AI.

`define(['N/machineTranslation'], (machineTranslation) ⇒ { ... })` ✓ 1 Code Sample(s) Scripts: Server scripts SuiteScript 2.1 only Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/machineTranslation'], (machineTranslation) ⇒ { // Access methods and properties on machineTranslation });
```

The content in this help topic pertains to SuiteScript 2.1.

Use the N/machineTranslation module to translate text into supported languages using generative AI. This module uses the Oracle Cloud Infrastructure (OCI) Language service to translate text in documents you provide. For more information about this service, see Language in the OCI documentation.

You can also use the N/llm Module to translate text by providing a suitable prompt when calling generative AI methods (such as llm.generateText(options)). However, using the N/machineTranslation module provides several benefits that make it a better choice for most translation use cases:

Reduced cost - The N/machineTranslation module supports unlimited translation requests, but methods in the N/llm module consume AI Units. For more information about AI Units, see NetSuite AI Units and NetSuite Features and AI Units FAQ.

No prompt engineering required - When you use the N/machineTranslation module, you don't need to write a prompt to generate translations. The module detects the source language automatically and translates provided documents into the language you specify in your request.

More straightforward limits - The N/machineTranslation module limits the length of each provided document to 5,000 characters, and it also limits the total length of all provided documents to 20,000 characters. The N/llm module provides limits that are based on tokens and the context window of the LLM you use, which can be more difficult to estimate.

More supported languages - The N/machineTranslation module can translate text into additional languages that may not be supported by LLMs and the N/llm module.

More reliable translations - The N/machineTranslation module uses the OCI Language service, which is designed for specific natural language processing (NLP) tasks, including translation. It uses pretrained models that can offer more predictable and reliable translation results compared to the N/llm module. The N/llm module uses the OCI Generative AI service, which is designed for content generation and may provide more varied translation results compared to a dedicated translation service.

This module is available in NetSuite by default when the Server SuiteScript feature is enabled. For more information, see Enabling Features.

N/machineTranslation Module Members

Return Type / Value Type

Supported Script Types

machineTranslation.Document

A document with untranslated text (to provide to machineTranslation.translate(options) for translation) or translated text (when part of a machineTranslation.Response object returned from the translation service).

machineTranslation.Error

An error returned from the translation service.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	machineTranslation.Document	Object	Server scripts	A document with untranslated text (to provide to machineTranslation.translate(options) for translation) or translated text (when part of a machineTranslation.Response object returned from the translation service).
OBJECT	machineTranslation.Error	Object	Server scripts	An error returned from the translation service.
OBJECT	machineTranslation.Response	Object	Server scripts	The response from the translation service, which contains machineTranslation.Document objects with translated text.
METHOD	machineTranslation.createDocument(options)	Object	Server scripts	Creates a document to be used as source content when calling the translation service.
METHOD	machineTranslation.translate(options)	Object	Server scripts	Translates a set of documents into the specified language.
METHOD	machineTranslation.translate.promise(options)	Promise	Server scripts	Asynchronously translates a set of documents into the specified language.
ENUM	machineTranslation.Language	enum	Server scripts	Holds string values for the source language (when creating a document) or target language (when calling machineTranslation.translate(options)). Use this enum to set the options.language parameter in machineTranslation.createDocument(options) or the options.targetLanguage parameter in machineTranslation.translate(options).

Document Object Members

Return Type / Value Type

Supported Script Types

The ID of the document.

Document.language

The language of the document.

The content of the document.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Document.id	string	Server scripts	The ID of the document.
PROPERTY	Document.language	string	Server scripts	The language of the document.
PROPERTY	Document.text	string	Server scripts	The content of the document.

Error Object Members

Return Type / Value Type

Supported Script Types

Error.documentId

The ID of the document in which the error occurred.

The error message returned from the translation service.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Error.documentId	string	Server scripts	The ID of the document in which the error occurred.
PROPERTY	Error.message	string	Server scripts	The error message returned from the translation service.

Response Object Members

Return Type / Value Type

Supported Script Types

machineTranslation.Error[]

The errors returned from the translation service.

Response.results

machineTranslation.Document[]

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Response.errors	machineTranslation.Error[]	Server scripts	The errors returned from the translation service.
PROPERTY	Response.results	machineTranslation.Document[]	Server scripts	The translated documents returned from the translation service.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: N/machineTranslation Module Script Samples

```
/**
*@NApiVersion 2.1
*/
require(['N/machineTranslation'],
    function(mt) {
        const myDocument = mt.createDocument({
            id: 'myDoc',
            text: 'Hello everyone! How are you today?',
            language: mt.Language.ENGLISH
        });

        const translationResults = mt.translate({
            documents: [myDocument],
            targetLanguage: mt.Language.CZECH
        });

        // Work with the translated document
        log.debug('Translated text', translationResults.results[0].text);
    }
);
```

28. N/manufacturing/productionCharges Module

Load the N/manufacturing/productionCharges module to update unit costs of manufacturing charges on Assembly Build, Work Order Close, Work Order Completion transactions.

`define(['N/manufacturing/productionCharges'], (productionCharges) ⇒ { ... })` ✓ 1 Code Sample(s) Scripts: Â Permissions: Â

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/manufacturing/productionCharges'], (productionCharges) ⇒ { // Access methods and properties on productionCharges });
```

Use the N/manufacturing/productionCharges module to update unit costs of manufacturing charges on Assembly Build, Work Order Close, Work Order Completion transactions.

The module is only usable in server-side SuiteScript 2.1 scripts.

You can use productionCharges.updateChargesToCustomUnitCost(options) method to update cost on a specific transaction line to a specified unit cost.

You can use productionCharges.updateChargesToItemPurchasePrice(options) method to update cost on a specific transaction line to a specified unit cost.

You can use productionCharges.updateAllChargesToItemPurchasePrice(options) method to update cost on all routing and non-inventory transaction lines on a specified transaction to the current price.

Prerequisites

Assembly Items feature must be enabled (for more information, see Enabling Assembly Items)

Allow bulk cost updates for Production Charges preference must be enabled

The user must have at least Edit permission for the following transactions:

with Manufacturing Work In Process feature DISABLED: Build Assemblies

with Manufacturing Work In Process feature ENABLED: Build Assemblies, Work Order Completion, Work Order Issue

Transactions modified by this module must be:

N/manufacturing/productionCharges Module Members

Return Type / Value Type

Supported Script Types

productionCharges.updateChargesToCustomUnitCost(options)

Updates cost on a specific transaction line to a specified unit cost.

productionCharges.updateChargesToItemPurchasePrice(options)

Updates cost on a specific transaction line according to the current purchase price of the originating item.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**productionCharges.updateChargesToCustomUnitCost(options)**	void	Server scripts	Updates cost on a specific transaction line to a specified unit cost.
METHOD	**productionCharges.updateChargesToItemPurchasePrice(options)**	void	Server scripts	Updates cost on a specific transaction line according to the current purchase price of the originating item.
METHOD	**productionCharges.updateAllChargesToItemPurchasePrice(options)**	void	Server scripts	Updates cost on all routing and non-inventory transaction lines on a specified transaction to the current price.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: Update Production Charges and Unit Costs on Assembly Transactions

```
/**
 * @NApiVersion 2.1
 * @NScriptType ScheduledScript
 */
define(['N/manufacturing/productionCharges', 'N/log'], (productionCharges, log) => {
    const execute = (context) => {
        try {
            // 1. Update cost on a specific transaction line to a custom unit cost
            productionCharges.updateChargesToCustomUnitCost({
                transactionId: 1045,
                line: 1,
                customUnitCost: 15.50
            });
            log.audit('Cost Update', 'Custom unit cost updated on line 1');

            // 2. Update cost on a specific line to item purchase price
            productionCharges.updateChargesToItemPurchasePrice({
                transactionId: 1045,
                line: 2
            });

            // 3. Update all routing and non-inventory charges to current item purchase price
            productionCharges.updateAllChargesToItemPurchasePrice({
                transactionId: 1045
            });
            log.audit('Bulk Update', 'All production charges updated to item purchase price');
        } catch (e) {
            log.error('Production Charges Error', e.message);
        }
    };
    return { execute };
});
```

29. N/pgp Module

Load the N/pgp module to send secure messages to one or multiple receivers.

`define(['N/pgp'], (pgp) ⇒ { ... })` ✓ 4 Code Sample(s) Scripts: Server scripts SuiteScript 2.1 only Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/pgp'], (pgp) ⇒ { // Access methods and properties on pgp });
```

The content in this help topic pertains to SuiteScript 2.1.

Use the N/pgp module to enable secure messaging, file encryption, and document signing. Based on OpenPGP encryption standards.

To use the N/pgp Module, you must first generate PGP keys from GnuPG, OpenPGP, or a third party source that supports pgp key generation. The generated keys must be stored in Secrets Management to securely manage and reference the keys. To store your generated keys, go to Setup > Company > API Secrets to create a new secret key.

For more information about API Secrets in NetSuite, see Secrets Management.

If you are new to PGP, use the following resources to learn more:

RFC 4880 OpenPGP Message Format - Provides background information about the PGP standard.

GnuPG Manual - A reliable resource for practical information.

OpenPGP - Learn more about the pgp standards with documentation resources for developers.

Limitations of N/pgp

As you are working with the N/pgp module, consider the following limitations:

You cannot generate, modify, or inspect PGP keys using the N/pgp module. You must generate keys from a third party source that supports PGP key generation.

You cannot create a message without readable PGP software.

You are limited to strings.

You are limited to data that fits into memory.

N/pgp Module Script Samples

N/pgp Module Members

Return Type / Value Type

Supported Script Types

General configuration options that can be used for message decryption.

Cryptographic keys and its metadata.

An octet scalar that identifies a subkey.

Processed PGP data.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	**pgp.Config**	Object	Server scripts	General configuration options that can be used for message decryption.
OBJECT	**pgp.Key**	Object	Server scripts	Cryptographic keys and its metadata.
OBJECT	**pgp.KeyId**	Object	Server scripts	An octet scalar that identifies a subkey.
OBJECT	**pgp.Message**	Object	Server scripts	Processed PGP data.
OBJECT	**pgp.MessageData**	Object	Server scripts	Message data.
OBJECT	**pgp.Verification**	Object	Server scripts	Verification results.
OBJECT	**pgp.VerificationSignature**	Object	Server scripts	A verification result for a single signature.
METHOD	**pgp.createConfig(options)**	pgp.Config	Server scripts	Creates a new configuration object.
METHOD	**pgp.createMessageData(options)**	pgp.MessageData	Server scripts	Creates new message data.
METHOD	**pgp.createSigner(options)**	certificate.Signer	Server scripts	Creates a certificate.Signer object for signing plain strings.
METHOD	**pgp.createVerification()**	pgp.Verification	Server scripts	Creates an empty verification object.
METHOD	**pgp.loadKeyFromSecret(options)**	pgp.Key	Server scripts	Loads a key whose contents are stored securely in secret.
METHOD	**pgp.parseMessage(options)**	pgp.Message	Server scripts	Parses a PGP message.
METHOD	**pgp.parseKey(options)**	pgp.Key	Server scripts	Parses an existing PGP key.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
ENUM	pgp.CompressionAlgorithm	Enum	Server scripts	Holds the values for available compression algorithms. Use this enum to set the value of the options.compressionAlgorithm parameter of the MessageData.encrypt(options) method.
ENUM	pgp.Format	Enum	Server scripts	Literal data packet type. Use this enum to set the value for the options.format parameter of the pgp.createMessageData(options) method.

Config Object Members

Return Type / Value Type

Supported Script Types

Config.allowInsecureDecryptionWithSigningKeys

Enables decryption that is not secured with signing keys.

Config.allowMessagesWithoutIntegrityProtection

Allows messages without integrity protection.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Config.allowInsecureDecryptionWithSigningKeys	boolean	Server scripts	Enables decryption that is not secured with signing keys.
PROPERTY	Config.allowMessagesWithoutIntegrityProtection	boolean	Server scripts	Allows messages without integrity protection.
PROPERTY	Config.useRelaxedSignatureParsing	boolean	Server scripts	Relaxed signature parsing for configuration objects.

KeyId Object Members

Return Type / Value Type

Supported Script Types

Returns a key ID as a hexadecimal string.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	KeyId.asHex()	string	Server scripts	Returns a key ID as a hexadecimal string.

Message Object Members

Return Type / Value Type

Supported Script Types

Message type that specifies how a message is processed.

Message.asArmored()

Converts a message to ASCII armored format.

Message.toMessageData()

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Message.type	boolean	Server Scripts	Message type that specifies how a message is processed.
METHOD	Message.asArmored()	string	Server scripts	Converts a message to ASCII armored format.
METHOD	Message.toMessageData()	pgp.MessageData	Server scripts	Converts a message to message data without processing. Works only if the message is not encrypted.
METHOD	Message.decrypt(options)	pgp.MessageData	Server scripts	Decrypts a message and optionally verifies the signatures.

MessageData Object Members

Return Type / Value Type

Supported Script Types

MessageData.filename

The name of a file.

MessageData.date

The date of a message or modification date of the file.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	MessageData.filename	string	Server scripts	The name of a file.
PROPERTY	MessageData.date	Date	Server scripts	The date of a message or modification date of the file.
PROPERTY	MessageData.format	pgp.Format	Server scripts	Literal data packet type.
METHOD	MessageData.getText()	string	Server scripts	Extracts the contents of a message as text.
METHOD	MessageData.toMessage()	pgp.Message	Server scripts	Creates a message with no signature, compression, or encryption.
METHOD	MessageData.encrypt(options)	pgp.Message	Server scripts	Creates a message that is encrypted and optionally signed.

Verification Object Members

Return Type / Value Type

Supported Script Types

VerificationSignature.verified

Indicates whether verification was successful.

Verification.signatures

null | Array<VerificationSignature>

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	VerificationSignature.verified	null | boolean	Server scripts	Indicates whether verification was successful.
PROPERTY	Verification.signatures	null | Array<VerificationSignature>	Server scripts	A list of individual verifications, one per signature.

VerificationSignature Object Members

Return Type / Value Type

Supported Script Types

VerificationSignature.keyId

ID of the (sub)key that was used for signing.

VerificationSignature.dateSigned

Date when the message was signed.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	VerificationSignature.keyId	pgp.KeyId	Server Scripts	ID of the (sub)key that was used for signing.
PROPERTY	VerificationSignature.dateSigned	Date	Server Scripts	Date when the message was signed.
PROPERTY	VerificationSignature.verified	boolean	Server scripts	Indicates whether verification was successful for a signature.
PROPERTY	VerificationSignature.problems	string[]	Server scripts	A list of problems for verification signatures.

▶ Official NetSuite Script Samples (4 Sample(s))

📄 Sample 1: Send a Message

```
/**
 * @NApiVersion 2.1
 */

require(['N/pgp', 'N/email'], (pgp, email) =&gt; {
  const keys = {
    ours: {
      pub: pgp.loadKeyFromSecret({
        secret: { scriptId: 'custsecret_pgp_key_ours_public' }
      }),
      pri: pgp.loadKeyFromSecret({
        secret: { scriptId: 'custsecret_pgp_key_ours_private' },
        password: { scriptId: 'custsecret_pgp_key_ours_private_password' }
      })
    }
  }

  const data = pgp.createMessageData({
    content: 'Hello, world!'
  })
  const message = data.encrypt({
    encryptionKeys: keys.ours.pub,
    signingKeys: keys.ours.pri
  })
  const payload = message.asArmored()

  const senderId = -5
  const recipientId = 'notify@myCompany.com'
  email.send({
    author: senderId,
    recipients: recipientId,
    subject: 'Test PGP',
    body: 'Payload: ' + payload
  })
})
```

📄 Sample 2: Receive a Message

```
/**
 * @NApiVersion 2.1
 */

require(['N/pgp', 'N/email'], (pgp, email) =&gt; {
  const keys = {
    ours: {
      pub: pgp.loadKeyFromSecret({
        secret: { scriptId: 'custsecret_pgp_key_ours_public' }
      }),
      pri: pgp.loadKeyFromSecret({
        secret: { scriptId: 'custsecret_pgp_key_ours_private' },
        password: { scriptId: 'custsecret_pgp_key_ours_private_password' }
      })
    }
  }
  const data = pgp.createMessageData({
    content: 'Hello, world!'
  })
  const message = data.encrypt({
    encryptionKeys: keys.ours.pub,
    signingKeys: keys.ours.pri
  })
  const payload = message.asArmored()
  const parseMessage = pgp.parseMessage({
    value: payload
  })
  const msgData = parseMessage.decrypt({
    decryptionKeys: keys.ours.pri,
    verificationKeys: keys.ours.pub
  })
  msgData.getText()
})
```

📄 Sample 3: Send a Message to Multiple Receivers

```
/**
 * @NApiVersion 2.1
 */

require(['N/pgp'], (pgp) =&gt; {
  // public and private keys for multiple receivers
  const keys = {
    alice: {
      pub: pgp.loadKeyFromSecret({
        secret: { scriptId: 'custsecret_pgp_key_alice_public' }
      }),
      pri: pgp.loadKeyFromSecret({
        secret: { scriptId: 'custsecret_pgp_key_alice_private' },
        password: { scriptId: 'custsecret_pgp_key_bob_private_password' }
      })
    },
    bob: {
      pub: pgp.loadKeyFromSecret({
        secret: { scriptId: 'custsecret_pgp_key_bob_public' }
      }),
      pri: pgp.loadKeyFromSecret({
        secret: { scriptId: 'custsecret_pgp_key_bob_private' },
        password: { scriptId: 'custsecret_pgp_key_bob_private_password' }
      })
    },
    netsuite: {
      pub: pgp.loadKeyFromSecret({
        secret: { scriptId: 'custsecret_pgp_netsuite_pub' }
      }),
      pri: pgp.loadKeyFromSecret({
        secret: { scriptId: 'custsecret_pgp_netsuite_pri' }
      })
    },
    example: {
      pri: pgp.loadKeyFromSecret({
        secret: { scriptId: 'custsecret_pgp_example_pri' }
      })
    }
  }
  const data = pgp.createMessageData({
    content: 'Hello Alice and Bob!'
  })
  const message = data.encrypt({
    encryptionKeys: [keys.alice.pub, keys.bob.pub],
    signingKeys: [keys.netsuite.pri, keys.example.pri]
  })
  /*
   * Alice decryption.
   */
  message.decrypt({
    decryptionKeys: keys.alice.pri,
    verificationKeys: [keys.netsuite.pub]
  })
  /*
   * Bob decryption.
   */
  message.decrypt({
    decryptionKeys: [keys.bob.pri],
    verificationKeys: keys.example.pub
  })
})
```

📄 Sample 4: Use a Cryptographic Key for a Signature

```
/**
 * @NApiVersion 2.1
 */

require(['N/pgp', 'N/crypto/certificate', 'N/encode'], (pgp, cryptoCertificate, encode) =&gt; {
  const keys = {
    ours: {
      pub: pgp.loadKeyFromSecret({
        secret: { scriptId: 'custsecret_pgp_key_ours_public' }
      }),
      pri: pgp.loadKeyFromSecret({
        secret: { scriptId: 'custsecret_pgp_key_ours_private' },
        password: { scriptId: 'custsecret_pgp_key_ours_private_password' }
      })
    }
  }
  const signer = pgp.createSigner({
    key: keys.ours.pri,
    algorithm: cryptoCertificate.HashAlg.SHA256
  })
  signer.update({
    input: 'Test'
  })
  const signature = signer.sign({
    outputEncoding: encode.Encoding.BASE_64_URL_SAFE
  })

  log.debug(signature)
})
```

30. N/piremoval Module

Load the N/piremoval module to remove personal information (PI) from system notes, workflow history, and specific field values.

`define(['N/piremoval'], (piremoval) => { ... })` ✓ 1 Code Sample(s) Scripts: Server scripts

Permissions: Remove Personal Information Create Remove Personal Information Run

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/piremoval'], (piremoval) => { // Access methods and properties on piremoval });
```

Use the N/piremoval module to remove personal information (PI) from system notes, workflow history, and specific field values. Use the N/piremoval module to comply with privacy regulations, specifically the right to be forgotten. You can remove personal information from system notes only, or you can also remove workflow history and field values on the record. Entity records, transactions, and custom records are supported.

N/piremoval Module Script Sample

You can use the piremoval.createTask(options) method to create a PI removal task, or use piremoval.loadTask(options) to load an existing PI removal task. Both of these methods return a piremoval.PiRemovalTask object that represents the task. Create a piremoval.PiRemovalTask object for each record type that requires removal of personal information. Use the PiRemovalTask.save() method to save the task, then use the PiRemovalTask.run() method to process the task and remove the personal information.

You can use the piremoval.getTaskStatus(options) method to check the status of a submitted PI removal task. This method returns a piremoval.PiRemovalTaskStatus object that describes the current status of the removal task. The piremoval.PiRemovalTaskStatus object uses an iterator to provide a list of log entries in the PiRemovalTaskStatus.logList object.

To use the N/piremoval module, the following requirements must be met:

Remove Personal Information Create permission is required to create a PI removal task.

Remove Personal Information Run permission is required to run a PI removal task.

For more information, see Personal Information (PI) Removal.

N/piremoval Module Members

Return Type / Value Type

Supported Script Types

piremoval.PiRemovalTask

Encapsulates a personal information removal task.

Use piremoval.createTask(options) to create this object.

piremoval.PiRemovalTaskLogItem

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	**piremoval.PiRemovalTask**	Object	Server scripts	Encapsulates a personal information removal task. Use piremoval.createTask(options) to create this object.
OBJECT	**piremoval.PiRemovalTaskLogItem**	Object	Server scripts	Encapsulates a log item of the personal information removal task status.
OBJECT	**piremoval.PiRemovalTaskStatus**	Object	Server scripts	Encapsulates the status of a personal information removal task. Use piremoval.getTaskStatus(options) to create this object.
METHOD	**piremoval.createTask(options)**	piremoval.PiRemovalTask	Server scripts	Creates a personal information removal task.
METHOD	**piremoval.deleteTask(options)**	void	Server scripts	Deletes a personal information removal task.
METHOD	**piremoval.getTaskStatus(options)**	piremoval.PiRemovalTaskStatus	Server scripts	Retrieves the status of a personal information removal task.
METHOD	**piremoval.loadTask(options)**	piremoval.PiRemovalTask	Server scripts	Loads a personal information removal task.

PiRemovalTask Object Members

The following members are available for a piremoval.PiRemovalTask object.

Return Type / Value Type

Supported Script Types

PiRemovalTask.deleteTask()

Deletes the personal information removal task.

PiRemovalTask.run()

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**PiRemovalTask.deleteTask()**	void	Server scripts	Deletes the personal information removal task.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	PiRemovalTask.run()	void	Server scripts	Runs the personal information removal task.
METHOD	PiRemovalTask.save()	void	Server scripts	Saves the personal information removal task.
PROPERTY	PiRemovalTask.fieldIds	string[] (read-only)	Server scripts	Represents the field IDs that are processed by the PI removal task.
PROPERTY	PiRemovalTask.historyOnly	boolean	Server scripts	Indicates whether the PI removal task removes system note information only, not field values or workflow history.
PROPERTY	PiRemovalTask.historyReplacement	string (read-only)	Server scripts	Represents the text used in system notes to replace the original values.
PROPERTY	PiRemovalTask.id	number (read-only)	Server scripts	Represents the ID of the personal information removal task.
PROPERTY	PiRemovalTask.recordIds	number[] (read-only)	Server scripts	Represents the record IDs that are processed by the PI removal task.
PROPERTY	PiRemovalTask.recordType	string (read-only)	Server scripts	Describes the record type updated by the PI removal task.
PROPERTY	PiRemovalTask.status	piremoval.PiRemovalTaskStatus	Server scripts	Describes the status of the submitted personal information removal task.
PROPERTY	PiRemovalTask.workflowIds	number[] (read-only)	Server scripts	Represents the workflow IDs whose history is processed by the PI removal task.

PiRemovalTaskLogItem Object Members

The following members are available for a piremoval.PiRemovalTaskLogItem object.

Return Type/Value Type

Support Script Type

PiRemovalTaskLogItem.exception

string (read-only)

Describes the exception for the log item, including and what caused it.

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORT SCRIPT TYPE	DESCRIPTION
PROPERTY	PiRemovalTaskLogItem.exception	string (read-only)	Server scripts	Describes the exception for the log item, including and what caused it.
PROPERTY	PiRemovalTaskLogItem.message	string (read-only)	Server scripts	Describes the message for the log item and an explanation for any errors.
PROPERTY	PiRemovalTaskLogItem.status	string (read-only)	Server scripts	Describes the status of the log item. This property takes its values from task.TaskStatus.
PROPERTY	PiRemovalTaskLogItem.type	string (read-only)	Server scripts	Describes the type of personal information that was removed, one of FieldValue, SystemNote, or Workflow.

PiRemovalTaskStatus Object Members

The following members are available for a piremoval.PiRemovalTaskStatus object.

Return Type/Value Type

Support Script Type

PiRemovalTaskStatus.logList

Represents a list of logs for the PI removal task job.

PiRemovalTaskStatus.status

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORT SCRIPT TYPE	DESCRIPTION
PROPERTY	PiRemovalTaskStatus.logList	list	Server scripts	Represents a list of logs for the PI removal task job.
PROPERTY	PiRemovalTaskStatus.status	string	Server scripts	Describes the status of the submitted personal information removal task.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: N/piremoval Module Script Sample

```
/**
 * @NApiVersion 2.1
 */

require(['N/piremoval'], function(piremoval) {
    function removePersonalInformation() {
        var piRemovalTask = piremoval.createTask({
            recordType: 'customer',
            recordIds: [11, 19],
            fieldIds: ['comments', 'phone'],
            workflowIds: [1],
            historyOnly: false,
            historyReplacement: 'removed_value'
        });

        piRemovalTask.save();
        var taskId = piRemovalTask.id;

        var piRemovalTaskInProgress = piremoval.loadTask(taskId);
        piRemovalTaskInProgress.run();

        var status = piremoval.getTaskStatus(taskId);
    };

    removePersonalInformation();
});
```

31. N/plugin Module

Load the N/plugin module to load custom plug-in implementations.

`define(['N/plugin'], (plugin) => { ... })` ✓ 2 Code Sample(s) Scripts: Server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/plugin'], (plugin) => { // Access methods and properties on plugin });
```

Use the N/plugin module to load custom plug-in implementations. For more information about custom plug-ins, see the following help topics:

Custom Plug-ins

Custom Plug-in Creation

Custom Plug-in Development

Custom Plug-in Implementation

N/plugin Module Script Sample

You cannot use the SuiteScript Debugger to debug a script on demand that uses the N/plugin module. You must use deployed debugging. To use deployed debugging, you must complete the steps described in Adding a Script that Instantiates a Custom Plug-in to NetSuite. For the complete process on creating a custom plugin, see Custom Plug-in Development. For additional information about ad-hoc and deployed debugging, see SuiteScript Debugger.

N/plugin Module Members

Return Type / Value Type

Supported Script Types

plugin.findImplementations(options)

Returns the script IDs of custom plug-in type implementations.

plugin.loadImplementation(options)

Instantiates an implementation of the custom plug-in type.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**plugin.findImplementations(options)**	`string[]`	Server scripts	Returns the script IDs of custom plug-in type implementations.
METHOD	**plugin.loadImplementation(options)**	`Object`	Server scripts	Instantiates an implementation of the custom plug-in type.

▶ Official NetSuite Script Samples (2 Sample(s))

📄 Sample 1: N/plugin Module Script Sample

```
/**
 * @NApiVersion 2.1
 * @NScriptType plugintypeimpl
 */
define(function() {
    return {
        doTheMagic: function(operand1, operand2) {
            return operand1 + operand2;
        }
    }
});
```

📄 Sample 2: N/plugin Module Script Sample

```
/**
 * @NApiVersion 2.1
 * @NScriptType Suitelet
 */
define(['N/plugin'], function(plugin) {
    function onRequest(context) {
        var impls = plugin.findImplementations({
            type: 'customscript_magic_plugin'
        });

        for (var i = 0; i &lt; impls.length; i++) {
            var pl = plugin.loadImplementation({
                type: 'customscript_magic_plugin',
                implementation: impls[i]
            });
            log.debug('impl ' + impls[i] + ' result = ' + pl.doTheMagic(10, 20));
        }

        var pl = plugin.loadImplementation({
            type: 'customscript_magic_plugin'
        });
        log.debug('default impl result = ' + pl.doTheMagic(10, 20));
    }

    return {
        onRequest: onRequest
    };
});
```

32. N/portlet Module

Load the N/portlet module to resize or refresh a form portlet.

`define(['N/portlet'], (portlet) ⇒ { ... })` ✓ 1 Code Sample(s) Scripts: Client scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/portlet'], (portlet) ⇒ { // Access methods and properties on portlet });
```

Use the N/portlet module to resize or refresh a form portlet. For more information about portlet scripts, see SuiteScript 2.1 Portlet Script Type.

N/portlet Module Script Sample

N/portlet Module Members

Supported Script Types

portlet.resize()

Resizes a form portlet immediately.

portlet.refresh()

Refreshes a form portlet immediately.

MEMBER TYPE	NAME	RETURN TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	`portlet.resize()`	`void`	Client scripts	Resizes a form portlet immediately.
METHOD	`portlet.refresh()`	`void`	Client scripts	Refreshes a form portlet immediately.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: N/portlet Module Script Sample

```
/**
 * @NApiVersion 2.x
 * @NScriptType Portlet
 * @NScriptPortletType form
 */

define([], function() {
    function render(context) {
        var portletObj = context.portlet;
        portletObj.title = 'Test Form Portlet';
        setComponentsForResize();
        setComponentsForRefresh();

        function setComponentsForResize() {
            var DEFAULT_HEIGHT = '50';
            var DEFAULT_WIDTH = '50';
            var inlineHTMLField = portletObj.addField({
                id: 'divfield',
                type: 'inlinehtml',
                label: 'Test inline HTML'
            });
            inlineHTMLField.defaultValue = '&lt;div id=\'divfield_elem\' style=\'border: 1px dotted red; height: ' + DEFAULT_HEIGHT + 'px; width: ' + DEFAULT
_WIDTH + 'px;\'&gt;&lt;/div&gt;';
            inlineHTMLField.updateLayoutType({
                layoutType: 'normal'
            });
            inlineHTMLField.updateBreakType({
                breakType: 'startcol'
            });
            var resizeHeight = portletObj.addField({
                id: 'resize_height',
                type: 'text',
                label: 'Resize Height'
            });
            resizeHeight.defaultValue = DEFAULT_HEIGHT;
            var resizeWidth = portletObj.addField({
                id: 'resize_width',
                type: 'text',
                label: 'Resize Width'
            });
            resizeWidth.defaultValue = DEFAULT_WIDTH;
            var resizeLink = portletObj.addField({
                id: 'resize_link',
                type: 'inlinehtml',
                label: 'Resize link'
            });
            resizeLink.defaultValue = resizeLink.defaultValue = '&lt;a id=\'resize_link\' onclick=\"require([\'SuiteScripts/portletApiTestHelper\'], function
(portletApiTestHelper) {portletApiTestHelper.resizePortlet(); }) \" href=\'#\'&gt;Resize&lt;/a&gt;&lt;br&gt;';
        }

        function setComponentsForRefresh() {
            var textField = portletObj.addField({
                id: 'refresh_output',
                type: 'text',
                label: 'Date.now().toString()'
            });
            textField.defaultValue = Date.now().toString();
            var refreshLink = portletObj.addField({
                id: 'refresh_link',
                type: 'inlinehtml',
                label: 'Refresh link'
            });
            refreshLink.defaultValue = '&lt;a id=\'refresh_link\' onclick=\'require([\"SuiteScripts/portletApiTestHelper\"], function(portletApiTestHelper)
{portletApiTestHelper.refreshPortlet(); }) \' href=\'#\'&gt;Refresh&lt;/a&gt;';
        }
    }

    return {
        render: render
    };
})

// portletApiTestHelper.js
define(['N/portlet'], function(portlet) {
    function refreshPortlet() {
        portlet.refresh();
    }

    function resizePortlet() {
        var div = document.getElementById('divfield_elem');
        var newHeight = parseInt(document.getElementById('resize_height').value);
        var newWidth = parseInt(document.getElementById('resize_width').value);
        div.style.height = newHeight + 'px';
        div.style.width = newWidth + 'px';
        portlet.resize();
    }

    return {
```

```
        refreshPortlet: refreshPortlet,
        resizePortlet: resizePortlet
    };
});
```

33. N/query Module

Load the N/query module to create and run searches using the SuiteAnalytics Workbook query process.

`define(['N/query'], (query) => { ... })` ✓ 6 Code Sample(s) Scripts: Client and server scripts Permissions: SuiteAnalytics Workbook

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/query'], (query) => { // Access methods and properties on query });
```

Use the N/query module to create and run queries using the SuiteAnalytics Workbook query process. For more information about SuiteAnalytics Workbook, see SuiteAnalytics Workbook Overview.

Using the query module, you can:

Use multilevel joins to create queries using field data from multiple record types.

Create conditions (filters) using AND, OR, and NOT logic, as well as formulas and relative dates.

Sort query results based on the values of multiple columns.

Load and delete existing saved queries that were created using the SuiteAnalytics Workbook interface.

View paged query results.

Use promises for asynchronous .

Convert query objects to SuiteQL queries and run arbitrary SuiteQL queries.

For more information about creating scripts using the N/query module, see the following help topics:

Scripting with the N/query Module

Formulas in the N/query Module

Relative Dates in the N/query Module

SuiteQL in the N/query Module

As you use the N/query module, keep the following considerations in mind:

The N/query module lets you create and run queries using the SuiteAnalytics Workbook query process. You can load and delete existing queries, but you can't save queries with this module. To save queries, use the SuiteAnalytics Workbook interface. For details, see Navigating SuiteAnalytics Workbook.

The N/query module uses a different data source than N/search. To find record types or field IDs for filters or result columns, use the Records Catalog (not the SuiteScript Records Browser). The Records Catalog lists everything available for SuiteAnalytics Workbook and N/query. For more, see Records Catalog Overview.

The N/query module doesn't work in unauthenticated client-side contexts.

For details, see the SuiteAnswers article Outbound HTTPs in an unauthenticated client-side context.

N/query Module Members

Return Type / Value Type

Supported Script Types

Client and server scripts

The field types (query result columns) that are displayed from the query results.

Use Query.createColumn(options) or Component.createColumn(options) to create this object.

Client and server scripts

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	**query.Column**	Object	Client and server scripts	The field types (query result columns) that are displayed from the query results. Use Query.createColumn(options) or Component.createColumn(options) to create this object.
OBJECT	**query.Component**	Object	Client and server scripts	One component of the query definition. The query definition always contains at least one component that encapsulates the initial query type. Queries with joins contain multiple components that encapsulate the join relationships. The initial component (Query.root) is automatically created with the query definition (query.Query). Use Query.autoJoin(options) or Component.autoJoin(options) to create subsequent components.
OBJECT	**query.Condition**	Object	Client and server scripts	A condition. A condition narrows the query results. Use Query.createCondition(options) or Component.createCondition(options) to create this object.
OBJECT	**query.Page**	Object	Client and server scripts	One page of the paged query results.
OBJECT	**query.PagedData**	Object	Client and server scripts	A set of paged query results. This object also contains information about the set of paged results it encapsulates.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	query.PageRange	Object	Client and server scripts	A range of pages from the paged query results.
OBJECT	query.Period	Object	Client and server scripts	A period of time to use in query conditions.
OBJECT	query.RelativeDate	Object	Client and server scripts	A relative date to use in query conditions.
OBJECT	query.Result	Object	Client and server scripts	A single row of the query result set.
OBJECT	query.ResultSet	Object	Client and server scripts	The set of results returned by the query.
OBJECT	query.Query	Object	Client and server scripts	The query definition. Use query.create(options) or query.load(options) to create this object. Important: The creation of this object is the first step in creating a query with the N/query Module.
OBJECT	query.Sort	Object	Client and server scripts	A sort that is placed on a particular query result column. Use Query.createSort(options) or Component.createSort(options) to create this object.
OBJECT	query.SuiteQL	Object	Client and server scripts	A SuiteQL query. Use Query.toSuiteQL() to create this object.
METHOD	query.create(options)	query.Query	Client and server scripts	Creates the query definition. Important: The invocation of this method is the first step in creating a query with the N/query Module.
METHOD	query.createPeriod(options)	query.Period	Client and server scripts	Creates a query.Period object that represents a period of time.
METHOD	query.createRelativeDate(options)	query.RelativeDate	Client and server scripts	Creates a query.RelativeDate object that represents a date relative to the current date.
METHOD	query.delete(options)	void	Client and server scripts	Deletes an existing query that was created using the SuiteAnalytics Workbook UI. The deleted query is no longer available and cannot be modified or executed.
METHOD	query.listTables(options)	<Object>	Client and server scripts	Lists the table view objects that are included in a workbook in SuiteAnalytics Workbook.
METHOD	query.load(options)	query.Query	Client and server scripts	Loads an existing query that was created using the SuiteAnalytics Workbook UI. The loaded query can be modified (for example, by setting additional property values), joined with other query types, and executed in the same way as queries created using query.create(options).
METHOD	query.load.promise(options)	Promise Object	Client and server scripts	Asynchronously loads an existing query that was created using the SuiteAnalytics Workbook UI.
METHOD	query.runSuiteQL(options)	query.ResultSet	Client and server scripts	Runs an arbitrary SuiteQL query.
METHOD	query.runSuiteQL.promise(options)	Promise Object	Client and server scripts	Asynchronously runs an arbitrary SuiteQL query.
METHOD	query.runSuiteQLPaged(options)	query.PagedData	Client and server scripts	Runs an arbitrary SuiteQL query as a paged query.
METHOD	query.runSuiteQLPaged.promise(options)	Promise Object	Client and server scripts	Asynchronously runs an arbitrary query as a paged query.
ENUM	query.Aggregate	enum	Client and server scripts	Holds the string values for aggregate functions supported with the N/query Module. This enum is used to pass the aggregate function argument to Component.createColumn(options), Component.createCondition(options), Query.createColumn(options), and Query.createCondition(options).
ENUM	query.DateId	enum	Client and server scripts	Holds the string values for supported date codes in relative dates. This enum is used to pass the date ID argument to query.createRelativeDate(options).
ENUM	query.FieldContext	enum	Client and server scripts	Holds the string values for the field context to use when creating a column. This enum is used to pass the context argument to Query.createColumn(options) and Component.createColumn(options).
ENUM	query.Operator	enum	Client and server scripts	Holds the string values for operators supported with the N/query Module. This enum is used to pass the operator argument to Query.createCondition(options) and Component.createCondition(options).
ENUM	query.PeriodAdjustment	enum	Client and server scripts	Holds the string values for adjustment types for a period. This enum is used to pass the adjustment argument to query.createPeriod(options).
ENUM	query.PeriodCode	enum	Client and server scripts	Holds the string values for period codes for a period. This enum is used to pass the code argument to query.createPeriod(options).
ENUM	query.PeriodType	enum	Client and server scripts	Holds the string values for period types for a period. This enum is used to pass the type argument to query.createPeriod(options).
ENUM	query.RelativeDateRange	enum	Client and server scripts	Holds query.RelativeDate object values for supported date ranges in relative dates. This enum is used to pass the values argument to Query.createCondition(options) and Component.createCondition(options).

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
ENUM	query.ReturnType	enum	Client and server scripts	Holds the string values for the formula return types supported with the N/query Module. This enum is used to pass the formula return type argument to Query.createColumn(options), Component.createColumn(options), Query.createCondition(options), and Component.createCondition(options).
ENUM	query.SortLocale	enum	Client and server scripts	Holds the string values for sort locales supported with the N/query Module. This enum is used to pass the sort locale argument to Query.createSort(options) and Component.createSort(options).
ENUM	query.Type	enum	Client and server scripts	Holds the string values for supported query types used in the query definition. This enum is used to pass the initial query type argument to query.create(options).

Column Object Members

The following members are available for a query.Column object.

Return Type/Value Type

Supported Script Types

Column.aggregate

string (read-only)

Client and server scripts

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Column.aggregate	string (read-only)	Client and server scripts	An aggregate function that is performed on the query result column. An aggregate function performs a calculation on the column values and returns a single value.
PROPERTY	Column.alias	string (read-only)	Client and server scripts	An alias for this column. An alias is an alternate name for a column, and the alias is used in mapped results.
PROPERTY	Column.component	query.Component (read-only)	Client and server scripts	A reference to the query.Component object to which this query result column belongs.
PROPERTY	Column.context	Object (read-only)	Client and server scripts	The field context for values in the query result column. The field context determines how field values are displayed in the column.
PROPERTY	Column.fieldId	string (read-only)	Client and server scripts	The name of the query result column. This property and the Column.formula property cannot be set at the same time.
PROPERTY	Column.formula	string (read-only)	Client and server scripts	The formula used to create the query result column. This property and the Column.fieldId property cannot be set at the same time.
PROPERTY	Column.groupBy	(read-only)	Client and server scripts	Whether the query results are grouped by this query result column.
PROPERTY	Column.label	string (read-only)	Client and server scripts	The label for the column.
PROPERTY	Column.type	string (read-only)	Client and server scripts	The return type of the formula used to create the query result column.

Component Object Members

The following members are available for a query.Component object.

Return Type/Value Type

Supported Script Types

Component.autoJoin(options)

Client and server scripts

Creates a join relationship.

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Component.autoJoin(options)	query.Component	Client and server scripts	Creates a join relationship. After you create the initial query definition, use Query.autoJoin(options) to create your first join. Then use this method to create each subsequent join. This method selects the correct join type automatically based on the record types that are being joined.
METHOD	Component.createColumn(options)	query.Column	Client and server scripts	Creates a query result column based on the component. Use this method to create columns based on the join relationships created with Query.autoJoin(options) and Component.autoJoin(options).
METHOD	Component.createCondition(options)	query.Condition	Client and server scripts	Creates a condition (filter column) based on the component. Use this method to create conditions based on the join relationships created with Query.autoJoin(options) and Component.autoJoin(options).

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**Component.createSort(options)**	query.Sort	Client and server scripts	Creates a sort based on the component. Use this method to create sorts based on the join relationships created with Query.autoJoin(options) and Component.autoJoin(options).
METHOD	**Component.join(options)**	query.Component	Client and server scripts	Creates a join relationship. This method is an alias to Component.autoJoin(options). After you create the initial query definition, use Query.autoJoin(options) to create your first join. Then use this method, or Component.autoJoin(options), to create each subsequent join.
METHOD	**Component.joinFrom(options)**	query.Component	Client and server scripts	Creates an explicit directional join relationship from another component to this component (an inverse join). This method sets the Component.source property on the returned query.Component object. After you create the initial query definition, use this method to create explicit directional joins from other components to this component.
METHOD	**Component.joinTo(options)**	query.Component	Client and server scripts	Creates an explicit directional join relationship to another component from this component (a join). You can use this method to specify the target of the join when a field can join multiple query types. This method sets the Component.target property on the returned query.Component object. After you create the initial query definition, use this method to create explicit directional joins to other components from this component.
PROPERTY	**Component.child**	Object (read-only)	Client and server scripts	The child components of the component. This property holds an object of key-value pairs. Each key is the name of a child component. Each value is the corresponding child query.Component object.
PROPERTY	**Component.parent**	string (read-only)	Client and server scripts	The parent query.Component object of the component.
PROPERTY	**Component.source**	string (read-only)	Client and server scripts	The source query type of the component. The value of this property is set when Component.joinFrom(options) is called to perform an explicit directional join from another component.
PROPERTY	**Component.target**	string (read-only)	Client and server scripts	The target query type of the component. The value of this property is set when Component.joinTo(options) is called to perform an explicit directional join to another component.
PROPERTY	**Component.type**	string (read-only)	Client and server scripts	The query type of the component.

Condition Object Members

The following members are available for a query.Condition object.

Return Type/Value Type

Supported Script Types

Condition.aggregate

string (read-only)

Client and server scripts

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Condition.aggregate**	string (read-only)	Client and server scripts	An aggregate function that is performed on the condition. An aggregate function performs a calculation on the condition values and returns a single value.
PROPERTY	**Condition.children**	query.Condition[] (read-only)	Client and server scripts	An array of child conditions used to create the parent condition.
PROPERTY	**Condition.component**	query.Component (read-only)	Client and server scripts	A reference to the query.Component object to which this condition belongs.
PROPERTY	**Condition.fieldId**	string (read-only)	Client and server scripts	The name of the field that is used in the condition.
PROPERTY	**Condition.formula**	string (read-only)	Client and server scripts	The formula used to create the condition.
PROPERTY	**Condition.operator**	string (read-only)	Client and server scripts	The name of the operator used to create the condition.
PROPERTY	**Condition.type**	string (read-only)	Client and server scripts	The return type of the formula used to create the condition.
PROPERTY	**Condition.values**	string | number | | <string | number | > (read-only)	Client and server scripts	Value or an array of values used by an operator to create the condition.

Page Object Members

The following members are available for a query.Page object.

Return Type/Value Type

Supported Script Types

query.ResultSet (read-only)

Client and server scripts

The query results contained in this page.

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Page.data**	query.ResultSet (read-only)	Client and server scripts	The query results contained in this page.
PROPERTY	**Page.isFirst**	boolean (read-only)	Client and server scripts	Whether this page is the first of the paged query results.
PROPERTY	**Page.isLast**	boolean (read-only)	Client and server scripts	Whether this page is the last of the paged query results.
PROPERTY	**Page.pagedData**	query.PagedData (read-only)	Client and server scripts	The set of paged query results that this page is from.
PROPERTY	**Page.pageRange**	query.PageRange (read-only)	Client and server scripts	The range of query results for this page.

PagedData Object Members

The following members are available for a query.PagedData object.

Return Type/Value Type

Supported Script Types

PagedData.iterator()

Client and server scripts

Standard SuiteScript 2.0 object for iterating through results.

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**PagedData.iterator()**	Iterator object	Client and server scripts	Standard SuiteScript 2.0 object for iterating through results.
METHOD	**PagedData.fetch(options)**	query.Page	Client and server scripts	Retrieves a page in the set of pages included in the PagedData object.
METHOD	**PagedData.fetch.promise(options)**	Promise Object	Client and server scripts	Asynchronously retrieves a page in the set of pages included in the PagedData object.
PROPERTY	**PagedData.count**	number (read-only)	Client and server scripts	The total number of paged query results.
PROPERTY	**PagedData.pageRanges**	query.PageRange[]	Client and server scripts	An array of page ranges for the set of paged query results.
PROPERTY	**PagedData.pageSize**	number (read-only)	Client and server scripts	The number of query result rows per page.

PageRange Object Members

The following members are available for a query.PageRange object.

Return Type/Value Type

Supported Script Types

number (read-only)

Client and server scripts

The index for this page range.

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**PageRange.index**	number (read-only)	Client and server scripts	The index for this page range.
PROPERTY	**PageRange.size**	number (read-only)	Client and server scripts	The number of query result rows in this page range.

Period Object Members

The following members are available for a query.Period object.

Return Type/Value Type

Supported Script Types

Period.adjustment

string (read-only)

Client and server scripts

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Period.adjustment**	string (read-only)	Client and server scripts	The adjustment of the period. This property uses values from the query.PeriodAdjustment enum.
PROPERTY	**Period.code**	string (read-only)	Client and server scripts	The code of the period. This property uses values from the query.PeriodCode enum.
PROPERTY	**Period.type**	string (read-only)	Client and server scripts	The type of the period. This property uses values from the query.PeriodType enum.

Query Object Members

The following members are available for a query.Query object.

Return Type/Value Type

Supported Script Types

Query.and(conditions)

Client and server scripts

Creates a new condition (a query.Condition object) that corresponds to a logical conjunction (AND) of the arguments passed to the method. The arguments must be one or more query.Condition objects.

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**Query.and(conditions)**	query.Condition	Client and server scripts	Creates a new condition (a query.Condition object) that corresponds to a logical conjunction (AND) of the arguments passed to the method. The arguments must be one or more query.Condition objects.
METHOD	**Query.autoJoin(options)**	query.Component	Client and server scripts	Creates a join relationship. After you create the initial query definition, use this method to create your first join or subsequent joins from the root component of the query. This method selects the correct join type automatically based on the record types that are being joined.
METHOD	**Query.createColumn(options)**	query.Column	Client and server scripts	Creates a query result column based on the query.Query object. Use this method to create columns on the initial query definition created with query.create(options).
METHOD	**Query.createCondition(options)**	query.Condition	Client and server scripts	Creates a condition (filter column) based on the query.Query object. Use this method to create conditions on the initial query definition created with query.create(options).
METHOD	**Query.createSort(options)**	query.Sort	Client and server scripts	Creates a sort based on the query.Query object. The query.Sort object describes a sort that is placed on a particular query result column or condition.
METHOD	**Query.join(options)**	query.Component	Client and server scripts	Creates a join relationship. This method is an alias to Query.autoJoin(options). After you create the initial query definition, use this method, or Query.autoJoin(options), to create your first join.
METHOD	**Query.joinFrom(options)**	query.Component	Client and server scripts	Creates an explicit directional join relationship from another component to the root component of the search definition (an inverse join). This method sets the Component.source property on the returned query.Component object. After you create the initial query definition, use this method to create your first join as an explicit directional join from another component to this component.
METHOD	**Query.joinTo(options)**	query.Component	Client and server scripts	Creates an explicit directional join relationship to another component from this component (a forward join). You can use this method to specify the target of the join when a field can join multiple query types. This method sets the Component.target property on the returned query.Component object. After you create the initial query definition, use this method to create your first join as an explicit directional join to another component from this component.
METHOD	**Query.not(condition)**	query.Condition	Client and server scripts	Creates a new condition (a query.Condition object) that corresponds to a logical negation (NOT) of the argument passed to the method. The argument must be a query.Condition object.
METHOD	**Query.or(conditions)**	query.Condition	Client and server scripts	Creates a new condition (a query.Condition object) that corresponds to a logical disjunction (OR) of the arguments passed to the method. The arguments must be one or more query.Condition objects.
METHOD	**Query.run(options)**	query.ResultSet	Client and server scripts	Executes the query and returns the query result set.
METHOD	**Query.run.promise()**	query.ResultSet	Client and server scripts	Executes the query asynchronously and returns the query result set.
METHOD	**Query.runPaged(options)**	query.PagedData	Client and server scripts	Executes the query and returns a set of paged results.
METHOD	**Query.runPaged.promise(options)**	query.PagedData	Client and server scripts	Executes the query asynchronously and returns a set of paged results.
METHOD	**Query.toSuiteQL()**	query.SuiteQL	Client and server scripts	Converts this query.Query object to its corresponding SuiteQL representation.
PROPERTY	**Query.child**	Object (read-only)	Client and server scripts	A reference to children of the root component of the query definition. The value of this property is an object of key-value pairs. Each key is the name of a child component. Each respective value is the corresponding query.Component object.
PROPERTY	**Query.columns**	query.Column[]	Client and server scripts	An array of query result columns returned from the query. Before you perform the query, you must assign all created columns as values to this property.
PROPERTY	**Query.condition**	query.Condition object	Client and server scripts	The parent condition that narrows the query results. Before you perform the query, you must assign your simple or complex conditions to this property.
PROPERTY	**Query.id**	number (read-only)	Client and server scripts	The ID of the query definition. This property has a value only for existing queries that are loaded using query.load(options). If you create a query using query.create(options) but do not save it, this property is null.
PROPERTY	**Query.name**	string (read-only)	Client and server scripts	The name of the query definition. This property has a value only for existing queries that are loaded using query.load(options). If you create a query using query.create(options) but do not save it, this property is null.

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Query.root**	query.Component (read-only)	Client and server scripts	The root component of the query definition.
PROPERTY	**Query.sort**	query.Column[] (read-only)	Client and server scripts	An array of query result columns used for sorting.
PROPERTY	**Query.type**	string (read-only)	Client and server scripts	The query type of the initial query definition.

RelativeDate Object Members

The following members are available for a query.RelativeDate object.

Return Type/Value Type

Supported Script Types

RelativeDate.dateId

string (read-only)

Client and server scripts

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**RelativeDate.dateId**	string (read-only)	Client and server scripts	The ID of the relative date.
PROPERTY	**RelativeDate.end**	Object (read-only)	Client and server scripts	The end point of the relative date.
PROPERTY	**RelativeDate.interval**	Object (read-only)	Client and server scripts	The interval of the relative date (from the RelativeDate.start point to the RelativeDate.end point).
PROPERTY	**RelativeDate.isRange**	boolean (read-only)	Client and server scripts	Whether the relative date represents a range of dates or a specific moment in time.
PROPERTY	**RelativeDate.start**	Object (read-only)	Client and server scripts	The start point of the relative date.
PROPERTY	**RelativeDate.value**	number (read-only)	Client and server scripts	The value of the relative date.

Result Object Members

The following members are available for a query.Result object.

Return Type/Value Type

Supported Script Types

Client and server scripts

Returns the query result as an object with the columns mapped to the result values.

Result.getValue(options)

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**Result.asMap()**	Object	Client and server scripts	Returns the query result as an object with the columns mapped to the result values.
METHOD	**Result.getValue(options)**	<boolean | number | string | Date | null> (read-only)	Client and server scripts	Gets the value at a given index in Result.values.
PROPERTY	**Result.values**	Array <boolean | number | string | Date | null> (read-only)	Client and server scripts	The result values.

ResultSet Object Members

The following members are available for a query.ResultSet object.

Return Type/Value Type

Supported Script Types

ResultSet.asMappedResults()

Client and server scripts

Returns a query result set as an array of mapped results.

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**ResultSet.asMappedResults()**	Object[]	Client and server scripts	Returns a query result set as an array of mapped results.
METHOD	**ResultSet.iterator()**	Iterator object	Client and server scripts	Standard SuiteScript 2.0 object for iterating through results.
PROPERTY	**ResultSet.columns**	query.Column[] (read-only)	Client and server scripts	An array of query result column references.

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**ResultSet.results**	query.Result[] (read-only)	Client and server scripts	An array of query.Result objects.
PROPERTY	**ResultSet.types**	string[] (read-only)	Client and server scripts	An array of the return types for ResultSet.results.

Sort Object Members

The following members are available for a query.Sort object.

Return Type/Value Type

Supported Script Types

Client and server scripts

Whether the sort direction is ascending.

Sort.caseSensitive

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Sort.ascending**	boolean	Client and server scripts	Whether the sort direction is ascending.
PROPERTY	**Sort.caseSensitive**	boolean	Client and server scripts	Whether the sort is case sensitive. If a sort is case sensitive (and the sort direction is ascending), rows with column values that start with uppercase letters are listed before rows with column values that start with lowercase letters. If a sort is not case sensitive, uppercase and lowercase letters are treated the same.
PROPERTY	**Sort.column**	query.Column (read-only)	Client and server scripts	The query result column that the query results are sorted by.
PROPERTY	**Sort.locale**	string	Client and server scripts	The locale to use for the sort. A locale represents a combination of language and region, and it can affect how certain values (such as strings) are sorted.
PROPERTY	**Sort.nullsLast**	boolean	Client and server scripts	Whether query results with null values are listed at the end of the query results.

SuiteQL Object Members

The following members are available for a query.SuiteQL object.

Return Type/Value Type

Supported Script Types

Client and server scripts

Runs the SuiteQL query and returns the query results.

SuiteQL.runPaged(options)

MEMBER TYPE	NAME	RETURN TYPE/VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**SuiteQL.run()**	query.ResultSet	Client and server scripts	Runs the SuiteQL query and returns the query results.
METHOD	**SuiteQL.runPaged(options)**	query.PagedData	Client and server scripts	Runs the SuiteQL query as a paged query and returns the paged query results.
PROPERTY	**SuiteQL.columns**	query.Column[]	Client and server scripts	Describes the result columns to be returned from the query.
PROPERTY	**SuiteQL.params**	<string | number | > (read-only)	Client and server scripts	Contains the parameters for the query.
PROPERTY	**SuiteQL.query**	string (read-only)	Client and server scripts	Holds the string representation of the query.
PROPERTY	**SuiteQL.type**	string (read-only)	Client and server scripts	Describes the type of the query. This property uses values from the query.Type enum.

▶ Official NetSuite Script Samples (6 Sample(s))

📄 Sample 1: Create a Query for Customer Records and Run It as a Non-Paged Query

```
/**
 * @NApiVersion 2.1
 */
require(['N/query'], query =&gt; {
    // Create a query definition for customer records
    let myCustomerQuery = query.create({
        type: query.Type.CUSTOMER
    });

    // Join the original query definition based on the salesrep field. In a customer
    // record, the salesrep field contains a reference to an employee record. When you
    // join based on this field, you are joining the query definition with the employee
    // query type, and you can access the fields of the joined employee record in
    // your query.
    let mySalesRepJoin = myCustomerQuery.autoJoin({
        fieldId: 'salesrep'
    });

    // Join the joined query definition based on the location field. In an employee
    // record, the location field contains a reference to a location record.
    let myLocationJoin = mySalesRepJoin.autoJoin({
        fieldId: 'location'
    });

    // Create conditions for the query
    let firstCondition = myCustomerQuery.createCondition({
        fieldId: 'id',
        operator: query.Operator.EQUAL,
        values: 107
    });
    let secondCondition = myCustomerQuery.createCondition({
        fieldId: 'id',
        operator: query.Operator.EQUAL,
        values: 2647
    });
    let thirdCondition = mySalesRepJoin.createCondition({
        fieldId: 'email',
        operator: query.Operator.START_WITH_NOT,
        values: 'example'
    });

    // Combine conditions using and() and or() operator methods. In this example,
    // the combined condition states that the id field of the customer record must
    // have a value of either 107 or 2647, and the email field of the employee
    // record (the record that is referenced in the salesrep field of the customer
    // record) must not start with 'example'.
    myCustomerQuery.condition = myCustomerQuery.and(
        thirdCondition, myCustomerQuery.or(firstCondition, secondCondition)
    );

    // Create query columns
    myCustomerQuery.columns = [
        myCustomerQuery.createColumn({
            fieldId: 'entityid'
        }),
        myCustomerQuery.createColumn({
            fieldId: 'id'
        }),
        mySalesRepJoin.createColumn({
            fieldId: 'entityid'
        }),
        mySalesRepJoin.createColumn({
            fieldId: 'email'
        }),
        mySalesRepJoin.createColumn({
            fieldId: 'hiredate'
        }),
        myLocationJoin.createColumn({
            fieldId: 'name'
        })
    ];

    // Sort the query results based on query columns
    myCustomerQuery.sort = [
        myCustomerQuery.createSort({
            column: myCustomerQuery.columns[3]
        }),
        myCustomerQuery.createSort({
            column: myCustomerQuery.columns[0],
            ascending: false
        })
    ];

    // Run the query
    let resultSet = myCustomerQuery.run();

    // Retrieve and log the results
    let results = resultSet.results;
```

```
    for (let i = results.length - 1; i &gt;= 0; i--)
        log.debug(results[i].values);
    log.debug(resultSet.types);
});
```

📄 Sample 2: Create a Query for Transaction Records and Run It as a Paged Query

```
/**
 * @NApiVersion 2.1
 */
require(['N/query'], query =&gt; {
    // Create a query definition for transaction records
    let myTransactionQuery = query.create({
        type: query.Type.TRANSACTION
    });

    // Join the original query definition based on the employee field. In a transaction
    // record, the employee field contains a reference to an employee record. When you
    // join based on this field, you are joining the query definition with the employee
    // query type, and you can access the fields of the joined employee record in
    // your query.
    let myEmployeeJoin = myTransactionQuery.autoJoin({
        fieldId: 'employee'
    });

    // Create a condition for the transaction query
    let transactionCondition = myTransactionQuery.createCondition({
        fieldId: 'isreversal',
        operator: query.Operator.IS,
        values: true
    });
    myTransactionQuery.condition = transactionCondition;

    // Create a query column
    myTransactionQuery.columns = [
        myEmployeeJoin.createColumn({
            fieldId: 'subsidiary'
        })
    ];

    // Sort the query results based on a query column
    myTransactionQuery.sort = [
        myTransactionQuery.createSort({
            column: myTransactionQuery.columns[0],
            ascending: false
        })
    ];

    // Run the query as a paged query with 10 results per page
    let results = myTransactionQuery.runPaged({
        pageSize: 10
    });

    log.debug(results.pageRanges.length);
    log.debug(results.count);

    // Retrieve the query results using an iterator
    let iterator = results.iterator();
    iterator.each(function(result) {
        let page = result.value;
        log.debug(page.pageRange.size);
        return true;
    })

    // Alternatively, retrieve the query results by looping through
    // each result
    for (let i = 0; i &lt; results.pageRanges.length; i++)  {
        let page = results.fetch(i);
        log.debug(page.pageRange.size);
    }
});
```

📄 Sample 3: Convert a Query to a SuiteQL and Run It

```
/**
 * @NApiVersion 2.x
 */
require(['N/query'], function(query) {
    var myCustomerQuery = query.create({
        type: query.Type.CUSTOMER
    });

    myCustomerQuery.columns = [
        myCustomerQuery.createColumn({
            fieldId: 'entityid'
        }),
        myCustomerQuery.createColumn({
            fieldId: 'email'
        })
    ];

    myCustomerQuery.condition = myCustomerQuery.createCondition({
        fieldId: 'isperson',
        operator: query.Operator.IS,
        values: [true]
    });

    var mySQLCustomerQuery = myCustomerQuery.toSuiteQL();

    var results = mySQLCustomerQuery.run();
});
```

📄 Sample 4: Run an Arbitrary SuiteQL Query

```
/**
 * @NApiVersion 2.x
 */
require(['N/query'], function(query) {
   var sql =
        "SELECT " +
        "  scriptDeployment.primarykey, scriptexecutioncontextmap.executioncontext " +
        " FROM " +
        "  scriptDeployment, scriptexecutioncontextmap " +
        " WHERE " +
        "  scriptexecutioncontextmap.scriptrecord = scriptDeployment.primarykey " +
        " AND " +
        "  scriptexecutioncontextmap.executioncontext IN ('WEBSTORE', 'WEBAPPLICATION')";

    var resultIterator = query.runSuiteQLPaged({
        query: sql,
        pageSize: 10
    }).iterator();

    resultIterator.each(function(page) {
        var pageIterator = page.value.data.iterator();
        pageIterator.each(function(row) {
            log.debug('ID: ' + row.value.getValue(0) + ', Context: ' + row.value.getValue(1));
            return true;
        });
        return true;
    });
});
```

📄 Sample 5: Create a Query for a Custom Field

```
/*
 * @NApiVersion 2.x
 */
require(['N/query'], function(query) {
    var customFieldIdQuery = query.create({
        type: query.Type.CUSTOM_FIELD
    });
    customFieldIdQuery.columns = [
        customFieldIdQuery.createColumn({
            fieldId: 'internalid'
        })
    ];
    customFieldIdQuery.condition = customFieldIdQuery.createCondition({
        fieldId: 'scriptid',
        operator: query.Operator.IS,
        values: 'custrecord_my_custom_field'
    });

    var results = customFieldIdQuery.run().asMappedResults();
    var customFieldInternalId = results[0].internalid;
    log.debug({
        title: 'Internal ID of the custom field is ',
        details: customFieldInternalId
    });
});
```

📄 Sample 6: Create a Query Using a Specific Record Field

```
/**
* @NApiVersion 2.x
*/
require(['N/query'], function(query) {
    var mfgComponent = query.create({
        type: query.Type.MANUFACTURING_COMPONENT
    });

    var mfgOperation = mfgComponent.autoJoin({
        fieldId: 'operationdisplaytext'
    });

    mfgComponent.columns = [
        mfgComponent.createColumn({
            fieldId: 'operationdisplaytext'
        }),
        mfgComponent.createColumn({
            fieldId: 'item'
        }),
        mfgOperation.createColumn({
            fieldId: 'operationsequence'
        })
    ];

    mfgComponent.condition = mfgComponent.and(
        mfgComponent.createCondition({
            fieldId: 'transaction',
            operator: query.Operator.ANY_OF,
            values: &lt;transactionId&gt;
        }),
        mfgOperation.createCondition({
            fieldId: 'operationsequence',
            operator: query.Operator.EQUAL,
            values: 20
        })
    );

    var results = mfgComponent.run();
});
```

34. N/record Module

Load the N/record module to work with NetSuite records.

`define(['N/record'], (record) => { ... })` ✓ 6 Code Sample(s) Scripts: Client and server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/record'], (record) => { // Access methods and properties on record });
```

Use the N/record module to work with NetSuite records. You can use this module to create, delete, copy, load, or make changes to a record.

For help finding a record's internal ID, see Finding Internal IDs of Records.

N/record Module Members

Return Type / Value Type

Supported Script Types

Client and server scripts

Encapsulates a column of a sublist on a standard or custom record.

Client and server scripts

Encapsulates a body or sublist field on a standard or custom record.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	record.Column	Object	Client and server scripts	Encapsulates a column of a sublist on a standard or custom record.
OBJECT	record.Field	Object	Client and server scripts	Encapsulates a body or sublist field on a standard or custom record.
OBJECT	record.Macro	Object	Client and server scripts	Encapsulates a NetSuite record macro.
OBJECT	Plain JavaScript Object	Object	Client and server scripts	A plain JavaScript object of record macros available for a record type. This object is returned by Record.getMacros().
OBJECT	record.Record	Object	Client and server scripts	Encapsulates a NetSuite record.
OBJECT	record.Sublist	Object	Client and server scripts	Encapsulates a sublist on a standard or custom record.
METHOD	record.attach(options)	void	Client and server scripts	Attaches a record to another record.
METHOD	record.attach.promise(options)	Promise	Client scripts	Attaches a record asynchronously to another record.
METHOD	record.copy(options)	record.Record	Client and server scripts	Creates a new record by copying an existing record in NetSuite.
METHOD	record.copy.promise(options)	Promise	Client scripts	Creates a new record asynchronously by copying an existing record in NetSuite.
METHOD	record.create(options)	record.Record	Client and server scripts	Creates a new record.
METHOD	record.create.promise(options)	Promise	Client scripts	Creates a new record asynchronously.
METHOD	record.delete(options)	number	Client and server scripts	Deletes a record.
METHOD	record.delete.promise(options)	Promise	Client scripts	Deletes a record asynchronously.
METHOD	record.detach(options)	void	Client and server scripts	Detaches a record from another record.
METHOD	record.detach.promise(options)	Promise	Client scripts	Detaches a record from another record asynchronously.
METHOD	record.load(options)	Object	Client and server scripts	Loads an existing record.
METHOD	record.load.promise(options)	Promise	Client scripts	Loads an existing record asynchronously.
METHOD	record.submitFields(options)	Object	Client and server scripts	Updates and submits one or more body fields on an existing record in NetSuite, and returns the internal ID of the parent record.
METHOD	record.submitFields.promise(options)	Promise	Client scripts	Updates and submits one or more body fields asynchronously on an existing record in NetSuite, and returns the internal ID of the parent record.
METHOD	record.transform(options)	record.Record	Client and server scripts	Transforms a record from one type into another, using data from an existing record.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	record.transform.promise(options)	Promise	Client scripts	Transforms a record from one type into another asynchronously, using data from an existing record.
ENUM	record.Type	enum	Client and server scripts	Holds the string values for supported record types. Use this enum to set the value of the Record.type property in cases where you are working with an instance of a standard NetSuite record type.

Column Object Members

Return Type / Value Type

Supported Script Types

string (read-only)

Client and server scripts

Returns the internal ID of the column.

Column.isDisabled

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Column.id	string (read-only)	Client and server scripts	Returns the internal ID of the column.
PROPERTY	Column.isDisabled	boolean	Client and server scripts	Indicates whether the column is disabled.
PROPERTY	Column.isDisplay	boolean (read-only)	Client and server scripts	Indicates whether the column is displayed.
PROPERTY	Column.isMandatory	boolean	Client and server scripts	Indicates whether the column is required.
PROPERTY	Column.isSortable	boolean (read-only)	Client and server scripts	Indicates whether the column is sortable.
PROPERTY	Column.label	string (read-only)	Client and server scripts	Returns the UI label for the column.
PROPERTY	Column.sublistId	string (read-only)	Client and server scripts	Returns the internal ID of the standard or custom sublist that contains the column.
PROPERTY	Column.type	string (read-only)	Client and server scripts	Returns the column type.

Field Object Members

Return Type / Value Type

Supported Script Types

Field.getSelectOptions(options)

Client and server scripts

Returns an array of available options on a standard or custom select, multiselect, or radio field as key-value pairs. Only the first 1,000 available options are returned.

string (read-only)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Field.getSelectOptions(options)	array	Client and server scripts	Returns an array of available options on a standard or custom select, multiselect, or radio field as key-value pairs. Only the first 1,000 available options are returned.
PROPERTY	Field.label	string (read-only)	Client and server scripts	Returns the UI label for a standard or custom field body or sublist field.
PROPERTY	Field.id	string (read-only)	Client and server scripts	Returns the internal ID of a standard or custom body or sublist field.
PROPERTY	Field.type	string (read-only)	Client and server scripts	Returns the type of a body or sublist field.
PROPERTY	Field.isMandatory	boolean	Client and server scripts	Returns true if the standard or custom field is required on the record form, or false otherwise.
PROPERTY	Field.sublistId	string (read-only)	Client and server scripts	Returns the ID of the sublist associated with the specified sublist field. This property is available only when working with a record in dynamic mode.
PROPERTY	Field.isDisplay	boolean	Client and server scripts	Returns true if the field is visible on the record form, or false if it is not. This property is available only when working with a record in dynamic mode.

Macro Object Members

The following members are called on the record.Macro object. For information about record macros, see Overview of Record Action and Macro APIs.

Return Type / Value Type

Supported Script Types

Macro.execute(options)

Client and server scripts

Performs a macro operation and returns its result in an object.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Macro.execute(options)	Object	Client and server scripts	Performs a macro operation and returns its result in an object.
METHOD	Macro.execute.promise(options)	Promise	Client scripts	Performs a macro operation asynchronously.
METHOD	Macro(options)	Object	Client and server scripts	Performs a macro operation and returns its result in an object.
METHOD	Macro.promise(options)	Promise	Client scripts	Performs a macro operation asynchronously.
PROPERTY	Macro.id	string	Client and server scripts	The ID of the macro. For a list of macro IDs, see Supported Record Macros
PROPERTY	Macro.label	string	Client and server scripts	The macro label.
PROPERTY	Macro.description	string	Client and server scripts	The macro description.
PROPERTY	Macro.attributes	Object	Client and server scripts	The macro defined attributes.

Record Object Members

The following members are called on the record.Record object.

Return Type / Value Type

Supported Script Types

Record.cancelLine(options)

Client and server scripts

Cancels the currently selected line on a sublist.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Record.cancelLine(options)	record.Record	Client and server scripts	Cancels the currently selected line on a sublist.
METHOD	Record.commitLine(options)	record.Record	Client and server scripts	Commits the currently selected line on a sublist.
METHOD	Record.executeMacro(options)	Object	Client and server scripts	Performs macro operation and returns its result in a plain JavaScript object.
METHOD	Record.getMacros()	Object	Client and server scripts	Provides a plain JavaScript object that contains macro objects defined for a record type, indexed by the Macro ID.
METHOD	Record.findMatrixSublistLineWithValue(options)	number	Client and server scripts	Returns the line number of the first instance where a specified value is found in a specified column of the matrix.
METHOD	Record.findSublistLineWithValue(options)	number	Client and server scripts	Returns the line number for the first occurrence of a field value in a sublist.
METHOD	Record.getCurrentMatrixSublistValue(options)	number | Date | string | array | boolean	Client and server scripts	Gets the value for the currently selected line in the matrix.
METHOD	Record.getCurrentSublistField(options)	record.Field	Client and server scripts	Returns a field object from a sublist.
METHOD	Record.getCurrentSublistIndex(options)	number	Client and server scripts	Returns the line number of the currently selected line.
METHOD	Record.getCurrentSublistSubrecord(options)	record.Record	Client and server scripts	Gets the subrecord for the associated sublist field on the current line.
METHOD	Record.getCurrentSublistText(options)	string	Client and server scripts	Returns a text representation of the field value in the currently selected line.
METHOD	Record.getCurrentSublistValue(options)	number | Date | string | array | boolean	Client and server scripts	Returns the value of a sublist field on the currently selected sublist line.
METHOD	Record.getField(options)	record.Field	Client and server scripts	Returns a field object from a record.
METHOD	Record.getFields()	string[]	Client and server scripts	Returns the body field names (internal ids) of all the fields in the record, including machine header field and matrix header fields.
METHOD	Record.getLineCount(options)	number	Client and server scripts	Returns the number of lines in a sublist.
METHOD	Record.getMacro(options)	record.Macro	Client and server scripts	Provides a macro to execute.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Record.getMacros()	Object	Client and server scripts	Provides a plain JavaScript object that contains macro objects defined for a record type, indexed by the Macro ID.
METHOD	Record.getMatrixHeaderCount(options)	number	Client and server scripts	Returns the number of columns for the specified matrix.
METHOD	Record.getMatrixHeaderField(options)	record.Field	Client and server scripts	Gets the field for the specified header in the matrix.
METHOD	Record.getMatrixHeaderValue(options)	number | Date | string | array | boolean	Client and server scripts	Gets the value for the associated header in the matrix.
METHOD	Record.getMatrixSublistField(options)	record.Field	Client and server scripts	Gets the field for the specified sublist in the matrix.
METHOD	Record.getMatrixSublistValue(options)	number | Date | string | array | boolean	Client and server scripts	Gets the value for the associated field in the matrix.
METHOD	Record.getSublist(options)	record.Sublist	Client and server scripts	Returns the specified sublist.
METHOD	Record.getSublists()	string[]	Client and server scripts	Returns all the names of all the sublists.
METHOD	Record.getSublistField(options)	record.Field	Client and server scripts	Returns a field object from a sublist.
METHOD	Record.getSublistFields(options)	string[]	Client and server scripts	Returns all the field names in a sublist.
METHOD	Record.getSublistSubrecord(options)	record.Record	Client and server scripts	Gets the subrecord associated with a sublist field. (standard mode only)
METHOD	Record.getSublistText(options)	string	Client and server scripts	Returns the value of a sublist field in a text representation.
METHOD	Record.getSublistValue(options)	number | Date | string | array | boolean	Client and server scripts	Returns the value of a sublist field.
METHOD	Record.getSubrecord(options)	record.Record	Client and server scripts	Gets the subrecord for the associated field.
METHOD	Record.getText(options)	string	Client and server scripts	Returns the text representation of a field value.
METHOD	Record.getValue(options)	number | Date | string | array | boolean	Client and server scripts	Returns the value of a field. Not to be used on custom password fields. Use crypto.checkPasswordField(options) instead.
METHOD	Record.hasCurrentSublistSubrecord(options)	boolean	Client and server scripts	Returns a value indicating whether the associated sublist field has a subrecord on the current line.
METHOD	Record.hasSublistSubrecord(options)	boolean	Client and server scripts	Returns a value indicating whether the associated sublist field contains a subrecord.
METHOD	Record.hasSubrecord(options)	boolean	Client and server scripts	Returns a value indicating whether the field contains a subrecord.
METHOD	Record.insertLine(options)	record.Record	Client and server scripts	Inserts a sublist line.
METHOD				
METHOD	Record.removeCurrentSublistSubrecord(options)	record.Record	Client and server scripts	Removes the subrecord for the associated sublist field on the current line.
METHOD	Record.removeLine(options)	record.Record	Client and server scripts	Removes a sublist line.
METHOD	Record.removeSublistSubrecord(options)	record.Record	Client and server scripts	Removes the subrecord for the associated sublist field. (standard mode only)
METHOD	Record.removeSubrecord(options)	record.Record	Client and server scripts	Removes the subrecord for the associated field.
METHOD	Record.save(options)	number	Client and server scripts	Submits a new record or saves edits to an existing record. This method is not available to subrecords.
METHOD	Record.save.promise(options)	number	Client scripts	Submits a new record asynchronously or saves edits to an existing record asynchronously. This method is not available to subrecords.
METHOD	Record.selectLine(options)	record.Record	Client and server scripts	Selects an existing line in a sublist.
METHOD	Record.selectNewLine(options)	record.Record	Client and server scripts	Selects a new line at the end of a sublist.
METHOD	Record.setCurrentMatrixSublistValue(options)	record.Record	Client and server scripts	Sets the value for the line currently selected in the matrix.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Record.setCurrentSublistText(options)	record.Record	Client and server scripts	Sets the value for the field in the currently selected line by a text representation.
METHOD	Record.setCurrentSublistValue(options)	record.Record	Client and server scripts	Sets the value for the field in the currently selected line.
METHOD	Record.setMatrixHeaderValue(options)	record.Record	Client and server scripts	Sets the value for the associated header in the matrix.
METHOD	Record.setMatrixSublistValue(options)	record.Record	Client and server scripts	Sets the value for the associated field in the matrix.
METHOD	Record.setSublistText(options)	record.Record	Client and server scripts	Sets the value of a sublist field by a text representation. (standard mode only)
METHOD	Record.setSublistValue(options)	record.Record	Client and server scripts	Sets the value of a sublist field. (standard mode only)
METHOD	Record.setText(options)	record.Record	Client and server scripts	Sets the value of the field by a text representation.
METHOD	Record.setValue(options)	record.Record	Client and server scripts	Sets the value of a field.
PROPERTY	Record.id	number (read-only)	Client and server scripts	The internal ID of a specific record. This property is not available to subrecords.
PROPERTY	Record.isDynamic	boolean (read-only)	Client and server scripts	Indicates whether the record is in dynamic or standard mode.
PROPERTY	Record.type	string (read-only)	Client and server scripts	The record type. This property is not available to subrecords.

Sublist Object Members

Return Type / Value Type

Supported Script Types

Sublist.getColumn(options)

Client and server scripts

Returns a column in the sublist.

string (read-only)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Sublist.getColumn(options)	record.Column	Client and server scripts	Returns a column in the sublist.
PROPERTY	Sublist.id	string (read-only)	Client and server scripts	Returns the internal ID of the sublist.
PROPERTY	Sublist.isChanged	boolean (read-only)	Client and server scripts	Indicates whether the sublist has changed on the record form.
PROPERTY	Sublist.isDisplay	boolean (read-only)	Client and server scripts	Indicates whether the sublist is displayed on the record form.
PROPERTY	Sublist.type	string (read-only)	Client and server scripts	Returns the sublist type.

N/record Default Values

You can specify record initialization parameters that default when creating, copying, loading, and transforming records. To enable this behavior, use the optional defaultValues parameter in the following APIs:

record.create(options)

record.copy(options)

record.transform(options)

record.load(options)

The following table lists initialization types that are available to certain SuiteScript-supported records and the values they can contain.

RECORD	INITIALIZATION TYPE	VALUES
ALL SUITESCRIPT-SUPPORTED RECORDS THAT SUPPORT FORM CUSTOMIZATION.	customform	<customformid>
ASSEMBLY BUILD	assemblyitem	<assemblyitemid>
CASH REFUND	entity	<entityid>
CASH SALE	entity	<entityid>
CHARGE RULE	entity	<entityid>
CHECK	entity	<entityid>

RECORD	INITIALIZATION TYPE	VALUES
CREDIT MEMO	entity	<entityid>
CUSTOMER PAYMENT	entity	<entityid>
CUSTOMER REFUND	entity	<entityid>
DEPOSIT	disablepaymentfilters	<disablepaymentfilters>
ESTIMATE	entity	<entityid>
EXPENSE REPORT	entity	<entityid>
INVOICE	entity	<entityid>
ITEM RECEIPT	entity	<entityid>
NON-INVENTORY PART	subtype	sale | resale | purchase
OPPORTUNITY	entity	<entityid>
OTHER CHARGE ITEM	subtype	sale | resale | purchase
PURCHASE ORDER	entity	<entityid>
RETURN AUTHORIZATION	entity	<entityid>
SALES ORDER	entity	<entityid>
SCRIPT DEPLOYMENT	script	<scriptid>
SERVICE	subtype	sale | resale | purchase
SUBSCRIPTION CHANGE ORDER		<scriptid> See Default Values.
TAX GROUP	nexuscountry	<countrycode> See Country Codes Used for Initialization Parameters.
TAX TYPE	country	<countrycode> See Country Codes Used for Initialization Parameters.
TOPIC	parenttopic	<parenttopicid>
VENDOR BILL	entity	<entityid>
VENDOR PAYMENT	entity	<entityid>
WORK ORDER	assemblyitem	<assemblyitemid>

Country Codes Used for Initialization Parameters

If you are scripting the Tax Group or Tax Type records, you can initialize the record to source all values related to a specific country. In your script, use the country code for the countrycodeid value, for example:

record.create('taxgroup', {nexuscountry: 'AR'}); Country Code

United Arab Emirates

Antigua and Barbuda

Bosnia and Herzegovina

Saint Barthélemy

COUNTRY CODE	COUNTRY NAME
AD	Andorra
AE	United Arab Emirates
AF	Afghanistan
AG	Antigua and Barbuda
AI	Anguilla
AL	Albania
AM	Armenia
AO	Angola
AQ	Antarctica
AR	Argentina
AS	American Samoa

COUNTRY CODE	COUNTRY NAME
AT	Austria
AU	Australia
AW	Aruba
AX	Åland Islands
AZ	Azerbaijan
BA	Bosnia and Herzegovina
BB	Barbados
BD	Bangladesh
BE	Belgium
BF	Burkina Faso
BG	Bulgaria
BH	Bahrain
BI	Burundi
BJ	Benin
BL	Saint Barthélemy
BM	Bermuda
BN	Brunei Darrussalam
BO	Bolivia (Plurinational State of)
BQ	Bonaire, Sint Eustatius and Saba
BR	Brazil
BS	Bahamas
BT	Bhutan
BV	Bouvet Island
BW	Botswana
BY	Belarus
BZ	Belize
CA	Canada
CC	Cocos (Keeling) Islands
CD	Congo (the Democratic Republic of the)
CF	Central African Republic
CG	Congo
CH	Switzerland
CI	Côte d'Ivoire
CK	Cook Islands
CL	Chile
CM	Cameroon
CN	China
CO	Colombia
CR	Costa Rica
CU	Cuba
CV	Cabo Verde
CW	Curacao
CX	Christmas Island

COUNTRY CODE	COUNTRY NAME
CY	Cyprus
CZ	Czechia
DE	Germany
DJ	Djibouti
DK	Denmark
DM	Dominica
DO	Dominican Republic
DZ	Algeria
EA	Ceuta and Melilla
EC	Ecuador
EE	Estonia
EG	Egypt
EH	Western Sahara
ER	Eritrea
ES	Spain
ET	Ethiopia
FI	Finland
FJ	Fiji
FK	Falkland Islands (Malvinas)
FM	Micronesia (Federated States of)
FO	Faroe Islands
FR	France
GA	Gabon
GB	United Kingdom
GD	Grenada
GE	Georgia
GF	French Guiana
GG	Guernsey
GH	Ghana
GI	Gibraltar
GL	Greenland
GM	Gambia
GN	Guinea
GP	Guadeloupe
GQ	Equatorial Guinea
GR	Greece
GS	South Georgia and the South Sandwich Islands
GT	Guatemala
GU	Guam
GW	Guinea-Bissau
GY	Guyana
HK	Hong Kong
HM	Heard Island and McDonald Islands

COUNTRY CODE	COUNTRY NAME
HN	Honduras
HR	Croatia
HT	Haiti
HU	Hungary
IC	Canary Islands
ID	Indonesia
IE	Ireland
IL	Israel
IM	Isle of Man
IN	India
IO	British Indian Ocean Territory
IQ	Iraq
IR	Iran (Islamic Republic of)
IS	Iceland
IT	Italy
JE	Jersey
JM	Jamaica
JO	Jordan
JP	Japan
KE	Kenya
KG	Kyrgyzstan
KH	Cambodia
KI	Kiribati
KM	Comoros
KN	Saint Kitts and Nevis
KP	Korea (the Democratic People's Republic of)
KR	Korea (the Republic of)
KW	Kuwait
KY	Cayman Islands
KZ	Kazakhstan
LA	Lao People's Democratic Republic
LB	Lebanon
LC	Saint Lucia
LI	Liechtenstein
LK	Sri Lanka
LR	Liberia
LS	Lesotho
LT	Lithuania
LU	Luxembourg
LV	Latvia
LY	Libya
MA	Morocco
MC	Monaco

COUNTRY CODE	COUNTRY NAME
MD	Moldova (the Republic of)
ME	Montenegro
MF	Saint Martin (French part)
MG	Madagascar
MH	Marshall Islands
MK	North Macedonia
ML	Mali
MM	Myanmar
MN	Mongolia
MO	Macao
MP	Northern Mariana Islands
MQ	Martinique
MR	Mauritania
MS	Montserrat
MT	Malta
MU	Mauritius
MV	Maldives
MW	Malawi
MX	Mexico
MY	Malaysia
MZ	Mozambique
NA	Namibia
NC	New Caledonia
NE	Niger
NF	Norfolk Island
NG	Nigeria
NI	Nicaragua
NL	Netherlands
NO	Norway
NP	Nepal
NR	Nauru
NU	Niue
NZ	New Zealand
OM	Oman
PA	Panama
PE	Peru
PF	French Polynesia
PG	Papua New Guinea
PH	Philippines
PK	Pakistan
PL	Poland
PM	Saint Pierre and Miquelon
PN	Pitcairn

COUNTRY CODE	COUNTRY NAME
PR	Puerto Rico
PS	Palestine, State of
PT	Portugal
PW	Palau
PY	Paraguay
QA	Qatar
RE	Réunion
RO	Romania
RS	Serbia
RU	Russian Federation
RW	Rwanda
SA	Saudi Arabia
SB	Solomon Islands
SC	Seychelles
SD	Sudan
SE	Sweden
SG	Singapore
SH	Saint Helena, Ascension and Tristan da Cunha
SI	Slovenia
SJ	Svalbard and Jan Mayen
SK	Slovakia
SL	Sierra Leone
SM	San Marino
SN	Senegal
SO	Somalia
SR	Suriname
SS	South Sudan
ST	Sao Tome and Principe
SV	El Salvador
SX	Sint Maarten (Dutch part)
SY	Syrian Arab Republic
SZ	Eswatini
TC	Turks and Caicos Islands
TD	Chad
TF	French Southern Territories
TG	Togo
TH	Thailand
TJ	Tajikistan
TK	Tokelau
TM	Turkmenistan
TN	Tunisia
TO	Tonga
TP	Timor-Leste

COUNTRY CODE	COUNTRY NAME
TR	Türkiye
TT	Trinidad and Tobago
TV	Tuvalu
TW	Taiwan (Province of China)
TZ	Tanzania, the United Republic of
UA	Ukraine
UG	Uganda
UM	United States Minor Outlying Islands
US	United States
UY	Uruguay
UZ	Uzbekistan
VA	Holy See
VC	Saint Vincent and the Grenadines
VE	Venezuela (Bolivarian Republic of)
VG	Virgin Islands (British)
VI	Virgin Islands (U.S.)
VN	Viet Nam
VU	Vanuatu
WF	Wallis and Futuna Islands
WS	Samoa
XK	Kosovo
YE	Yemen
YT	Mayotte
ZA	South Africa
ZM	Zambia
ZW	Zimbabwe

▶ Official NetSuite Script Samples (6 Sample(s))

📄 Sample 1: Create and Save a Contact Record

```
/**
 * @NApiVersion 2.1
 */
require(['N/record'], record =&gt; {
    // Create an object to hold name data for the contact
    const nameData = {
        firstname: 'John',
        middlename: 'Doe',
        lastname: 'Smith'
    };

    // Create a contact record
    let objRecord = record.create({
        type: record.Type.CONTACT,
        isDynamic: true
    });

    // Set the values of the subsidiary, firstname, middlename,
    // and lastname properties
    objRecord.setValue({
        fieldId: 'subsidiary',
        value: '1'
    });
    for (let key in nameData) {
        if (nameData.hasOwnProperty(key)) {
            objRecord.setValue({
                fieldId: key,
                value: nameData[key]
            });
        }
    }

    // Save the record
    let recordId = objRecord.save({
        enableSourcing: false,
        ignoreMandatoryFields: false
    });
});
```

📄 Sample 2: Create and Save a Contact Record Asynchronously Using Promise Methods

```
/**
 * @NApiVersion 2.1
 * @NScriptType ClientScript
 */
require(['N/record'], record =&gt; {
    // Create an object to hold name data for the contact
    const nameData = {
        firstname: 'John',
        middlename: 'Doe',
        lastname: 'Smith'
    };

    // Create a contact record using the promise method
    let createRecordPromise = record.create.promise({
        type: record.Type.CONTACT,
        isDynamic: true
    });

    // When the promise is fulfilled, set the values of the subsidiary,
    // firstname, middlename, and lastname properties, and save the
    // record
    createRecordPromise.then(objRecord =&gt; {
        log.debug('Start evaluating promise content...');
        objRecord.setValue({
            fieldId: 'subsidiary',
            value: '1'
        });
        for (let key in nameData) {
            if (nameData.hasOwnProperty(key)) {
                objRecord.setValue({
                    fieldId: key,
                    value: nameData[key]
                });
            }
        }
        let recordId = objRecord.save({
            enableSourcing: false,
            ignoreMandatoryFields: false
        });
    }, function(e) {
        log.error('Unable to create contact', e.name);
    });
});
```

📄 Sample 3: Create Multiple Sales Records Using a Scheduled Script

```
/**
* @NApiVersion 2.1
* @NScriptType ScheduledScript
*/

// This script creates multiple sales records and logs the record creation progress.
define(['N/runtime', 'N/record'], function(runtime, record) {
    return {
        execute: function(context) {
            var script = runtime.getCurrentScript();
            for (x = 0; x &lt; 500; x++) {
                var rec = record.create({
                    type: record.Type.SALES_ORDER
                });
                script.percentComplete = (x * 100)/500;
                log.debug({
                    title: 'New Sales Orders',
                    details: 'Record creation progress: ' + script.percentComplete + '%'
                });
            }
         }
    };
});
```

📄 Sample 4: Access Sublists and a Subrecord from a Record

```
/**
 * @NApiVersion 2.x
 */

require(['N/record'], function(record) {
    function createPurchaseOrder() {
        var rec = record.create({
            type: 'purchaseorder',
            isDynamic: true
        });
        rec.setValue({
            fieldId: 'entity',
            value: 52
        });
        rec.setValue({
            fieldId: 'location',
            value: 2
        });
        rec.selectNewLine({
            sublistId: 'item'
        });
        rec.setCurrentSublistValue({
            sublistId: 'item',
            fieldId: 'item',
            value: 190
        });
        rec.setCurrentSublistValue({
            sublistId: 'item',
            fieldId: 'quantity',
            value: 2
        });
        subrecordInvDetail = rec.getCurrentSublistSubrecord({
            sublistId: 'item',
            fieldId: 'inventorydetail'
        });
        subrecordInvDetail.selectNewLine({
            sublistId: 'inventoryassignment'
        });
        subrecordInvDetail.setCurrentSublistValue({
            sublistId: 'inventoryassignment',
            fieldId: 'receiptinventorynumber',
            value: 'myinventoryNumber'
        });
        subrecordInvDetail.commitLine({
            sublistId: 'inventoryassignment'
        });
        subrecordInvDetail.selectLine({
            sublistId: 'inventoryassignment',
            line: 0
        });
        var myInventoryNumber = subrecordInvDetail.getCurrentSublistValue({
            sublistId: 'inventoryassignment',
            fieldId: 'receiptinventorynumber'
        });
        rec.commitLine({
            sublistId: 'item'
        });
        var recordId = rec.save();
    }
    createPurchaseOrder();
});
```

📄 Sample 5: Access Sublists and a Subrecord from a Record Asynchronously Using Promise Methods

```
/**
 * @NApiVersion 2.x
 */

require(['N/record'], function(record) {
    function createPurchaseOrder() {
        var createRecordPromise = record.create.promise({
            type: 'purchaseorder',
            isDynamic: true
        });
        createRecordPromise.then(function(rec) {
            rec.setValue({
                fieldId: 'entity',
                value: 52
            });
            rec.setValue({
                fieldId: 'location',
                value: 2
            });
            rec.selectNewLine({
                sublistId: 'item'
            });
            rec.setCurrentSublistValue({
                sublistId: 'item',
                fieldId: 'item',
                value: 190
            });
            rec.setCurrentSublistValue({
                sublistId: 'item',
                fieldId: 'quantity',
                value: 2
            });
            subrecordInvDetail = rec.getCurrentSublistSubrecord({
                sublistId: 'item',
                fieldId: 'inventorydetail'
            });
            subrecordInvDetail.selectNewLine({
                sublistId: 'inventoryassignment'
            });
            subrecordInvDetail.setCurrentSublistValue({
                sublistId: 'inventoryassignment',
                fieldId: 'receiptinventorynumber',
                value: 'myinventoryNumber'
            });
            subrecordInvDetail.commitLine({
                sublistId: 'inventoryassignment'
            });
            subrecordInvDetail.selectLine({
                sublistId: 'inventoryassignment',
                line: 0
            });
            var myInventoryNumber = subrecordInvDetail.getCurrentSublistValue({
                sublistId: 'inventoryassignment',
                fieldId: 'receiptinventorynumber'
            });
            rec.commitLine({
                sublistId: 'item'
            });
              var recordId = rec.save();
        }, function(err) {
            log.error('Unable to create purchase order!', err.name);
        });
    }
    createPurchaseOrder();
});
```

📄 Sample 6: Call a Macro on a Sales Order Record

```
/**
 * @NApiVersion 2.x
 */

require(['N/record'],
    function(record) {

     var recordObj = record.create({
         type: record.Type.SALES_ORDER,
         isDynamic: true
     });

     var ENTITY_VALUE = 1;
     var ITEM_VALUE = 1;
     recordObj.setValue({
         fieldId: 'entity',
         value: ENTITY_VALUE
    });
    recordObj.selectNewLine({
         sublistId: 'item'
    });
    recordObj.setCurrentSublistValue({
         sublistId: 'item',
         fieldId: 'item',
         value: ITEM_VALUE
    });
    recordObj.setCurrentSublistValue({
         sublistId: 'item',
         fieldId: 'quantity',
         value: 1
    });
    recordObj.commitLine({
         sublistId:'item'
    });

    var totalBeforeTax = recordObj.getValue({fieldId: 'total'});

    // get macros available on the record
    var macros = recordObj.getMacros();

    // execute the macro
    if ('calculateTax' in macros)
    {
        macros.calculateTax(); // For promise version use: macros.calculateTax.promise()
    }
    // Alternative (direct) macro execution
    // var calculateTax = recordObj.getMacro({id: 'calculateTax'});
    // calculateTax(); // For promise version use: calculateTax.promise()
    var totalAfterTax = recordObj.getValue({fieldId: 'total'});

    var recordId = recordObj.save({
       enableSourcing: false,
       ignoreMandatoryFields: false
    });
});
```

35. N/recordContext Module

Load the N/recordContext module to get the context type of a record.

`define(['N/recordContext'], (recordContext) => { ... })` ✓ 1 Code Sample(s) Scripts: Client and server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/recordContext'], (recordContext) => { // Access methods and properties on recordContext });
```

Use the N/recordContext module to get all the available context types of the record, such as localization. The localization context type indicates which country a script is using for execution. You can also use the N/recordContext module to create conditional statements within a script so that the script behaves differently based on the context.

For more information about localization context, see Localization Context.

N/recordContext Module Sample

N/recordContext Module Members

Return Type / Value Type

Supported Script Types

recordContext.RecordContext

Object (read-only)

Client and server scripts

Contains key-value pairs that represent context types and their values.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	**recordContext.RecordContext**	Object (read-only)	Client and server scripts	Contains key-value pairs that represent context types and their values.
METHOD	**recordContext.getContext(options)**	recordContext.RecordContext	Client and server scripts	Returns the record context object for a record.
ENUM	**recordContext.ContextType**	enum (read-only)	Client and server scripts	Holds the values for the context type. Used to set the value for the contextTypes parameter of the recordContext.getContext(options) method.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: N/recordContext Module Sample

```
/**
 *@NApiVersion 2.x
 */
// This example calls the getContext() API with options recordType, recordId and  record.
require(['N/record', 'N/recordContext'],
    function(record, recordContext) {
        // Create record
        var employee = record.create({
            type : record.Type.EMPLOYEE,
            isDynamic: true
        })
        // Setup CA subsidiary
        employee.setValue('subsidiary', 2);
        employee.setValue('entityid', 'test_emp_' + Date.now())
        employeeId = employee.save()

        // getContext() with options recordType, recordId and contextTypes
        var employeeContext = recordContext.getContext({
            recordType : record.Type.EMPLOYEE,
            recordId: employeeId,
            contextTypes: [recordContext.ContextType.LOCALIZATION]
        })
        log.debug(employeeContext);  // expected log will list {"localization":["CA"]}

         // Change employee subsidiary to AU
         employee.setValue('subsidiary', 3);

        // getContext() with options record and contextTypes
        var employeeContext = recordContext.getContext({
            record : employee,
            contextTypes: [recordContext.ContextType.LOCALIZATION]
        })
        log.debug(employeeContext);  // expected log will list {"localization":["AU"]}

        // Delete record
        record.delete({
            type : record.Type.EMPLOYEE,
            id: 1
        })
    });
```

36. N/redirect Module

Load the N/redirect module to redirect users to a URL, a Suitelet, a record, a task link, a saved search, or an unsaved search.

`define(['N/redirect'], (redirect) => { ... })` ✓ 1 Code Sample(s) Scripts: Server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/redirect'], (redirect) => { // Access methods and properties on redirect });
```

Use the N/redirect module to customize navigation within NetSuite by setting up a redirect URL that resolves to a NetSuite resource or external URL. You can redirect users to one of the following:

URL

Suitelet

Record

Task link

Saved search

Unsaved search

N/redirect Module Script Sample

Suitelets, beforeLoad user events, and synchronous afterSubmit user events are supported. This module does not support beforeSubmit or asynchronous afterSubmit user events. This module is only supported when triggered from the UI. Backend contexts such as CSV Import and Scheduled Scripts are not supported.

N/redirect Module Members

Supported Script Types

redirect.redirect(options)

Suitelets, beforeLoad user events, and synchronous afterSubmit user events

Redirects to the URL of a Suitelet that is available externally (available without login).

redirect.toRecord(options)

Suitelets, beforeLoad user events, and synchronous afterSubmit user events

MEMBER TYPE	NAME	RETURN TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	`redirect.redirect(options)`	`void`	Suitelets, beforeLoad user events, and synchronous afterSubmit user events	Redirects to the URL of a Suitelet that is available externally (available without login).
METHOD	`redirect.toRecord(options)`	`void`	Suitelets, beforeLoad user events, and synchronous afterSubmit user events	Redirects to a NetSuite record.
METHOD	`redirect.toRecordTransform(options)`	`void`	workflow action scripts	Redirects to a standard or custom transaction instance.
METHOD	`redirect.toSavedSearch(options)`	`void`	afterSubmit user events	Redirects to a saved search.
METHOD	`redirect.toSavedSearchResult(options)`	`void`	afterSubmit user events	Redirects to a saved search result.
METHOD	`redirect.toSearch(options)`	`void`	afterSubmit user events	Redirects to search.
METHOD	`redirect.toSearchResult(options)`	`void`	afterSubmit user events	Redirects to search results.
METHOD	`redirect.toSuitelet(options)`	`void`	Suitelets, beforeLoad user events, and synchronous afterSubmit user events	Redirects to a Suitelet.
METHOD	`redirect.toTaskLink(options)`	`void`	Suitelets, beforeLoad user events, and synchronous afterSubmit user events	Redirects to a tasklink.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: N/redirect Module Script Sample

```
/**
 * @NApiVersion 2.x
 */

require(['N/record', 'N/redirect'], function(record, redirect) {
    function redirectToTaskRecord() {
        var taskTitle = 'New Opportunity';
        var taskRecord = record.create({
            type: record.Type.TASK
        });
        taskRecord.setValue('title', taskTitle);

        var taskRecordId = taskRecord.save();

        redirect.toRecord({
            type: record.Type.TASK,
            id: taskRecordId
        });
    }

    redirectToTaskRecord();
});
```

37. N/render Module

Load the N/render module to create forms or email from templates and to print to PDF or HTML.

`define(['N/render'], (render) => { ... })` ✓ 4 Code Sample(s) Scripts: Server scripts Permissions: Advanced PDF/HTML Templates Custom PDF Layouts

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/render'], (render) => { // Access methods and properties on render });
```

Use the N/render module for printing, PDF creation, form creation from templates, and email creation from templates.

Direct manipulation of the print URL is not supported.

N/render Module Members

Return Type / Value Type

Supported Script Types

render.EmailMergeResult

Encapsulates an email merge result.

render.TemplateRenderer

Encapsulates a template object that produces HTML and PDF printed forms utilizing advanced PDF/HTML template capabilities.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	render.EmailMergeResult	Object	Server scripts	Encapsulates an email merge result.
OBJECT	render.TemplateRenderer	Object	Server scripts	Encapsulates a template object that produces HTML and PDF printed forms utilizing advanced PDF/HTML template capabilities.
METHOD	render.bom(options)	file.File	Server scripts	Creates a PDF or HTML file object containing a bill of materials.
METHOD	render.create()	render.TemplateRenderer	Server scripts	Creates a render.TemplateRenderer object.
METHOD	render.mergeEmail(options)	render.EmailMergeResult	Server scripts	Creates a render.EmailMergeResult object.
METHOD	render.packingSlip(options)	file.File	Server scripts	Creates a PDF or HTML file object containing a packing slip.
METHOD	render.pickingTicket(options)	file.File	Server scripts	Creates a PDF or HTML file object containing a picking ticket.
METHOD	render.statement(options)	file.File	Server scripts	Creates a PDF or HTML file object containing a statement.
METHOD	render.transaction(options)	file.File	Server scripts	Creates a PDF or HTML file object containing a transaction.
METHOD	render.xmlToPdf(options)	file.File	Server scripts	Passes XML to the BFO tag library (which is stored by NetSuite), and returns a PDF file.
ENUM	render.DataSource	enum	Server scripts	Holds the string values for supported data source types. Use this enum to set the options.format parameter of the TemplateRenderer.addCustomDataSource(options) method.
ENUM	render.PrintMode	enum	Server scripts	Holds the string values for supported print output types. Use this enum to set the options.printMode parameter of the render.bom(options), render.pickingTicket(options), and render.statement(options) methods.

EmailMergeResult Object Members

Return Type / Value Type

Supported Script Types

EmailMergeResult.body

string (read-only)

The body of the email distribution in string format.

EmailMergeResult.subject

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	EmailMergeResult.body	string (read-only)	Server scripts	The body of the email distribution in string format.
PROPERTY	EmailMergeResult.subject	string (read-only)	Server scripts	The subject of the email distribution in string format.

TemplateRenderer Object Members

Return Type / Value Type

Supported Script Types

TemplateRenderer.addCustomDataSource(options)

Adds an XML file or JSON object to an advanced template as a custom data source.

TemplateRenderer.addQuery(options)

Uses Query as the renderer's data source.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	TemplateRenderer.addCustomDataSource(options)	void	Server scripts	Adds an XML file or JSON object to an advanced template as a custom data source.
METHOD	TemplateRenderer.addQuery(options)	void	Server scripts	Uses Query as the renderer's data source.
METHOD	TemplateRenderer.addRecord(options)	void	Server scripts	Binds a record to a template variable.
METHOD	TemplateRenderer.addSearchResults(options)	void	Server scripts	Binds a search result to a template variable.
METHOD	TemplateRenderer.renderAsPdf()	Object	Server scripts	Uses an advanced template to produce a PDF printed form.
METHOD	TemplateRenderer.renderPdfToResponse(options)	void	Server scripts	Renders PDF template content as a server response.
METHOD	TemplateRenderer.renderAsString()	string	Server scripts	Returns template content in string form.
METHOD	TemplateRenderer.setTemplateById(options)	void	Server scripts	Sets the template using the internal ID.
METHOD	TemplateRenderer.setTemplateByScriptId(options)	void	Server scripts	Sets the template using the script ID.
METHOD	TemplateRenderer.renderToResponse(options)	void	Server scripts	Renders HTML template content as a server response.
PROPERTY	TemplateRenderer.templateContent	string	Server scripts	Content of the template.

▶ Official NetSuite Script Samples (4 Sample(s))

📄 Sample 1: Generate a PDF File from a Raw XML String

```
/**
 * @NApiVersion 2.x
 */

require(['N/render'],
    function(render) {
        function generatePdfFileFromRawXml() {
            var xmlStr = "&lt;?xml version=\"1.0\"?&gt;\n" +
                "&lt;!DOCTYPE pdf PUBLIC \"-//big.faceless.org//report\" \"report-1.1.dtd\"&gt;\n" +
                "&lt;pdf&gt;\n&lt;body font-size=\"18\"&gt;\nHello World!\n&lt;/body&gt;\n&lt;/pdf&gt;";
            var pdfFile = render.xmlToPdf({
                xmlString: xmlStr
            });
        }
        generatePdfFileFromRawXml();
    });
```

📄 Sample 2: Render a Transaction Record Into an HTML Page

```
/**
 * @NApiVersion 2.x
 */

require(['N/render'],
    function(render) {
        function renderTransactionToHtml() {
            var transactionFile = render.transaction({
            entityId: 23,
            printMode: render.PrintMode.HTML
            });
        }
        renderTransactionToHtml();
    });
```

📄 Sample 3: Render an Invoice Into a PDF File Using an XML Template

```
/**
 * @NApiVersion 2.x
 */

// This sample shows how to render an invoice into a PDF file using an XML template in the file cabinet.
// Note that this example requires the Advanced PDF/HTML Templates feature.
require(['N/render', 'N/file', 'N/record'],
    function(render, file, record) {
        function renderRecordToPdfWithTemplate() {
            var xmlTemplateFile = file.load('Templates/PDF Templates/invoicePDFTemplate.xml');
            var renderer = render.create();
            renderer.templateContent = xmlTemplateFile.getContents();
            renderer.addRecord('record', record.load({
                type: record.Type.INVOICE,
                id: 37
            }));
            var invoicePdf = renderer.renderAsPdf();
        }
        renderRecordToPdfWithTemplate();
    });
```

📄 Sample 4: Render Search Results Into a PDF File

```
/**
 * @NApiVersion 2.1
 * @NScriptType Suitelet
 */
// This sample shows how to render search results into a PDF file.
define(['N/render', 'N/search'], function(render, search) {
    function onRequest(options) {
        var request = options.request;
        var response = options.response;

        var xmlStr = '&lt;?xml version=\"1.0\" encoding=\"UTF-8\"?&gt;\n' +
            '&lt;!DOCTYPE pdf PUBLIC \"-//big.faceless.org//report\" \"report-1.1.dtd\"&gt;\n' +
            '&lt;pdf lang=\"ru-RU\" xml:lang=\"ru-RU\"&gt;\n" + "&lt;head&gt;\n' +
            '&lt;link name=\"russianfont\" type=\"font\" subtype=\"opentype\" ' +
            'src=\"NetSuiteFonts/verdana.ttf\" " + "src-bold=\"NetSuiteFonts/verdanab.ttf\"' +
            'src-italic=\"NetSuiteFonts/verdanai.ttf\" " + "src-bolditalic=\"NetSuiteFonts/verdanabi.ttf\"' +
            'bytes=\"2\"/&gt;\n" + "&lt;/head&gt;\n' +
            '&lt;body font-family=\"russianfont\" font-size=\"18\"&gt;\n??????? ?????&lt;/body&gt;\n" + "&lt;/pdf&gt;';

        var rs = search.create({
            type: search.Type.TRANSACTION,
            columns: ['trandate', 'amount', 'entity'],
            filters: []
        }).run();

        var results = rs.getRange(0, 1000);
        var renderer = render.create();
        renderer.templateContent = xmlStr;
        renderer.addSearchResults({
            templateName: 'results',
            searchResult: results
        });

        var newfile = renderer.renderAsPdf();
        response.writeFile(newfile, false);
    }

    return {
        onRequest: onRequest
    };
});
```

38. N/runtime Module

Load the N/runtime module to access runtime settings for company, script, session, system, user, or version.

`define(['N/runtime'], (runtime) => { ... })` ✓ 2 Code Sample(s) Scripts: Client and server scripts Permissions: Â

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/runtime'], (runtime) => { // Access methods and properties on runtime });
```

Use the N/runtime module to view runtime settings for the script, the session, or the user. You can also use this module to set a session key and to see whether a particular feature is enabled in your account.

N/runtime Module Members

Return Type / Value Type

Supported Script Types

Client and server scripts

Encapsulates the runtime settings of the currently executing script.

Client and server scripts

Encapsulates the user session for the currently executing script.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	runtime.Script	Object	Client and server scripts	Encapsulates the runtime settings of the currently executing script.
OBJECT	runtime.Session	Object	Client and server scripts	Encapsulates the user session for the currently executing script.
OBJECT	runtime.User	Object	Client and server scripts	Encapsulates the properties and preferences of the user currently executing the script.
METHOD	runtime.getCurrentScript()	runtime.Script	Client and server scripts	Returns a runtime.Script object that represents the currently executing script.
METHOD	runtime.getCurrentSession()	runtime.Session	Client and server scripts	Returns a runtime.Session object that represents the user session for the currently executing script.
METHOD	runtime.getCurrentUser()	runtime.User	Client and server scripts	Returns a runtime.User object that represents the properties and preferences of the user currently executing the script.
METHOD	runtime.isFeatureInEffect(options)	boolean	Client and server scripts	Indicates whether a particular feature is enabled in a NetSuite account. These are the features that appear on the Enable Features page.
METHOD	runtime.isNextActive()	boolean	Client and server scripts	Returns whether NetSuite Next is active for the user running the script.
PROPERTY	runtime.accountId	string (read-only)	Client and server scripts	The account ID for the current user.
PROPERTY	runtime.country	string (read-only)	Client and server scripts	The country for the current company.
PROPERTY	runtime.envType	string (read-only)	Client and server scripts	The current environment in which the script is executing. This property uses values from the runtime.EnvType enum.
PROPERTY	runtime.executionContext	string (read-only)	Client and server scripts	The trigger of the current script. This property uses values from the runtime.ContextType enum.
PROPERTY	runtime.processorCount	number (read-only)	Client and server scripts	The number of processors available to the current account.
PROPERTY	runtime.queueCount	number (read-only)	Client and server scripts	The number of scheduled script queues available to the current account.
PROPERTY	runtime.version	string (read-only)	Client and server scripts	The version of NetSuite that the method is called in. For example, this property in an account running NetSuite 2023.2 is 2023.2.
ENUM	runtime.ContextType	enum	Client and server scripts	Holds the context values for script triggers. This is the type for the runtime.executionContext property.
ENUM	runtime.EnvType	enum	Client and server scripts	Holds all possible environment types that the current script can execute in. This is the type for the runtime.envType property.
ENUM	runtime.Permission	enum	Client and server scripts	Holds the user permission level for a specific permission ID. This is the type returned by the User.getPermission(options) method.

Script Object Members

Return Type / Value Type

Supported Script Types

Script.getParameter(options)

number | Date | string | boolean

Client and server scripts

Returns the value of a script parameter for the currently executing script.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**Script.getParameter(options)**	`number | Date | string | boolean`	Client and server scripts	Returns the value of a script parameter for the currently executing script.
METHOD	**Script.getRemainingUsage()**	`number`	Client and server scripts	Returns the number of units remaining (per governance limitations) for the currently executing script.
PROPERTY	**Script.apiVersion**	`string (read-only)`	Client and server scripts	The current script's runtime version.
PROPERTY	**Script.bundleIds**	`Array (read-only)`	Client and server scripts	An array of bundle IDs for the bundles that include the currently executing script.
PROPERTY	**Script.deploymentId**	`string (read-only)`	Server scripts	The deployment ID for the script deployment of the currently executing script.
PROPERTY	**Script.id**	`string (read-only)`	Client and server scripts	The script ID for the currently executing script.
PROPERTY	**Script.logLevel**	`string (read-only)`	Server scripts	The script logging level for the currently executing script.
PROPERTY	**Script.percentComplete**	`number`	Client and server scripts	The percent complete for the current scheduled script execution. This value will appear in the % Complete column on the Scheduled Script Status page.

Session Object Members

Return Type / Value Type

Supported Script Types

Session.get(options)

Returns the user-defined session object value associated with the session object key. Both the session object value and associated key are defined using Session.set(options).

Session.set(options)

Sets a key and value for a user-defined session object.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**Session.get(options)**	`string | null`	Server scripts	Returns the user-defined session object value associated with the session object key. Both the session object value and associated key are defined using Session.set(options).
METHOD	**Session.set(options)**	`void`	Server scripts	Sets a key and value for a user-defined session object.

User Object Members

Return Type / Value Type

Supported Script Types

User.getPermission(options)

Client and server scripts

Returns a runtime.Permission user permission level for the specified permission.

User.getPreference(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**User.getPermission(options)**	`string`	Client and server scripts	Returns a runtime.Permission user permission level for the specified permission.
METHOD	**User.getPreference(options)**	`string`	Client and server scripts	Returns the value of a NetSuite preference. Currently only General Preferences and Accounting Preferences are exposed in SuiteScript. For more information about these preferences, see General Preferences and Accounting Preferences.
PROPERTY	**User.contact**	`number(read-only)`	Client and server scripts	The internal ID of the currently logged-in contact.
PROPERTY	**User.department**	`number (read-only)`	Client and server scripts	The internal ID of the department for the current user.
PROPERTY	**User.email**	`string (read-only)`	Client and server scripts	The email address of the current user.
PROPERTY	**User.id**	`number (read-only)`	Client and server scripts	The internal ID of the current user.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	User.location	number (read-only)	Client and server scripts	The internal ID of the location of the current user.
PROPERTY	User.name	string (read-only)	Client and server scripts	The name of the current user.
PROPERTY	User.role	number (read-only)	Client and server scripts	The internal ID of the role for the current user.
PROPERTY	User.roleCenter	string (read-only)	Client and server scripts	The string value of the center type, or role center, for the current user.
PROPERTY	User.roleId	string (read-only)	Client and server scripts	The custom scriptId of the role for the current user.
PROPERTY	User.subsidiary	number (read-only)	Client and server scripts	The internal ID of the subsidiary for the current user.

▶ Official NetSuite Script Samples (2 Sample(s))

📄 Sample 1: Return User and Session Information

```
/**
 * @NApiVersion 2.1
 * @NScriptType Suitelet
 */

// This script writes user and session information for the currently executing script to the response.
define(['N/runtime'], function(runtime) {
    function onRequest(context) {
        var remainingUsage = runtime.getCurrentScript().getRemainingUsage();
        var userRole = runtime.getCurrentUser().role;
        var currentSession = runtime.getCurrentSession();

        // Set the current sessions's scope
        currentSession.set({
            name: 'scope',
            value: 'global'
        });

        var sessionScope = runtime.getCurrentSession().get({
            name: 'scope'
        });

        log.debug('Remaining Usage:', remainingUsage);
        log.debug('Role:', userRole);
        log.debug('Session Scope:', sessionScope);

        context.response.write('Executing under role: ' + userRole
            + '. Session scope: ' + sessionScope + '.');
    }
    return {
        onRequest: onRequest
    };
});
```

📄 Sample 2: Create Multiple Sales Records Using a Scheduled Script

```
/**
* @NApiVersion 2.1
* @NScriptType ScheduledScript
*/

// This script creates multiple sales records and logs the record creation progress.
define(['N/runtime', 'N/record'], function(runtime, record) {
    return {
        execute: function(context) {
            var script = runtime.getCurrentScript();
            for (x = 0; x &lt; 500; x++) {
                var rec = record.create({
                    type: record.Type.SALES_ORDER
                });
                script.percentComplete = (x * 100)/500;
                log.debug({
                    title: 'New Sales Orders',
                    details: 'Record creation progress: ' + script.percentComplete + '%'
                });
            }
         }
    };
});
```

39. N/scriptTypes/restlet Module

Load the N/scriptTypes/restlet module to create custom handling for your RESTlet script.

`define(['N/scriptTypes/restlet'], (restlet) ⇒ { ... })` ✓ 1 Code Sample(s) Scripts: RESTlet scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/scriptTypes/restlet'], (restlet) ⇒ { // Access methods and properties on restlet });
```

Use the N/scriptTypes/restlet module to create custom HTTP responses for your RESTlet script.

This module is available only to the RESTlet script type.

N/scriptTypes/restlet Module Script Sample

N/scriptTypes/restlet Module Members

Return Type / Value Type

Supported Script Types

restlet.Response

Object (read-only)

An HTTP response of a RESTlet script.

restlet.createResponse(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	**restlet.Response**	Object (read-only)	RESTlet script	An HTTP response of a RESTlet script.
METHOD	**restlet.createResponse(options)**	restlet.Response	RESTlet script	Creates a custom HTTP response for a RESTlet script.

Response Object Members

The following members are called on the restlet.Response object.

Return Type / Value Type

Supported Script Types

Response.content

string (read-only)

The content of the RESTlet HTTP response.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Response.content**	string (read-only)	RESTlet script	The content of the RESTlet HTTP response.
PROPERTY	**Response.contentType**	string (read-only)	RESTlet script	The Content-Type header of the RESTlet HTTP response.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: N/scriptTypes/restlet Module Script Sample

```
/**
 * @NApiVersion 2.1
 * @NScriptType RESTlet
 */
define(['N/scriptTypes/restlet', 'N/query'],
    function(restlet, query) {

    // RESTlet entry point
    const get = function (requestParams) {
        // Create Employee query
        const qObj = query.create({
            type: query.Type.EMPLOYEE
        });
        // Set query condition for the Supervisor field on the employee record, it should not be empty
        qObj.condition = qObj.createCondition({
            fieldId: 'supervisor',
            operator: query.Operator.EMPTY_NOT
        });
        // Add supervisor value in the query results
        qObj.columns = [qObj.createColumn({
            fieldId: 'supervisor'
        })]

        const returnArray = [];

        // Retrieve the results
        const results = qObj.run().results;
        for (var i = results.length - 1; i &gt;= 0; i--) {
            const obj = results[i];
            returnArray.push(obj.values[0]);
        }

        // Create a RESTlet custom response to set the Content-Type header to JSON object
        const response = restlet.createResponse({
            content: JSON.stringify(returnArray),
            contentType: 'application/json'
        });

        log.debug('response', JSON.stringify(response))

        return response;
    }

    return { get };
});
```

40. N/search Module

Load the N/search module to create and run on-demand or saved searches and analyze and iterate through the search results. You can search for a single record by keywords, create saved searches, search for duplicate records, or return a set of records that match filters you define.

`define(['N/search'], (search) => { ... })` ✓ 9 Code Sample(s) Scripts: Client and server scripts

Permissions: Perform Search Persist Search Publish Search

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/search'], (search) => { // Access methods and properties on search });
```

Use the N/search module to create and run on-demand or saved searches, then analyze and work through the results. With this module, you can do things like:

Search for a single record using keywords

Create and save searches

Load and run saved searches

Search for duplicate records

Return a set of records that match your defined filter criteria

You can also paginate results and add navigation to jump between pages. This makes it great for handling large result sets.

The N/search module doesn't work in unauthenticated client-side contexts.

For details, see the SuiteAnswers Outbound HTTPs in an unauthenticated client-side context.

N/search Module Members

Return Type / Value Type

Supported Script Types

Client and server scripts

Encapsulates a single search column in a search.Search object. Use the methods and properties available to the Column object to get or set Column properties.

Client and server scripts

Encapsulates a search filter used in a search. Use the properties for the Filter object to get and set the filter properties.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	**search.Column**	Object	Client and server scripts	Encapsulates a single search column in a search.Search object. Use the methods and properties available to the Column object to get or set Column properties.
OBJECT	**search.Filter**	Object	Client and server scripts	Encapsulates a search filter used in a search. Use the properties for the Filter object to get and set the filter properties.
OBJECT	**search.Page**	Object	Client and server scripts	Encapsulates a set of search results for a single search page.
OBJECT	**search.PagedData**	Object	Client and server scripts	Holds metadata about a paginated query.
OBJECT	**search.PageRange**	Object	Client and server scripts	Defines the page range to bound the result set for a paginated query.
OBJECT	**search.Result**	Object	Client and server scripts	Encapsulate a single search result row. Use the methods and properties for the Result object to get the column values for the result row.
OBJECT	**search.ResultSet**	Object	Client and server scripts	Encapsulates a set of search results returned by Search.run().
OBJECT	**search.Search**	Object	Client and server scripts	Encapsulates a NetSuite search. Use the methods available to the Search object to create a search, run a search, or save a search.
OBJECT	**search.Setting**	Object	Client and server scripts	Encapsulates a search setting. Search settings let you specify search parameters that are typically available only in the UI.
METHOD	**search.create(options)**	search.Search	Client and server scripts	Creates a new search and returns it as a search.Search object.
METHOD	**search.create.promise(options)**	search.Search	Client and server scripts	Creates a new search asynchronously and returns it as a search.Search object.
METHOD	**search.createColumn(options)**	search.Column	Client and server scripts	Creates a new search column as a search.Column object.
METHOD	**search.createFilter(options)**	search.Filter	Client and server scripts	Creates a new search filter as a search.Filter object.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	search.createSetting(options)	search.Setting	Client and server scripts	Creates a new search setting and returns it as a search.Setting object.
METHOD	search.delete(options)	void	Client and server scripts	Deletes an existing saved search asynchronously and returns it as a search.Search object.
METHOD	search.delete.promise(options)	void	Client and server scripts	Deletes an existing saved search and returns it as a search.Search object.
METHOD	search.duplicates(options)	search.Result[]	Client and server scripts	Performs a search for duplicate records based on the duplicate detection configuration for the account. Returns an array of search.Result objects.
METHOD	search.duplicates.promise(options)	search.Result[]	Client and server scripts	Performs a search for duplicate records asynchronously based on the duplicate detection configuration for the account. Returns an array of search.Result objects.
METHOD	search.global(options)	search.Result[]	Client and server scripts	Performs a global search against a single keyword or multiple keywords.
METHOD	search.global.promise(options)	search.Result[]	Client and server scripts	Performs a global search asynchronously against a single keyword or multiple keywords.
METHOD	search.load(options)	search.Search	Client and server scripts	Loads an existing saved search and returns it as a search.Search object.
METHOD	search.load.promise(options)	search.Search	Client and server scripts	Loads an existing saved search asynchronously and returns it as a search.Search object.
METHOD	search.lookupFields(options)	Object | array	Client and server scripts	Performs a search for one or more body fields on a record. Returns select fields as an object with value and text properties. Returns multiselect fields as an object with value:text pairs.
METHOD	search.lookupFields.promise(options)	Object | array	Client and server scripts	Performs a search asynchronously for one or more body fields on a record. Returns select fields as an object with value and text properties. Returns multiselect fields as an object with value:text pairs.
ENUM	search.Operator	enum	Client and server scripts	Holds the values for search operators to use with the search.Filter.
ENUM	search.Sort	enum	Client and server scripts	Holds the values for supported sorting directions used with search.createColumn(options).
ENUM	search.Summary	enum	Client and server scripts	Holds the values for summary types used by the Column.summary object.
ENUM	search.Type	enum	Client and server scripts	Holds the string values for search types supported in the N/search Module. Use this enum to set the value for the options.type parameter of the search.create(options) method.

Column Object Members

The following members are available for a search.Column object.

Return Type / Value Type

Supported Script Types

Column.setWhenOrderedBy(options)

Client and server scripts

Returns the search column for which the minimal or maximal value should be found when returning the search.Column value.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Column.setWhenOrderedBy(options)	search.Column	Client and server scripts	Returns the search column for which the minimal or maximal value should be found when returning the search.Column value.
PROPERTY	Column.formula	string	Client and server scripts	Formula used for a search column as a string.
PROPERTY	Column.function	string	Client and server scripts	Special function used in the search column as a string.
PROPERTY	Column.join	string (read-only)	Client and server scripts	Join ID for a search column as a string.
PROPERTY	Column.label	string	Client and server scripts	Label used for the search column. You can only get or set custom labels with this property.
PROPERTY	Column.name	string (read-only)	Client and server scripts	Name of a search column as a string.
PROPERTY	Column.summary	string (read-only)	Client and server scripts	Returns the summary type for a search column.

Filter Object Members

The following members are available for a search.Filter object.

Return Type / Value Type

Supported Script Types

Client and server scripts

Formula used by the search filter.

string (read-only)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Filter.formula	string	Client and server scripts	Formula used by the search filter.
PROPERTY	Filter.join	string (read-only)	Client and server scripts	Join ID for the search filter.
PROPERTY	Filter.name	string (read-only)	Client and server scripts	Name or internal ID of the search field.
PROPERTY	Filter.operator	string (read-only)	Client and server scripts	Operator used for the search filter.
PROPERTY	Filter.summary	search.Summary	Client and server scripts	Summary type for the search filter.

Page Object Members

The following members are available for a search.Page object.

Return Type / Value Type

Supported Script Types

Client and server scripts

Gets the next segment of data from a paginated search

Page.next.promise()

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Page.next()	void	Client and server scripts	Gets the next segment of data from a paginated search
METHOD	Page.next.promise()	void	Client scripts	Asynchronously gets the next segment of data from a paginated search
METHOD	Page.prev()	void	Client and server scripts	Gets the previous segment of data from a paginated search
METHOD	Page.prev.promise()	void	Client scripts	Asynchronously gets the previous segment of data from a paginated search
PROPERTY	Page.data	search.Result[]	Client and server scripts	The results from a paginated search.
PROPERTY	Page.isFirst	boolean (read-only)	Client and server scripts	Indicates whether a page is the first page of data for a result set
PROPERTY	Page.isLast	boolean (read-only)	Client and server scripts	Indicates whether a page is the last page of data for a result set.
PROPERTY	Page.pagedData	search.PagedData (read-only)	Client and server scripts	The PagedData Object used to fetch this Page Object.
PROPERTY	Page.pageRange	search.PageRange (read-only)	Client and server scripts	The PageRange Object used to fetch this Page Object.

PagedData Object Members

The following members are available for a search.PagedData object.

Return Type / Value Type

Supported Script Types

PagedData.fetch(options)

Client and server scripts

Retrieves the data within the specified page range.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	PagedData.fetch(options)	search.Page	Client and server scripts	Retrieves the data within the specified page range.
METHOD	PagedData.fetch.promise()	search.Page	Client scripts	Asynchronously retrieves the data within the specified page range.
PROPERTY	PagedData.count	number (read-only)	Client and server scripts	The total number of results when Search.runPaged(options) was executed.
PROPERTY	PagedData.pageRanges	search.PageRange[] (read-only)	Client and server scripts	The collection of PageRange objects that divide the entire result set into smaller groups.
PROPERTY	PagedData.pageSize	number (read-only)	Client and server scripts	The maximum number of entries per page
PROPERTY	PagedData.searchDefinition	search.Search (read-only)	Client and server scripts	The search criteria used when Search.runPaged(options) was executed.

PageRange Object Members

The following members are available for a search.PageRange object.

Return Type / Value Type

Supported Script Types

PageRange.compoundLabel

string (read-only)

Client and server scripts

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	PageRange.compoundLabel	string (read-only)	Client and server scripts	Human-readable label with beginning and ending range identifiers
PROPERTY	PageRange.index	number (read-only)	Client and server scripts	The index of this page range.

Result Object Members

The following members are available for a search.Result object.

Return Type / Value Type

Supported Script Types

Result.getText(column)

Client and server scripts

The text value for a search.Column if it is a stored select field.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Result.getText(column)	string	Client and server scripts	The text value for a search.Column if it is a stored select field.
METHOD	Result.getText(options)	string	Client and server scripts	The UI display name, or text value, for a search result column. This method is supported only for non-stored select, image, and document fields.
METHOD	Result.getValue(options)	string	Client and server scripts	Used on formula fields and non-formula (standard) fields to get the value of a specified search return column.
METHOD	Result.getValue(column)	string	Client and server scripts	Used on formula and non-formula (standard) fields. Returns the string value of a specified search result column. For convenience, this method takes a single search.Column Object.
PROPERTY	Result.columns	search.Column[]	Client and server scripts	Array of search.Column objects that encapsulate the columns returned in the search result row.
PROPERTY	Result.id	string (read-only)	Client and server scripts	The internal ID for the record returned in a search result row.
PROPERTY	Result.recordType	string (read-only)	Client and server scripts	The type of record returned in a search result row.

ResultSet Object Members

The following members are available for a search.ResultSet object.

Return Type / Value Type

Supported Script Types

ResultSet.each(callback)

Client and server scripts

Use a developer-defined function to invoke on each row in the search results, up to 4000 results at a time.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	ResultSet.each(callback)	void	Client and server scripts	Use a developer-defined function to invoke on each row in the search results, up to 4000 results at a time.
METHOD	ResultSet.each.promise(callback)	void	Client scripts	Asynchronously use a developer-defined function to invoke on each row in the search results, up to 4000 results at a time.
METHOD	ResultSet.getRange(options)	search.Result[]	Client and server scripts	Retrieve a slice of the search result as an array of search.Result objects.
METHOD	ResultSet.getRange.promise(options)	search.Result[]	Client scripts	Asynchronously retrieve a slice of the search result as an array of search.Result objects.
PROPERTY	ResultSet.columns	search.Column[]	Client and server scripts	An array of search.Column objects that represent the columns returned in the search results.

Search Object Members

The following members are available for a search.Search object.

Return Type / Value Type

Supported Script Types

search.ResultSet

Client and server scripts

Runs an on demand search created with search.create(options) or a search loaded with search.load(options), returning the results as a search.ResultSet.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**Search.run()**	search.ResultSet	Client and server scripts	Runs an on demand search created with search.create(options) or a search loaded with search.load(options), returning the results as a search.ResultSet.
METHOD	**Search.runPaged(options)**	search.PagedData	Client and server scripts	Runs the current search and returns a search.PagedData Object.
METHOD	**Search.runPaged.promise(options)**	search.PagedData	Client and server scripts	Asynchronously runs the current search and returns a search.PagedData Object.
METHOD	**Search.save()**	number	Client and server scripts	Saves a search created by search.create(options) or loaded with search.load(options). Returns the internal ID of the saved search.
METHOD	**Search.save.promise()**	number	Client and server scripts	Asynchronously saves a search created by search.create(options) or loaded with search.load(options). Returns the internal ID of the saved search.
PROPERTY	**Search.columns**	search.Column[] | string[]	Client and server scripts	Columns to return for this search as an array of search.Column objects or a string array of column names.
PROPERTY	**Search.filterExpression**	Object[]	Client and server scripts	Search filter expression for the search as an array of expression objects.
PROPERTY	**Search.filters**	search.Filter[]	Client and server scripts	Filters for the search as an array of search.Filter objects.
PROPERTY	**Search.id**	string	Client and server scripts	Script ID for a saved search, starting with customsearch.
PROPERTY	**Search.isPublic**	boolean	Client and server scripts	Value is true if the search is public, or false if it is not.
PROPERTY	**Search.packageId**	string	Client and server scripts	The application ID for the search.
PROPERTY	**Search.searchId**	number	Client and server scripts	Internal ID of a search.
PROPERTY	**Search.searchType**	string	Client and server scripts	Search type on which a search is based.
PROPERTY	**Search.settings**	search.Setting[] | string[]	Client and server scripts	Search settings for this search as an array of search.Setting objects or a string array of column names.
PROPERTY	**Search.title**	string	Client and server scripts	Title for a saved search. Use this property to set the title for a search before you save it for the first time.

Setting Object Members

The following members are available for a search.Setting object.

Return Type / Value Type

Supported Script Types

string (read-only)

Client and server scripts

The name of the search parameter.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Setting.name**	string (read-only)	Client and server scripts	The name of the search parameter.
PROPERTY	**Setting.value**	string (read-only)	Client and server scripts	The value of the search parameter.

▶ Official NetSuite Script Samples (9 Sample(s))

📄 Sample 1: Search for Customer Records and Log First 50 Results

```
/**
 * @NApiVersion 2.1
 */

require(['N/search'], function(search) {
    var mySearch = search.create({
        type: search.Type.CUSTOMER,
        columns: ['entityid', 'firstname', 'lastname', 'salesrep'],
        filters: ['entityid', 'contains', 'Adam']
    });

    var myResultSet = mySearch.run();

    var resultRange = myResultSet.getRange({
        start: 0,
        end: 50
    });

    for (var i = 0; i &lt; resultRange.length; i++) {
        log.debug(resultRange[i]);
    }
});
```

📄 Sample 2: Search for Sales Order Records

```
/**
 * @NApiVersion 2.x
 */

require(['N/search'], function(search) {
    function createSearch() {
        var mySalesOrderSearch = search.create({
            type: search.Type.SALES_ORDER,
            title: 'My SalesOrder Search',
            id: 'customsearch_my_so_search',
            columns: ['entity', 'subsidiary', 'name', 'currency'],
            filters: [
                ['mainline', 'is', 'T'],
                'and', ['subsidiary.name', 'contains', 'CAD']
            ]
        });

        mySalesOrderSearch.save();
    }

    createSearch();
});
```

📄 Sample 3: Load a Search for Sales Order Records and Use a Callback Function to Process Results

```
/**
 * @NApiVersion 2.x
 */

require(['N/search'], function(search) {
    function loadAndRunSearch() {
        var mySearch = search.load({
            id: 'customsearch_my_so_search'
        });

        mySearch.run().each(function(result) {
            var entity = result.getValue({
                name: 'entity'
            });
            var subsidiary = result.getValue({
                name: 'subsidiary'
            });

            return true;
        });
    }

    loadAndRunSearch();
});
```

📄 Sample 4: Load a Search for Sales Order Records and Return the First 100 Search Results

```
/**
 * @NApiVersion 2.x
 */

require(['N/search'], function(search) {
    function runSearchAndFetchResult() {
        var mySearch = search.load({
            id: 'customsearch_my_so_search'
        });

        var searchResult = mySearch.run().getRange({
            start: 0,
            end: 100
        });
        for (var i = 0; i &lt; searchResult.length; i++) {
            var entity = searchResult[i].getValue({
                name: 'entity'
            });
            var subsidiary = searchResult[i].getValue({
                name: 'subsidiary'
            });
        }
    }

    runSearchAndFetchResult();
});
```

📄 Sample 5: Load and Run a Paginated Search and Process the Results

```
/**
 * @NApiVersion 2.x
 */

require(['N/search'], function(search) {
    function loadAndRunSearch() {
        var mySearch = search.load({
            id: 'customsearch_my_so_search'
        });

        var myPagedData = mySearch.runPaged();
        myPagedData.pageRanges.forEach(function(pageRange){
            var myPage = myPagedData.fetch({index: pageRange.index});
            myPage.data.forEach(function(result){
                var entity = result.getValue({
                    name: 'entity'
                });
                var subsidiary = result.getValue({
                    name: 'subsidiary'
                });
            });
        });
    }

    loadAndRunSearch();
});
```

📄 Sample 6: Create a Search for a Custom Record Type

```
/**
 * @NApiVersion 2.x
 */

require(['N/search'], function(search) {
    var myCustomRecordSearch = search.create({
        type: search.Type.CUSTOM_RECORD + '6',
        title: 'My Search Title',
        columns: ['custrecord1']
    }).run().each(function(result) {
        // Process each result
        return true;
    });
});
```

📄 Sample 7: Search for Items in a Custom List

```
/**
 * @NApiVersion 2.x
 */

require(['N/search'], function(search) {
    var internalId = -1;
    var myCustomListSearch = search.create({
        type: 'customlist_mylist',
        columns: [
            { name : 'internalId' },
            { name : 'abbreviation' }
        ]
    });

    myCustomListSearch.filters = [
        search.createFilter({
            name: 'formulatext',
            formula: '{abbreviation}',
            operator: search.Operator.IS,
            values: abbreviation
        });
    ]

    var resultSet = myCustomListSearch.run();
    var results = resultSet.getRange({
        start: 0,
        end: 1
    });
    for(var i in results) {
        // log.debug('Found custom list record', results[i]);
        internalId = results[i].getValue({
            name:'internalId'
        });
    };
});
```

📄 Sample 8: Delete a Saved Search

```
/**
 * @NApiVersion 2.x
 */

require(['N/search'], function(search) {
    function deleteSearch() {
        search.delete({
            id: 'customsearch_my_so_search'
        });
    }

    deleteSearch();
});
```

📄 Sample 9: Search Using a Specific Record Field

```
/**
* @NApiVersion 2.x
*/
require(['N/search'], function(search) {
    var mySearch = search.create({
        type: search.Type.TRANSACTION,
        columns: ['item', 'operationdisplaytext'],
        filters: [
            ['internalid', search.Operator.ANYOF, &lt;transactionId&gt;],
            'and',
            ['operationdisplaytext', search.Operator.EQUALTO, 20]]
    });

    var myResultSet = mySearch.run();
    var resultRange = myResultSet.getRange({
        start: 0,
        end: 20
    });
});
```

41. N/sftp Module

Load the N/sftp module to connect to a remote FTP server using SFTP and transfer files.

`define(['N/sftp'], (sftp) => { ... })` ✓ 3 Code Sample(s) Scripts: Server scripts

Permissions: Key Access (needed when public key authentication is used)

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/sftp'], (sftp) => { // Access methods and properties on sftp });
```

Use the N/sftp module to manage folders and upload or download files from external SSH file transfer (SFTP) servers. You can perform the following SFTP functions using the N/sftp module:

SFTP servers can be hosted by your organization or by a third party. NetSuite does not provide SFTP server functionality. All SFTP transfers to or from NetSuite must originate from SuiteScript. It is not possible for external clients to initiate file transfers using SFTP.

Use SSH keys to establish an SFTP connection. By using the keys, you can manage files and directories by using the SFTP protocol. For more information, see SSH Keys for SFTP. For more information about working with keys in SuiteScript, see the N/keyControl Module, see N/keyControl Module.

All paths, directories, and filenames that contain wildcards such as ? and * must have those characters escaped, unless these characters are specifically intended to work as wildcards.

To use an external server to initiate a NetSuite file transfer that doesn't use SFTP, you can use RESTlets or SOAP web services. In SuiteScript, RESTlets can respond to requests containing file data and save them in the File Cabinet. RESTlets can also respond to requests for file data by loading the contents from the File Cabinet and returning them in the response. Note that binary file content must be received or sent as Base64 encoded Strings. See SuiteScript 2.1 RESTlet Script Type for more information.

In SOAP web services, applications can invoke CRUD operations on the file record to populate or change the contents of the File Cabinet. See SuiteTalk SOAP Web Services Platform Guide and File for more information.

N/sftp Module Members

Return Type / Value Type

Supported Script Types

Represents a connection to the account on the remote FTP server.

sftp.createConnection(options)

Establishes a connection to a remote FTP server.

sftp.MAX_CONNECT_TIMEOUT

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	**sftp.Connection**	Object	Server scripts	Represents a connection to the account on the remote FTP server.
METHOD	**sftp.createConnection(options)**	sftp.Connection	Server scripts	Establishes a connection to a remote FTP server.
ENUM	**sftp.MAX_CONNECT_TIMEOUT**	enum	Server scripts	Holds the values for maximum connection timeout.
ENUM	**sftp.MIN_CONNECT_TIMEOUT**	enum	Server scripts	Holds the values for minimum connection timeout.
ENUM	**sftp.MAX_PORT_NUMBER**	enum	Server scripts	Holds the values for the maximum port number.
ENUM	**sftp.MIN_PORT_NUMBER**	enum	Server scripts	Holds the values for the minimum port number.
ENUM	**sftp.DEFAULT_PORT_NUMBER**	enum	Server scripts	Holds the values for the default port number.
ENUM	**sftp.Sort**	enum	Server scripts	Holds the values to be used to sort the listed directory. Use this enum to set the value of the options.sort parameter of the Connection.list(options) method.

Connection Object Members

The following members are called on the sftp.Connection object.

Return Type / Value Type

Supported Script Types

Connection.download(options)

Downloads a file from the remote FTP server.

Connection.upload(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**Connection.download(options)**	file.File	Server scripts	Downloads a file from the remote FTP server.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Connection.upload(options)	void	Server scripts	Uploads a file to the remote FTP server.
METHOD	Connection.makeDirectory(options)	string	Server scripts	Creates an empty directory.
METHOD	Connection.removeDirectory(options)	void	Server scripts	Removes an empty directory.
METHOD	Connection.removeFile(options)	void	Server scripts	Removes a file in a directory.
METHOD	Connection.move(options)	void	Server scripts	Moves a file or directory from one location to another.
METHOD	Connection.list(options)	Array<Object>	Server scripts	Lists the remote directory.
ENUM	Connection.MAX_FILE_SIZE	enum	Server scripts	Holds the values for the maximum file size.
ENUM	Connection.MAX_TRANSFER_TIMEOUT	enum	Server scripts	Holds the values for the maximum transfer timeout.

▶ Official NetSuite Script Samples (3 Sample(s))

📄 Sample 1: Upload and Download a File

```
/**
 * @NApiVersion 2.1
 */
require(['N/sftp', 'N/file'], (sftp, file) =&gt; {
    const myPwdGuid = "B34672495064525E5D65032D63B52301";
    const myHostKey = "AAA1234567890Q=";

    // Establish a connection to a remote FTP server
    let connection = sftp.createConnection({
        username: 'myuser',
        passwordGuid: myPwdGuid,
        url: 'host.somewhere.com',
        directory: 'myuser/wheres/my/file',
        hostKey: myHostKey
    });

    // Create a file to upload using the N/file module
    let myFileToUpload = file.create({
        name: 'originalname.js',
        fileType: file.Type.PLAINTEXT,
        contents: 'I am a test file.'
    });

    // Upload the file to the remote server
    connection.upload({
        directory: 'relative/path/to/remote/dir',
        filename: 'newFileNameOnServer.js',
        file: myFileToUpload,
        replaceExisting: true
    });

    // Download the file from the remote server
    let downloadedFile = connection.download({
        directory: 'relative/path/to/file',
        filename: 'downloadMe.js'
    });
});
```

📄 Sample 2: Manage Files and Directories

```
/**
 * @NApiVersion 2.1
 */
require(['N/sftp', 'N/file'], (sftp, file) =&gt; {
    // Establish a connection
    log.debug('Establishing SFTP connection...');
    let connection = sftp.createConnection({
        username: 'sftpuser',
        keyId: 'custkeysftp_nft_demo_key',
        url: 'myurl',
        port: 22,
        directory: 'inbound',
        hostKey: 'myhostkey'
    });
    log.debug('Connection established!');

    // List the directory and log the number of elements there
    let list = connection.list({
        path: 'yyy/test'
    });
    log.debug('Items in directory "test" at the beginning: ' + list.length);

    // Generate the test file
    log.debug('Generating test file...');
    let myFileToUpload = file.create({
        name: 'asdf.txt',
        fileType: file.Type.PLAINTEXT,
        contents: 'I am a test file.'
    });
    log.debug('Test file generated, uploading to "test" directory...');

    // Upload the test file
    connection.upload({
        directory: 'yyy/test',
        filename: 'af.txt',
        file: myFileToUpload,
        replaceExisting: true
    });
    log.debug('Upload complete!');

    // List the directory to confirm there is one more file than before
    list = connection.list({
        path: 'yyy/test'
    });
    log.debug('Items in directory "test" after the upload: ' + list.length);

    // Create a new directory
    log.debug('Creating directory "test2"...');
    try {
        connection.makeDirectory({
            path: 'yyy/test2'
        });
        log.debug('Directory created.');
    } catch (e) {
        log.debug('Directory not created.');
        log.error(e.message);
    }
    list = connection.list({
        path: 'yyy/test2'
    });
    log.debug('Items in directory "test2": ' + list.length);

    // Move the test file to the new directory
    log.debug('Moving the test file from "test" to "test2"...');
    connection.move({
        from: 'yyy/test/af.txt',
        to: 'yyy/test2/af.txt'
    })
    log.debug('File moved!');

    // List the original directory again to see the file is moved
    list = connection.list({
        path: 'yyy/test'
    });
    log.debug('Items in directory "test" after the upload: ' + list.length);

    // List the new directory for the file
    list = connection.list({path: 'yyy/test2'});
    log.debug('Items in directory "test2" after the upload: ' + list.length);
    log.debug(JSON.stringify(list));

    // Try to remove the directory
    log.debug('Removing directory "test2"...');
    try {
        connection.removeDirectory({
            path: 'yyy/test2'
        });
        log.debug('Directory removed!');
    } catch (e) {
        log.debug('Directory not removed!');
        log.error(e.message);
```

```
    }

    // The directory is not empty so delete the file first
    log.debug('Removing test file from "test2" directory...');
    connection.removeFile({
        path: 'yyy/test2/af.txt'
    });
    log.debug('Test file removed!');

    list = connection.list({
        path: 'yyy/test2'
    });
    log.debug('Items in directory "test2": ' + list.length);

    // Try to remove the directory again
    log.debug('Removing directory "test2"...');
    try {
        connection.removeDirectory({
            path: 'yyy/test2'
        });
        log.debug('Directory removed!');
    } catch (e) {
        log.debug('Directory not removed!');
        log.error(e.message);
    }

    // Try to list the removed directory
    log.debug('Trying to list directory "test2"...');
    try {
        list = connection.list({
            path: 'yyy/test2'
        });
    } catch (e) {
        log.error(e.message);
    }
});
```

📄 Sample 3: Set Conditional Default Settings Using N/sftp Enums

```
/**
 * @NApiVersion 2.1
 * @NScriptType UserEventScript
 * @NModuleScope SameAccount
 */

define(['N/file', 'N/sftp', 'N/error'],
function(file, sftp, error) {
    return {
        beforeLoad: function(){
            var portNumber = -1;
            var connectTimeout = -1;
            var transferTimeout = -1;
            //these variables can be taken as parameters of the script instead

            if (portNumber &lt; sftp.MIN_PORT_NUMBER || portNumber &gt; sftp.MAX_PORT_NUMBER)
                portNumber = sftp.DEFAULT_PORT_NUMBER;
            if (connectTimeout &lt; sftp.MIN_CONNECT_TIMEOUT)
                connectTimeout = sftp.MIN_CONNECT_TIMEOUT;
            else if (connectTimeout &gt; sftp.MAX_CONNECT_TIMEOUT)
                connectTimeout = sftp.MAX_CONNECT_TIMEOUT;

            var connection = sftp.createConnection({
                username: 'sftpuser',
                keyId: 'custkey1',
                url: '192.168.0.100',
                port: portNumber,
                directory: 'inbound',
                timeout: connectTimeout,
                hostKey: "AAAAB3NzaC1yc2EAAAADAQABAAABAQDMifKH2vTxdiype8nem7+lS3x7dTQR/A67KdsR/5C2WUcDipBzYhHbnG6Am12Nd2tlM01LnaBZA6/8P4Y9x/sGTxtsdE/MzeGDUBn
6HBlQvgIrhX62wgoKGQ+P2lEAO1+Vz8y3/MB1NmD7Fc62cJ9Mu88YA6jwJOIPZeHYNVyIm9OrY6VyzYyvSJhH0x7SXyvGnijJQF4G8C4c8u/UVpF/sE16xKZtly2Rx0aDL2FsDRtpyPmM602/R6ISbsmgab3M
zzAEIu+zLDMdIBJn3cDhNt1F7Rar6Tu0u18KCkk8GPxbnxDuG4sCNOnXPYkDXSMUbM/ocRjYGtqdZUMmeTf3"
            });

            // can also be a big file (created for example by async search)
            var myFileToUpload = file.create({
                name: 'originalname.txt',
                fileType: file.Type.PLAINTEXT,
                contents: 'I am a test file.'
            });

            if (myFileToUpload.size &gt; connection.MAX_FILE_SIZE)
                throw error.create({name:"FILE_IS_TOO_BIG", message:"The file you are trying to upload is too big"});

            var minTransferTimeout = 10;
            if (transferTimeout &gt; connection.MAX_TRANSFER_TIMEOUT)
                transferTimeout = connection.MAX_TRANSFER_TIMEOUT;
            else if (transferTimeout &lt; minTransferTimeout)
                transferTimeout = minTransferTimteout;

            connection.upload({
                directory: 'files',
                filename: 'test.txt',
                file: myFileToUpload,
                replaceExisting: true,
                timeout: transferTimeout
            });
        }
    };
});
```

42. N/sso Module

Note: As of NetSuite 2025.1, the support ended for the SuiteSignOn feature, which means the N/sso module is also no longer supported. Load the N/sso module to generate outbound single sign-on (SuiteSignOn) tokens.

`define(['N/sso'], (sso) ⇒ { ... })` ✓ 1 Code Sample(s) Scripts: Client and server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/sso'], (sso) ⇒ { // Access methods and properties on sso });
```

As of NetSuite 2025.1, support ended for the SuiteSignOn feature, which includes the N/sso module. If you still use SuiteSignOn or the N/sso module in your customizations, you must transition to a supported alternative, such as SuiteScript OAuth 2.0 or OpenID Connect (OIDC) via N/auth and N/https modules.

Deprecation & Migration Notice

Status: Deprecated as of NetSuite release 2025.1.

Alternative: Use N/auth and N/https modules for OAuth 2.0 and OpenID Connect authentication workflows.

Legacy Capabilities: Previously provided single sign-on authentication token generation.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: Migration from N/sso to OAuth 2.0 / OIDC Authentication

```
/**
 * @NApiVersion 2.1
 * @NScriptType UserEventScript
 *
 * Note: N/sso is deprecated as of NetSuite 2025.1.
 * This sample demonstrates the official supported migration to OAuth 2.0 / OIDC using N/https and N/auth.
 */
define(['N/https', 'N/auth', 'N/log'], (https, auth, log) ⇒ {
    const beforeSubmit = (context) ⇒ {
        // Modern replacement: Authenticate using OAuth 2.0 client credentials or token exchange
        try {
            const tokenResponse = https.post({
                url: 'https://oauth2.provider.com/token',
                headers: {
                    'Content-Type': 'application/x-www-form-urlencoded'
                },
                body: {
                    grant_type: 'client_credentials',
                    client_id: 'YOUR_CLIENT_ID',
                    client_secret: 'YOUR_CLIENT_SECRET'
                }
            });

            const tokenData = JSON.parse(tokenResponse.body);
            log.audit('OAuth 2.0 Auth', 'Obtained token: ' + tokenData.access_token);
        } catch (e) {
            log.error('Authentication Error', e);
        }
    };
    return { beforeSubmit };
});
```

43. N/suiteAppInfo Module

Load the N/suiteAppInfo module when you want to access information related to SuiteApps and Bundles.

`define(['N/suiteAppInfo'], (suiteAppInfo) ⇒ { ... })` ✓ 1 Code Sample(s) Scripts: Client and server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/suiteAppInfo'], (suiteAppInfo) ⇒ { // Access methods and properties on suiteAppInfo
});
```

Use the N/suiteAppInfo module to access information related to SuiteApps and Bundles. This module is available for all script types.

N/suiteAppInfo Module Script Sample

N/suiteAppInfo Module Members

Return Type / Value Type

Supported Script Types

suiteAppInfo.isBundleInstalled(options)

Client and server scripts

Returns true if the specific bundle is installed.

suiteAppInfo.isSuiteAppInstalled(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**suiteAppInfo.isBundleInstalled(options)**	`boolean`	Client and server scripts	Returns true if the specific bundle is installed.
METHOD	**suiteAppInfo.isSuiteAppInstalled(options)**	`boolean`	Client and server scripts	Returns true if the specified SDF SuiteApp is installed.
METHOD	**suiteAppInfo.listBundlesContainingScripts(options)**	`Object`	Client and server scripts	Returns the IDs for bundles that contain the specified script, for each script specified.
METHOD	**suiteAppInfo.listInstalledBundles()**	`Object[]`	Client and server scripts	Returns a list of bundles that are installed.
METHOD	**suiteAppInfo.listInstalledSuiteApps()**	`Object[]`	Client and server scripts	Returns a list of SDF SuiteApps that are installed.
METHOD	**suiteAppInfo.listSuiteAppsContainingScripts(options)**	`Object`	Client and server scripts	Returns the ID for the SDF SuiteApp that contains the specified script, for each script specified.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: N/suiteAppInfo Module Script Sample

```
/*
 * @NApiVersion 2.x
 *
 */

// This script sample uses each method available in the N/suiteAppInfo module to retrieve information about installed bundles and SuiteApps installed in the
current account.

require(['N/suiteAppInfo'], function (suiteAppInfo){
    var isBundleInstalled = suiteAppInfo.isBundleInstalled({
        bundleId: '1234'
    });

   var isSuiteAppInstalled = suiteAppInfo.isSuiteAppInstalled({
        suiteAppId: '789'
    });

    var allMyBundlesInstalled = suiteAppInfo.listInstalledBundles();

    var allMySuiteAppsInstalled = suiteAppInfo.listInstalledSuiteApps();

    var myScripts = {"scriptA", "scriptB", "scriptC"};
    var scriptInBundles = suiteAppInfo.listBundlesContainingScripts({
        scriptIds: myScripts
    });

    var scriptInSuiteApps = suiteAppInfo.listSuiteAppsContainingScripts({
        scriptsIds: myScripts
    });
})
```

44. N/task Module

Load the N/task module to create tasks and place them in the internal NetSuite scheduling or task queue. Use this module to schedule scripts, run Map/Reduce scripts, import CSV files, merge duplicate records, and execute asynchronous workflows.

`define(['N/task'], (task) => { ... })` ✓ 5 Code Sample(s) Scripts: Server scripts Permissions: Tasks

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/task'], (task) => { // Access methods and properties on task });
```

Use the N/task module to create tasks and place them in the internal NetSuite scheduling or task queue. You can use this module to create tasks for the following:

To submit a scheduled script

To run a map/reduce script

To import CSV files

To merge duplicate records

To execute asynchronous searches, asynchronous document capture tasks, constructed queries, SuiteQL queries, and workflows

Each task is a specific task type (task.TaskType) and each task type has its own corresponding object types. Use the methods available to each object type to configure, submit, and monitor the tasks.

Regardless of task type, tasks are always triggered asynchronously.

N/task Module Members

Return Type / Value Type

Supported Script Types

task.CsvImportTask

The properties of a CSV import task.

Use the methods and properties for this object to submit a CSV import task into the task queue and asynchronously import record data into NetSuite.

task.CsvImportTaskStatus

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	task.CsvImportTask	Object	Server scripts	The properties of a CSV import task. Use the methods and properties for this object to submit a CSV import task into the task queue and asynchronously import record data into NetSuite.
OBJECT	task.CsvImportTaskStatus	Object	Server scripts	The status of a CSV import task placed into the NetSuite scheduling queue.
OBJECT	task.DocumentCaptureTask	Object	Server scripts	The properties of a document capture task. Use the methods and properties of this object to submit a document capture task into the NetSuite task queue.
OBJECT	task.DocumentCaptureTaskStatus	Object	Server scripts	The status of a document capture task placed into the NetSuite task queue.
OBJECT	task.EntityDeduplicationTask	Object	Server scripts	All the properties of a merge duplicate records task request. Use the methods and properties of this object to submit a merge duplicate record job task into the NetSuite task queue.
OBJECT	task.EntityDeduplicationTaskStatus	Object	Server scripts	The status of a merge duplicate record task placed into the NetSuite task queue.
OBJECT	task.MapReduceScriptTask	Object	Server scripts	A map/reduce script deployment.
OBJECT	task.MapReduceScriptTaskStatus	Object	Server scripts	The status of a map/reduce script deployment that has been submitted for processing.
OBJECT	task.QueryTask	Object	Server scripts	The properties of a query task. Use the methods and properties of this object to submit a query task into the NetSuite task queue.
OBJECT	task.QueryTaskStatus	Object	Server scripts	The status of a query task placed into the NetSuite task queue.
OBJECT	task.RecordActionTask	Object	Server scripts	The properties of a record action task. Use this object to place a record action task into the NetSuite scheduling queue.
OBJECT	task.RecordActionTaskStatus	Object	Server scripts	The status of a record action task in the NetSuite scheduling queue.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	task.ScheduledScriptTask	Object	Server scripts	All the properties of a scheduled script task in SuiteScript. Use this object to place a scheduled script deployment into the NetSuite scheduling queue.
OBJECT	task.ScheduledScriptTaskStatus	Object	Server scripts	The status of a scheduled script placed into the NetSuite scheduling queue.
OBJECT	task.SearchTask	Object	Server scripts	The properties required to initiate an asynchronous search.
OBJECT	task.SearchTaskStatus	Object	Server scripts	The status of an asynchronous search initiation task that is placed into the NetSuite task queue.
OBJECT	task.SuiteQLTask	Object	Server scripts	The properties of a SuiteQL task. Use the methods and properties of this object to submit a query task into the NetSuite task queue.
OBJECT	task.SuiteQLTaskStatus	Object	Server scripts	The status of a SuiteQL task placed into the NetSuite task queue.
OBJECT	task.WorkflowTriggerTask	Object	Server scripts	All the properties required to asynchronously initiate a workflow. Use WorkflowTriggerTask to create a task that initiates an instance of a specific workflow.
OBJECT	task.WorkflowTriggerTaskStatus	Object	Server scripts	The status of an asynchronous workflow initiation task placed into the NetSuite task queue.
METHOD	task.checkStatus(options)	task.CsvImportTaskStatus | task.DocumentCaptureTaskStatus | task.EntityDeduplicationTaskStatus | task.MapReduceScriptTaskStatus | task.QueryTaskStatus | task.RecordActionTaskStatus | task.ScheduledScriptTaskStatus | task.SearchTaskStatus | task.SuiteQLTaskStatus |task.WorkflowTriggerTaskStatus	Server scripts	Returns a task status object associated with a specific task ID.
METHOD	task.create(options)	task.CsvImportTask | task.DocumentCaptureTask | task.EntityDeduplicationTask | task.MapReduceScriptTask | task.QueryTask | task.RecordActionTask | task.ScheduledScriptTask | task.SearchTask | task.SuiteQLTask | task.WorkflowTriggerTask	Server scripts	Creates an object for a specific task type and returns the task object.
ENUM	task.ActionCondition	enum	Server scripts	Holds the string values for the possible record action conditions. This enum is returned by RecordActionTask.condition.
ENUM	task.DedupeEntityType	enum	Server scripts	Holds the string values for entity types for which you can merge duplicate records with task.EntityDeduplicationTask.
ENUM	task.DedupeMode	enum	Server scripts	Holds the string values for available deduplication modes when merging duplicate records with task.EntityDeduplicationTask. Use this enum to set the EntityDeduplicationTask.entityType.
ENUM	task.MapReduceStage	enum	Server scripts	Holds the string values for the stages of a map/reduce script deployment, which is encapsulated by the task.MapReduceScriptTask object. This enum is returned by MapReduceScriptTaskStatus.stage.
ENUM	task.MasterSelectionMode	enum	Server scripts	Holds the string values for supported master selection modes when merging duplicate records with task.EntityDeduplicationTask. Use this enum to set the EntityDeduplicationTask.masterSelectionMode property.
ENUM	task.TaskStatus	enum	Server scripts	Holds the string values for the possible status of tasks created and submitted with the N/task Module.
ENUM	task.TaskType	enum	Server scripts	Holds the string values for the types of task objects you can create using task.create(options). Use this enum to set the value for the options.taskType parameter of the task.create(options) method.

CsvImportTask Object Members

The following members are available for a task.CsvImportTask object.

Return Type / Value Type

Supported Script Types

CsvImportTask.submit()

Directs NetSuite to place a CSV import task into the NetSuite task queue and returns a unique ID for the task.

You can use this method only in bundle installation scripts, scheduled scripts, and RESTlets.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	CsvImportTask.submit()	string	Server scripts	Directs NetSuite to place a CSV import task into the NetSuite task queue and returns a unique ID for the task. You can use this method only in bundle installation scripts, scheduled scripts, and RESTlets.
PROPERTY	CsvImportTask.id	string	Server scripts	The ID of the task.
PROPERTY	CsvImportTask.importFile	file.File | string	Server scripts	CSV file to import. Use a file.File object or a string that represents the CSV text to be imported.
PROPERTY	CsvImportTask.linkedFiles	Object	Server scripts	A map of key-value pairs that sets the data to be imported in a linked file for a multi-file import job, by referencing a file in the File Cabinet or the raw CSV data to import.
PROPERTY	CsvImportTask.mappingId	number | string	Server scripts	Script ID or internal ID of the saved import map that you created when you ran the Import Assistant.
PROPERTY	CsvImportTask.name	string	Server scripts	Name for the CSV import task.
PROPERTY	CsvImportTask.queueId	number	Server scripts	Overrides the Queue Number property under Advanced Options on the Import Options page of the Import Assistant.

CsvImportTaskStatus Object Members

The following members are available for a task.CsvImportTaskStatus object.

Return Type / Value Type

Supported Script Types

CsvImportTaskStatus.status

string (read-only)

Status for a CSV import task. Returns a task.TaskStatus enum value.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	CsvImportTaskStatus.status	string (read-only)	Server scripts	Status for a CSV import task. Returns a task.TaskStatus enum value.
PROPERTY	CsvImportTaskStatus.taskId	string (read-only)	Server scripts	The task ID associated with the specified task.

DocumentCaptureTask Object Members

The following members are available for a task.DocumentCaptureTask object.

Return Type / Value Type

Supported Script Types

DocumentCaptureTask.addInboundDependency(options)

Adds a scheduled script task to the document capture task as a dependent task.

DocumentCaptureTask.submit()

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	DocumentCaptureTask.addInboundDependency(options)	void	Server scripts	Adds a scheduled script task to the document capture task as a dependent task.
METHOD	DocumentCaptureTask.submit()	string	Server scripts	Submits the document capture task for asynchronous processing and returns the task ID.
PROPERTY	DocumentCaptureTask.documentType	string	Server scripts	The document type.
PROPERTY	DocumentCaptureTask.features	string[]	Server scripts	The features to extract from the document (such as fields, tables, or text).
PROPERTY	DocumentCaptureTask.id	string	Server scripts	The ID of the task.
PROPERTY	DocumentCaptureTask.inboundDependencies	Object[]	Server scripts	Key-value pairs that contain information about the dependent tasks added to the document capture task.
PROPERTY	DocumentCaptureTask.inputFile	file.File	Server scripts	The document to extract content from.
PROPERTY	DocumentCaptureTask.language	string	Server scripts	The language of the document.
PROPERTY	DocumentCaptureTask.outputFilePath	string	Server scripts	The path of the JSON file to export document capture results to.
PROPERTY	DocumentCaptureTask.ociConfig	Object	Server scripts	Important: This object is no longer supported. Any values specified in this object are ignored.

DocumentCaptureTaskStatus Object Members

The following members are available for a task.DocumentCaptureTaskStatus object.

Return Type / Value Type

Supported Script Types

DocumentCaptureTaskStatus.status

string (read-only)

The status for a document capture task (as a task.TaskStatus enum value).

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	DocumentCaptureTaskStatus.status	string (read-only)	Server scripts	The status for a document capture task (as a task.TaskStatus enum value).
PROPERTY	DocumentCaptureTaskStatus.taskId	string (read-only)	Server scripts	The task ID associated with the specified task.

EntityDeduplicationTask Object Members

The following members are available for a task.EntityDeduplicationTask object.

Return Type / Value Type

Supported Script Types

EntityDeduplicationTask.submit()

Directs NetSuite to place the merge duplicate records task into the NetSuite task queue and returns a unique ID for the task.

EntityDeduplicationTask.dedupeMode

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	EntityDeduplicationTask.submit()	string	Server scripts	Directs NetSuite to place the merge duplicate records task into the NetSuite task queue and returns a unique ID for the task.
PROPERTY	EntityDeduplicationTask.dedupeMode	string	Server scripts	The mode in which to merge or delete duplicate records. Use values from the task.DedupeMode enum.
PROPERTY	EntityDeduplicationTask.entityType	string	Server scripts	The type of entity on which you want to merge duplicate records. Use a task.DedupeEntityType enum to set the value.
PROPERTY	EntityDeduplicationTask.id	string	Server scripts	The ID of the task.
PROPERTY	EntityDeduplicationTask.masterRecordId	number	Server scripts	Master record ID. When you merge duplicate records, you can delete all duplicates for a record or merge information from the duplicate records into the master record.
PROPERTY	EntityDeduplicationTask.masterSelectionMode	string	Server scripts	Master selection mode. Use values from the task.MasterSelectionMode enum.
PROPERTY	EntityDeduplicationTask.recordIds	number[]	Server scripts	Number array of record internal IDs to perform the merge or delete operation on.

EntityDeduplicationTaskStatus Object Members

The following members are available for a task.EntityDeduplicationTaskStatus object.

Return Type / Value Type

Supported Script Types

EntityDeduplicationTaskStatus.status

string (read-only)

Status for a merge duplicate record task. Returns a task.TaskStatus enum value.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	EntityDeduplicationTaskStatus.status	string (read-only)	Server scripts	Status for a merge duplicate record task. Returns a task.TaskStatus enum value.
PROPERTY	EntityDeduplicationTaskStatus.taskId	string (read-only)	Server scripts	The task ID associated with the specified task.

MapReduceScriptTask Object Members

The following members are available for a task.MapReduceScriptTask object.

Return Type / Value Type

Supported Script Types

MapReduceScriptTask.submit()

Submits a map/reduce script deployment for processing.

MapReduceScriptTask.deploymentId

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	MapReduceScriptTask.submit()	string	Server scripts	Submits a map/reduce script deployment for processing.
PROPERTY	MapReduceScriptTask.deploymentId	string	Server scripts	Script ID (as a string), for the script deployment record for a map/reduce script.
PROPERTY	MapReduceScriptTask.id	string	Server scripts	The ID of the task.
PROPERTY	MapReduceScriptTask.params	Object	Server scripts	Object that represents key-value pairs that override static script parameter field values on the script deployment record.
PROPERTY	MapReduceScriptTask.scriptId	number | string	Server scripts	Internal ID (as a number), or script ID (as a string), for the map/reduce script record.

MapReduceScriptTaskStatus Object Members

The following members are available for a task.MapReduceScriptTaskStatus object.

Return Type / Value Type

Supported Script Types

MapReduceScriptTaskStatus.getCurrentTotalSize()

Returns the total size in bytes of all stored work in progress by a task.MapReduceScriptTask.

MapReduceScriptTaskStatus.getPendingMapCount()

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	MapReduceScriptTaskStatus.getCurrentTotalSize()	number	Server scripts	Returns the total size in bytes of all stored work in progress by a task.MapReduceScriptTask.
METHOD	MapReduceScriptTaskStatus.getPendingMapCount()	number	Server scripts	Returns the total number of records or rows not yet processed by the map stage of a task.MapReduceScriptTask.
METHOD	MapReduceScriptTaskStatus.getPendingMapSize()	number	Server scripts	Returns the total number of bytes not yet processed by the map stage, as a component of total size, of a task.MapReduceScriptTask.
METHOD	MapReduceScriptTaskStatus.getPendingOutputCount()	number	Server scripts	Returns the total number of records or rows not yet processed by a task.MapReduceScriptTask.
METHOD	MapReduceScriptTaskStatus.getPendingOutputSize()	number	Server scripts	Returns the total size in bytes of all key-value pairs written as output, as a component of total size, by a task.MapReduceScriptTask.
METHOD	MapReduceScriptTaskStatus.getPendingReduceCount()	number	Server scripts	Returns the total number of records or rows not yet processed by the reduce stage of a task.MapReduceScriptTask.
METHOD	MapReduceScriptTaskStatus.getPendingReduceSize()	number	Server scripts	Returns the total number of bytes not yet processed by the reduce stage, as a component of total size, of a task.MapReduceScriptTask.
METHOD	MapReduceScriptTaskStatus.getPercentageCompleted()	number	Server scripts	Returns the current percentage complete for the current stage of a task.MapReduceScriptTask.
METHOD	MapReduceScriptTaskStatus.getTotalMapCount()	number	Server scripts	Returns the total number of records or rows passed as input to the map stage of a task.MapReduceScriptTask.
METHOD	MapReduceScriptTaskStatus.getTotalOutputCount()	number	Server scripts	Returns the total number of records or rows passed as inputs to the output phase of a task.MapReduceScriptTask.
METHOD	MapReduceScriptTaskStatus.getTotalReduceCount()	number	Server scripts	Returns the total number of record or row inputs to the reduce stage of a task.MapReduceScriptTask.
PROPERTY	MapReduceScriptTaskStatus.deploymentId	string (read-only)	Server scripts	Script ID for a script deployment record associated with a specific task.MapReduceScriptTask.
PROPERTY	MapReduceScriptTaskStatus.scriptId	number (read-only)	Server scripts	Internal ID for a map/reduce script record associated with a specific task.MapReduceScriptTask.
PROPERTY	MapReduceScriptTaskStatus.stage	string (read-only)	Server scripts	The current stage of a map/reduce script deployment that is being processed. See task.MapReduceStage for supported values.
PROPERTY	MapReduceScriptTaskStatus.status	string (read-only)	Server scripts	Status for a map/reduce script task. Returns a task.TaskStatus enum value.
PROPERTY	MapReduceScriptTaskStatus.taskId	string (read-only)	Server scripts	The task ID associated with the specified task.

QueryTask Object Members

The following members are available for a task.QueryTask object.

Return Type / Value Type

Supported Script Types

QueryTask.addInboundDependency(options)

Adds a scheduled script task or map/reduce script task to the query task as a dependent task.

QueryTask.submit()

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	QueryTask.addInboundDependency(options)	void	Server scripts	Adds a scheduled script task or map/reduce script task to the query task as a dependent task.
METHOD	QueryTask.submit()	string	Server scripts	Submits the query task for asynchronous processing and returns the task ID.
PROPERTY	QueryTask.fileId	number	Server scripts	Internal ID of the CSV file to export query results to. This property is mutually exclusive with the QueryTask.filePath parameter.
PROPERTY	QueryTask.filePath	string	Server scripts	Path of the CSV file to export query results to. This property is mutually exclusive with the QueryTask.fileId property.
PROPERTY	QueryTask.id	string	Server scripts	The ID of the task.
PROPERTY	QueryTask.inboundDependencies	Object[]	Server scripts	Key-value pairs that contain information about the dependent tasks added to the query task.
PROPERTY	QueryTask.query	string	Server scripts	Query definition for the query task.

QueryTaskStatus Object Members

The following members are available for a task.QueryTaskStatus object.

Return Type / Value Type

Supported Script Types

QueryTaskStatus.fileId

number (read-only)

Internal ID of the CSV file that query results are exported to.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	QueryTaskStatus.fileId	number (read-only)	Server scripts	Internal ID of the CSV file that query results are exported to.
PROPERTY	QueryTaskStatus.query	query.Query (read-only)	Server scripts	Query definition for the submitted query task.
PROPERTY	QueryTaskStatus.status	string (read-only)	Server scripts	Status of the submitted query task.
PROPERTY	QueryTaskStatus.taskId	string (read-only)	Server scripts	ID of the submitted query task.

RecordActionTask Object Members

The following members are available for a task.RecordActionTaskStatus object.

Return Type / Value Type

Supported Script Types

RecordActionTask.submit()

Submits a record action task for processing and returns its task ID.

RecordActionTask.action

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	RecordActionTask.submit()	string	Server scripts	Submits a record action task for processing and returns its task ID.
PROPERTY	RecordActionTask.action	string	Server scripts	The ID of the action to be invoked.
PROPERTY	RecordActionTask.condition	Object	Server scripts	The condition used to select record IDs of records for which the action is to be executed. Only the action.ALL_QUALIFIED_INSTANCES constant is currently supported.
PROPERTY	RecordActionTask.id	string	Server scripts	The ID of the task.
PROPERTY	RecordActionTask.paramCallback	Object	Server scripts	Function that takes record ID and returns the parameter object for the specified record ID.
PROPERTY	RecordActionTask.params	Object[]	Server scripts	An array of parameter objects. Each object corresponds to one record ID of the record for which the action is to be executed. The object has the following form: {recordId: 1, someParam: 'example1', otherParam: 'example2'}
PROPERTY	RecordActionTask.recordType	string	Server scripts	The record type on which the action is to be performed. For a list of record types, see record.Type.

RecordActionTaskStatus Object Members

The following members are available for a task.RecordActionTaskStatus object.

Return Type / Value Type

Supported Script Types

RecordActionTaskStatus.complete

number (read-only)

The number of record action tasks with a completed status.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	RecordActionTaskStatus.complete	number (read-only)	Server scripts	The number of record action tasks with a completed status.
PROPERTY	RecordActionTaskStatus.errors	Object (read-only)	Server scripts	The error details of failed action executions. The value of the property is the record instance ID and the corresponding error details. The error details are returned in an unnamed object with two properties: code and message.
PROPERTY	RecordActionTaskStatus.failed	number (read-only)	Server scripts	The number of record action tasks with a failed status.
PROPERTY	RecordActionTaskStatus.pending	number (read-only)	Server scripts	The number of record action tasks with a pending status.
PROPERTY	RecordActionTaskStatus.results	Object (read-only)	Server scripts	The results of successfully executed record action tasks. The value of the property is the task instance ID and the corresponding action result.
PROPERTY	RecordActionTaskStatus.status	string (read-only)	Server scripts	Represents the record action task status. Returns a value from the task.TaskStatus enum.
PROPERTY	RecordActionTaskStatus.succeeded	number (read-only)	Server scripts	The number of record action tasks with a succeeded status.
PROPERTY	RecordActionTaskStatus.taskId	string (read-only)	Server scripts	The task ID associated with the specified task.

ScheduledScriptTask Object Members

The following members are available for a task.ScheduledScriptTask object.

Return Type / Value Type

Supported Script Types

ScheduledScriptTask.submit()

Directs NetSuite to place a scheduled script deployment into the NetSuite scheduling queue and returns a unique ID for the task.

ScheduledScriptTask.deploymentId

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	ScheduledScriptTask.submit()	string	Server scripts	Directs NetSuite to place a scheduled script deployment into the NetSuite scheduling queue and returns a unique ID for the task.
PROPERTY	ScheduledScriptTask.deploymentId	string	Server scripts	Script ID (as a string), for the script deployment record associated with a task.ScheduledScriptTask object.
PROPERTY	ScheduledScriptTask.id	string	Server scripts	The ID of the task.
PROPERTY	ScheduledScriptTask.params	Object	Server scripts	Object with key-value pairs that override the static script parameter field values on the script deployment.
PROPERTY	ScheduledScriptTask.scriptId	number | string	Server scripts	Internal ID (as a number), or script ID (as a string) for the script record associated with a task.ScheduledScriptTask object.

ScheduledScriptTaskStatus Object Members

The following members are available for a task.ScheduledScriptTaskStatus object.

Return Type / Value Type

Supported Script Types

ScheduledScriptTaskStatus.deploymentId

string (read-only)

Script ID for a script deployment record associated with a specific task.ScheduledScriptTask object.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	ScheduledScriptTaskStatus.deploymentId	string (read-only)	Server scripts	Script ID for a script deployment record associated with a specific task.ScheduledScriptTask object.
PROPERTY	ScheduledScriptTaskStatus.scriptId	number (read-only)	Server scripts	Internal ID for a script record associated with a specific task.ScheduledScriptTask object.
PROPERTY	ScheduledScriptTaskStatus.status	string (read-only)	Server scripts	Status for a scheduled script task. Returns a task.TaskStatus enum value.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	ScheduledScriptTaskStatus.taskId	string (read-only)	Server scripts	The task ID associated with the specified task.

SearchTask Object Members

The following members are available for a task.SearchTask object.

Return Type / Value Type

Supported Script Types

SearchTask.addInboundDependency()

Adds a scheduled script task or map/reduce script task to the search task as a dependent script. Dependent scripts are processed automatically when the search task is complete. For more information, see SuiteCloud Processors.

SearchTask.submit()

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	SearchTask.addInboundDependency()	void	Server scripts	Adds a scheduled script task or map/reduce script task to the search task as a dependent script. Dependent scripts are processed automatically when the search task is complete. For more information, see SuiteCloud Processors.
METHOD	SearchTask.submit()	string	Server scripts	Places the asynchronous search initiation task into the SuiteScript task queue, and returns a unique ID for the task.
PROPERTY	SearchTask.fileId	number	Server scripts	ID of the CSV file to export search results into.
PROPERTY	SearchTask.filePath	string	Server scripts	Path of the CSV file to export search results into.
PROPERTY	SearchTask.id	string	Server scripts	The ID of the task.
PROPERTY	SearchTask.inboundDependencies	Object[] (read-only)	Server scripts	Key-value pairs to describe the dependent scripts added to the search task.
PROPERTY	SearchTask.savedSearchId	number	Server scripts	ID of the saved search to be executed during the task.

SearchTaskStatus Object Members

The following members are available for a task.SearchTaskStatus object.

Return Type / Value Type

Supported Script Types

SearchTaskStatus.fileId

number (read-only)

ID of the CSV file into which search results are exported.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	SearchTaskStatus.fileId	number (read-only)	Server scripts	ID of the CSV file into which search results are exported.
PROPERTY	SearchTaskStatus.savedSearchId	number (read-only)	Server scripts	ID of the saved search executed during the task.
PROPERTY	SearchTaskStatus.status	string (read-only)	Server scripts	Status of an asynchronous search task placed in the NetSuite task queue. Returns one of the task.TaskStatus enum values.
PROPERTY	SearchTaskStatus.taskId	string (read-only)	Server scripts	ID of the asynchronous task.

SuiteQLTask Object Members

The following members are available for a task.SuiteQLTask object.

Return Type / Value Type

Supported Script Types

SuiteQLTask.addInboundDependency(options)

Adds a scheduled script task or map/reduce script task to the SuiteQL task as a dependent task.

SuiteQLTask.submit()

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	SuiteQLTask.addInboundDependency(options)	void	Server scripts	Adds a scheduled script task or map/reduce script task to the SuiteQL task as a dependent task.
METHOD	SuiteQLTask.submit()	string	Server scripts	Submits the SuiteQL task for asynchronous processing and returns the task ID.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	SuiteQLTask.fileId	number	Server scripts	Internal ID of the CSV file to export SuiteQL query results to. This property is mutually exclusive with the SuiteQLTask.filePath parameter.
PROPERTY	SuiteQLTask.filePath	string	Server scripts	Path of the CSV file to export SuiteQL query results to. This property is mutually exclusive with the SuiteQLTask.fileId property.
PROPERTY	SuiteQLTask.id	string	Server scripts	The ID of the task.
PROPERTY	SuiteQLTask.inboundDependencies	Object[]	Server scripts	Key-value pairs that contain information about the dependent tasks added to the SuiteQL task.
PROPERTY	SuiteQLTask.params	Array<string | boolean | number>	Server scripts	Parameters for the SuiteQL query.
PROPERTY	SuiteQLTask.query	string	Server scripts	SuiteQL query definition for the SuiteQL task.

SuiteQLTaskStatus Object Members

The following members are available for a task.SuiteQLTaskStatus object.

Return Type / Value Type

Supported Script Types

SuiteQLTaskStatus.fileId

number (read-only)

Internal ID of the CSV file that SuiteQL query results are exported to.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	SuiteQLTaskStatus.fileId	number (read-only)	Server scripts	Internal ID of the CSV file that SuiteQL query results are exported to.
PROPERTY	SuiteQLTaskStatus.params	Array<string | boolean | number> (read-only)	Server scripts	Parameters for the SuiteQL query.
PROPERTY	SuiteQLTaskStatus.query	string (read-only)	Server scripts	SuiteQL query definition for the SuiteQL task.
PROPERTY	SuiteQLTaskStatus.status	string (read-only)	Server scripts	Status of the SuiteQL task.
PROPERTY	SuiteQLTaskStatus.taskId	string (read-only)	Server scripts	ID of the submitted SuiteQL task.

WorkflowTriggerTask Object Members

The following members are available for a task.WorkflowTriggerTask object.

Return Type / Value Type

Supported Script Types

WorkflowTriggerTask.submit()

Directs NetSuite to place the asynchronous workflow initiation task into the NetSuite scheduling queue and returns a unique ID for the task.

WorkflowTriggerTask.id

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	WorkflowTriggerTask.submit()	string	Server scripts	Directs NetSuite to place the asynchronous workflow initiation task into the NetSuite scheduling queue and returns a unique ID for the task.
PROPERTY	WorkflowTriggerTask.id	string	Server scripts	The ID of the task.
PROPERTY	WorkflowTriggerTask.params	Object	Server scripts	Object that contains key-value pairs to set default values on fields specific to the workflow.
PROPERTY	WorkflowTriggerTask.recordId	number	Server scripts	Internal ID of the workflow definition base record. For example, 55 or 124.
PROPERTY	WorkflowTriggerTask.recordType	string	Server scripts	Record type of the workflow base record. For example, customer, salesorder, or lead.
PROPERTY	WorkflowTriggerTask.workflowId	number | string	Server scripts	Internal ID (as a number), or script ID (as a string), for the workflow definition.

WorkflowTriggerTaskStatus Object Members

The following members are available for a task.WorkflowTriggerTaskStatus object.

Return Type / Value Type

Supported Script Types

WorkflowTriggerTaskStatus.status

string (read-only)

Status for a asynchronous workflow placed in the NetSuite task queue. Returns a value from the task.TaskStatus enum.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	WorkflowTriggerTaskStatus.status	string (read-only)	Server scripts	Status for a asynchronous workflow placed in the NetSuite task queue. Returns a value from the task.TaskStatus enum.
PROPERTY	WorkflowTriggerTaskStatus.taskId	string (read-only)	Server scripts	The task ID associated with the specified task.

▶ Official NetSuite Script Samples (5 Sample(s))

📄 Sample 1: Create and Submit a Map/Reduce Script Task

```
/**
 * @NApiVersion 2.1
 */
require(['N/task', 'N/runtime', 'N/email'], (task, runtime, email) =&gt; {
    // Store the script ID of the script to submit
    //
    // Update the following statement so it uses the script ID
    // of the map/reduce script record you want to submit
    const mapReduceScriptId = 'customscript_test_mapreduce_script';

    // Create a map/reduce task
    //
    // Update the deploymentId parameter to use the script ID of
    // the deployment record for your map/reduce script
    let mrTask = task.create({
        taskType: task.TaskType.MAP_REDUCE,
        scriptId: mapReduceScriptId,
        deploymentId: 'customdeploy_test_mapreduce_script'
    });

    // Submit the map/reduce task
    let mrTaskId = mrTask.submit();

    // Check the status of the task, and send an email if the
    // task has a status of FAILED
    //
    // Update the authorId value with the internal ID of the user
    // who is the email sender. Update the recipientEmail value
    // with the email address of the recipient.
    let taskStatus = task.checkStatus(mrTaskId);
    if (taskStatus.status === 'FAILED') {
        const authorId = -5;
        const recipientEmail = 'notify@myCompany.com';
        email.send({
            author: authorId,
            recipients: recipientEmail,
            subject: 'Failure executing map/reduce job!',
            body: 'Map reduce task: ' + mapReduceScriptId + ' has failed.'
        });
    }
});
```

📄 Sample 2: Create and Submit an Asynchronous Search Task and Export the Results into a CSV File

```
/**
 * @NApiVersion 2.1
 */
require(['N/task'], task =&gt; {
    // Do one of the following:
    //
    // - Create a saved search and capture its ID. To do this, you can use
    //   the following code snippet (replacing the type, id, filters, and
    //   columns values as appropriate):
    //
    //   let mySearch = search.create({
    //       type: search.Type.SALES_ORDER,
    //       id: 'customsearch_my_search',
    //       filters: [...],
    //       columns: [...]
    //   });
    //   mySearch.save();
    //   let savedSearchId = mySearch.searchId;
    //
    // - Use the ID of an existing saved search. This is the approach that
    //   this script sample uses. Update the following statement with the
    //   internal ID of the search you want to use.
    const savedSearchId = 669;

    // Create the search task
    let myTask = task.create({
        taskType: task.TaskType.SEARCH
    });
    myTask.savedSearchId = savedSearchId;

    // Specify the ID of the file that search results will be exported into
    //
    // Update the following statement so it uses the internal ID of the file
    // you want to use
    myTask.fileId = 448;

    // Submit the search task
    let myTaskId = myTask.submit();

    // Retrieve the status of the search task
    let taskStatus = task.checkStatus({
        taskId: myTaskId
    });

    // Optionally, add logic that executes when the task is complete
    if (taskStatus.status === task.TaskStatus.COMPLETE) {
        // Add any code that is appropriate. For example, if this script created
        // a saved search, you may want to delete it.
    }
});
```

📄 Sample 3: Create and Submit a Task with Dependent Scripts

```
/**
 * @NApiVersion 2.1
 */

require(['N/task'], (task) =&gt; {
    // Specify a file for the search results
    const asyncSearchResultFile = 'SuiteScripts/ExportFile.csv';

    // Create a map/reduce script task
    const mapReduceScript = task.create({
        taskType: task.TaskType.MAP_REDUCE
    });
    mapReduceScript.scriptId = 'customscript_mr_create_submit';
    mapReduceScript.deploymentId = 'customdeploy_mr_create_submit';
    mapReduceScript.params = {
        'custscript_mr_as_srch_res' : asyncSearchResultFile
    };

    // Create the search task
    const asyncTask = task.create({
        taskType: task.TaskType.SEARCH
    };
    asyncTask.savedSearchId = 'customsearch35';
    asyncTask.filePath = asyncSearchResultFile;

    // Add dependent scripts to the search task before it is submitted
    asyncTask.addInboundDependency(mapReduceScript);

    // Submit the search task
    const asyncTaskId = asyncTask.submit();
});
```

📄 Sample 4: Create and Submit a Task with Dependent Scripts

```
/**
 * @NApiVersion 2.1
 * @NScriptType MapReduceScript
 * @NModuleScope SameAccount
 */

define(['N/runtime', 'N/file', 'N/log', 'N/email'], (runtime, file, log, email) =&gt; {
    // Load the search results file, count the number of letters in the file, and
    // store this count in another file

    function getInputData() {
        // Retrieve the ID of the search results file
        //
        // Update the completionScriptParameterName value to use the script
        // ID of the original search task
        const fileId = runtime.getCurrentScript().getParameter({
            name: 'custscript_mr_new_submit_task_field'
        });

        if (!fileId) {
            log.error({
                details: 'fileId is not valid. Please check the script parameter stored in the completionScriptParameterName variable in getInputData().'
            });
        }

        return {
            type: 'file',
            id: fileId
        };
    }

    function map(context) {
        log.debug('map context: ', context);
        context.write({
           key: context.key,
           value: 1
        });
    }

    function reduce(context) {
        log.debug('reduce context: ', context);
        log.debug('context.values.length', context.values.length);

        // Count the number of rows
        let rowCount = 0;
        rowCount = context.values.length - 1 // Subtracting 1 to remove header row from the count.
        log.debug('rowCount: ', rowCount);

       // Send an email to the user who ran the script, and attach the
       // CSV file with the search results
       const completionScriptParameterName = 'custscript_mr_new_submit_task_field';
       const resFileId = runtime.getCurrentScript().getParameter({
          name: completionScriptParameterName
       });
       const fileObj = file.load({
          id: resFileId
       });
       const userId = runtime.getCurrentUser().id;
       email.send({
          author: userId,
          recipients: userId,
          subject: 'Search completed',
          body: 'CSV file attached, ' + rowCount + ' record(s) found.',
          attachments: [fileObj]
       });
    }

    function summarize(summary) {
        const type = summary.toString();
        log.audit({ title: type + ' Usage Consumed ', details: summary.usage });
        log.audit({ title: type + ' Concurrency Number ', details: summary.concurrency });
        log.audit({ title: type + ' Number of Yields ', details: summary.yields });
    }
    return {
        getInputData: getInputData,
        map: map,
        reduce: reduce,
        summarize: summarize
    };
});
```

📄 Sample 5: Submit a Record Action Task and Check Status

```
/**
 * @NApiVersion 2.x
 */

require(['N/task'], function(task) {
    var recordActionTask = task.create({
        taskType: task.TaskType.RECORD_ACTION
    });
    recordActionTask.recordType = 'timebill';
    recordActionTask.action = 'approve';
    recordActionTask.params = [
        {recordId: 1, note: 'This is a note for 1'},
        {recordId: 5, note: 'This is a note for 5'},
        {recordId: 23, note: 'This is a note for 23'}
    ];

    var handle = recordActionTask.submit();

    var res = task.checkStatus({
        taskId: handle
    });   // Returns a RecordActionTaskStatus object
    log.debug('Initial status: ' + res.status);
});
```

45. N/task/accounting/recognition Module

Load the task/accounting/recognition module to merge revenue arrangements or revenue elements. This module lets you combine revenue arrangements or revenue elements from multiple sources to represent a single contract for revenue allocation and recognition.

`define(['N/task/accounting/recognition'], (recognition) ⇒ { ... })` ✓ 3 Code Sample(s) Scripts: Server scripts

Permissions: (Transactions) Revenue Arrangement

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/task/accounting/recognition'], (recognition) ⇒ { // Access methods and properties on recognition });
```

Use the N/task/accounting/recognition module to merge revenue arrangements or revenue elements. A revenue arrangement is a transaction that records the details of a sale for the purposes of revenue allocation and recognition. The N/task/accounting/recognition module lets you combine revenue arrangements or revenue elements from multiple sources to represent a single contract obligation for revenue allocation and recognition.

You can use the recognition.create(options) method to create a merge task that combines entire revenue arrangements or individual revenue elements. This method returns a recognition.MergeArrangementsTask object (when merging revenue arrangements) or recognition.MergeElementsTask object (when merging revenue elements). After you obtain one of these objects, you can set its properties, such as the list of arrangements or elements to merge, the date on the merged revenue arrangement, whether to prospectively merge arrangements, and so on. You can use these properties to specify the same input data that you can specify when you merge revenue arrangements using the NetSuite UI. After you set its properties, you can submit the task for processing. Merge tasks are processed asynchronously.

You cannot merge more than 10,000 revenue elements at one time. An error is thrown in the script if you attempt to merge more than 10,000 revenue elements. For UI related limitations, see Limitations for Creating Transactions.

You can use the recognition.checkStatus(options) method to check the status of a submitted merge task. This method returns a recognition.MergeArrangementsTaskStatus object that describes the current status of the merge task (pending, processing, complete, or failed). This object represents the current status for either a recognition.MergeArrangementsTask or a recognition.MergeElementsTask. If the task completes successfully, this object includes the ID of the merged revenue arrangement record that was created. If the task fails, this object includes an error message that describes the failure.

To merge revenue arrangements or revenue elements using the N/task/accounting/recognition module, the following requirements must be met:

The Advanced Revenue Management feature must be enabled in your account. For more information, see Enabling the Advanced Revenue Management (Essentials) Feature.

Your role must have the (Transactions) Revenue Arrangement permission assigned at a level of Create or higher. For more information, see NetSuite Permissions Overview.

For more information about revenue arrangements, see the following help topics:

Revenue Arrangement Management - This topic describes revenue arrangements in general.

Combination and Modification of Performance Obligations - This topic describes the different types of merge results (combined revenue arrangements and prospective change orders).

Revenue Arrangement - This topic describes the revenue arrangement record type, including scripting considerations, supported script types, and sublist fields.

N/task/accounting/recognition Module Members

Return Type / Value Type

Supported Script Types

recognition.MergeArrangementsTask

Encapsulates a task to merge all of the revenue elements from a specified list of revenue arrangements.

Use recognition.create(options) to create this object.

recognition.MergeArrangementsTaskStatus

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	**recognition.MergeArrangementsTask**	`Object`	Server scripts	Encapsulates a task to merge all of the revenue elements from a specified list of revenue arrangements. Use recognition.create(options) to create this object.
OBJECT	**recognition.MergeArrangementsTaskStatus**	`Object`	Server scripts	Encapsulates the current status of a submitted merge task. Use recognition.checkStatus(options) to create this object.
OBJECT	**recognition.MergeElementsTask**	`Object`	Server scripts	Encapsulates a task to merge all of the specified revenue elements. Use recognition.create(options) to create this object.
METHOD	**recognition.checkStatus(options)**	`recognition.MergeArrangementsTaskStatus`	Server scripts	Checks the status of a submitted merge task.
METHOD	**recognition.create(options)**	`recognition.MergeArrangementsTask | recognition.MergeElementsTask`	Server scripts	Creates a merge task that combines entire revenue arrangements or individual revenue elements. Use values in the recognition.TaskType enum to specify the type of merge task to create.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
ENUM	recognition.TaskStatus	enum	Server scripts	Holds the string values for supported merge task statuses. This enum is used to represent the task status in a recognition.MergeArrangementsTaskStatus object.
ENUM	recognition.TaskType	enum	Server scripts	Holds the string values for supported merge task types. This enum is used to pass the task type argument to recognition.create(options).

MergeArrangementsTask Object Members

The following members are available for a recognition.MergeArrangementsTask object.

Return Type / Value Type

Supported Script Types

MergeArrangementsTask.submit()

number (read-only)

Submits the merge task for processing.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	MergeArrangementsTask.submit()	number (read-only)	Server scripts	Submits the merge task for processing. This method returns a task ID that uniquely identifies the merge task.
PROPERTY	MergeArrangementsTask.arrangements	Array<number | string> (read-only)	Server scripts	Holds an array of internal IDs of the revenue arrangement records to merge.
PROPERTY	MergeArrangementsTask.contractAcquisitionDeferredExpenseAccount	number | string (read-only)	Server scripts	References the contract acquisition deferred expense account for the new revenue arrangement. This property is valid only if the accounting preference Enable Advanced Cost Amortization is enabled. For more information, see Advanced Cost Amortization. The default value is the account specified by the accounting preference Contract Acquisition Deferred Expense Account in your account.
PROPERTY	MergeArrangementsTask.contractAcquisitionExpenseAccount	number | string (read-only)	Server scripts	References the contract acquisition expense account for the new revenue arrangement. This property is valid only if the accounting preference Enable Advanced Cost Amortization is enabled. For more information, see Advanced Cost Amortization. The default value is the account specified by the accounting preference Contract Acquisition Expense Account in your account.
PROPERTY	MergeArrangementsTask.contractCostAccrualDate	JavaScript Date (read-only)	Server scripts	Describes the contract cost accrual date to use for the new revenue arrangement. This property is valid only if the accounting preference Enable Advanced Cost Amortization is enabled. For more information, see Advanced Cost Amortization. The default value is today's date.
PROPERTY	MergeArrangementsTask.mergeResidualRevenueAmounts	boolean (read-only)	Server scripts	Indicates whether the revenue arrangements are merged prospectively. For more information about prospective merges, see Prospective Merges. The default value is false.
PROPERTY	MergeArrangementsTask.recalculateResidualFairValue	boolean (read-only)	Server scripts	Indicates whether to recalculate the fair value on residual elements when revenue arrangements are prospectively merged. For more information about prospective merges, see Prospective Merges. This property can be set to true only if the MergeArrangementsTask.mergeResidualRevenueAmounts property is also set to true. The default value is false.
PROPERTY	MergeArrangementsTask.revenueArrangementDate	JavaScript Date (read-only)	Server scripts	Describes the date of the new revenue arrangement. The default value is today's date.

MergeArrangementsTaskStatus Object Members

The following members are available for a recognition.MergeArrangementsTaskStatus object.

Return Type / Value Type

Supported Script Types

MergeArrangementsTaskStatus.errorMessage

string (read-only)

Holds an error message that describes the failure of the merge task. This property is valid only if the value of the status property is TaskStatus.FAILED.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	MergeArrangementsTaskStatus.errorMessage	string (read-only)	Server scripts	Holds an error message that describes the failure of the merge task. This property is valid only if the value of the status property is TaskStatus.FAILED.
PROPERTY	MergeArrangementsTaskStatus.inputArrangements	number[] (read-only)	Server scripts	Holds an array of internal IDs of the revenue arrangement records to merge. This property is valid only if the merge task was created using a task type of TaskType.MERGE_ARRANGEMENTS_TASK.
PROPERTY	MergeArrangementsTaskStatus.inputElements	number[] (read-only)	Server scripts	Holds an array of internal IDs of the revenue elements to merge. This property is valid only if the merge task was created using a task type of TaskType.MERGE_ELEMENTS_TASK.
PROPERTY	MergeArrangementsTaskStatus.resultingArrangement	number | string (read-only)	Server scripts	References the internal ID of the new revenue arrangement that was created. This property is valid only if the value of the status property is TaskStatus.COMPLETE.
PROPERTY	MergeArrangementsTaskStatus.status	string (read-only)	Server scripts	Represents the current status of the merge task. This property uses values in the recognition.TaskStatus enum.
PROPERTY	MergeArrangementsTaskStatus.submissionId	number | string (read-only)	Server scripts	References the submission ID of the merge arrangements bulk process.
PROPERTY	MergeArrangementsTaskStatus.taskId	number | string (read-only)	Server scripts	Holds the task ID of the merge task. The task ID is assigned to the merge task when you call recognition.create(options).

MergeElementsTask Object Members

The following members are available for a recognition.MergeElementsTask object.

Return Type / Value Type

Supported Script Types

MergeElementsTask.submit()

number (read-only)

Submits the merge task for processing.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	MergeElementsTask.submit()	number (read-only)	Server scripts	Submits the merge task for processing. This method returns a task ID that uniquely identifies the merge task.
PROPERTY	MergeElementsTask.contractAcquisitionDeferredExpenseAccount	number | string (read-only)	Server scripts	References the contract acquisition deferred expense account for the new revenue arrangement. This property is valid only if the accounting preference Enable Advanced Cost Amortization is enabled. For more information, see Advanced Cost Amortization. The default value is the account specified by the accounting preference Contract Acquisition Deferred Expense Account in your account.
PROPERTY	MergeElementsTask.contractAcquisitionExpenseAccount	number | string (read-only)	Server scripts	References the contract acquisition expense account for the new revenue arrangement. This property is valid only if the accounting preference Enable Advanced Cost Amortization is enabled. For more information, see Advanced Cost Amortization. The default value is the account specified by the accounting preference Contract Acquisition Expense Account in your account.
PROPERTY	MergeElementsTask.contractCostAccrualDate	JavaScript Date (read-only)	Server scripts	Describes the contract cost accrual date to use for the new revenue arrangement. This property is valid only if the accounting preference Enable Advanced Cost Amortization is enabled. For more information, see Advanced Cost Amortization. The default value is today's date.
PROPERTY	MergeElementsTask.elements	Array<number | string> (read-only)	Server scripts	Holds an array of internal IDs of the revenue element records to merge.
PROPERTY	MergeElementsTask.revenueArrangementDate	JavaScript Date (read-only)	Server scripts	Describes the date of the new revenue arrangement. The default value is today's date.

▶ Official NetSuite Script Samples (3 Sample(s))

📄 Sample 1: Merge Revenue Elements Using Internal IDs

```
/**
 * @NApiVersion 2.x
 */

require(['N/task/accounting/recognition'], function(recognition){
    var elementsList = [];
    elementsList.push(401);
    elementsList.push(402);

    var recognitionTask = recognition.create({
        taskType: recognition.TaskType.MERGE_ELEMENTS_TASK
    });
    recognitionTask.elements = elementsList;
    var taskStatusId = recognitionTask.submit();

    var mergeTaskState = recognition.checkStatus({
        taskId: taskStatusId
    });

    log.debug('Submission ID = ' + mergeTaskState.submissionId);
    log.debug('Resulting Arrangement ID = ' + mergeTaskState.resultingArrangement);
    log.debug('status = ' + mergeTaskState.status);
});
```

📄 Sample 2: Merge Revenue Arrangements Using a Saved Search

```
/**
 * @NApiVersion 2.x
 */

require(['N/task/accounting/recognition', 'N/search'], function(recognition, search){
    var mySearch = search.load({
        id: 'customsearch22'
    });

    var elementsList = [];
    mySearch.run().each(function(result) {
        var id = result.getValue({
            name: 'internalid'
        });
        elementsList.push(id);
    });

    var recognitionTask = recognition.create({
        taskType: recognition.TaskType.MERGE_ARRANGEMENTS_TASK
    });

    recognitionTask.arrangements = elementsList;
    recognitionTask.revenueArrangementDate = new Date(2019, 2, 10);

    var taskStatusId = recognitionTask.submit();
    log.debug('taskId = ' + taskStatusId);

    var mergeTaskState = recognition.checkStatus({
        taskId: taskStatusId
    });

    log.debug('Submission ID = ' + mergeTaskState.submissionId);
    log.debug('Resulting Arrangement ID = ' + mergeTaskState.resultingArrangement);
    log.debug('status = ' + mergeTaskState.status);
    log.debug('Error message = ' + mergeTaskState.errorMessage);
});
```

📄 Sample 3: Merge Revenue Arrangements Using an Ad-Hoc Search

```
/**
 * @NApiVersion 2.x
 */

require(['N/task/accounting/recognition', 'N/search'], function(recognition, search) {
    var elementsList = [];
    var rs = search.create({
        type: 'revenueelement',
        columns: [
            'internalid'
        ]
    }).run();

    var results = rs.getRange(0, 50);
    for (var i = 0; i &lt; results.length; i++) {
        var id = result.getValue('elementsList');
        elementsList.push(id);
    }

    var t = recognition.create({
        taskType: recognition.TaskType.MERGE_ELEMENTS_TASK
    });
    t.elements = elementsList;
    t.revenueArrangementDate = new Date(2019, 1, 1);

    var taskId = t.submit();
    log.debug('Initial status: ' + res.status);
});
```

46. N/transaction Module

Load the N/transaction module to void transactions.

`define(['N/transaction'], (transaction) => { ... })` ✓ 1 Code Sample(s) Scripts: Client and server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/transaction'], (transaction) => { // Access methods and properties on transaction });
```

Use the N/transaction module to void transactions.

When you void a transaction, the total and all the line items for the transaction are set to zero. The transaction is not removed from the system. NetSuite supports two types of voids: direct voids and voids by reversing journal. For additional information, see Voiding, Deleting, or Closing Transactions.

The type of void performed with your script depends on the targeted account's preference settings:

If the Using Reversing Journals preference is disabled, a direct void is performed.

If the Using Reversing Journals preference is enabled, a void by reversing journal is performed.

After you successfully void a transaction, you can no longer make changes to the transaction that impact the general ledger.

N/transaction Module Script Sample

N/transaction Module Members

Return Type / Value Type

Supported Script Types

transaction.void(options)

Client and server scripts

Voids a transaction record.

transaction.void.promise(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**transaction.void(options)**	number	Client and server scripts	Voids a transaction record.
METHOD	**transaction.void.promise(options)**	number	Client and server scripts	Voids a transaction record asynchronously.
ENUM	**transaction.Type**	enum	Client and server scripts	Holds the string values for supported record types. This enum is used for the options.type parameter of the transaction.void(options) method.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: N/transaction Module Script Sample

```
/**
 * @NApiVersion 2.x
 */

require(['N/transaction', 'N/config', 'N/record'], function(transaction, config, record) {
    function voidSalesOrder() {
        var accountingConfig = config.load({
            type: config.Type.ACCOUNTING_PREFERENCES
        });
        accountingConfig.setValue({
            fieldId: 'REVERSALVOIDING',
            value: false
        });

        accountingConfig.save();

        var salesOrderObj = record.create({
            type: 'salesorder',
            isDynamic: false
        });
        salesOrderObj.setValue({
            fieldId: 'entity',
            value: 107
        });
        salesOrderObj.setSublistValue({
            sublistId: 'item',
            fieldId: 'item',
            value: 233,
            line: 0
        });
        salesOrderObj.setSublistValue({
            sublistId: 'item',
            fieldId: 'amount',
            value: 1,
            line: 0
        });

        var salesOrderId = salesOrderObj.save();

        var voidSalesOrderId = transaction.void({
            type: record.Type.SALES_ORDER,
            id: salesOrderId
        });

        var salesOrder = record.load({
            type: 'salesorder',
            id: voidSalesOrderId
        });

        // The value of the memo field is 'VOID'
        var memo = salesOrder.getValue({
            fieldId: 'memo'
        });
    }

    voidSalesOrder();
});
```

47. N/translation Module

Load the N/translation module to load NetSuite Translation Collections in SuiteScript.

`define(['N/translation'], (translation) => { ... })` ✓ 5 Code Sample(s) Scripts: Client and server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/translation'], (translation) => { // Access methods and properties on translation });
```

Use the N/translation module to allow SuiteScript developers to interact with NetSuite Translation Collections programmatically. For more information about Translation Collections, see Translation Collections Overview.

You can watch a video that demonstrates how to use the N/translation module to work with Translation Collections.

The N/translation module provides read-only access. If you want to create or modify Translation Collections, you can do so in the NetSuite UI at Customization > Translations > Manage Translations

For more information about managing Translation Collections in the UI, see Manage Translations.

A Translation Collection is encapsulated in the translation.Handle object. The translation.Handle object is a hierarchical object, which means that each node in the object is either another translation.Handle object or a translation.Translator function. Translator functions combine strings with parameters. When you create a Translation Collection in the NetSuite UI, you can include parameter placeholders in your translation strings. The translator function injects the specified parameter values into the placeholders in the returned translation string.

In your scripts, use translation.get(options) to get a translation.Translator function you can use to obtain specific translated strings in a collection. Consider the following code sample:

// key HELLO_1 = 'Hello, {1}' message: translation.get({ collection: 'custcollection_my_strings', key: 'HELLO_1' })({ params: ['NetSuite'] }) In this sample, if the string value of the HELLO_1 key is "Hello, {1}", the translation.Translator function combines the string with the params parameter value and returns "Hello, NetSuite". You can also use translation.load(options) to load translation strings from one or more Translation Collections. For information about the way strings are added to and formatted in collections, see Working with Translation Collection Strings.

You can load collections in different language locales by using the locales parameter of translation.load(options). You can also use translation.selectLocale(options) to create a translation.Handle object in a specific locale from an existing translation.Handle object.

N/translation Module Members

Return Type / Value Type

Supported Script Type

translation.Handle

Client and server scripts

Encapsulates a Translation Collection for a locale.

translation.Translator

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPE	DESCRIPTION
OBJECT	**translation.Handle**	`Object`	Client and server scripts	Encapsulates a Translation Collection for a locale.
OBJECT	**translation.Translator**	`Object / Function`	Client and server scripts	Represents a translator function that returns translated strings. The translated strings include variables that are passed as parameters to the translator function.
METHOD	**translation.get(options)**	`translation.Translator`	Client and server scripts	Creates a translator function for a key in the specified Translation Collection and locale.
METHOD	**translation.load(options)**	`translation.Handle`	Client and server scripts	Creates a translation.Handle object with translations for the specified Translation Collections and locales.
METHOD	**translation.selectLocale(options)**	`translation.Handle`	Client and server scripts	Creates a translation.Handle object in the specified locale from an existing translation.Handle object.
ENUM	**translation.Locale**	`enum`	Client and server scripts	Holds the supported locales for Translation Collections. Use this enum to pass the locale argument to translation.get(options) and translation.selectLocale(options).

▶ Official NetSuite Script Samples (5 Sample(s))

📄 Sample 1: Access Translation Strings

```
/**
 * @NApiVersion 2.x
 */

require(['N/ui/message', 'N/translation'],
    function(message, translation) {

        // Create a message with translated strings
        var myMsg = message.create({
            title: translation.get({
                collection: 'custcollection_my_strings',
                key: 'MY_TITLE'
            })(),
            message: translation.get({
                collection: 'custcollection_my_strings',
                key: 'MY_MESSAGE'
            })(),
            type: message.Type.CONFIRMATION
        });

        // Show the message for 5 seconds
        myMsg.show({
            duration: 5000
        });
});
```

📄 Sample 2: Access Translation Strings Using a Non-Default Locale

```
/**
 * @NApiVersion 2.x
 */

require(['N/ui/message', 'N/translation'],
    function(message, translation) {

        // Create a message with translated strings
        var myMsg = message.create({
            title: translation.get({
                collection: 'custcollection_my_strings',
                key: 'MY_TITLE',
                locale: translation.Locale.COMPANY_DEFAULT
            })(),
            message: translation.get({
                collection: 'custcollection_my_strings',
                key: 'MY_MESSAGE',
                locale: translation.Locale.COMPANY_DEFAULT
            })(),
            type: message.Type.CONFIRMATION
        });

        // Show the message for 5 seconds
        myMsg.show({
            duration: 5000
        });
});
```

📄 Sample 3: Access Parameterized Translation Strings

```
/**
 * @NApiVersion 2.x
 */

require(['N/ui/message', 'N/translation'],
    function(message, translation) {

        // Create a message with translated strings
        var myMsg = message.create({
            title: translation.get({
                collection: 'custcollection_my_strings',
                key: 'MY_TITLE'
            })(),
            message: translation.get({
                collection: 'custcollection_my_strings',
                key: 'HELLO_1'
            })({
                params: ['NetSuite']
            }),
            type: message.Type.CONFIRMATION
        });

        // Show the message for 5 seconds
        myMsg.show({
            duration: 5000
        });
});
```

📄 Sample 4: Load Specific Translation Strings from a Collection

```
/**
 * @NApiVersion 2.x
 */

require(['N/ui/message', 'N/translation'],
    function(message, translation) {

        // Load translation strings by key
        var localizedStrings = translation.load({
            collections: [{
                alias: 'myCollection',
                collection: 'custcollection_my_strings',
                keys: ['MY_TITLE', 'MY_MESSAGE']
            }]
        });

        // Create a message with translated strings
        var myMsg = message.create({
            title: localizedStrings.myCollection.MY_TITLE(),
            message: localizedStrings.myCollection.MY_MESSAGE(),
            type: message.Type.CONFIRMATION
        });

        // Show the message for 5 seconds
        myMsg.show({
            duration: 5000
        });
});
```

📄 Sample 5: Load Translation Strings By Key from Multiple Translation Collections

```
/**
 * @NApiVersion 2.x
 */

require(['N/ui/message', 'N/translation'],
    function(message, translation) {

        // Load two Translation Collections
        var localizedStrings = translation.load({
            collections: [{
                alias: 'myCollection',
                collection: 'custcollection_my_strings',
                keys: ['MY_TITLE']
            },{
                alias: 'myOtherCollection',
                collection: 'custcollection_other_strings',
                keys: ['MY_OTHER_MESSAGE']
            }]
        });

        // Create a message with translated strings
        var myMsg = message.create({
            title: localizedStrings.myCollection.MY_TITLE(),
            message: localizedStrings.myOtherCollection.MY_OTHER_MESSAGE(),
            type: message.Type.CONFIRMATION
        });

        // Show the message for 5 seconds
        myMsg.show({
            duration: 5000
        });
});
```

48. N/ui/dialog Module

Load the N/dialog module to create a modal dialog that is displayed until a button on the dialog is pressed.

`define(['N/ui/dialog'], (dialog) => { ... })` ✓ 4 Code Sample(s) Scripts: Client scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/ui/dialog'], (dialog) => { // Access methods and properties on dialog });
```

Use the N/ui/dialog module to create a modal dialog that persists until a button on the dialog is pressed.

N/ui/dialog Module Members

Property Type / Method Return Type

Supported Script Types

dialog.alert(options)

Creates an Alert dialog with an OK button.

dialog.confirm(options)

Creates a Confirm dialog with OK and Cancel buttons.

MEMBER TYPE	NAME	PROPERTY TYPE / METHOD RETURN TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**dialog.alert(options)**	Promise	Client scripts	Creates an Alert dialog with an OK button.
METHOD	**dialog.confirm(options)**	Promise	Client scripts	Creates a Confirm dialog with OK and Cancel buttons.
METHOD	**dialog.create(options)**	Promise	Client scripts	Creates a dialog with specified buttons.

▶ Official NetSuite Script Samples (4 Sample(s))

📄 Sample 1: Create an Alert Dialog

```
/**
 * @NApiVersion 2.1
 * @NScriptType ClientScript
 */
define(['N/ui/dialog'], (dialog) =&gt; {
    function pageInit() {
        let options = {
            title: 'I am an Alert',
            message: 'Click OK to continue.'
        };

        function success(result) {
            console.log('Success with value ' + result);
        }

        function failure(reason) {
            console.log('Failure: ' + reason);
        }

        dialog.alert(options).then(success).catch(failure);
    }

    return {
        pageInit: pageInit
    };

});
```

📄 Sample 2: Create a Confirmation Dialog

```
/**
 * @NApiVersion 2.1
 * @NScriptType ClientScript
 */
define(['N/ui/dialog'], (dialog) =&gt; {
    function pageInit() {
        let options = {
            title: 'I am a Confirmation',
            message: 'Press OK or Cancel'
        };

        function success(result) {
            console.log('Success with value ' + result);
        }

        function failure(reason) {
            console.log('Failure: ' + reason);
        }

        dialog.confirm(options).then(success).catch(failure);
    }

    return {
        pageInit: pageInit
    };

});
```

📄 Sample 3: Create a Dialog with Buttons

```
/**
 * @NApiVersion 2.1
 * @NScriptType ClientScript
 */
define(['N/ui/dialog'], (dialog) =&gt; {
    function pageInit() {
        let button1 = {
            label: 'I am A',
            value: 1
        };
        let button2 = {
            label: 'I am B',
            value: 2
        };
        let button3 = {
            label: 'I am C',
            value: 3
        };
        let options = {
            title: 'Alphabet Test',
            message: 'Which One?',
            buttons: [button1, button2, button3]
        };

        function success(result) {
            console.log('Success with value ' + result);
        }

        function failure(reason) {
            console.log('Failure: ' + reason);
        }

        dialog.create(options).then(success).catch(failure);
    }

    return {
        pageInit: pageInit
    };

});
```

📄 Sample 4: Create a Dialog that Includes a Default Button

```
/**
 * @NApiVersion 2.1
 * @NScriptType ClientScript
 */
define(['N/ui/dialog'], (dialog) =&gt; {
    function pageInit() {
        let options = {
            title: 'I am a Dialog with the default button',
            message: 'Click a button to continue.',
        };

        function success(result) {
            console.log('Success with value ' + result);
        }

        function failure(reason) {
            console.log('Failure: ' + reason);
        }

        dialog.create(options).then(success).catch(failure);
    }

    return {
        pageInit: pageInit
    };

});
```

49. N/ui/message Module

Load the N/message module to display a message at the top of the screen under the menu bar.

`define(['N/ui/message'], (message) => { ... })` ✓ 1 Code Sample(s) Scripts: Client scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/ui/message'], (message) => { // Access methods and properties on message });
```

Use the N/ui/message module to display a message at the top of the screen under the menu bar.

N/ui/message Module Script Sample

N/ui/message Members

Return Type / Value Type

Supported Script Types

Encapsulates the Message object that gets created when calling the message.create(options) method.

message.create(options)

Creates a message that can be displayed or hidden near the top of the page.

Indicates the type of message to create and display, which specifies the background color of the message and other message indicators.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	message.Message	void	Client scripts	Encapsulates the Message object that gets created when calling the message.create(options) method.
METHOD	message.create(options)	message.Message	Client scripts	Creates a message that can be displayed or hidden near the top of the page.
ENUM	message.Type	enum	Client scripts	Indicates the type of message to create and display, which specifies the background color of the message and other message indicators. Use this enum to set the value of the options.type parameter of the message.create(options) method.

Message Object Members

Return Type / Value Type

Supported Script Types

Hides the message.

Message.show(options)

Shows the message.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Message.hide()	void	Client scripts	Hides the message.
METHOD	Message.show(options)	void	Client scripts	Shows the message.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: N/ui/message Module Script Sample

```
/**
 * @NApiVersion 2.1
 */
define(['N/ui/message'], (message) =&gt; {
    let myMsg = message.create({
        title: 'My Title',
        message: 'My Message',
        type: message.Type.CONFIRMATION
    });
    myMsg.show({
        duration: 5000 // will disappear after 5s
    });

    let myMsg2 = message.create({
        title: 'My Title 2',
        message: 'My Message 2',
        type: message.Type.INFORMATION
    });
    myMsg2.show();
    setTimeout(myMsg2.hide, 15000); // will disappear after 15s

    let myMsg3 = message.create({
        title: 'My Title 3',
        message: 'My Message 3',
        type: message.Type.WARNING,
        duration: 20000
    });
    myMsg3.show(); // will disappear after 20s

    let myMsg4 = message.create({
        title: 'My Title 4',
        message: 'My Message 4',
        type: message.Type.ERROR
    });
    myMsg4.show(); // will stay up until hide is called.
});
```

50. N/ui/serverWidget Module

Load the N/serverWidget module to work with the user interface within NetSuite.

`define(['N/ui/serverWidget'], (serverWidget) ⇒ { ... })` ✓ 10 Code Sample(s) Scripts: Server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/ui/serverWidget'], (serverWidget) ⇒ { // Access methods and properties on serverWidget });
```

Use the N/ui/serverWidget module to work with the user interface within NetSuite. You can use Suitelets to build custom pages and wizards that have a NetSuite look-and-feel. You can also create various components of the NetSuite UI (for example, forms, fields, sublists, tabs).

When you add a UI object to an existing NetSuite page, to minimize the occurrence of field/object name conflicts, the internal ID that references the object must be prefixed with custpage.

N/ui/serverWidget Module Members

Return Type / Value Type

Supported Script Types

serverWidget.Assistant

A scriptable, multi-step NetSuite assistant.

serverWidget.AssistantStep

A step within a custom NetSuite assistant.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	serverWidget.Assistant	Object	Suitelets	A scriptable, multi-step NetSuite assistant.
OBJECT	serverWidget.AssistantStep	Object	Suitelets	A step within a custom NetSuite assistant.
OBJECT	serverWidget.Button	Object	Suitelets and beforeLoad user events	A button that appears in a UI object.
OBJECT	serverWidget.Field	Object	Suitelets and beforeLoad user events	A NetSuite field.
OBJECT	serverWidget.FieldGroup	Object	Suitelets and beforeLoad user events	A field group.
OBJECT	serverWidget.Form	Object	Suitelets and beforeLoad user events	A NetSuite form.
OBJECT	serverWidget.List	Object	Suitelets	A list.
OBJECT	serverWidget.ListColumn	Object	Suitelets	A list column.
OBJECT	serverWidget.Sublist	Object	Suitelets and beforeLoad user events	A NetSuite sublist.
OBJECT	serverWidget.Tab	Object	Suitelets and beforeLoad user events	A NetSuite tab and subtabs.
METHOD	serverWidget.createAssistant(options)	serverWidget.Assistant	Suitelets	Creates and returns a new assistant object.
METHOD	serverWidget.createForm(options)	serverWidget.Form	Suitelets	Creates and returns a new form object.
METHOD	serverWidget.createList(options)	serverWidget.List	Suitelets	Creates a List object (specifying the title, and whether to hide the navigation bar).
ENUM	serverWidget.AssistantSubmitAction	string (read-only)	Suitelets	Holds the string values for submit actions performed by the user. This enum is used to set the value of the Assistant.getLastAction().
ENUM	serverWidget.FieldBreakType	string (read-only)	Suitelets and beforeLoad user events	Holds the string values for supported field break types. This enum is used to set the value of the Field.updateBreakType(options) property.
ENUM	serverWidget.FieldDisplayType	string (read-only)	Suitelets and beforeLoad user events	Holds the string values for supported field display types. This enum is used to set the value of the Field.updateDisplayType(options) property.
ENUM	serverWidget.FieldLayoutType	string (read-only)	Suitelets and beforeLoad user events	Holds the string values for the supported types of field layouts. This enum is used to set the value of the Field.updateLayoutType(options) property.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
ENUM	**serverWidget.FieldType**	string (read-only)	Suitelets and beforeLoad user events	Holds the values for supported field types. This enum is used to set the value of the type parameter when Form.addField(options) is called.
ENUM	**serverWidget.FormPageLinkType**	string (read-only)	Suitelets and beforeLoad user events	Holds the string values for supported page link types on a form. This enum is used to set the value of the type parameter for Form.addPageLink(options).
ENUM	**serverWidget.LayoutJustification**	string (read-only)	Suitelets and beforeLoad user events	Holds the string values for supported justification layouts. This enum is used to set the value of the align parameter when List.addColumn(options) is called.
ENUM	**serverWidget.ListStyle**	string (read-only)	Suitelets	Holds the string values for supported list styles. This enum is used to set the value of the List.style property.
ENUM	**serverWidget.SublistDisplayType**	string (read-only)	Suitelets and beforeLoad user events	Holds the string values for supported sublist display types. This enum is used to set the value of the Sublist.displayType property.
ENUM	**serverWidget.SublistType**	string (read-only)	Suitelets and beforeLoad user events	Holds the string values for valid sublist types. This enum is used to define the type parameter when Form.addSublist(options) is called.

Assistant Object Members

The following members are called on the serverWidget.Assistant object.

Return Type / Value Type

Supported Script Types

Assistant.addField(options)

serverWidget.Field

Adds a field to an assistant.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**Assistant.addField(options)**	serverWidget.Field	Suitelets	Adds a field to an assistant.
METHOD	**Assistant.addFieldGroup(options)**	serverWidget.FieldGroup	Suitelets	Adds a field group to an assistant.
METHOD	**Assistant.addStep(options)**	serverWidget.AssistantStep	Suitelets	Adds a step to an assistant.
METHOD	**Assistant.addSublist(options)**	serverWidget.Sublist	Suitelets	Adds a sublist to an assistant.
METHOD	**Assistant.getField(options)**	serverWidget.Field	Suitelets	Gets a field object.
METHOD	**Assistant.getFieldGroup(options)**	serverWidget.FieldGroup	Suitelets	Gets a field group object.
METHOD	**Assistant.getFieldGroupIds()**	string[]	Suitelets	Gets all the field group IDs in an assistant.
METHOD	**Assistant.getFieldIds()**	string[]	Suitelets	Gets all the field IDs in an assistant.
METHOD	**Assistant.getFieldIdsByFieldGroup(fieldGroup)**	string[]	Suitelets	Gets all field IDs in the assistant field group.
METHOD	**Assistant.getLastAction()**	string	Suitelets	Gets the last action submitted by the user.
METHOD	**Assistant.getLastStep()**	serverWidget.AssistantStep	Suitelets	Gets the step that the last submitted action came from.
METHOD	**Assistant.getNextStep()**	serverWidget.AssistantStep	Suitelets	Gets the next step prompted by the assistant.
METHOD	**Assistant.getStep(options)**	serverWidget.AssistantStep	Suitelets	Returns a step in an assistant.
METHOD	**Assistant.getStepCount()**	number	Suitelets	Gets the total count of steps in the assistant.
METHOD	**Assistant.getSteps()**	serverWidget.AssistantStep[]	Suitelets	Gets all the steps in the assistant.
METHOD	**Assistant.getSublist(options)**	serverWidget.Sublist	Suitelets	Get a Sublist object from its ID.
METHOD	**Assistant.getSublistIds()**	string[]	Suitelets	Gets all the sublist IDs in an assistant.
METHOD	**Assistant.hasErrorHtml()**	Boolean	Suitelets	Indicates whether the assistant has an error message to display.
METHOD	**Assistant.isFinished()**	Boolean	Suitelets	Indicates the status of the assistant. If set to true, the assistant is finished.
METHOD	**Assistant.sendRedirect(options)**	void	Suitelets	Manages redirects in an assistant.
METHOD	**Assistant.setSplash(options)**	void	Suitelets	Define a splash message.
METHOD	**Assistant.updateDefaultValues(values)**	void	Suitelets	Sets the default values of an array of fields that are specific to the assistant.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Assistant.clientScriptFileId	number	Suitelets	The File Cabinet ID of client script file to be used in this assistant.
PROPERTY	Assistant.clientScriptModulePath	string	Suitelets	The relative path to the client script file to be used in this assistant.
PROPERTY	Assistant.currentStep	serverWidget.AssistantStep (read-only)	Suitelets	The current step.
PROPERTY	Assistant.errorHtml	string	Suitelets	The error message text.
PROPERTY	Assistant.finishedHtml	string	Suitelets	The text displayed after an assistant is finished.
PROPERTY	Assistant.hideAddToShortcutsLink	Boolean	Suitelets	Whether the Add to Shortcuts Link is displayed in the UI.
PROPERTY	Assistant.hideStepNumber	Boolean	Suitelets	Whether the current and total step numbers are displayed in the UI.
PROPERTY	Assistant.isNotOrdered	Boolean	Suitelets	Whether assistant steps are ordered or unordered.
PROPERTY	Assistant.title	string	Suitelets	The title of the assistant.

AssistantStep Object Members

The following members are called on the serverWidget.AssistantStep object.

Return Type / Value Type

Supported Script Types

AssistantStep.getFieldIds()

Gets all the field IDs in an assistant step.

AssistantStep.getLineCount(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	AssistantStep.getFieldIds()	string[]	Suitelets	Gets all the field IDs in an assistant step.
METHOD	AssistantStep.getLineCount(options)	number	Suitelets	Gets the number of lines previously entered by a user in a step.
METHOD	AssistantStep.getLineCount(options)	string[]	Suitelets	Gets all the field IDs in a list.
METHOD	AssistantStep.getSublistValue(options)	string	Suitelets	Gets the current value of a sublist field (line item) in a step.
METHOD	AssistantStep.getSubmittedSublistIds()	string[]	Suitelets	Gets the IDs for all the sublist fields (line items) in a step.
METHOD	AssistantStep.getValue(options)	string | string[]	Suitelets	Gets the current value of a field.
PROPERTY	AssistantStep.helpText	string	Suitelets	The help text for a step.
PROPERTY	AssistantStep.id	string (read-only)	Suitelets	The internal ID of the step.
PROPERTY	AssistantStep.label	string	Suitelets	The label for a step.
PROPERTY	AssistantStep.stepNumber	number	Suitelets	Indicates where this step appears sequentially in an assistant.

Button Object Members

The following members are called on the serverWidget.Button object.

Supported Script Types

Button.isDisabled

Suitelets and beforeLoad user events

Determines whether a button is dimmed.

Suitelets and beforeLoad user events

MEMBER TYPE	NAME	PROPERTY TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Button.isDisabled	Boolean	Suitelets and beforeLoad user events	Determines whether a button is dimmed.
PROPERTY	Button.isHidden	Boolean	Suitelets and beforeLoad user events	Determines whether the button is hidden in the UI.
PROPERTY	Button.label	string	Suitelets and beforeLoad user events	The label for the button.

Field Object Members

The following members are called on the serverWidget.Field object.

Return Type / Value Type

Supported Script Types

Field.addSelectOption(options)

Suitelets and beforeLoad user events

Adds a select option to a dropdown list for a selectable field.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**Field.addSelectOption(options)**	void	Suitelets and beforeLoad user events	Adds a select option to a dropdown list for a selectable field.
METHOD	**Field.getSelectOptions(options)**	Object[]	Suitelets and beforeLoad user events	Returns the internal ID and label of the options for a select field as name/value pairs.
METHOD	**Field.setHelpText(options)**	serverWidget.Field	Suitelets and beforeLoad user events	Sets the help text that appears in the field help popup.
METHOD	**Field.updateBreakType(options)**	serverWidget.Field	Suitelets and beforeLoad user events	Updates the break type used to add a break in flow layout for the field.
METHOD	**Field.updateDisplaySize(options)**	serverWidget.Field	Suitelets and beforeLoad user events	Updates the height and width for the field.
METHOD	**Field.updateDisplayType(options)**	serverWidget.Field	Suitelets and beforeLoad user events	Updates the type of display for the field.
METHOD	**Field.updateLayoutType(options)**	serverWidget.Field	Suitelets and beforeLoad user events	Updates the layout type for the field.
PROPERTY	**Field.alias**	string	Suitelets and beforeLoad user events	The alias used to set the field value.
PROPERTY	**Field.defaultValue**	string	Suitelets and beforeLoad user events	The default value for the field.
PROPERTY	**Field.helpText**	string (read-only)	Suitelets and beforeLoad user events	The help text for the field.
PROPERTY	**Field.id**	string (read-only)	Suitelets and beforeLoad user events	The internal ID for the field.
PROPERTY	**Field.isMandatory**	Boolean	Suitelets and beforeLoad user events	Whether the field is required.
PROPERTY	**Field.label**	string	Suitelets and beforeLoad user events	The label for the field.
PROPERTY	**Field.linkText**	string	Suitelets and beforeLoad user events	The text displayed for a link in place of the URL.
PROPERTY	**Field.maxLength**	number	Suitelets and beforeLoad user events	The maximum length, in characters, for the field.
PROPERTY	**Field.padding**	number	Suitelets and beforeLoad user events	The number of empty vertical character spaces above the field.
PROPERTY	**Field.richTextHeight**	number	Suitelets and beforeLoad user events	The height of a rich text field, in pixels.
PROPERTY	**Field.richTextWidth**	number	Suitelets and beforeLoad user events	The width of a rich text field, in pixels.
PROPERTY	**Field.type**	string (read-only)	Suitelets and beforeLoad user events	The type of field.

FieldGroup Object Members

The following members are called on the serverWidget.FieldGroup object.

Supported Script Types

FieldGroup.isBorderHidden

Suitelets and beforeLoad user events

Whether a border appears around the field group.

FieldGroup.isCollapsible

MEMBER TYPE	NAME	PROPERTY TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**FieldGroup.isBorderHidden**	Boolean	Suitelets and beforeLoad user events	Whether a border appears around the field group.
PROPERTY	**FieldGroup.isCollapsible**	Boolean	Suitelets and beforeLoad user events	Whether the field group is collapsible.
PROPERTY	**FieldGroup.isCollapsed**	Boolean	Suitelets and beforeLoad user events	Whether the field group is initially collapsed or expanded in the default view.

MEMBER TYPE	NAME	PROPERTY TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	FieldGroup.isSingleColumn	Boolean	Suitelets and beforeLoad user events	Whether the field group is displayed in a single column.
PROPERTY	FieldGroup.label	string	Suitelets and beforeLoad user events	The label for the field group.

Form Object Members

The following members are called on the serverWidget.Form object.

Return Type / Value Type

Supported Script Types

Form.addButton(options)

serverWidget.Button

Suitelets and beforeLoad user events

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Form.addButton(options)	serverWidget.Button	Suitelets and beforeLoad user events	Adds a button to the form.
METHOD	Form.addCredentialField(options)	serverWidget.Field	Suitelets and beforeLoad user events	Adds a field that store credentials in NetSuite for invoking services provided by third parties.
METHOD	Form.addField(options)	serverWidget.Field	Suitelets and beforeLoad user events	Adds a field to the form.
METHOD	Form.addFieldGroup(options)	serverWidget.FieldGroup	Suitelets and beforeLoad user events	Adds a group of fields to the form.
METHOD	Form.addPageInitMessage(options)	void	Suitelets and beforeLoad user events	Shows a message on a form in view mode. You can use this method to show a message on a form based on its user event script context.
METHOD	Form.addPageLink(options)	void	Suitelets and beforeLoad user events	Adds a link to a form.
METHOD	Form.addResetButton(options)	serverWidget.Button	Suitelets and beforeLoad user events	Adds a Reset button to a form that clears the values of all fields.
METHOD	Form.addSecretKeyField(options)	serverWidget.Field	Suitelets and beforeLoad user events	Add a secret key field to the form.
METHOD	Form.addSublist(options)	serverWidget.Sublist	Suitelets and beforeLoad user events	Adds a sublist to the form.
METHOD	Form.addSubmitButton(options)	serverWidget.Button	Suitelets and beforeLoad user events	Adds a submit button to a form that saves user inputs.
METHOD	Form.addSubtab(options)	serverWidget.Tab	Suitelets and beforeLoad user events	Adds a subtab to a form.
METHOD	Form.addTab(options)	serverWidget.Tab	Suitelets and beforeLoad user events	Adds a tab to a form.
METHOD	Form.getButton(options)	serverWidget.Button	Suitelets and beforeLoad user events	Returns a button by internal ID.
METHOD	Form.getField(options)	serverWidget.Field	Suitelets and beforeLoad user events	Returns a field by internal ID.
METHOD	Form.getSublist(options)	serverWidget.Sublist	Suitelets and beforeLoad user events	Returns a sublist by internal ID.
METHOD	Form.getSubtab(options)	serverWidget.Tab	Suitelets and beforeLoad user events	Returns a subtab by internal ID.
METHOD	Form.getTab(options)	serverWidget.Tab	Suitelets and beforeLoad user events	Returns a tab object from its internal ID.
METHOD	Form.getTabs()	string[]	Suitelets and beforeLoad user events	Returns the internal IDs of all tabs.
METHOD	Form.insertField(options)	void	Suitelets and beforeLoad user events	Inserts a field before another field within a form.
METHOD	Form.insertSublist(options)	void	Suitelets and beforeLoad user events	Inserts a sublist before another sublist on a form.
METHOD	Form.insertSubtab(options)	void	Suitelets and beforeLoad user events	Inserts a subtab before another subtab on a form.
METHOD	Form.insertTab(options)	void	Suitelets and beforeLoad user events	Inserts a tab before another tab on a form.
METHOD	Form.removeButton(options)	void	Suitelets and beforeLoad user events	Removes a button from a form.
METHOD	Form.updateDefaultValues(options)	void	Suitelets and beforeLoad user events	Sets the default values of many fields on a form.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Form.clientScriptFileId**	number	Suitelets and beforeLoad user events	The File Cabinet ID of client script file to be used in this form.
PROPERTY	**Form.clientScriptModulePath**	string	Suitelets and beforeLoad user events	The relative path to the client script file to be used in this form.
PROPERTY	**Form.title**	string	Suitelets and beforeLoad user events	The title used for the form.

List Object Members

The following members are called on the serverWidget.List object.

Return Type / Value Type

Supported Script Types

List.addButton(options)

serverWidget.Button

Adds a button to a list.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**List.addButton(options)**	serverWidget.Button	Suitelets	Adds a button to a list.
METHOD	**List.addColumn(options)**	serverWidget.ListColumn	Suitelets	Adds a column to a list.
METHOD	**List.addEditColumn(options)**	serverWidget.ListColumn	Suitelets	Adds a column containing Edit or Edit/View links to a Suitelet or Portlet list.
METHOD	**List.addPageLink(options)**	serverWidget.List	Suitelets	Adds a link to a list.
METHOD	**List.addRow(options)**	serverWidget.List	Suitelets	Adds a single row to a list.
METHOD	**List.addRows(options)**	serverWidget.List	Suitelets	Adds multiple rows to a list.
PROPERTY	**List.clientScriptFileId**	number	Suitelets	The File Cabinet ID of client script file to be used in this list.
PROPERTY	**List.clientScriptModulePath**	string	Suitelets	The relative path to the client script file to be used in this list.
PROPERTY	**List.style**	string	Suitelets	The display style for this list.
PROPERTY	**List.title**	string	Suitelets	The List title.

ListColumn Object Members

The following members are called on the serverWidget.ListColumn object.

Return Type / Value Type

Supported Script Types

ListColumn.addParamToURL(options)

serverWidget.ListColumn

Adds a URL parameter (optionally defined per row) to the list column's URL.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**ListColumn.addParamToURL(options)**	serverWidget.ListColumn	Suitelets	Adds a URL parameter (optionally defined per row) to the list column's URL.
METHOD	**ListColumn.setURL(options)**	serverWidget.ListColumn	Suitelets	Sets the base URL for the list column.
PROPERTY	**ListColumn.label**	string	Suitelets	The label of this list column.

Sublist Object Members

The following members are called on the serverWidget.Sublist object.

Return Type / Value Type

Supported Script Types

Sublist.addButton(options)

serverWidget.Button

Suitelets and beforeLoad user events

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**Sublist.addButton(options)**	serverWidget.Button	Suitelets and beforeLoad user events	Adds a button to a sublist.
METHOD	**Sublist.addField(options)**	serverWidget.Field	Suitelets and beforeLoad user events	Add a field to a sublist.
METHOD	**Sublist.addMarkAllButtons()**	serverWidget.Button[]	Suitelets and beforeLoad user events	Adds a Mark All or Unmark All button.
METHOD	**Sublist.addRefreshButton()**	serverWidget.Button	Suitelets and beforeLoad user events	Adds a Reset button.
METHOD	**Sublist.getField(options)**	serverWidget.Field	Suitelets and beforeLoad user events	Returns a Field object on a specified sublist.
METHOD	**Sublist.getSublistValue(options)**	string	Suitelets and beforeLoad user events	Gets a field value on a sublist.
METHOD	**Sublist.insertField(options)**	void	Suitelets and beforeLoad user events	Inserts a field before another field on a sublist.
METHOD	**Sublist.setSublistValue(options)**	void	Suitelets and beforeLoad user events	Sets the value of a sublist field.
METHOD	**Sublist.updateTotallingFieldId(options)**	serverWidget.Sublist	Suitelets and beforeLoad user events	Updates the ID of a field designated as a totalling column, which is used to calculate and display a running total for the sublist.
METHOD	**Sublist.updateUniqueFieldId(options)**	serverWidget.Sublist	Suitelets and beforeLoad user events	Updates a field ID that is to have unique values across the rows in the sublist.
PROPERTY	**Sublist.displayType**	string	Suitelets and beforeLoad user events	The display style for a sublist.
PROPERTY	**Sublist.helpText**	string	Suitelets and beforeLoad user events	The inline help text for a sublist.
PROPERTY	**Sublist.label**	string	Suitelets and beforeLoad user events	The label for a sublist.
PROPERTY	**Sublist.lineCount**	number (read-only)	Suitelets and beforeLoad user events	The number of line items in a sublist.

Tab Object Members

The following members are called on the serverWidget.Tab object.

Supported Script Types

Suitelets and beforeLoad user events

The inline help text for a tab or subtab.

Suitelets and beforeLoad user events

The label for a tab or subtab.

MEMBER TYPE	NAME	VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Tab.helpText**	string	Suitelets and beforeLoad user events	The inline help text for a tab or subtab.
PROPERTY	**Tab.label**	string	Suitelets and beforeLoad user events	The label for a tab or subtab.

▶ Official NetSuite Script Samples (10 Sample(s))

📄 Sample 1: Create a Form with a Credential Field

```
/**
 * @NApiVersion 2.1
 * @NScriptType Suitelet
 */

// This script creates a form with a credential field.
define(['N/ui/serverWidget', 'N/https', 'N/url'], (serverWidget, https, url) =&gt; {
    function onRequest(context) {
        if (context.request.method === 'GET') {
            const form = serverWidget.createForm({
             title: 'Password Form'
            });

            const credField = form.addCredentialField({
                id: 'password',
                label: 'Password',
                restrictToDomains: ['&lt;accountID&gt;.app.netsuite.com'],
                restrictToCurrentUser: false,
                restrictToScriptIds: 'customscript_my_script'
            });

            credField.maxLength = 32;

            form.addSubmitButton();

            context.response.writePage({
                pageObject: form
                });
        }
        else {
            // Request to an existing Suitelet with credentials
            let passwordGuid = context.request.parameters.password;

            // Replace SCRIPTID and DEPLOYMENTID with the internal ID of the suitelet script and deployment in your account
            let baseUrl = url.resolveScript({
                scriptId: SCRIPTID,
                deploymentId: DEPLOYMENTID,
                returnExternalURL: true
            });

            let authUrl = baseUrl + '&amp;pwd={' + passwordGuid + '}';

            let secureStringUrl = https.createSecureString({
                input: authUrl
            });

            let headers = ({
               'pwd': passwordGuid
            });

            let response = https.post({
                credentials: [passwordGuid],
                url: secureStringUrl,
                body: {authorization:' '+ passwordGuid + '', data:'anything can be here'},
                headers: headers
            });
        }
    }
    return {
        onRequest: onRequest
    };
});
```

📄 Sample 2: Complete Custom List Page Sample Script

```
/**
 *@NApiVersion 2.1
 *@NScriptType Suitelet
*/
```

📄 Sample 3: Complete Custom List Page Sample Script

```
/**
 *@NApiVersion 2.1
 *@NScriptType Suitelet
*/
define(['N/ui/serverWidget'],
    function(serverWidget) {
};
```

📄 Sample 4: Complete Custom List Page Sample Script

```
/**
 *@NApiVersion 2.1
 *@NScriptType Suitelet
*/
define(['N/ui/serverWidget'],
    function(serverWidget) {
        function onRequest(context){
            if(context.request.method === 'GET'){
                // Section One - List - See 'Steps for Creating a List', Step Five
                // Section Two - Columns  - See 'Steps for Creating a List', Step Seven
                context.response.writePage(list);
            }else{
            }
        }
    return {
        onRequest: onRequest
    }

});
```

📄 Sample 5: Complete Custom List Page Sample Script

```
var list = serverWidget.createList({
    title: 'Purchase History'
});
list.style = serverWidget.ListStyle.REPORT;
```

📄 Sample 6: Complete Custom List Page Sample Script

```
list.addButton({
    id : 'buttonid',
    label : 'Test',
    functionName: '' // the function called when the button is pressed
});
```

📄 Sample 7: Complete Custom List Page Sample Script

```
var datecol = list.addColumn({
               id : 'column1',
               type : serverWidget.FieldType.DATE,
               label : 'Date',
               align : serverWidget.LayoutJustification.RIGHT
            });

               list.addColumn({
               id : 'column2',
               type : serverWidget.FieldType.TEXT,
               label : 'Product',
               align : serverWidget.LayoutJustification.RIGHT
            });

               list.addColumn({
               id : 'column3',
               type : serverWidget.FieldType.INTEGER,
               label : 'Quantity',
               align : serverWidget.LayoutJustification.RIGHT
            });

               list.addColumn({
               id : 'column4',
               type : serverWidget.FieldType.CURRENCY,
               label : 'Unit Cost',
               align : serverWidget.LayoutJustification.RIGHT
            });
```

📄 Sample 8: Complete Custom List Page Sample Script

```
list.addRow({
    row : { column1 : '02/12/2018', column2 : 'Widget', column3: '4', column4:'4.50' }
});
```

📄 Sample 9: Complete Custom List Page Sample Script

```
list.addRows({
    rows :[{column1 : '02/12/2018', column2 : 'Widget', column3: '4', column4:'4.50'},
        {column1 : '02/14/2018', column2 : 'Sprocket', column3: '6', column4:'11.50'},
        {column1 : '02/16/2018', column2 : 'Gizmo', column3: '9', column4:'1.25'}]
});
```

📄 Sample 10: Complete Custom List Page Sample Script

```
/**
 * @NApiVersion 2.1
 * @NScriptType Suitelet
 */
define(['N/ui/serverWidget'], function(serverWidget) {
    function onRequest(context){
        if(context.request.method === 'GET'){
            // Section One - List - See 'Steps for Creating a Custom List Page', Step Five
            var list = serverWidget.createList({
                title: 'Purchase History'
            });

            list.style = serverWidget.ListStyle.REPORT;

            list.addButton({
                id: 'buttonid',
                label: 'Test',
                functionName: '' // the function called when the button is pressed
            });

            // Section Two - Columns  - See 'Steps for Creating a Custom List Page', Step Seven
            var datecol = list.addColumn({
                id: 'column1',
                type: serverWidget.FieldType.DATE,
                label: 'Date',
                align: serverWidget.LayoutJustification.RIGHT
            });

            list.addColumn({
                id: 'column2',
                type: serverWidget.FieldType.TEXT,
                label: 'Product',
                align: serverWidget.LayoutJustification.RIGHT
            });

            list.addColumn({
                id: 'column3',
                type: serverWidget.FieldType.INTEGER,
                label: 'Quantity',
                align: serverWidget.LayoutJustification.RIGHT
            });

            list.addColumn({
                id: 'column4',
                type: serverWidget.FieldType.CURRENCY,
                label: 'Unit Cost',
                align: serverWidget.LayoutJustification.RIGHT
            });

            list.addRows({
                rows: [{column1: '05/30/2018', column2: 'Widget', column3: '4', column4: '4.50'},
                    {column1: '05/30/2018', column2: 'Sprocket', column3: '6', column4: '11.50'},
                    {column1: '05/30/2018', column2: 'Gizmo', column3: '9', column4: '1.25'}]
            });
            context.response.writePage(list);
        }
    }

    return {
        onRequest: onRequest
    }
});
```

51. N/url Module

Load the N/url module to determine URL navigation paths within NetSuite or format URL strings.

`define(['N/url'], (url) => { ... })` ✓ 4 Code Sample(s) Scripts: Client and server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/url'], (url) => { // Access methods and properties on url });
```

Use the N/url module to determine URL navigation paths within NetSuite and format URL strings.

N/url Module Members

Return Type / Value Type

Supported Script Types

url.format(options)

Client and server scripts

Converts (serializes) URL query parameters into a string.

url.resolveDomain(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**url.format(options)**	string	Client and server scripts	Converts (serializes) URL query parameters into a string.
METHOD	**url.resolveDomain(options)**	string	Client and server scripts	Returns a domain name for a NetSuite account.
METHOD	**url.resolveRecord(options)**	string	Client and server scripts	Returns an internal URL to a NetSuite record.
METHOD	**url.resolveScript(options)**	string	Client and server scripts	Returns an external or internal URL to a script.
METHOD	**url.resolveTaskLink(options)**	string	Client and server scripts	Returns an internal URL for a tasklink.
ENUM	**url.HostType**	enum	Client and server scripts	Holds the string values that describe a category of domain name. Use this enum to set the value of the hostType parameter of the url.resolveDomain(options) method.

▶ Official NetSuite Script Samples (4 Sample(s))

📄 Sample 1: Retrieve the Relative URL of a Record

```
/**
 * @NApiVersion 2.x
 */

// This script retrieves the relative URL of a record.
require(['N/url'], function(url) {
    var output = url.resolveRecord({
        recordType: 'salesorder',
        recordId: 6,
        isEditMode: true
    });
});
```

📄 Sample 2: Generate an Absolute URL to a Specific Resource

```
/**
 * @NApiVersion 2.x
 */

// This script generates an absolute URL to a specific resource.
require(['N/url', 'N/record'], function(url, record) {
    function resolveRecordUrl() {
        var scheme = 'https://';
        var host = url.resolveDomain({
            hostType: url.HostType.APPLICATION
        });
        var relativePath = url.resolveRecord({
            recordType: record.Type.SALES_ORDER,
            recordId: 6,
            isEditMode: true
        });
        var myURL = scheme + host + relativePath;
    }
    resolveRecordUrl();
});
```

📄 Sample 3: Retrieve the Domain for Calling a RESTlet

```
/**
 * @NApiVersion 2.x
 */

// This script retrieves the domain for calling a RESTlet.
require(['N/url'], function(url) {
    function resolveDomainUrl() {
        var sCompId = 'MSTRWLF';
        var output = url.resolveDomain({
            hostType: url.HostType.RESTLET,
            accountId: sCompId
        });
    }
    resolveDomainUrl();
});
```

📄 Sample 4: Create a URL and Send a Secure HTTPS Post Request to the URL

```
/**
 * @NApiVersion 2.x
 */

// This script creates a URL, sends a secure HTTPS POST request to that URL, and logs the server's response.
require(['N/url', 'N/https'], function(url, https) {
    var script = 'customscript1';
    var deployment = 'customdeploy1';
    var parameters = '';
    try {
        var suiteletURL = url.resolveScript({
            scriptId: script,
            deploymentId: deployment
        });
        var response = https.post({
            url: suiteletURL,
            body: parameters
        });
        log.debug(response.body.toString());
    }
    catch(e) {
        log.error(e.toString());
    }
});
```

52. N/util Module

Load the N/util module to manually access util methods. Module members are also supported by the global util Object.

`define(['N/util'], (util) => { ... })` ✓ 1 Code Sample(s) Scripts: Client and server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/util'], (util) => { // Access methods and properties on util });
```

Use the N/util module to manually access methods that verify object and primitive types in a SuiteScript 2.x script. These methods can also be accessed using the global util object. For more information about the global util object, see SuiteScript 2.1 Global Objects.

N/util Module Script Sample

Use the N/util module to access methods that verify object and primitive types in a SuiteScript 2.x script.

Each method (for example, util.isArray(obj)) returns a boolean value, based on evaluation of the obj parameter.

If you need to identify a type specific to SuiteScript 2.x, use the toString() global method.

N/util Module Members

Return Type / Value Type

Supported Script Types

util.each(iterable, callback)

Client and server scripts

Iterates over each member in an Object or Array.

util.extend(receiver, contributor)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	util.each(iterable, callback)	Object or Array	Client and server scripts	Iterates over each member in an Object or Array.
METHOD	util.extend(receiver, contributor)	Object	Client and server scripts	Copies the properties in a source object to a destination object.
METHOD	util.isArray(obj)	boolean	Client and server scripts	Returns true if the obj parameter is a JavaScript Array object and false otherwise.
METHOD	util.isAsyncFunction(obj)	boolean	Client and server scripts	Returns true if the obj parameter is JavaScript Async Function and false otherwise.
METHOD	util.isBoolean(obj)	boolean	Client and server scripts	Returns true if the obj parameter is a JavaScript Boolean and false otherwise.
METHOD	util.isDate(obj)	boolean	Client and server scripts	Returns true if the obj parameter is a JavaScript Date object and false otherwise.
METHOD	util.isFunction(obj)	boolean	Client and server scripts	Returns true if the obj parameter is a JavaScript Function or Async Function and false otherwise.
METHOD	util.isNumber(obj)	boolean	Client and server scripts	Returns true if the obj parameter is a JavaScript Number object or a value that evaluates to a Number object, and false otherwise.
METHOD	util.isObject(obj)	boolean	Client and server scripts	Returns true if the obj parameter is a plain JavaScript object (new Object() or {} for example), and false otherwise.
METHOD	util.isRegExp(obj)	boolean	Client and server scripts	Returns true if the obj parameter is a JavaScript RegExp object or a value that evaluates to a RegExp object, and false otherwise.
METHOD	util.isString(obj)	boolean	Client and server scripts	Returns true if the obj parameter is a JavaScript String object or a value that evaluates to a String object, and false otherwise.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: N/util Module Script Sample

```
/**
 * @NApiVersion 2.x
 */

require(['N/record'], function(record){
    // Create a sales order
    var rec = record.create({
        type: 'salesorder',
        isDynamic: true
    });
    rec.setValue({
        fieldId: 'entity',
        value: 107
    });

    // Set up an object containing an item's internal ID and the corresponding quantity
    var itemList = {
        39: 5,
        38: 1
    }

    // Iterate through the object and set the key-value pairs on the record
    util.each(itemList, function(quantity, itemId){     // (5, 39) and (1, 38)
        rec.selectNewLine('item');
        rec.setCurrentSublistValue('item','item',itemId);
        rec.setCurrentSublistValue('item','quantity',quantity);
        rec.commitLine('item');
    });

    var id = rec.save();
});
```

53. N/workbook Module

Load the N/workbook module to create and manage workbooks in SuiteAnalytics Workbook. Use this module with the N/dataset module to manage all aspects of your datasets and workbooks in SuiteAnalytics Workbook.

`define(['N/workbook'], (workbook) => { ... })` ✓ 2 Code Sample(s) Scripts: Server scripts Permissions: SuiteAnalytics Workbook

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/workbook'], (workbook) => { // Access methods and properties on workbook });
```

Use the N/workbook module to create new workbooks, load existing ones, or list all available workbooks.

A workbook can contain:

Pivots

Tables with columns, filters, and field contexts

Charts with axes and legends

Selectors

Sections

Data dimensions

Sorts

Conditional and limiting filters

Expressions

Data measures and calculated measures

Workbooks help you analyze your dataset query results using different components, like table views. Every workbook is based on a dataset. For more, see the N/dataset Module. To learn more about SuiteAnalytics workbooks and datasets, check out:

SuiteAnalytics Workbook Overview

Getting Started with SuiteAnalytics Workbook

Custom Workbooks and Datasets

The N/workbook module doesn't work in unauthenticated client-side contexts.

For details, see the SuiteAnswers Outbound HTTPs in an unauthenticated client-side context.

N/workbook Module Members

Return Type / Value Type

Supported Script Types

workbook.CalculatedMeasure

A calculated measure.

workbook.Category

A chart category.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	workbook.Aspect	Object	Server scripts	An aspect.
OBJECT	workbook.CalculatedMeasure	Object	Server scripts	A calculated measure.
OBJECT	workbook.Category	Object	Server scripts	A chart category.
OBJECT	workbook.Chart	Object	Server scripts	A chart.
OBJECT	workbook.ChartAxis	Object	Server scripts	A chart axis object which is used when you create a category or a legend.
OBJECT	workbook.ChildNodesSelector	Object	Server scripts	A selector for child nodes.
OBJECT	workbook.Color	Object	Server scripts	A color.
OBJECT	workbook.ConditionalFilter	Object	Server scripts	A conditional filter.
OBJECT	workbook.ConditionalFormat	Object	Server scripts	A conditional format.
OBJECT	workbook.ConditionalFormatRule	Object	Server scripts	A conditional format rule.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	workbook.Currency	Object	Server scripts	A currency amount and currency type.
OBJECT	workbook.DataDimension	Object	Server scripts	A data dimension.
OBJECT	workbook.DataDimensionItem	Object	Server scripts	A data dimension item.
OBJECT	workbook.DataDimensionItemValue	Object	Server scripts	The value of a data dimension item.
OBJECT	workbook.DataDimensionValue	Object	Server scripts	The value of a data dimension.
OBJECT	workbook.DataMeasure	Object	Server scripts	A data measure.
OBJECT	workbook.DescendantorSelfNodesSelector	Object	Server scripts	A selector for descendant or self nodes.
OBJECT	workbook.DimensionSelector	Object	Server scripts	A dimension selector.
OBJECT	workbook.Duration	Object	Server scripts	A duration.
OBJECT	workbook.Expression	Object	Server scripts	An expression.
OBJECT	workbook.FieldContext	Object	Server scripts	A field context.
OBJECT	workbook.FontSize	Object	Server scripts	A font size.
OBJECT	workbook.Legend	Object	Server scripts	A chart legend.
OBJECT	workbook.LimitingFilter	Object	Server scripts	A limiting filter.
OBJECT	workbook.MeasureSelector	Object	Server scripts	A measure selector.
OBJECT	workbook.MeasureValue	Object	Server scripts	A measure value.
OBJECT	workbook.MeasureValueSelector	Object	Server scripts	A measure value selector.
OBJECT	workbook.PathSelector	Object	Server scripts	A path selector.
OBJECT	workbook.Pivot	Object	Server scripts	A pivot definition. A pivot is a workbook component that enables you to pivot your dataset query results by defining measures and dimensions, so that you can analyze different subsets of data.
OBJECT	workbook.PivotAxis	Object	Server scripts	A pivot axis.
OBJECT	workbook.PivotIntersection	Object	Server scripts	A pivot intersection.
OBJECT	workbook.PositionPercent	Object	Server scripts	A position defined by percentages of the x and y axes.
OBJECT	workbook.PositionUnits	Object	Server scripts	A position defined by units.
OBJECT	workbook.PositionValues	Object	Server scripts	A position defined by horizontal and vertical position values.
OBJECT	workbook.Range	Object	Server scripts	A date or date-time range.
OBJECT	workbook.Record	Object	Server scripts	A record.
OBJECT	workbook.RecordKey	Object	Server scripts	A record key.
OBJECT	workbook.ReportStyle	Object	Server scripts	A report style.
OBJECT	workbook.ReportStyleRule	Object	Server scripts	A report style rule.
OBJECT	workbook.Section	Object	Server scripts	A workbook section.
OBJECT	workbook.SectionValue	Object	Server scripts	A section value.
OBJECT	workbook.Series	Object	Server scripts	A series in a workbook. A series is used when you create a chart definition.
OBJECT	workbook.Sort	Object	Server scripts	A sort.
OBJECT	workbook.SortByDataDimensionItem	Object	Server scripts	A sort based on a data dimension item.
OBJECT	workbook.SortByMeasure	Object	Server scripts	A sort based on a measure.
OBJECT	workbook.SortDefinition	Object	Server scripts	A sort definition.
OBJECT	workbook.Style	Object	Server scripts	A style.
OBJECT	workbook.Table	Object	Server scripts	A table.
OBJECT	workbook.TableColumn	Object	Server scripts	A table column.
OBJECT	workbook.TableColumnCondition	Object	Server scripts	Condition for a table view column.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	workbook.TableColumnFilter	Object	Server scripts	A table column filter.
OBJECT	workbook.Workbook	Object	Server scripts	A workbook. Workbooks are where you analyze the results of your dataset queries using different components, such as table views and pivots. All workbooks are based on a dataset, and a single dataset can be used as the basis for multiple workbooks.
METHOD	workbook.create(options)	workbook.Workbook	Server scripts	Creates a workbook. Workbooks are where you analyze the results of your dataset queries using different components, such as table views and pivots. All workbooks are based on a dataset, and a single dataset can be used as the basis for multiple workbooks.
METHOD	workbook.createAspect(options)	workbook.Aspect	server scripts	Creates an aspect for a chart series. An aspect includes a measure and an aspect type.
METHOD	workbook.createCalculatedMeasure(options)	workbook.CalculatedMeasure	Server scripts	Creates a calculated measure.
METHOD	workbook.createCategory(options)	workbook.Category	Server scripts	Creates a chart category, which includes an axis, a data root, and a sort definition. A chart category is used in a workbook.Chart.
METHOD	workbook.createChart(options)	workbook.Chart	Server scripts	Creates a chart.
METHOD	workbook.createChartAxis(options)	workbook.ChartAxis	Server scripts	Creates an X-axis or a Y-axis for the chart.
METHOD	workbook.createColor(options)	workbook.Color	Server scripts	Creates a color.
METHOD	workbook.createComplexRecordKey	workbook.RecordKey	Server scripts	Creates a complex RecordKey object from another object.
METHOD	workbook.createConditionalFilter(options)	workbook.ConditionalFilter	Server scripts	Creates a conditional filter.
METHOD	workbook.createConditionalFormat(options)	workbook.ConditionalFormat	Server scripts	Creates a conditional format.
METHOD	workbook.createConditionalFormatRule(options)	workbook.ConditionalFormatRule	Server scripts	Creates a conditional format rule.
METHOD	workbook.createConstant(options)	workbook.Expression	Server scripts	Creates a constant expression.
METHOD	workbook.createCurrency(options)	workbook.Currency	Server scripts	Creates a currency.
METHOD	workbook.createDataDimension(options)	workbook.DataDimension	Server scripts	Creates a data dimension.
METHOD	workbook.createDataDimensionItem(options)	workbook.DataDimensionItem	Server scripts	Creates a data dimension item.
METHOD	workbook.createDataMeasure(options)	workbook.DataMeasure	Server scripts	Creates a data measure.
METHOD	workbook.createDimensionSelector(options)	workbook.DimensionSelector	Server scripts	Creates a dimension selector.
METHOD	workbook.createDuration(options)	workbook.Duration	Server scripts	Creates a duration.
METHOD	workbook.createExpression(options)	workbook.Expression	Server scripts	Creates an expression.
METHOD	workbook.createFieldContext(options)	workbook.FieldContext	Server scripts	Creates a field context for table column.
METHOD	workbook.createFontSize(options)	workbook.FontSize	Server scripts	Creates a font size.
METHOD	workbook.createLegend(options)	workbook.Legend	Server scripts	Creates a chart legend.
METHOD	workbook.createLimitingFilter(options)	workbook.LimitingFilter	Server scripts	Creates a limiting filter.
METHOD	workbook.createMeasureSelector(options)	workbook.MeasureSelector	Server scripts	Creates a measure selector.
METHOD	workbook.createMeasureValueSelector(options)	workbook.MeasureValueSelector	Server scripts	Creates a measure value selector.
METHOD	workbook.createPathSelector(options)	workbook.PathSelector	Server scripts	Creates a path selector.
METHOD	workbook.createPivot(options)	workbook.Pivot	Server scripts	Creates a pivot definition. A pivot is a workbook component that enables you to pivot your dataset query results by defining measures and dimensions, so that you can analyze different subsets of data.
METHOD	workbook.createPivotAxis(options)	workbook.PivotAxis	Server scripts	Creates a pivot axis, which includes a data root and a sort definition.
METHOD	workbook.createPositionPercent(options)	workbook.PositionPercent	Server scripts	Creates a position defined by percentages of the x and y axes.
METHOD	workbook.createPositionUnits(options)	workbook.PositionUnits	Server scripts	Creates a position defined by units.
METHOD	workbook.createPositionValues(options)	workbook.PositionValues	Server scripts	Creates a position defined by horizontal and vertical position values.
METHOD	workbook.createRange(options)	workbook.Range	Server scripts	Creates a range.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	workbook.createReportStyle(options)	workbook.ReportStyle	Server scripts	Creates a report style.
METHOD	workbook.createReportStyleRule(options)	workbook.ReportStyleRule	Server scripts	Creates a report style rule.
METHOD	workbook.createSection(options)	workbook.Section	Server scripts	Creates a section.
METHOD	workbook.createSeries(options)	workbook.Series	Server scripts	Creates a chart series, which is a set of aspects.
METHOD	workbook.createSimpleRecordKey	workbook.RecordKey	Server scripts	Creates a record key.
METHOD	workbook.createSort(options)	workbook.Sort	Server scripts	Creates a sort.
METHOD	workbook.createSortByDataDimensionItem(options)	workbook.SortByDataDimensionItem	Server scripts	Creates a sort based on a data dimension item.
METHOD	workbook.createSortByMeasure(options)	workbook.SortByMeasure	Server scripts	Creates a sort based on a measure.
METHOD	workbook.createSortDefinition(options)	workbook.SortDefinition	Server scripts	Creates a sort definition.
METHOD	workbook.createStyle(options)	workbook.Style	Server scripts	Creates a style.
METHOD	workbook.createTable(options)	workbook.Table	Server scripts	Creates a table view.
METHOD	workbook.createTableColumn(options)	workbook.TableColumn	Server scripts	Creates a table column.
METHOD	workbook.createTableColumnCondition(options)	workbook.TableColumnCondition	Server scripts	Creates a table column condition.
METHOD	workbook.createTableColumnFilter(options)	workbook.TableColumnFilter	Server scripts	Creates a table filter.
METHOD	workbook.createTranslation(options)	workbook.Expression	Server scripts	Creates a translation (a translation expression).
METHOD	workbook.list()	Object[]	Server scripts	Lists all existing workbooks.
METHOD	workbook.listPaged(options)	PagedInfoData	Server scripts	Retrieves a set of pages with metadata about workbooks.
METHOD	workbook.loadWorkbook(options)	workbook.Workbook	Server scripts	Loads an existing workbook.
ENUM	workbook.Aggregation	enum	Server scripts	Holds string values for aggregation types. Used to set the value of the options.aggregation parameter of the workbook.createDataMeasure(options) method.
ENUM	workbook.AspectType	enum	Server scripts	Holds string values for aspect types. Used to set the options.type parameter of the workbook.createAspect(options) method.
ENUM	workbook.ChartType	enum	Server scripts	Holds string values for chart types. Used to pass the type value to workbook.createChart(options)
ENUM	workbook.Color	enum	Server scripts	Holds string values for colors. Used to set the options.backgroundcolor, options.color, and options.textDecorationColor parameters of the workbook.createStyle(options) method.
ENUM	workbook.ConstantType	enum	Server scripts	Holds string values for constant types. Used to set the value of the options.type parameter of the workbook.createConstant(options) method.
ENUM	workbook.DateTimeHierarchy	enum	Server scripts	Holds string values for workbook date-time hierarchy types.
ENUM	workbook.DateTimeProperty	enum	Server scripts	Holds string values for workbook date-time property types. Used to set the value of the DATE_TIME_PROPERTY in workbook.ExpressionType.
ENUM	workbook.ExpressionType	enum	Server scripts	Holds string values for workbook expression types. Use these values for the options.functionId parameter when creating an expression using workbook.createExpression(options)
ENUM	workbook.FontSize	enum	Server scripts	Holds string values for font sizes. Used to set the value for the options.fontSize parameter of the workbook.createStyle(options) method.
ENUM	workbook.FontStyle	enum	Server scripts	Holds string values for font sizes. Used to set the value for the options.fontSize parameter of the workbook.createStyle(options) method.
ENUM	workbook.FontWeight	enum	Server scripts	Holds string values for font weights. Used to set the value for the options.fontWeight parameter of the workbook.createStyle(options) method.
ENUM	workbook.Image	enum	Server scripts	Holds string values for images that you can use in workbooks. Used as a value for the Style.backgroundImage property.
ENUM	workbook.Position	enum	Server scripts	Holds string values for positions. Used to set the value for the PositionValues.horizontal and PositionValues.vertical properties.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
ENUM	workbook.Stacking	enum	Server scripts	Holds stacking types. Used to pass the stacking value to workbook.createChart(options).
ENUM	workbook.TemporalUnit	enum	Server scripts	Holds string values for temporal units, such as hours or minutes. Used to set the value of the options.start and options.end parameters of the workbook.createDuration(options) method.
ENUM	workbook.TextAlign	enum	Server scripts	Holds string values for text alignments. Used to set the value for the options.textAlign parameter of the workbook.createStyle(options) method.
ENUM	workbook.TextDecorationLine	enum	Server scripts	Holds string values for text decoration line types, such as underline and strikethrough. Used to set the value for the options.textDecorationLine parameter of the workbook.createStyle(options) method.
ENUM	workbook.TextDecorationStyle	enum	Server scripts	Holds string values for text decoration line styles, such as solid and dashed. Used to set the value for the options.textDecoractionStyle parameter of the workbook.createStyle(options) method.
ENUM	workbook.TotalLine	enum	Server scripts	Holds string values for predefined total line formats. Used to pass the totalLine value to workbook.createDataDimension(options) and to workbook.createSection(options).
ENUM	workbook.Unit	enum	Server scripts	Holds string values for units of measurement. Used to set the options.unit parameter in the workbook.createPositionUnits(options) method.

Aspect Object Members

The following members are available for a workbook.Aspect object.

Return Type / Value Type

Supported Script Types

workbook.CalculatedMeasure | workbook.DataMeasure

The measure of the aspect.

The type of the aspect.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Aspect.measure	workbook.CalculatedMeasure | workbook.DataMeasure	Server scripts	The measure of the aspect.
PROPERTY	Aspect.type	string	Server scripts	The type of the aspect. Set this value using workbook.AspectType.

CalculatedMeasure Object Members

The following members are available for a workbook.CalculatedMeasure object.

Return Type / Value Type

Supported Script Types

CalculatedMeasure.expression

workbook.Expression

The expression for the calculated measure.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	CalculatedMeasure.expression	workbook.Expression	Server scripts	The expression for the calculated measure.
PROPERTY	CalculatedMeasure.label	string | workbook.Expression	Server scripts	The label of the calculated measure.

Category Object Members

The following members are available for a object.

Return Type / Value Type

Supported Script Types

workbook.ChartAxis

The axis of the category.

workbook.DataDimension | workbook.Section

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Category.axis	workbook.ChartAxis	Server scripts	The axis of the category.
PROPERTY	Category.root	workbook.DataDimension | workbook.Section	Server scripts	The section or data dimension (that is, the fields for the x-axis).
PROPERTY	Category.sortDefinitions	workbook.SortDefinition[]	Server scripts	The sort definitions of the category.

ChartAxis Object Members

The following members are available for a workbook.ChartAxis object.

Return Type / Value Type

Supported Script Types

The title of the chart axis.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	ChartAxis.title	string	Server scripts	The title of the chart axis.

Chart Object Members

The following members are available for a workbook.Chart object.

Return Type / Value Type

Supported Script Types

Chart.aggregationFilters

Array<workbook.LimitingFilter | workbook.ConditionalFilter>

Limiting and conditional filters for the chart.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Chart.aggregationFilters	Array<workbook.LimitingFilter | workbook.ConditionalFilter>	Server scripts	Limiting and conditional filters for the chart.
PROPERTY	Chart.category	string	Server scripts	The category of the chart.
PROPERTY	Chart.dataset	dataset.Dataset	Server scripts	The underlying dataset for the chart.
PROPERTY	Chart.filterExpressions	workbook.Expression[]	Server scripts	The filter expressions of the chart.
PROPERTY	Chart.id	string	Server scripts	The ID of the chart.
PROPERTY	Chart.legend	workbook.Legend	Server scripts	The legend of the chart.
PROPERTY	Chart.name	string	Server scripts	The name of chart.
PROPERTY	Chart.series	workbook.Series	Server scripts	The series of the chart.
PROPERTY	Chart.stacking	string	Server scripts	The stacking type for the chart.
PROPERTY	Chart.subTitle	string	Server scripts	The subtitle of the chart.
PROPERTY	Chart.title	string	Server scripts	The title of chart.
PROPERTY	Chart.type	workbook.ChartType	Server scripts	The type of the chart.

Color Object Members

The following members are available for a workbook.Color object.

Return Type / Value Type

Supported Script Types

The opacity, or transparency, of the color.

The blue portion of the color.

The green portion of the color.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Color.alpha	number	Server scripts	The opacity, or transparency, of the color.
PROPERTY	Color.blue	number	Server scripts	The blue portion of the color.
PROPERTY	Color.green	number	Server scripts	The green portion of the color.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Color.red	number	Server scripts	The red portion of the color.

ConditionalFilter Object Members

The following members are available for a workbook.ConditionalFilter object.

Return Type / Value Type

Supported Script Types

ConditionalFilter.columnSelector

workbook.DescendantorSelfNodesSelector| workbook.PathSelector | workbook.DimensionSelector | workbook.ChildNodesSelector

The column selector.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	ConditionalFilter.columnSelector	workbook.DescendantorSelfNodesSelector| workbook.PathSelector | workbook.DimensionSelector | workbook.ChildNodesSelector	Server scripts	The column selector.
PROPERTY	ConditionalFilter.filteredNodesSelector	workbook.PathSelector | workbook.DimensionSelector	Server scripts	The selected filters.
PROPERTY	ConditionalFilter.measure	workbook.CalculatedMeasure | workbook.DataMeasure	Server scripts	The measure of the filter.
PROPERTY	ConditionalFilter.otherAxisSelector	workbook.PathSelector | workbook.DimensionSelector	Server scripts	The filter selector for the other axis.
PROPERTY	ConditionalFilter.predicate	workbook.Expression	Server scripts	The predicate which indicates if the condition is met.
PROPERTY	ConditionalFilter.row	boolean	Server scripts	The row axis indicator.
PROPERTY	ConditionalFilter.rowSelector	workbook.DescendantorSelfNodesSelector| workbook.PathSelector | workbook.DimensionSelector | workbook.ChildNodesSelector	Server scripts	The row selector.

ConditionalFormat Object Members

The following members are available for a workbook.ConditionalFormat object.

Return Type / Value Type

Supported Script Types

ConditionalFormat.rules

workbook.ConditionalFormatRule[]

The conditional formatting rules that are included in the conditional format.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	ConditionalFormat.rules	workbook.ConditionalFormatRule[]	Server scripts	The conditional formatting rules that are included in the conditional format.

ConditionalFormatRule Object Members

The following members are available for a workbook.ConditionalFormatRule object.

Return Type / Value Type

Supported Script Types

ConditionalFormatRule.filter

workbook.TableColumnFilter

The filter that determines which rows or cells to apply the conditional format to.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	ConditionalFormatRule.filter	workbook.TableColumnFilter	Server scripts	The filter that determines which rows or cells to apply the conditional format to.
PROPERTY	ConditionalFormatRule.style	workbook.Style	Server scripts	The style to apply as the conditional format.

Currency Object Members

The following members are available for a workbook.Currency object.

Return Type / Value Type

Supported Script Types

The amount of the currency.

The ID of the currency (for example, USD, EUR, GBP, and so on).

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Currency.amount**	number	Server scripts	The amount of the currency.
PROPERTY	**Currency.id**	string	Server scripts	The ID of the currency (for example, USD, EUR, GBP, and so on).

DataDimension Object Members

The following members are available for a workbook.DataDimension object.

Return Type / Value Type

Supported Script Types

DataDimension.children

Array<workbook.DataDimension | workbook.Section>

The children of the data dimension.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**DataDimension.children**	Array<workbook.DataDimension | workbook.Section>	Server scripts	The children of the data dimension.
PROPERTY	**DataDimension.items**	workbook.DataDimensionItem[]	Server scripts	The items of the data dimension.
PROPERTY	**DataDimension.totalLine**	string	Server scripts	The formatting option for the total line. Set this value using workbook.TotalLine.

DataDimensionItem Object Members

The following members are available for a workbook.DataDimensionItem object.

Return Type / Value Type

Supported Script Types

DataDimensionItem.expression

workbook.Expression

The expression of data dimension item.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**DataDimensionItem.expression**	workbook.Expression	Server scripts	The expression of data dimension item.
PROPERTY	**DataDimensionItem.label**	string	Server scripts	The label of the data dimension item.

DataDimensionItemValue Object Members

The following members are available for a workbook.DataDimensionItemValue object.

Return Type / Value Type

Supported Script Types

DataDimensionItemValue.item

workbook.DataDimensionItem

The data dimension item.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**DataDimensionItemValue.item**	workbook.DataDimensionItem	Server scripts	The data dimension item.
PROPERTY	**DataDimensionItemValue.value**	string | number | boolean | workbook.Record | workbook.Currency | workbook.Range | workbook.Duration	Server scripts	The value of the data dimension item.

DataDimensionValue Object Members

The following members are available for a workbook.DataDimensionValue object.

Return Type / Value Type

Supported Script Types

DataDimensionValue.dataDimension

workbook.DataDimension

The data dimension.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**DataDimensionValue.dataDimension**	`workbook.DataDimension`	Server scripts	The data dimension.
PROPERTY	**DataDimensionValue.itemValues**	`workbook.DataDimensionItemValue[]`	Server scripts	The item values for the data dimension.

DataMeasure Object Members

The following members are available for a workbook.DataMeasure object.

Return Type / Value Type

Supported Script Types

DataMeasure.aggregation

The aggregation of the data measure.

DataMeasure.expression

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**DataMeasure.aggregation**	`string`	Server scripts	The aggregation of the data measure.
PROPERTY	**DataMeasure.expression**	`workbook.Expression`	Server scripts	The expression for the data measure. This property is used if the data measure is a single-expression measure.
PROPERTY	**DataMeasure.expressions**	`workbook.Expression[]`	Server scripts	The expressions for the data measure. This property is used if the data measure is a multiple-expression measure.
PROPERTY	**DataMeasure.label**	`string | workbook.Expression`	Server scripts	The label of the data measure.

DimensionSelector Object Members

The following members are available for a workbook.DimensionSelector object.

Return Type / Value Type

Supported Script Types

DimensionSelector.dimension

workbook.DataDimension | workbook.Section

The dimension of the dimension selector.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**DimensionSelector.dimension**	`workbook.DataDimension | workbook.Section`	Server scripts	The dimension of the dimension selector.

Duration Object Members

The following members are available for a workbook.Duration object.

Return Type / Value Type

Supported Script Types

The amount of the duration.

The units of the duration.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Duration.amount**	`number`	Server scripts	The amount of the duration.
PROPERTY	**Duration.units**	`Object`	Server scripts	The units of the duration.

Expression Object Members

The following members are available for a workbook.Expression object.

Return Type / Value Type

Supported Script Types

Expression.functionId

The ID of the function used in the expression.

Expression.parameters

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Expression.functionId**	`string`	Server scripts	The ID of the function used in the expression.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Expression.parameters**	Object	Server scripts	The parameters of the expression.

FieldContext Object Members

The following members are available for a workbook.FieldContext object.

Return Type / Value Type

Supported Script Types

FieldContext.name

The name of the field context (for example, DISPLAY or CONSOLIDATED)

FieldContext.parameters

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**FieldContext.name**	string	Server scripts	The name of the field context (for example, DISPLAY or CONSOLIDATED)
PROPERTY	**FieldContext.parameters**	Object	Server scripts	The parameters of the field context.

FontSize Object Members

The following members are available for a workbook.FontSize object.

Return Type / Value Type

Supported Script Types

The numeric size of the font size.

The unit of the font size.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**FontSize.size**	number	Server scripts	The numeric size of the font size.
PROPERTY	**FontSize.unit**	string	Server scripts	The unit of the font size.

Legend Object Members

The following members are available for a object.

Return Type / Value Type

Supported Script Types

workbook.ChartAxis[]

The axes of the legend.

workbook.DataDimension | workbook.Section

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Legend.axes**	workbook.ChartAxis[]	Server scripts	The axes of the legend.
PROPERTY	**Legend.root**	workbook.DataDimension | workbook.Section	Server scripts	The section or data dimension (that is., the fields for the y-axis).
PROPERTY	**Legend.sortDefinitions**	workbook.SortDefinition[]	Server scripts	The sort definitions of the legend.

LimitingFilter Object Members

The following members are available for a workbook.LimitingFilter object.

Return Type / Value Type

Supported Script Types

LimitingFilter.filteredNodesSelector

workbook.PathSelector | workbook.DimensionSelector

The selected filter.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**LimitingFilter.filteredNodesSelector**	workbook.PathSelector | workbook.DimensionSelector	Server scripts	The selected filter.
PROPERTY	**LimitingFilter.limit**	number	Server scripts	The limit number for the filter.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	LimitingFilter.row	boolean	Server scripts	The row axis indicator for the filter.
PROPERTY	LimitingFilter.sortBys	Array<workbook.SortByDataDimensionItem | workbook.SortByMeasure>	Server scripts	The ordering elements of the filter.

MeasureSelector Object Members

The following members are available for a workbook.MeasureSelector object.

Return Type / Value Type

Supported Script Types

MeasureSelector.measures

workbook.CalculatedMeasure[] | workbook.DataMeasure

The measures for the measure selector.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	MeasureSelector.measures	workbook.CalculatedMeasure[] | workbook.DataMeasure	Server scripts	The measures for the measure selector.

MeasureValue Object Members

The following members are available for a workbook.MeasureValue object.

Return Type / Value Type

Supported Script Types

MeasureValue.measure

workbook.MeasureValue

The measure to use for the measure value.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	MeasureValue.measure	workbook.MeasureValue	Server scripts	The measure to use for the measure value.
PROPERTY	MeasureValue.value	string | number | boolean | workbook.Record | workbook.Currency | workbook.Range | workbook.Duration	Server scripts	The value to use for the measure value.

MeasureValueSelector Object Members

The following members are available for a workbook.MeasureValueSelector object.

Return Type / Value Type

Supported Script Types

MeasureValueSelector.columnSelector

workbook.DimensionSelector | workbook.PathSelector | workbook.DescendantorSelfNodesSelector

The column selector.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	MeasureValueSelector.columnSelector	workbook.DimensionSelector | workbook.PathSelector | workbook.DescendantorSelfNodesSelector	Server scripts	The column selector.
PROPERTY	MeasureValueSelector.measureSelector	workbook.MeasureSelector[]	Server scripts	The measure selectors.
PROPERTY	MeasureValueSelector.rowSelector	workbook.DimensionSelector | workbook.PathSelector | workbook.DescendantorSelfNodesSelector	Server scripts	The row selector.

PathSelector Object Members

The following members are available for a workbook.PathSelector object.

Return Type / Value Type

Supported Script Types

PathSelector.elements

workbook.DimensionSelector

The elements denoting 'xpath' of the selector.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**PathSelector.elements**	workbook.DimensionSelector	Server scripts	The elements denoting 'xpath' of the selector.

PivotAxis Object Members

The following members are available for a workbook.PivotAxis object.

Return Type / Value Type

Supported Script Types

workbook.DataDimension | workbook.Section

The data for the pivot axis.

PivotAxis.sortDefinitions

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**PivotAxis.root**	workbook.DataDimension | workbook.Section	Server scripts	The data for the pivot axis.
PROPERTY	**PivotAxis.sortDefinitions**	workbook.SortDefinition[]	Server scripts	The sort definitions of the pivot axis.

Pivot Object Members

The following members are available for a workbook.Pivot object.

Return Type / Value Type

Supported Script Types

Pivot.aggregationFilters

Array<workbook.ConditionalFilter |workbook.LimitingFilter>

The limiting and conditional filters of the pivot definition.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Pivot.aggregationFilters**	Array<workbook.ConditionalFilter |workbook.LimitingFilter>	Server scripts	The limiting and conditional filters of the pivot definition.
PROPERTY	**Pivot.columnAxis**	workbook.PivotAxis	Server scripts	The column axis of the pivot definition.
PROPERTY	**Pivot.dataset**	dataset.Dataset	Server scripts	The underlying dataset of the pivot definition.
PROPERTY	**Pivot.datasetLink**	datasetLink.DatasetLink	Server scripts	Underlying dataset for the pivot.
PROPERTY	**Pivot.filterExpressions**	workbook.Expression	Server scripts	The filter expressions of the pivot definition.
PROPERTY	**Pivot.id**	string	Server scripts	The ID of the pivot definition.
PROPERTY	**Pivot.name**	string	Server scripts	The name of the pivot definition.
PROPERTY	**Pivot.portletName**	string | workbook.Expression	Server scripts	The name of the portlet for the pivot.
PROPERTY	**Pivot.reportStyles**	workbook.ReportStyle[]	Server scripts	Report styles for the pivot.
PROPERTY	**Pivot.rowAxis**	workbook.PivotAxis	Server scripts	The row axis of the pivot definition.

PivotIntersection Object Members

The following members are available for a workbook.PivotIntersection object.

Return Type / Value Type

Supported Script Types

PivotIntersection.column

workbook.DataDimensionValue | workbook.SectionValue

The column dimension value.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**PivotIntersection.column**	workbook.DataDimensionValue | workbook.SectionValue	Server scripts	The column dimension value.
PROPERTY	**PivotIntersection.measureValues**	workbook.MeasureValue[]	Server scripts	The measure values in the pivot intersection.
PROPERTY	**PivotIntersection.row**	workbook.DataDimensionValue | workbook.SectionValue	Server scripts	The row dimension value.

PositionPercent Object Members

The following members are available for a workbook.PositionPercent object.

Return Type / Value Type

Supported Script Types

PositionPercent.percentX

The percentage of the x dimension.

PositionPercent.percentY

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	PositionPercent.percentX	number	Server scripts	The percentage of the x dimension.
PROPERTY	PositionPercent.percentY	number	Server scripts	The percentage of the y dimension.

PositionUnits Object Members

The following members are available for a workbook.PositionUnits object.

Return Type / Value Type

Supported Script Types

PositionUnits.unit

The units for the position.

The x value of the position.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	PositionUnits.unit	string	Server scripts	The units for the position.
PROPERTY	PositionUnits.x	number	Server scripts	The x value of the position.
PROPERTY	PositionUnits.y	number	Server scripts	The y value of the position.

PositionValues Object Members

The following members are available for a workbook.PositionValues object.

Return Type / Value Type

Supported Script Types

PositionValues.horizontal

The horizontal value of the position.

PositionValues.vertical

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	PositionValues.horizontal	string	Server scripts	The horizontal value of the position.
PROPERTY	PositionValues.vertical	string	Server scripts	The vertical value of the position.

Range Object Members

The following members are available for a workbook.Range object.

Return Type / Value Type

Supported Script Types

The end date or date-time of the range.

The start date or date-time of the range.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Range.end	string	Server scripts	The end date or date-time of the range.
PROPERTY	Range.start	string	Server scripts	The start date or date-time of the range.

Record Object Members

The following members are available for a workbook.Record object.

Return Type / Value Type

Supported Script Types

The name of the record type for the record.

Record.primaryKey

The primary key of the record.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Record.name	string	Server scripts	The name of the record type for the record.
PROPERTY	Record.primaryKey	number	Server scripts	The primary key of the record.
PROPERTY	Record.properties	Object	Server scripts	The properties of the record.

RecordKey Object Members

The following members are available for a workbook.RecordKey object.

Return Type / Value Type

Supported Script Types

RecordKey.properties

The properties of the record key.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	RecordKey.properties	Object	Server scripts	The properties of the record key.

ReportStyle Object Members

The following members are available for a workbook.ReportStyle object.

Return Type / Value Type

Supported Script Types

ReportStyle.rules

workbook.ReportStyleRule[]

The formatting rules for the report style.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	ReportStyle.rules	workbook.ReportStyleRule[]	Server scripts	The formatting rules for the report style.
PROPERTY	ReportStyle.selectors	workbook.MeasureValueSelector[]	Server scripts	The selectors for the report style.

ReportStyleRule Object Members

The following members are available for a workbook.ReportStyleRule object.

Return Type / Value Type

Supported Script Types

ReportStyleRule.expression

workbook.Expression

A boolean expression indicating whether the style should be applied.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	ReportStyleRule.expression	workbook.Expression	Server scripts	A boolean expression indicating whether the style should be applied.
PROPERTY	ReportStyleRule.style	workbook.Style	Server scripts	The style to be applied.

Section Object Members

The following members are available for a workbook.Section object.

Return Type / Value Type

Supported Script Types

Section.children

Array<workbook.CalculatedMeasure | workbook.DataMeasure | workbook.DataDimension | workbook.DataDimensionItem>

The children of the section.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	Section.children	Array<workbook.CalculatedMeasure | workbook.DataMeasure | workbook.DataDimension | workbook.DataDimensionItem>	Server scripts	The children of the section.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Section.totalLine**	string	Server scripts	The formatting option for the total line. Set this value using workbook.TotalLine.

SectionValue Object Members

The following members are available for a workbook.SectionValue object.

Return Type / Value Type

Supported Script Types

SectionValue.section

workbook.Section

The section of the section value.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**SectionValue.section**	workbook.Section	Server scripts	The section of the section value.

Series Object Members

The following members are available for a workbook.Series object.

Return Type / Value Type

Supported Script Types

The aspects for the series.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Series.aspects**	string	Server scripts	The aspects for the series.

Sort Object Members

The following members are available for a workbook.Sort object.

Return Type / Value Type

Supported Script Types

When set to true, indicates the sort is in ascending order.

Sort.caseSensitive

When set to true, indicates the sort is case sensitive.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Sort.ascending**	boolean	Server scripts	When set to true, indicates the sort is in ascending order.
PROPERTY	**Sort.caseSensitive**	boolean	Server scripts	When set to true, indicates the sort is case sensitive.
PROPERTY	**Sort.locale**	query.SortLocale (read-only)	Server scripts	The locale of the sort.
PROPERTY	**Sort.nullsLast**	boolean	Server scripts	When set to true, indicates that nulls are placed last in the sort.
PROPERTY	**Sort.order**	number	Server scripts	Sort order indicator.

SortDefinition Object Members

The following members are available for a workbook.SortDefinition object.

Return Type / Value Type

Supported Script Types

SortDefinition.selector

workbook.DimensionSelector | workbook.PathSelector

The selector for the sort definition.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**SortDefinition.selector**	workbook.DimensionSelector | workbook.PathSelector	Server scripts	The selector for the sort definition.
PROPERTY	**SortDefinition.sortBys**	Array<workbook.SortByDataDimensionItem | workbook.SortByMeasure>	Server scripts	The sort order for the sort definition.

SortByDataDimensionItem Object Members

The following members are available for a workbook.SortByDataDimensionItem object.

Return Type / Value Type

Supported Script Types

SortByDataDimensionItem.item

workbook.DataDimensionItem

The data dimension item to use for the sort.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**SortByDataDimensionItem.item**	`workbook.DataDimensionItem`	Server scripts	The data dimension item to use for the sort.
PROPERTY	**SortByDataDimensionItem.sort**	`workbook.Sort`	Server scripts	The sort to use.

SortByMeasure Object Members

The following members are available for a workbook.SortByMeasure object.

Return Type / Value Type

Supported Script Types

SortByMeasure.measure

workbook.CalculatedMeasure | workbook.DataMeasure

The measure for the sort.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**SortByMeasure.measure**	`workbook.CalculatedMeasure | workbook.DataMeasure`	Server scripts	The measure for the sort.
PROPERTY	**SortByMeasure.otherAxisSelector**	`workbook.DescendantorSelfNodesSelector | workbook.PathSelector | workbook.DimensionSelector`	Server scripts	The selector for the axis that is not defined in the associated sort definition.
PROPERTY	**SortByMeasure.sort**	`workbook.Sort`	Server scripts	The sort to use.

Style Object Members

The following members are available for a workbook.Style object.

Return Type / Value Type

Supported Script Types

Style.backgroundColor

string | workbook.Color

The background color of the style.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Style.backgroundColor**	`string | workbook.Color`	Server scripts	The background color of the style.
PROPERTY	**Style.backgroundImage**	`string`	Server scripts	The background image of the style.
PROPERTY	**Style.backgroundPosition**	`workbook.PositionPercent | workbook.PositionUnits | workbook.PositionValues`	Server scripts	The background position of the style.
PROPERTY	**Style.color**	`string | workbook.Color`	Server scripts	The color of the style.
PROPERTY	**Style.fontSize**	`string | workbook.FontSize`	Server scripts	The font size of the style.
PROPERTY	**Style.fontStyle**	`string`	Server scripts	The font style of the style.
PROPERTY	**Style.fontWeight**	`string`	Server scripts	The font weight of the style.
PROPERTY	**Style.textAlign**	`string`	Server scripts	The text alignment of the style.
PROPERTY	**Style.textDecorationColor**	`string | workbook.Color`	Server scripts	The text decoration color of the style.
PROPERTY	**Style.textDecorationLine**	`string`	Server scripts	The text decoration line of the style.
PROPERTY	**Style.textDecorationStyle**	`string`	Server scripts	The text decoration style of the style.

Table Object Members

The following members are available for a workbook.Table object.

Return Type / Value Type

Supported Script Types

workbook.TableColumn

The columns in the table view.

The dataset for the table view.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Table.columns**	workbook.TableColumn	Server scripts	The columns in the table view.
PROPERTY	**Table.dataset**	dataset.Dataset	Server scripts	The dataset for the table view.
PROPERTY	**Table.id**	string	Server scripts	The ID of the table view.
PROPERTY	**Table.name**	string | workbook.Expression	Server scripts	The label of the table view.

TableColumn Object Members

The following members are available for a workbook.TableColumn object.

Return Type / Value Type

Supported Script Types

TableColumn.alias

The alias for the table column.

TableColumn.datasetColumnAlias

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**TableColumn.alias**	string	Server scripts	The alias for the table column.
PROPERTY	**TableColumn.datasetColumnAlias**	string	Server scripts	The alias of the dataset column from which the table column was created.
PROPERTY	**TableColumn.fieldContext**	workbook.FieldContext	Server scripts	The field context for the field used in the table column.
PROPERTY	**TableColumn.filters**	workbook.TableColumnFilter	Server scripts	The filters for the table column.
PROPERTY	**TableColumn.label**	string	Server scripts	The label for the table column.
PROPERTY	**TableColumn.sort**	workbook.Sort	Server scripts	The sort for the table column.
PROPERTY	**TableColumn.width**	number	Server scripts	The desired width of the table column when displayed in the UI.

TableColumnCondition Object Members

The following members are available for a workbook.TableColumnCondition object.

Return Type / Value Type

Supported Script Types

TableColumnCondition.filters

workbook.TableColumnFilter[]

The filters for the condition

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**TableColumnCondition.filters**	workbook.TableColumnFilter[]	Server scripts	The filters for the condition
PROPERTY	**TableColumnCondition.operator**	string	Server scripts	The operator for the condition.

TableColumnFilter Object Members

The following members are available for a workbook.TableColumnFilter object.

Return Type / Value Type

Supported Script Types

TableColumnFilter.operator

The operator for the table column filter.

TableColumnFilter.values

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**TableColumnFilter.operator**	`string`	Server scripts	The operator for the table column filter.
PROPERTY	**TableColumnFilter.values**	`Array<null | Object | boolean | number | string | Date>`	Server scripts	The values for the table column filter.

Workbook Object Members

The following members are available for a workbook.Workbook object.

Return Type / Value Type

Supported Script Types

Workbook.runPivot(options)

workbook.PivotIntersection[]

Runs a pivot in the workbook and returns the results as a set of row-column intersections.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**Workbook.runPivot(options)**	`workbook.PivotIntersection[]`	Server scripts	Runs a pivot in the workbook and returns the results as a set of row-column intersections.
PROPERTY	**Workbook.description**	`string`	Server scripts	The description of the workbook.
PROPERTY	**Workbook.id**	`string`	Server scripts	The ID of the workbook.
PROPERTY	**Workbook.name**	`string`	Server scripts	The name of the workbook.
PROPERTY	**Workbook.pivots**	`workbook.Pivot[]`	Server scripts	The pivots in the workbook.
PROPERTY	**Workbook.tables**	`workbook.Table[]`	Server scripts	The tables in the workbook.

▶ Official NetSuite Script Samples (2 Sample(s))

📄 Sample 1: Create Datasets, Dataset Links, and a Workbook with a Pivot and Run the Workbook

```
/**
 * @NApiVersion 2.x
 */
// The following script creates a simple workbook that contains a simple table
require(['N/workbook', 'N/dataset', 'N/datasetLink'], function(nWorkbook, nDataset, datasetLink) {
    var join = nDataset.createJoin({
        fieldId: "budgetmachine"
    });

    var period = nDataset.createColumn({
        join: join,
        fieldId: "period",
        alias: "budgetmachineperiod",
        label: "Accounting Period"
    });

    var department = nDataset.createColumn({
        fieldId: "department",
        alias: "department",
        label: "Department"
    });

    var total = nDataset.createColumn({
        fieldId: "total",
        alias: "total",
        label: "Amount (Total)"
    });

    var budget = nDataset.create({
        type: 'budgets',
        columns: [period, department, total]
    });

    var postingperiod = nDataset.createColumn({
        fieldId: "postingperiod",
        alias: "postingperiod",
        label: "Posting Period"
    });

    var amount = nDataset.createColumn({
        fieldId: "amount",
        alias: "amount",
        label: "Amount"
    });

    var sales = nDataset.create({
        type: 'salesinvoiced',
        columns: [postingperiod, department, amount],
    });


    var budgetmachineperiod = budget.getExpressionFromColumn({
        alias:"budgetmachineperiod"
    });
    var postingperiodExpression = sales.getExpressionFromColumn({
        alias:"postingperiod"
    });
    var link = datasetLink.create({
        datasets: [budget, sales],
        expressions: [[budgetmachineperiod, postingperiodExpression]],
        id: "link"
    });

    var postingPeriodItem = nWorkbook.createDataDimensionItem({
        expression: postingperiodExpression
    });
    var postingPeriodDimension = nWorkbook.createDataDimension({
        items: [postingPeriodItem]
    });
    var rowSection = nWorkbook.createSection({
        children: [postingPeriodDimension]
    });

    var departmentItem = nWorkbook.createDataDimensionItem({
        expression: budget.getExpressionFromColumn({
            alias: "department"
        })
    });
    var departmentDimension = nWorkbook.createDataDimension({
        items: [departmentItem]
    });

    var sumTotal = nWorkbook.createDataMeasure({
        label: 'Sum Total',
        expression: budget.getExpressionFromColumn({
            alias: 'total'
        }),
        aggregation: 'SUM'
```

```
    });

    var sumAmountNet = nWorkbook.createDataMeasure({
        label: 'Sum Amount',
        expression: sales.getExpressionFromColumn({
            alias: 'amount'
        }),
        aggregation: 'SUM'
    });

    var columnSection = nWorkbook.createSection({
        children: [departmentDimension, sumTotal, sumAmountNet]
    });

    var pivot = nWorkbook.createPivot({
        id: "pivot",
        rowAxis:  nWorkbook.createPivotAxis({
            root: rowSection
        }),
        columnAxis: nWorkbook.createPivotAxis({
            root: columnSection
        }),
        name: "Pivot",
        datasetLink: link
    });

    var wb = nWorkbook.create({
        pivots: [pivot]
    });

    wb.runPivot.promise("pivot").then(function(intersections){
        for (var i in intersections)
        {
            var intersection = intersections[i];
            if (intersection.row.itemValues) //skip header
            {
                console.log("Period: " + intersection.row.itemValues[0].value.name);
                console.log(intersection.column.section.children[1].label + ":");
                console.log(intersection.measureValues[0] ? intersection.measureValues[0].value.amount : 0);
                console.log(intersection.column.section.children[2].label + ":");
                console.log(intersection.measureValues[1] ? intersection.measureValues[1].value.amount : 0);
            }
        }
    })
});
```

📄 Sample 2: Create a Comprehensive Workbook

```
/**
 * @NApiVersion 2.x
 */
// The following script creates a workbook that includes a chart, a table, and a pivot. This sample uses a dataset that is not included in your account, so y
ou will need to change the dataset id to a valid value from your account. This dataset needs to include columns for 'id', 'name', 'date', and 'total'.

require(['N/workbook', 'N/dataset'], function(workbook, dataset){
    var myDataset = dataset.load({
        id: 'dataset_7'
    });

    var theIDExpression = myDataset.getExpressionFromColumn({
        alias: 'id'
    });
    var sort = workbook.createSort({
        ascending: false
    });
    var columnID = workbook.createTableColumn({
        datasetColumnAlias: 'id'
    });
    var columnName = workbook.createTableColumn({
        datasetColumnAlias: 'name'
    });
    var columnDate = workbook.createTableColumn({
        datasetColumnAlias: 'date'
    });
    var columnTotal = myBasicWorkbook.createTableColumn({
        datasetColumnAlias: 'total'
    });
    var tableView = workbook.createTableDefinition({
        id: 'view',
        name: 'View',
        dataset: myDataset,
        columns: [columnID, columnName, columnDate, columnTotal]
    });
    var theDateExpression = dataset.getExpressionFromColumn({
        alias: 'date'
    });
    var rowItem = workbook.createDataDimensionitem({
        label: 'A',
        expression: theDateExpression
    });
    var rowDataDimension = workbook.createDataDimension({
        items: [rowItem]
    });
    var rowSection = workbook.createSection({
        children: [rowDataDimension]
    });
    var theTotalExpression = dataset.getExpressionFromColumn({
        alias: 'total'
    });
    var columnItem = workbook.createDataDimensionItem({
        label: 'B',
        expression: theTotalExpression
    });    var columnDataDimension = workbook.createDataDimension({
        items: [columnItem]
    });
    var columnMeasure = workbook.createMeasure({
        label: 'M',
        expression: theIDExpression,
        aggregation: workbook.Aggregation.MAX
    });
    var columnSection = workbook.createSection({
        children: [columnDataDimension, columnMeasure]
    });
    var constExpr = workbook.createConstant({
        constant: 1
    });
    var anyOfExpr = workbook.createExpression({
        functionId: workbook.ExpressionType.AND,
        parameters: {
            expression: theIDExpression,
            set: [constExpr]
        }
    });
    var notExpr = workbook.createExpression({
        functionId: workbook.ExpressionType.NOT,
        parameters: {
            a: anyOfExpr
        }
    });
    var allSubNodesSelector = workbook.createAllSubNodesSelector();
    var rowItemSelector = workbook.createDimensionSelector({
        dimension: rowDataDimension
    });
    var columnItemSelector = workbook.createDimensionSelector({
        dimension: columnDataDimension
    });
    var rowSelector = workbook.createPathSelector({
        elements: [allSubNodesSelector, rowItemSelector]
    });
```

```
    var columnSelector = workbook.createPathSelector({
        elements: [allSubNodesSelector, columnItemSelector]
    });
    var rowSort = workbook.createDimensionSort({
        item:rowItem,
        sort:sort
    });
    var columnSort = workbook.createMeasureSort({
        measure: columnMeasure,
        sort: sort,
        otherAxisSelector: allSubNodesSelector
    });
    var rowSortDefinition = workbook.createSortDefinition({
        sortBys: [rowSort],
        selector: rowSelector
    });
    var columnSortDefinition = workbook.createSortDefinition({
        sortBys: [columnSort],
        selector: columnSelector
    });
    var rowAxis = workbook.createPivotAxis({
        root: rowSection,
        sortDefinitions: [rowSortDefinition]
    });
    var columnAxis = workbook.createPivotAxis({
        root: columnSection,
        sortDefinitions: [columnSortDefinition]
    });
    var limitingFilter = workbook.createLimitingFilter({
        row: true,
        filteredNodesSelector: rowSelector,
        limit: 1,
        sortBys: [rowSort]
    });
    var conditionalFilter = workbook.createConditionalFilter({
        row: false,
        filteredNodesSelector: rowSelector,
        otherAxisSelector: columnSelector,
        measure: columnMeasure,
        predicate: notExpr
    });
    var pivot = workbook.createPivotDefinition({
        id: 'pivot',
        name: 'Pivot',
        dataset: myDataset,
        rowAxis: rowAxis,
        columnAxis: columnAxis,
        filterExpressions: [notExpr],
        aggregationFilters: [limitingFilter, conditionalFilter]
    });
    var firstAxis = workbook.createChartAxis({
        title: 'First axis'
    });
    var secondAxis = workbook.createChartAxis({
        title: 'Second axis'
    });
    var category = workbook.createCategory({
        axis: firstAxis,
        root: rowSection
    });
    var legend = workbook.createLegend({
        axes: [secondAxis],
        root: columnSection
    });
    var aspect = workbook.createAspect({
        measure: columnMeasure
    });
    var series = workbook.createSeries({
        aspects: [aspect]
    });
    var chart = workbook.createChartDefinition({
        id: 'chart',
        name: 'Chart',
        type: workbook.ChartType.AREA,
        dataset: myDataset,
        category: category,
        legend: legend,
        series: [series]
    });
    var myNewWorkbook = workbook.create({
        description: 'My new updated workbook',
        name: 'Workbook Updated',
        tableDefinitions: [tableView],
        pivotDefinitions: [pivot],
        chartDefinitions: [chart]
    });
    var workbookList = workbook.list();

    log.debug({
        title: "MyNewWorkbook",
        details: myNewWorkbook
    });
});
```

54. N/workflow Module

Load the N/workflow module to initiate new workflow instances or trigger existing workflow instances.

`define(['N/workflow'], (workflow) ⇒ { ... })` ✓ 1 Code Sample(s) Scripts: Server scripts Permissions: Workflow

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/workflow'], (workflow) ⇒ { // Access methods and properties on workflow });
```

Use the N/workflow module to initiate new workflow instances or trigger existing workflow instances.

N/workflow Module Script Sample

N/workflow Module Members

Return Type / Value Type

Supported Script Types

workflow.initiate(options)

Initiates a workflow on-demand. This method is the programmatic equivalent of the Initiate Workflow Action action in SuiteFlow.

Returns the internal ID of the workflow instance used to track the workflow against the record.

workflow.trigger(options)

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	`workflow.initiate(options)`	`number`	Server scripts	Initiates a workflow on-demand. This method is the programmatic equivalent of the Initiate Workflow Action action in SuiteFlow. Returns the internal ID of the workflow instance used to track the workflow against the record.
METHOD	`workflow.trigger(options)`	`number`	Server scripts	Triggers a workflow on a record. The actions and transitions of the workflow are evaluated for the record in the workflow instance, based on the current state for the workflow instance. Returns the internal ID of the workflow instance used to track the workflow against the record.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: N/workflow Module Script Sample

```
/**
 * @NApiVersion 2.1
 */

require(['N/workflow', 'N/search', 'N/error', 'N/record'],
    function(workflow, search, error, record) {
        function initiateWorkflow() {
            var workflowInstanceId = workflow.initiate({
                recordType: 'customer',
                recordId: 24,
                workflowId: 'customworkflow_myWorkFlow'
            });
            var customerRecord = record.load({
                type: record.Type.CUSTOMER,
                id: 24
            });
        }
        initiateWorkflow();
    });
```

55. N/xml Module

Load the N/xml module to validate, parse, read, and modify XML documents.

`define(['N/xml'], (xml) => { ... })` ✓ 1 Code Sample(s) Scripts: Client and server scripts Permissions: -

```
/** * @NApiVersion 2.1 * @NScriptType UserEventScript */ define(['N/xml'], (xml) => { // Access methods and properties on xml });
```

Use the N/xml module to validate, parse, read, and modify XML documents.

N/xml Module Members

Return Type / Value Type

Supported Script Types

Client and server scripts

Represents an attribute node of an xml.Element object.

Client and server scripts

Represents an entire XML document. The XML DOM presents a document as a hierarchy of node objects. Use the methods and properties available to the xml.Document object to manipulate the XML document and the nodes in the document tree.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
OBJECT	**xml.Attr**	Object	Client and server scripts	Represents an attribute node of an xml.Element object.
OBJECT	**xml.Document**	Object	Client and server scripts	Represents an entire XML document. The XML DOM presents a document as a hierarchy of node objects. Use the methods and properties available to the xml.Document object to manipulate the XML document and the nodes in the document tree.
OBJECT	**xml.Element**	Object	Client and server scripts	Represents an element in an XML document. Elements may contain attributes, other elements, or text. If an element contains text, the text is represented in a text node of type TEXT_NODE.
OBJECT	**xml.Node**	Object	Client and server scripts	Represents a generic XML node in an XML document. A node can be a Document, Element, or Attribute.
OBJECT	**xml.Parser**	Object	Client and server scripts	Encapsulates the functionality used by NetSuite to parse XML.
OBJECT	**xml.XPath**	Object	Client and server scripts	Encapsulates the functionality used by NetSuite to run XPath expressions. XPath is a standard for enumerating paths in an XML document collection.
METHOD	**xml.escape(options)**	string	Client and server scripts	Prepares a string for use in XML by escaping XML markup, such as angle brackets, quotation marks, and ampersands.
METHOD	**xml.validate(options)**	void	Server scripts	Validates an XML document against an XML Schema (XSD).
ENUM	**xml.NodeType**	string (read-only)	Client and server scripts	Holds the string values for the supported node types. Use this enum to set the Node.nodeType property.

Attr Object Members

The following members are called on the xml.Attr object.

Return Type / Value Type

Supported Script Types

Attr.ownerElement

xml.Element (read-only)

Client and server scripts

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
PROPERTY	**Attr.ownerElement**	xml.Element (read-only)	Client and server scripts	The xml.Element object that is the parent of the xml.Attr object.
PROPERTY	**Attr.name**	string (read-only)	Client and server scripts	The name of an attribute.
PROPERTY	**Attr.specified**	boolean	Client and server scripts	Returns true if the attribute value is set in the parsed XML document, and false if it is a default value in a DTD or Schema.
PROPERTY	**Attr.value**	string	Client and server scripts	Value of an attribute. The value of the attribute is returned as a string. Character and general entity references are replaced with their values.

Document Object Members

In addition to the Document object members, Document objects inherit the members of the Node object. The methods and properties associated with a Node object can be used as members of a Document object. For more information, see Node Object Members.

The following members are called on the xml.Document object.

Return Type / Value Type

Supported Script Types

Document.adoptNode(options)

Client and server scripts

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**Document.adoptNode(options)**	xml.Node	Client and server scripts	Attempts to adopt a node from another document to this document.
METHOD	**Document.createAttribute(options)**	xml.Attr	Client and server scripts	Creates an attribute node of type ATTRIBUTE_NODE with the optional specified value.
METHOD	**Document.createAttributeNS(options)**	xml.Attr	Client and server scripts	Creates an attribute node of type ATTRIBUTE_NODE, with the specified namespace value and optional specified value.
METHOD	**Document.createCDATASection(options)**	xml.Node	Client and server scripts	Creates a CDATA section node of type DOCUMENT_FRAGMENT_NODE with the specified data.
METHOD	**Document.createComment(options)**	xml.Node	Client and server scripts	Creates a Comment node of type COMMENT_NODE with the specified string.
METHOD	**Document.createDocumentFragment()**	xml.Node	Client and server scripts	Creates a node of type DOCUMENT_FRAGMENT_NODE.
METHOD	**Document.createElement(options)**	xml.Element	Client and server scripts	Creates a new node of type ELEMENT_NODE with the specified name.
METHOD	**Document.createElementNS(options)**	xml.Element	Client and server scripts	Creates a new node of type ELEMENT_NODE with the specified namespace URI and name.
METHOD	**Document.createProcessingInstruction(options)**	xml.Node	Client and server scripts	Creates a new node of type PROCESSING_INSTRUCTION_NODE with the specified target and data.
METHOD	**Document.createTextNode(options)**	xml.Node	Client and server scripts	Creates a new node of type TEXT_NODE.
METHOD	**Document.getElementById(options)**	xml.Element	Client and server scripts	Returns the element that has an ID attribute with the specified value as an xml.Element object.
METHOD	**Document.getElementsByTagName(options)**	xml.Element[]	Client and server scripts	Returns an array of xml.Element objects with a specific tag name, in the order in which they appear in the XML document.
METHOD	**Document.getElementsByTagNameNS(options)**	xml.Element[]	Client and server scripts	Returns an array of xml.Element objects with a specific tag name and namespace, in the order in which they appear in the XML document.
METHOD	**Document.importNode(options)**	xml.Node	Client and server scripts	Imports a node from another document to this document. Creates a new copy of the source node.
PROPERTY	**Document.doctype**	Object (read-only)	Client and server scripts	Returns a node of type DOCUMENT_TYPE_NODE that represents the doctype of the XML document.
PROPERTY	**Document.documentElement**	xml.Element (read-only)	Client and server scripts	Root node of the XML document.
PROPERTY	**Document.documentURI**	string (read-only)	Client and server scripts	Location of the document or null if undefined.
PROPERTY	**Document.inputEncoding**	string (read-only)	Client and server scripts	Encoding used for an XML document at the time the document was parsed.
PROPERTY	**Document.xmlEncoding**	string (read-only)	Client and server scripts	Part of the XML declaration, the XML encoding of the XML document.
PROPERTY	**Document.xmlStandalone**	boolean	Client and server scripts	Part of the XML declaration, returns true if the current XML document is standalone or returns false if it is not.
PROPERTY	**Document.xmlVersion**	string	Client and server scripts	Part of the XML declaration, the version number of the XML document.

Element Object Members

In addition to the Element object members, Element objects inherit the members of the Node object. The methods and properties associated with a Node object can be used as members of a Element object. For more information, see Node Object Members.

The following members are called on the xml.Element object.

Return Type / Value Type

Supported Script Types

Element.getAttribute(options)

Client and server scripts

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**Element.getAttribute(options)**	string	Client and server scripts	Returns the value of the specified attribute.
METHOD	**Element.getAttributeNode(options)**	xml.Attr	Client and server scripts	Retrieves an attribute node by name.
METHOD	**Element.getAttributeNodeNS(options)**	string	Client and server scripts	Returns an attribute node with the specified namespace URI and local name.
METHOD	**Element.getAttributeNS(options)**	xml.Attr	Client and server scripts	Returns an attribute value with the specified namespace URI and local name.
METHOD	**Element.getElementsByTagName(options)**	xml.Element[]	Client and server scripts	Returns an array of descendant xml.Element objects with a specific tag name, in the order in which they appear in the XML document.
METHOD	**Element.getElementsByTagNameNS(options)**	xml.Element[]	Client and server scripts	Returns an array of descendant xml.Element objects with a specific tag name and namespace, in the order in which they appear in the XML document.
METHOD	**Element.hasAttribute(options)**	boolean	Client and server scripts	Returns true if the current element has an attribute with the specified name or if that attribute has a default value. Otherwise, returns false.
METHOD	**Element.hasAttributeNS(options)**	boolean	Client and server scripts	Returns true if the current element has an attribute with the specified local name and namespace or if that attribute has a default value. Otherwise, returns false.
METHOD	**Element.removeAttribute(options)**	void	Client and server scripts	Removes the attribute with the specified name.
METHOD	**Element.removeAttributeNode(options)**	xml.Attr	Client and server scripts	Removes the attribute specified as a xml.Attr object.
METHOD	**Element.removeAttributeNS(options)**	void	Client and server scripts	Removes the attribute with the specified namespace URI and local name.
METHOD	**Element.setAttribute(options)**	void	Client and server scripts	Adds a new attribute with the specified name. If an attribute with that name is already present in the element, its value is changed to the value specified in method argument.
METHOD	**Element.setAttributeNode(options)**	xml.Attr	Client and server scripts	Adds the specified attribute node. If an attribute with the same name is already present in the element, it is replaced by the new one.
METHOD	**Element.setAttributeNodeNS(options)**	xml.Attr	Client and server scripts	Adds the specified attribute node. If an attribute with the same local name and namespace URI is already present in the element, it is replaced by the new one.
METHOD	**Element.setAttributeNS(options)**	void	Client and server scripts	Adds a new attribute with the specified name and namespace URI. If an attribute with the same name and namespace URI is already present in the element, its value is changed to the value specified in method argument.
PROPERTY	**Element.tagName**	string (read-only)	Client and server scripts	The tag name of this xml.Element object.

Node Object Members

The following members are called on the xml.Node object.

Return Type / Value Type

Supported Script Types

Node.appendChild(options)

Client and server scripts

Appends a node after the last child node of a specific element node. Returns the new child node.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	**Node.appendChild(options)**	xml.Node	Client and server scripts	Appends a node after the last child node of a specific element node. Returns the new child node.
METHOD	**Node.cloneNode(options)**	xml.Node	Client and server scripts	Creates a copy of a node. Returns the copied node.
METHOD	**Node.compareDocumentPosition(options)**	number	Client and server scripts	Returns a number that reflects where two nodes are located, compared to each other.
METHOD	**Node.hasAttributes()**	boolean	Client and server scripts	Returns true if the current node has any attributes. Note that only element nodes can have attributes.
METHOD	**Node.hasChildNodes()**	boolean	Client and server scripts	Returns true if the current node has child nodes or returns false if the current node does not have child nodes.
METHOD	**Node.insertBefore(options)**	xml.Node	Client and server scripts	Inserts a new child node before an existing child node for the current node.
METHOD	**Node.isDefaultNamespace(options)**	boolean	Client and server scripts	Returns true if the specified namespace uniform resource identifier (URI) is the default namespace for the current node or returns false if the specified namespace is not the default namespace.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Node.isEqualNode(options)	boolean	Client and server scripts	Returns true if two nodes are equal or returns false if two nodes are not equal.
METHOD	Node.isSameNode(options)	boolean	Client and server scripts	Returns true if two nodes reference the same object or returns false if two nodes do not reference the same object.
METHOD	Node.lookupNamespaceURI(options)	string	Client and server scripts	Returns the namespace uniform resource identifier (URI) that matches the specified namespace prefix.
METHOD	Node.lookupPrefix(options)	string	Client and server scripts	Returns the namespace prefix associated with the specified namespace uniform resource identifier (URI).
METHOD	Node.normalize()	void	Client and server scripts	Puts all text nodes underneath a node, including attribute nodes, into a normal form.
METHOD	Node.removeChild(options)	xml.Node	Client and server scripts	Removes the specified child node. Returns the removed child node.
METHOD	Node.replaceChild(options)	xml.Node	Client and server scripts	Replaces a specific child node with another child node in a list of child nodes.
PROPERTY	Node.attributes	Object (read-only)	Client and server scripts	Key-value pairs for all attributes for an xml.Element node. Returns null for all other node types.
PROPERTY	Node.baseURI	string (read-only)	Client and server scripts	Absolute base uniform resource identifier (URI) of a node or null if the URI cannot be determined.
PROPERTY	Node.childNodes	xml.Node[] (read-only)	Client and server scripts	Array of all child nodes of a node or an empty array if there are no child nodes.
PROPERTY	Node.firstChild	xml.Node (read-only)	Client and server scripts	First child node for a specific node or null if there are no child nodes.
PROPERTY	Node.lastChild	xml.Node (read-only)	Client and server scripts	Last child node for a specific node or null if there is no last child node.
PROPERTY	Node.localName	string (read-only)	Client and server scripts	The local part of the qualified name of a node.
PROPERTY	Node.namespaceURI	string (read-only)	Client and server scripts	The namespace uniform resource identifier (URI) of a node or null if there is no namespace URI for the node.
PROPERTY	Node.nextSibling	xml.Node (read-only)	Client and server scripts	The next node in a node list or null if the current node is the last node.
PROPERTY	Node.nodeName	string (read-only)	Client and server scripts	Name of a node, depending on the type. For example, for a node of type xml.Element, the name is the name of the element.
PROPERTY	Node.nodeType	string	Client and server scripts	The type of node defined as a value from the xml.NodeType enum.
PROPERTY	Node.nodeValue	string	Client and server scripts	The value of a node, depending on its type.
PROPERTY	Node.ownerDocument	xml.Document (read-only)	Client and server scripts	The root element for a node as a xml.Document object.
PROPERTY	Node.parentNode	xml.Node (read-only)	Client and server scripts	The parent node of a node.
PROPERTY	Node.prefix	string	Client and server scripts	The namespace prefix of the node, or null if the node does not have a namespace.
PROPERTY	Node.previousSibling	xml.Node (read-only)	Client and server scripts	The previous node in a node list or null if the current node is the first node.
PROPERTY	Node.textContent	string	Client and server scripts	The textual content of a node and its descendants.

Parser Object Members

The following members are called on the xml.Parser object.

Return Type / Value Type

Supported Script Types

Parser.fromString(options)

Client and server scripts

Parses a string into a W3C XML document object.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	Parser.fromString(options)	xml.Document	Client and server scripts	Parses a string into a W3C XML document object.
METHOD	Parser.toString(options)	string	Client and server scripts	Converts (serializes) an xml.Document object into a string.

XPath Object Members

The following members are called on the xml.XPath object.

Return Type / Value Type

Supported Script Types

XPath.select(options)

Client and server scripts

Selects an array of nodes from an XML document using an XPath expression.

MEMBER TYPE	NAME	RETURN TYPE / VALUE TYPE	SUPPORTED SCRIPT TYPES	DESCRIPTION
METHOD	XPath.select(options)	xml.Node[]	Client and server scripts	Selects an array of nodes from an XML document using an XPath expression.

▶ Official NetSuite Script Samples (1 Sample(s))

📄 Sample 1: N/xml Module Script Samples

```
&lt;bookstore xmlns:b="http://www.qualifiednamespace.com/book"&gt;
    &lt;b:book category="cooking"&gt;
        &lt;b:title lang="en"&gt;Everyday Italian&lt;/b:title&gt;
        &lt;b:author&gt;Giada De Laurentiis&lt;/b:author&gt;
        &lt;b:year&gt;2005&lt;/b:year&gt;
        &lt;b:price&gt;30.00&lt;/b:price&gt;
    &lt;/b:book&gt;
    &lt;b:book category="children"&gt;
        &lt;b:title lang="en"&gt;Harry Potter&lt;/b:title&gt;
        &lt;b:author&gt;J K. Rowling&lt;/b:author&gt;
        &lt;b:year&gt;2005&lt;/b:year&gt;
        &lt;b:price&gt;29.99&lt;/b:price&gt;
    &lt;/b:book&gt;
    &lt;b:book category="web"&gt;
        &lt;b:title lang="en"&gt;XQuery Kick Start&lt;/b:title&gt;
        &lt;b:author&gt;James McGovern&lt;/b:author&gt;
        &lt;b:author&gt;Per Bothner&lt;/b:author&gt;
        &lt;b:author&gt;Kurt Cagle&lt;/b:author&gt;
        &lt;b:author&gt;James Linn&lt;/b:author&gt;
        &lt;b:author&gt;Vaidyanathan Nagarajan&lt;/b:author&gt;
        &lt;b:year&gt;2003&lt;/b:year&gt;
        &lt;b:price&gt;49.99&lt;/b:price&gt;
    &lt;/b:book&gt;
    &lt;b:book category="web" cover="paperback"&gt;
        &lt;b:title lang="en"&gt;Learning XML&lt;/b:title&gt;
        &lt;b:author&gt;Erik T. Ray&lt;/b:author&gt;
        &lt;b:year&gt;2003&lt;/b:year&gt;
        &lt;b:price&gt;39.95&lt;/b:price&gt;
    &lt;/b:book&gt;
&lt;/bookstore&gt;
```